

POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Sieci Teleinformatycznych

Praca dyplomowa inżynierska

PROJEKT I WDROŻENIE APLIKACJI DO ZBIERANIA I WIZUALIZACJI DANYCH SIECIOWYCH W KLASTRZE KUBERNETES NA PLATFORMIE MICROSOFT AZURE

Franciszek Mazoń, 156192
Jakub Medyński, 156172
Klaudiusz Zdrojewski, 156921

Promotor
dr inż. Michał Weissenberg

POZNAŃ 2026

Spis treści

1 Wstęp **1**

1.1 Uzasadnienie wyboru tematu . . . 1
1.2 Cel pracy . . . 2
1.3 Struktura pracy . . . 2
1.4 Problemy i trudności przy realizacji pracy . . . 3

2 Analiza problemu i wymagań **4**

2.1 Opis domeny problemu . . . 4
2.2 Wymagania funkcjonalne i niefunkcjonalne . . . 4
2.2.1 Wymagania funkcjonalne . . . 4
2.2.2 Wymagania niefunkcjonalne . . . 5
2.3 Założenia projektowe . . . 7
2.3.1 Założenia ogólne . . . 7
2.3.2 Założenia technologiczne . . . 7
2.4 Przegląd podobnych systemów i rozwiązań . . . 7
2.4.1 Podstawowe informacje o systemach . . . 9
2.4.2 Zakres i parametry danych technicznych . . . 10
2.4.3 Funkcjonalność i wizualizacja danych . . . 12
2.4.4 Aspekty prywatności i bezpieczeństwa . . . 13
2.4.5 Architektura systemu . . . 14
2.4.6 Podsumowanie . . . 15

3 Projekt systemu **16**

3.1 Ogólna koncepcja architektury . . . 17
3.2 Architektura sieciowa . . . 18
3.2.1 Ogólny model sieciowy systemu . . . 18
3.2.2 Model sieciowy przesyłu danych telemetrycznych . . . 19
3.2.3 Sieć klastra Kubernetes . . . 21
3.3 Omówienie technologii i narzędzi wykorzystanych w implementacji . . . 22
3.3.1 Platforma chmurowa i model wdrożenia . . . 22
Microsoft Azure jako środowisko uruchomieniowe . . . 22
Warstwa obliczeniowa: AKS i Azure Functions . . . 22
Warstwa danych i kolejkowania . . . 23
Rejestr obrazów i artefakty uruchomieniowe . . . 24
3.3.2 Automatyzacja wdrożeń i zarządzanie konfiguracją . . . 24
Zarządzanie projektem i kodem źródłowym . . . 24
Pipelines CI/CD i Infrastructure as Code . . . 25

Wdrożenie na Kubernetes: Helm i GitOps . . . 25
Zarządzanie konfiguracją aplikacji . . . 25
3.3.3 Bezpieczeństwo i segmentacja sieciowa . . . 26
Warstwa brzegowa: WAF i API Management . . . 26
Ruch przychodzący do klastra i TLS . . . 26
Sekrety i dane wrażliwe . . . 27
Dostęp prywatny do usług: Private Link, Private Endpoint i DNS . . . 27
Monitoring i obserwowalność . . . 27
3.3.4 Technologie warstwy aplikacyjnej . . . 28
Aplikacja mobilna Android . . . 28
Backend: Python i Flask . . . 29
Frontend: React i deck.gl . . . 29
3.4 Projekt klastra Kubernetes . . . 30
3.5 Model danych . . . 32
3.6 Projekt przepływu danych . . . 33
3.6.1 Generowanie i przesył danych z aplikacji mobilnej . . . 34
3.6.2 Warstwa brzegowa i integracyjna API . . . 34
3.6.3 Walidacja i normalizacja danych w Function App . . . 34
3.6.4 Asynchroniczne buforowanie danych w kolejce . . . 34
3.6.5 Przetwarzanie danych i trwały zapis w bazie danych . . . 35
3.6.6 Podsumowanie . . . 35

4 Implementacja **36**
4.1 Konfiguracja klastra Kubernetes . . . 36
4.1.1 Podział plików konfiguracyjnych . . . 36
4.1.2 Opis działania Flux i podstawowe pliki . . . 38
4.1.3 Aplikacja - konfiguracja . . . 38
Aplikacja backendowa - konfiguracja . . . 38
Aplikacja frontendowa - konfiguracja . . . 39
4.1.4 Flux - konfiguracja . . . 40
4.1.5 Infrastruktura - konfiguracja . . . 41
4.2 Implementacja warstwy gromadzenia i przetwarzania danych . . . 44
4.2.1 Implementacja aplikacji mobilnej SignalMap . . . 45
Architektura systemu . . . 46
Mechanizmy gromadzenia danych telemetrycznych . . . 49
Adaptacyjna polityka próbkowania i praca w tle . . . 51
Moduł testów wydajnościowych . . . 52
Komunikacja z infrastrukturą chmurową . . . 54
Projekt interfejsu użytkownika . . . 56
Monitorowanie sygnału i testy wydajnościowe . . . 57
4.2.2 Zarządzanie infrastrukturą API . . . 61
Architektura fasady i abstrakcja backendu . . . 61
Mechanizmy bezpieczeństwa i uwierzytelniania . . . 61
Kontrola przepływu i ochrona przed przeciążeniem . . . 61
Optymalizacja odpowiedzi i zarządzanie pamięcią podręczną . . . 62
Transformacja URI . . . 62
4.2.3 Implementacja warstwy odbioru i walidacji danych . . . 63

Mechanizmy identyfikacji i autoryzacji 63
Przetwarzanie pomiarów sygnału i telemetrii 63
Obsługa testów wydajnościowych 63
4.2.4 Asynchroniczna magistrala komunikatów (Azure Service Bus) 64
Konfiguracja kolejki i mechanizmy buforujące 64
Struktura komunikatu i metadane 64
4.2.5 Implementacja warstwy procesowania i utrwalania danych 65
Architektura wyzwalania i przetwarzanie wsadowe 65
Normalizacja danych i mapowanie operatorów 65
Integracja z silnikiem przestrzennym PostGIS 65
Trwałość danych i obsługa transakcji 65
4.3 Implementacja relacyjnej bazy danych 66
4.3.1 Relacje fizyczne i powiązania logiczne 67
4.3.2 Typy danych i optymalizacja przestrzenna 67
4.3.3 Optymalizacja zapytań i indeksowanie 67
4.3.4 Rozszerzenia i funkcje systemowe 68
4.4 Implementacja backendu 68
4.4.1 Architektura aplikacji i routing 68
4.4.2 Zarządzanie konfiguracją 69
4.4.3 Komunikacja z bazą danych i pula połączeń 69
4.4.4 Monitoring dostępności bazy danych 69
4.4.5 Implementacja endpointów danych 70
Endpoint telemetrii podstawowej 70
Endpoint telemetrii z dopasowaniem BTS 70
4.4.6 Bezpieczeństwo i walidacja danych 71
4.4.7 Health checks i integracja z Kubernetes 72
4.4.8 Kontrolowane zakończenie działania aplikacji 72
4.4.9 Skalowanie i wdrażanie w Kubernetes 73
4.4.10 Proces wdrażania nowej wersji 73
Przywracanie poprzedniej wersji 74
Zasoby i limity 74
Automatyczne skalowanie 74
4.5 Implementacja frontendu 75
4.5.1 Architektura aplikacji 75
4.5.2 Komunikacja z API i zarządzanie danymi 76
4.5.3 Komponent nagłówka 77
4.5.4 Panel boczny z filtrami 77
4.5.5 Komponent mapy i system wizualizacji 78
4.5.6 System interakcji użytkownika 79
4.5.7 Komponent statystyk 79
4.5.8 Zarządzanie stanem i optymalizacja wydajności 80
4.5.9 Proces budowania i wdrożenia 80
4.5.10 Wdrożenie w Kubernetes 80
4.5.11 Proces wdrażania nowej wersji 81
4.6 CI/CD i zarządzanie kontenerami 81
4.7 Implementacja infrastruktury przez Terraform 82
4.8 Napotkane problemy i ich rozwiązanie 83

4.8.1 Problem: Niedokładne dopasowanie stacji BTS do pomiarów 83
4.8.2 Problem: Ograniczenia domyślnej autoryzacji platformy Azure 84
4.8.3 Migracja i ujednolicanie nieustrukturyzowanych danych BTSearch 85
4.8.4 Zapewnienie ciągłości transmisji danych w warunkach braku łączności . . . 86
4.9 Testy implementacji 88
4.9.1 Weryfikacja procesu rejestracji i autoryzacji urządzeń 88
Walidacja identyfikatorów sprzętowych Android ID 88
Autoryzacja wydawania tokenów 89
Weryfikacja autoryzacji tokenem JWT i przesyłania danych 90
Walidacja typów i zakresów danych telemetrycznych 91
Eliminacja duplikatów i mechanizm Duplicate Detection 92
4.9.2 Weryfikacja mechanizmów ochrony przed przeciążeniem (Rate Limiting) . . 93

5 Bezpieczeństwo, autoryzacja oraz monitoring klastra 95
5.1 Analiza zagrożeń i strategia ochrony danych 95
5.2 Zabezpieczenie warstwy brzegowej 96
5.3 Autoryzacja dostępu i mechanizm JWT 97
5.3.1 Proces wydawania i struktura tokena JWT 97
5.3.2 Logika weryfikacji i zabezpieczenie punktów końcowych 98
5.3.3 Implementacja klienta mobilnego i mechanizm ponawiania 99
5.3.4 Zarządzanie kluczem sygnującym 99
5.4 Prywatność i anonimizacja użytkowników 99
5.4.1 Proces deterministycznej anonimizacji 99
5.4.2 Generowanie identyfikatora ShortCode 100
5.4.3 Idempotencja i retencja danych 100
5.4.4 Ciągłość tożsamości po reinstalacji aplikacji 101
5.5 Ciągłość działania i ochrona przed utratą danych 101
5.5.1 Asynchroniczne przetwarzanie z wykorzystaniem Azure Service Bus 102
5.5.2 Mechanizmy ochrony danych w relacyjnej bazie danych 102
5.6 Monitoring 103
5.6.1 Architektura systemu monitoringu 103
5.6.2 Prometheus - zbieranie i przechowywanie metryk 103
5.6.3 Grafana - wizualizacja danych 103
5.6.4 PostgreSQL Exporter - monitoring bazy danych 104
5.6.5 Node Exporter i Kube State Metrics 104
5.6.6 Ingress Nginx Controller 104
5.6.7 Prometheus Operator i automatyzacja konfiguracji 105
5.6.8 Przepływ danych w systemie monitoringu 105
5.6.9 Przykładowe wizualizacje metryk w Grafanie 105

6 Wnioski i kierunki rozwoju 113
6.1 Ocena rozwiązania 113
6.2 Możliwości dalszego rozwoju 113
6.2.1 Aplikacja mobilna 113
6.2.2 Warstwa zbierania danych i infrastruktura chmurowa 114
6.2.3 Warstwa bazy danych 115
6.2.4 Wizualizacja mapy i moduł analityczny 116

6.2.5 Mechanizmy klastra Kubernetes 117
6.3 Podsumowanie 118

Literatura **120**

A Repozytorium kodu **132**

Streszczenie

Celem pracy było stworzenie systemu do gromadzenia i analizy danych telemetrycznych (m.in. parametrów radiowych RSRP, RSRQ oraz informacji o stacjach BTS) oraz testów prędkości transmisji danych. Rozwiązanie oparto na natywnej aplikacji mobilnej dla systemu Android oraz infrastrukturze chmurowej Microsoft Azure.

Warstwa obliczeniowa wykorzystuje klaster Azure Kubernetes Service (AKS) działający na skalowalnych zestawach maszyn wirtualnych VMSS. Bezpieczeństwo punktu wejścia zapewnia zapora WAF oraz brama APIM, natomiast routing wewnętrzny i zarządzanie certyfikatami obsługują obiekty Ingress (Nginx) oraz Cert-Manager. Dane przesyłane są asynchronicznie przez usługi Azure Function App i kolejkę Service Bus, trafiając ostatecznie do relacyjnej bazy danych PostgreSQL z rozszerzeniem PostGIS. Zebrane pomiary są przetwarzane i udostępniane przez warstwę backendową zaimplementowaną w języku Python, a następnie prezentowane użytkownikowi w aplikacji frontendowej zbudowanej w oparciu o bibliotekę React w formie interaktywnej mapy. Dane wrażliwe i sekrety chronione są w usłudze Azure Key Vault i dostarczane do kontenerów przez Secrets Store CSI Driver.

Proces wdrożeniowy zautomatyzowano za pomocą potoków Azure Pipelines (CI/CD), obrazów w Azure Container Registry (ACR) oraz metodologii GitOps. System monitorowany jest przez stos technologiczny Prometheus i Grafana.

Abstract

The project presents a scalable system for collecting and analyzing mobile telemetry data (including radio parameters such as RSRP, RSRQ and BTS information) and speedtests. The solution integrates a native Android application with Microsoft Azure cloud infrastructure.

The backend is hosted on Azure Kubernetes Service (AKS) utilizing VMSS nodes for scalability. Inbound security is managed via WAF and APIM, while internal traffic routing and SSL/TLS management are handled by Ingress controllers and Cert-Manager. Data flows asynchronously through Azure Functions and Service Bus queues to a relational PostgreSQL (PostGIS) database. The collected measurements are processed and exposed by a backend layer implemented in Python, and subsequently presented to the user in a frontend application built using the React library in the form of an interactive map. Sensitive information and secrets are secured in Azure Key Vault and delivered to pods via Secrets Store CSI Driver.

The software lifecycle is automated using Azure Pipelines (CI/CD), Azure Container Registry (ACR), and GitOps methodology. System observability is ensured by Prometheus and Grafana, providing real-time infrastructure monitoring.

Podział pracy w zespole

Praca ta została wykonana wspólnie przez trzy osoby. Poniżej znajduje się dokładny podział prac, jakie wykonano.

Jakub Medyński w ramach tego projektu wykonał następujące fragmenty:

- zarządzał platformą Microsoft Azure oraz Azure DevOps, stworzył Serive Connection łączący obie platformy oraz stworzył szablon repozytoriów;
- opracował rozdział 1 dotyczący ogólnego zarysu pracy i problemów;
- opracował sekcje opisująca ogólny koncept architektury i wdrożenia systemu wraz z przygotowaniem schematem architektury ogólnej systemu (3.1);
- wykonywał schemat ogólny modelu sieciowego systemu (3.2.1);
- opracował schemat opisująca działanie sieci klastra Kubernetes (3.2.3) wraz z opisem działania sieci w narzędziu Kubernetes;
- opracował opisy technologii w rozdziale 3.3, w tym sekcje: 3.3.1, 3.3.3;
- opracował i wykonał projekt z rozdziału 3.4;
- wykonał i opracował rozdział 4.1, w którym znajdowało się wykonanie konfiguracji narzędzi zaimplementowanych w strukturę klastra Kubernetes, przedstawił sposób użycia poszczególnych narzędzi w całym systemie
- opracował opis poszczególnych plików konfiguracyjnych klastra Kubernetes (4.1.1, 4.1.2, 4.1.3. 4.1.4, 4.1.5, 4.5.10);
- wykonał i opracował mechanizmy CI/CD na platformie Azure DevOps (4.5.11, 4.6) wraz z przygotowaniem agentów do wykonywania pipelinów. Odpowiadał za stworzenie zasobów ACR i ich konfiguracje do współpracy z AKS
- wykonał i opracował mechanizmy wdrażania zasobów na platformę Microsoft Azure za pomocą pipelinów z narzędziem Terraform (4.7), stworzył pliki Terraform opisujące poszczególne zasoby;
- wykonywał wdrożenie monitoringu dla usług wdrożony na plafromie Azure (5.6) oraz opracował opis przykładowych dashboardów, odpowiadał za stworzenie i dostosowanie Grafany do środowiska projektu (4.1.5, 5.6);
- opracował możliwości rozwoju dla klastra Kubernetes (6.2.5);

Franciszek Mazoń w ramach tego projektu wykonał następujące fragmenty:

- opracował rozdział 2 obejmujący opis domeny problemu (2.1), analizę wymagań funkcjonalnych i niefunkcjonalnych systemu (2.2–2.2.2) oraz założenia projektowe wraz z założeniami ogólnymi (2.3–2.3.2);
- współuczestniczył w opracowaniu koncepcji architektury systemu oraz przygotowaniu schematu konceptualnego architektury (3.1);
- opracował opisy technologii i narzędzi wykorzystanych w warstwie frontendowej systemu w rozdziale 3.3, w tym technologie React i deck.gl, komunikację z backendem, zarządzanie stanem aplikacji oraz proces budowania i wdrażania;
- współuczestniczył w przygotowaniu schematu przepływu danych telemetrycznych (rys. 3.6);
- zaprojektował i zaimplementował backend systemu, obejmujący architekturę aplikacji i routing (4.4.1), zarządzanie konfiguracją (4.4.2), komunikację z bazą danych i pulę połączeń (4.4.3), monitoring dostępności bazy danych (4.4.4) oraz implementację endpointów danych (4.4.5);
- zaimplementował mechanizmy bezpieczeństwa i walidacji danych (4.4.6), health checks oraz integrację aplikacji z klastrem Kubernetes (4.4.7), kontrolowane zakończenie działania aplikacji (4.4.8) oraz procesy skalowania i wdrażania w środowisku Kubernetes (4.4.9–4.4.10), w tym mechanizmy przywracania poprzedniej wersji i automatycznego skalowania;
- zaprojektował i zaimplementował warstwę frontendową aplikacji webowej, obejmującą architekturę aplikacji (4.5.1), komunikację z API i zarządzanie danymi (4.5.2), komponenty interfejsu użytkownika (4.5.3–4.5.7), system interakcji użytkownika (4.5.6), zarządzanie stanem i optymalizację wydajności (4.5.8) oraz proces budowania i wdrażania aplikacji (4.5.9–4.5.11);
- opracował opis napotkanego problemu niedokładnego dopasowania stacji BTS do punktów pomiarowych wraz z zastosowanym rozwiązaniem (4.8.1);
- opracował rozdział 5.6 dotyczący monitoringu systemu, obejmujący architekturę systemu monitoringu (5.6.1), zbieranie i przechowywanie metryk z wykorzystaniem Prometheus (5.6.2), wizualizację danych w Grafanie (5.6.3), monitoring bazy danych (5.6.4), monitorowanie klastra Kubernetes (5.6.5–5.6.6), automatyzację konfiguracji z użyciem Prometheus Operator (5.6.7), przepływ danych w systemie monitoringu (5.6.8) oraz przykładowe wizualizacje metryk (5.6.9);
- opracował ocenę rozwiązania (6.1), wnioski dotyczące rozwoju modułu wizualizacji mapowej (6.2.4) oraz podsumowanie całej pracy (6.3).

Klaudiusz Zdrojewski w ramach tego projektu wykonał następujące fragmenty:

- opracował rozdział 2.4 dotyczący przeglądu podobnych systemów i istniejących rozwiązań rynkowych;
- współtworzył ogólny koncept architektury systemu (rys. 3.1) oraz opracował koncepcję merytoryczną modelu sieciowego (sekcje 3.2.1–3.2.2), projektując kompleksowy schemat przesyłu danych od urządzeń mobilnych do warstwy składowania (rys. 3.2 i 3.3);

- opracował opisy technologii w rozdziale 3.3, w tym sekcje: 3.3.1 (Warstwa danych i kolejkowania oraz Azure Functions), 3.3.3 (Bezpieczeństwo i segmentacja sieciowa: API Management, Private Link, Private Endpoint i DNS) oraz 3.3.4 (Technologie warstwy aplikacyjnej — aplikacja mobilna Android);
- zaprojektował model danych (3.5) oraz szczegółowy projekt przepływu danych telemetrycznych (3.6.1–3.6.5) wraz z grafiką schematu przepływu (rys. 3.6);
- zrealizował pełny cykl implementacji warstwy gromadzenia i przetwarzania danych (sekcje 4.2.1–4.2.5) — od rozwoju natywnej aplikacji mobilnej SignalMap wraz z adaptacyjnym algorytmem próbkowania (rys. 4.1), po budowę logiki procesowania i utrwalania komunikatów, a także wdrożył strukturę relacyjnej bazy danych (rozdział 4.3);
- opracował dokumentację napotkanych problemów (4.8), w tym: ograniczenia autoryzacji platformy Azure (4.8.2), migrację danych z systemu BTSearch (4.8.3) oraz zapewnienie ciągłości transmisji w warunkach braku łączności (4.8.4);
- zaplanował i przeprowadził kompleksowe testy weryfikacyjne systemu (sekcje 4.9.1–4.9.3), obejmujące walidację mechanizmów bezpieczeństwa i autoryzacji, testy integralności oraz idempotentności danych, a także weryfikację skuteczności polityk ograniczania przepływu (*Rate Limiting*);
- opracował strategię bezpieczeństwa i zapewnienia ciągłości działania systemu (sekcje 5.1–5.5), obejmującą analizę zagrożeń, architekturę autoryzacji JWT (rys. 5.1), deterministyczną anonimizację danych oraz proces generowania identyfikatora *ShortCode* (rys. 5.2), a także mechanizmy ochrony przed utratą danych;
- sformułował wytyczne dotyczące kierunków rozwoju systemu (sekcje 6.2.1–6.2.4), obejmujące ewolucję aplikacji mobilnej, skalowanie warstwy zbierania oraz składowania danych w bazie, a także koncepcję wdrożenia modułów analitycznych i wizualizacji mapy;

Rozdział 1

Wstęp

1.1 Uzasadnienie wyboru tematu

Dynamiczny rozwój sieci telekomunikacyjnych, w szczególności sieci komórkowych LTE, 5G oraz perspektywicznie 6G, powoduje gwałtowny wzrost wymagań względem jakości usług oraz doświadczenia użytkownika końcowego QoE, co wymusza stosowanie zaawansowanych mechanizmów monitorowania parametrów sieciowych i usługowych w czasie rzeczywistym [1, 2]. Narzędzia te analizują m.in. kluczowe wskaźniki jakości usług QoS, takie jak opóźnienie, przepustowość, wskaźniki niepowodzeń połączeń czy dostępność usług, a także syntetyczne miary QoE, będące podstawą oceny satysfakcji użytkowników i spełniania wymogów regulacyjnych [3, 4].

Najwięksi operatorzy i dostawcy rozwiązań sieciowych, tacy jak Orange czy Nokia, dysponują rozbudowanymi, wewnętrznymi platformami klasy OSS/NMS służącymi do monitorowania i raportowania jakości działania sieci w skali krajowej i międzynarodowej. Przykładem są systemy monitoringu ruchu SS7/LTE/IPX oferowane przez Orange, zapewniające wielowymiarową analizę KPI w czasie rzeczywistym, alarmowanie progowe oraz długoterminową analizę historyczną [5]. Z kolei platforma Nokia NetAct stanowi kompleksowy system zarządzania i nadzoru sieci wielotechnologicznych, integrujący dane o alarmach, wydajności, ruchu i konfiguracji, co umożliwia globalny wgląd w stan sieci, korelację zdarzeń i automatyzację działań utrzymaniowych [6]. Systemy tej klasy pozwalają operatorom na bieżące śledzenie poziomu obciążenia, identyfikację wąskich gardeł, wczesne wykrywanie degradacji jakości usług oraz planowanie rozbudowy infrastruktury w oparciu o dane empiryczne, co bezpośrednio przekłada się na utrzymanie wysokiego poziomu QoS i QoE [4, 7].

Z uwagi na fakt, że narzędzia te są wykorzystywane wyłącznie wewnętrznie przez operatorów i nie są dostępne dla użytkowników końcowych, wybór operatora sieci komórkowej przez klientów indywidualnych i małe przedsiębiorstwa opiera się zazwyczaj na informacjach marketingowych zawartych w ofertach handlowych oraz na subiektywnych opiniach użytkowników. Brakuje przy tym transparentnych, niezależnych źródeł obiektywnych danych o jakości usług w konkretnych lokalizacjach geograficznych [8]. W literaturze oraz dokumentach branżowych wskazuje się, że jedną z obiecujących metod wypełnienia tej luki są pomiary crowdsourcingowe z wykorzystaniem urządzeń końcowych użytkowników, które pozwalają na budowę niezależnych map jakości pokrycia i parametrów usług w różnych sieciach [9].

Istnieje zatem potrzeba stworzenia niezależnych i scentralizowanych narzędzi umożliwiających zbieranie, agregację i wizualizację danych pomiarowych pochodzących z wielu urządzeń mobilnych oraz porównywanie parametrów usług telekomunikacyjnych w różnych obszarach geograficznych. Tego rodzaju rozwiązanie dostarcza użytkownikom realny punkt odniesienia przy wyborze przy-

szłego usługodawcy, a jednocześnie może stanowić uzupełnienie wewnętrznych systemów monitorujących operatorów, dostarczając dodatkowego źródła informacji o jakości sieci z perspektywy użytkownika końcowego [1, 9]

1.2 Cel pracy

Istotnym problemem w tworzeniu takiego systemu jest brak bezpośredniego dostępu do danych sieciowych poszczególnych operatorów. Jedynym realnym źródłem informacji są urządzenia mobilne aktualnie podłączone do infrastruktury dostawców usług. W niniejszej pracy do pozyskiwania danych wykorzystano aplikację mobilną, której zadaniem jest zanonimizowane zbieranie istotnych parametrów sieciowych. Zebrane informacje są następnie przechowywane w bazach danych oraz prezentowane w przejrzystej formie za pomocą aplikacji webowej utrzymywanej w środowisku chmurowym Microsoft Azure.

Ze względu na dynamiczny rozwój rozwiązań chmurowych oraz technologii konteneryzacji, system został oparty na platformie Kubernetes. Pozwoliło to na uzyskanie skalowalności oraz elastyczności aplikacji w warunkach zmiennego obciążenia systemu. W pracy zwrócono również uwagę na odpowiednią organizację zarządzania kodem źródłowym projektu, automatyzację procesów wdrożeniowych z wykorzystaniem platformy Azure DevOps oraz podjęto działania mające na celu stworzenie podstawowej infrastruktury umożliwiającej monitorowanie stanu systemu w czasie rzeczywistym.

Co za tym idzie, głównym celem pracy było zaprojektowanie i wdrożenie systemu SignalMap do zbierania, przetwarzania i wizualizacji danych sieciowych z urządzeń mobilnych, działającego w klastrze Kubernetes na platformie Microsoft Azure.

Do celów szczegółowych należało:

- opracowanie architektury chmurowej (AKS, Service Bus, Function Apps, PostgreSQL/PostGIS) zapewniającej skalowalność i wysoką dostępność,
- implementacja aplikacji mobilnej do anonimizowanego zbierania pomiarów,
- implementacja backendu + frontendu z interaktywną mapą i filtrami,
- przygotowanie pipeline'ów CI/CD w Azure DevOps,
- zaprojektowanie podstawowego monitoringu i mechanizmów bezpieczeństwa (HTTPS, private endpoints, Key Vault, autoryzacja API).

W ramach pracy zaproponowano również szereg działań, które pozwolą na zwiększenie niezawodności oraz bezpieczeństwa systemu.

1.3 Struktura pracy

Praca składa się z siedmiu rozdziałów:

- W rozdziale 1 przedstawiono wprowadzenie do tematyki pracy, jej cel i zakres, uzasadnienie wyboru tematu oraz ogólną strukturę opracowania.
- Rozdział 2 poświęcony jest analizie problemu oraz określeniu wymagań funkcjonalnych i niefunkcjonalnych projektowanego systemu, a także przeglądowi podobnych rozwiązań dostępnych na rynku.

- W rozdziale 3 opisano projekt systemu, w tym ogólną koncepcję architektury, projekt klastra Kubernetes, przepływ danych oraz model danych.
- Rozdział 4 zawiera opis implementacji systemu, obejmujący konfigurację środowiska, warstwę zbierania danych, warstwę wizualizacji oraz procesy automatyzacji i wdrażania.
- Rozdział 5 dotyczy zagadnień bezpieczeństwa, monitoringu, opisane będą poszczególne dane zbieranie i prezentowane przez narzędzie Grafana.
- Rozdział 6 stanowi podsumowanie pracy, zawiera ocenę zaprojektowanego rozwiązania oraz wskazuje możliwe kierunki dalszego rozwoju systemu.

1.4 Problemy i trudności przy realizacji pracy

W celu stworzenia zaproponowanego systemu wykorzystano platformę Microsoft Azure. W tym celu założono konto Azure For Students. Konto to pozwala na wykorzystanie 100 dolarów w ciągu roku na przeróżne usługi chmurowe. Podczas korzystania z tego typu konta można także uruchomić konkretne usługi na określony czas za darmo. Dopiero po tym czasie płaci się za nie odliczają kwotę od 100 dolarów. Był to czynnik, który wymusił ciągłą analizę kosztów generowanych przez Azure.

Podczas tworzenia pracy postanowiono w aplikacji webowej dodać elemety zawierające informację o stacjach BTS (Base Transceiver Station). W tym celu skontaktowano się z przedstawicielem serwisu btsearch.pl w celu uzyskana plików zawierających informację o lokalizacjach wszystkich dostępnym w serwisie stacji BTS w Polsce. Dane te udało się pozyskać i wykorzystać w pracy.

Podczas konfiguracji wykupiono domenę *signalmap.com.pl*, którą zarejestrowana firma *lh.pl*. Skonfigurowano, aby adres publiczny AKS był rozwiązywany na tą domenę. Dodatkowo dla zagadnień monitoringu dodano subdomenę *grafana.signalmap.com.pl*.

Rozdział 2

Analiza problemu i wymagań

2.1 Opis domeny problemu

Współczesne systemy telekomunikacyjne, w szczególności sieci komórkowe, stanowią fundamentalną infrastrukturę społeczeństwa informacyjnego, umożliwiając miliardom ludzi dostęp do internetu i kluczowych usług cyfrowych [10]. Powszechne wykorzystanie tych systemów sprawia, że utrzymanie odpowiedniej jakości połączenia ma kluczowe znaczenie. W celu zapewnienia niezawodności i utrzymania odpowiednich parametrów sieci niezbędne jest ich stałe monitorowanie oraz analiza parametrów jakościowych.

Do podstawowych parametrów opisujących sieć bezprzewodową należą przede wszystkim zasięg, przepustowość oraz siła sygnału. Wartość siły sygnału jest zmienna i zależy od wielu czynników środowiskowych, takich jak odległość od stacji bazowej, ukształtowanie terenu, przeszkody architektoniczne, warunki atmosferyczne, zjawiska propagacyjne (np. wielodrogowość), interferencje oraz chwilowe obciążenie komórki sieciowej [11]. Z tego względu wiarygodne dane pomiarowe odnoszące się do siły sygnału w różnych lokalizacjach mogą dostarczyć cennych informacji na temat rzeczywistego pokrycia sieci oraz miejsc wymagających optymalizacji infrastruktury.

Celem projektu jest opracowanie systemu umożliwiającego automatyczne zbieranie informacji między innymi o sile sygnału sieci komórkowej w połączeniu z lokalizacją geograficzną urządzenia dokonującego pomiaru. Dane te będą zbierane za pomocą urządzeń mobilnych, a następnie wizualizowane w postaci interaktywnej mapy. Takie podejście umożliwia analizę parametrów sieci w czasie rzeczywistym oraz może stanowić podstawę do badań nad optymalizacją rozmieszczenia nadajników lub diagnostyką problemów z jakością połączenia.

Operatorzy sieci mogą wykorzystać tego typu informacje do planowania infrastruktury i diagnozowania problemów z pokryciem, natomiast użytkownicy indywidualni mogą dzięki nim ocenić jakość usług na danym obszarze. Rozwiązanie oparte na użyciu chmury obliczeniowej oraz automatyzacji procesów, umożliwia stworzenie skalowalnego i niezawodnego systemu monitorującego.

2.2 Wymagania funkcjonalne i niefunkcjonalne

2.2.1 Wymagania funkcjonalne

Wymagania funkcjonalne definiują, jakie funkcje system ma realizować oraz jakie operacje będą dostępne dla poszczególnych jego komponentów. Projektowany system składa się z trzech głównych elementów: aplikacji mobilnej, architektury systemu w chmurze oraz aplikacji wizualizującej dane w postaci mapy.

Aplikacja mobilna W wyniku analizy zidentyfikowano wymagania funkcjonalne dotyczące aplikacji mobilnej, odpowiedzialnej za zbieranie danych pomiarowych. Określono, że:

- aplikacja musi umożliwiać automatyczne pozyskiwanie informacji dotyczących sieci komórkowej, w tym danych o typie sieci, identyfikatorze operatora oraz sile sygnału,
- przewiduje się rejestrowanie bieżącej lokalizacji urządzenia na podstawie danych GPS (ang. *Global Positioning System*),
- dane pomiarowe powinny być cyklicznie gromadzone i przekazywane do środowiska chmurowego, gdzie są przetwarzane przez moduł odpowiedzialny za kolejkowanie pomiarów,
- konieczne jest zapewnienie możliwości uruchamiania i zatrzymywania procesu zbierania danych przez użytkownika,
- aplikacja powinna działać w tle w sposób minimalizujący zużycie energii i zasobów systemowych,
- wymagane jest uzyskanie zgody użytkownika na dostęp do funkcji lokalizacji GPS oraz odczytu stanu połączenia sieciowego, zgodnie z zasadami prywatności danych.

Warstwa środowiska chmurowego Dla warstwy chmurowej zidentyfikowano wymagania związane z odbiorem, przetwarzaniem i przechowywaniem danych pomiarowych. W szczególności:

- wymagane jest udostępnienie interfejsu API (ang. *Application Programming Interface*) umożliwiającego odbieranie i kolejkowanie danych z urządzeń mobilnych,
- dane przesyłane do środowiska chmurowego powinny być walidowane, parsowane i zapisywane w relacyjnej bazie danych PostgreSQL,
- określono potrzebę agregacji danych oraz ich udostępniania w ustrukturyzowanej formie dla aplikacji webowej,
- przetwarzanie danych powinno odbywać się w środowisku kontenerowym zarządzanym przez klaster Kubernetes.

Aplikacja wizualizująca pomiary Zidentyfikowano również wymagania dotyczące aplikacji webowej, odpowiedzialnej za prezentację i analizę danych pomiarowych:

- aplikacja musi umożliwiać wizualizację danych pomiarowych na interaktywnej mapie,
- przewiduje się możliwość filtrowania danych według siły sygnału, typu sieci oraz daty pomiaru,
- aplikacja powinna pobierać dane z komponentu backendowego za pomocą interfejsu API,
- dane pomiarowe muszą być prezentowane w sposób czytelny i intuicyjny, w postaci punktów pomiarowych naniesionych na mapę.

2.2.2 Wymagania niefunkcjonalne

Po zdefiniowaniu wymagań funkcjonalnych określono również zestaw wymagań niefunkcjonalnych, opisujących ogólne właściwości projektowanego systemu. Obejmują one aspekty związane ze skalowalnością, bezpieczeństwem, niezawodnością oraz użytecznością interfejsu.

Skalowalność i wydajność

- zidentyfikowano potrzebę obsługi wielu równoczesnych pomiarów pochodzących z różnych urządzeń mobilnych,
- system musi umożliwiać równoczesne przetwarzanie wielu zapytań dotyczących wizualizacji danych,
- architektura rozwiązania powinna zapewniać możliwość automatycznego skalowania zasobów obliczeniowych w zależności od zapotrzebowania,
- komponenty systemu muszą być odseparowane w niezależnych kontenerach, co zwiększa elastyczność oraz ułatwia wdrażanie.

Bezpieczeństwo

- dane przesyłane z aplikacji mobilnej do środowiska chmurowego muszą być autoryzowane przy użyciu wygenerowanych tokenów uwierzytelniających,
- dostęp do bazy danych nie może być udostępniany publicznie i powinien być możliwy jedynie z poziomu zasobów wewnętrznych klastra,
- komponent backendowy nie powinien być bezpośrednio dostępny publicznie, a jego wywołania muszą następować wyłącznie z poziomu aplikacji frontendowej,
- aplikacja webowa powinna być dostępna wyłącznie przy użyciu protokołu HTTPS, z wykorzystaniem ważnego certyfikatu TLS/SSL [12].

Niezawodność i utrzymanie

- określono konieczność pełnej automatyzacji procesu wdrażania systemu z wykorzystaniem narzędzi DevOps [13], takich jak Azure DevOps [14], Helm [15], Flux [16] i Kustomization [17],
- obrazy aplikacji muszą być wersjonowane i przechowywane w repozytorium Azure Container Registry [18],
- wdrożenia systemu w postaci pakietów Helm powinny być wersjonowane i możliwe do ponownego zastosowania,
- w przypadku awarii lub wzrostu obciążenia system musi automatycznie dostosowywać przydział zasobów w celu utrzymania ciągłości działania.

Użyteczność i interfejs

- przewiduje się, że aplikacja webowa będzie dostępna pod publiczną domeną internetową,
- interfejs użytkownika powinien być responsywny i dostosowany do różnych rozdzielczości ekranów,
- prezentacja danych na mapie musi być czytelna i umożliwiać intuicyjną interakcję z użytkownikiem.

2.3 Założenia projektowe

W rozdziale przedstawiono założenia projektowe opracowywanego systemu, obejmujące jego strukturę, charakter oraz zastosowane technologie. Projekt zakłada opracowanie skalowalnego i niezawodnego systemu opartego na technologii Kubernetes [19], umożliwiającego gromadzenie, przetwarzanie oraz wizualizację danych dotyczących parametrów sieci komórkowych w określonych lokalizacjach. System składa się z aplikacji mobilnej, architektury w warstwie chmurowej oraz aplikacji webowej wizualizującej zebrane dane.

2.3.1 Założenia ogólne

Opracowany system ma charakter demonstracyjny i badawczy, jego głównym celem jest analiza danych z sieci komórkowych. Zakłada się, że dane pomiarowe pochodzą z urządzeń mobilnych. Dostęp do systemu ma charakter publiczny w zakresie aplikacji webowej, natomiast warstwa przechowywania i przetwarzania danych jest dostępna wyłącznie wewnątrz klastra. Dane gromadzone w systemie nie zawierają informacji pozwalających na identyfikację użytkownika.

2.3.2 Założenia technologiczne

W systemie wykorzystano platformę Microsoft Azure [20] jako środowisko chmurowe. Zarządzanie infrastrukturą realizowane jest z użyciem Kubernetes, Helm, Kustomization oraz Flux. Dane przechowywane są w relacyjnej bazie danych PostgreSQL. Wdrożenia i zarządzanie kodem źródłowym realizowane są w środowisku Azure DevOps, z wykorzystaniem pipeline'ów CI/CD. Aplikacja webowa korzysta z publicznej domeny internetowej i jest ogólnodostępna umożliwiając interaktywną wizualizację punktów pomiarowych na mapie. Aplikacja mobilna dedykowana jest urządzeniom z systemem Android.

2.4 Przegląd podobnych systemów i rozwiązań

Na rynku funkcjonuje obecnie wiele narzędzi umożliwiających pomiar i analizę parametrów sieci komórkowych z poziomu urządzeń mobilnych. Tego typu narzędzia są wykorzystywane zarówno przez użytkowników indywidualnych, jak i techników oraz operatorów sieci, stanowiąc ważne źródło informacji o jakości usług (ang. *Quality of Service*, QoS) [21] oraz jakości doświadczenia użytkownika (ang. *Quality of Experience*, QoE) [22].

Jakość usług (QoS) odnosi się do obiektywnych, mierzalnych parametrów technicznych sieci telekomunikacyjnej, takich jak opóźnienia transmisji, przepustowość łącza, poziom strat pakietów czy stabilność połączenia [23]. Parametry te opisują sposób funkcjonowania sieci z punktu widzenia infrastruktury i protokołów komunikacyjnych. Z kolei jakość doświadczenia użytkownika (QoE) odzwierciedla subiektywne odczucia odbiorcy końcowego związane z korzystaniem z usług sieciowych, obejmując m.in. płynność transmisji wideo, jakość połączeń głosowych czy czas reakcji aplikacji [24]. W praktyce QoE jest silnie uzależnione od parametrów QoS, dlatego ich łączna analiza pozwala na pełniejszą ocenę jakości działania sieci komórkowej.

Większość rozwiązań pomiarowych powstaje z myślą o systemie **Android**, który umożliwia aplikacjom dostęp do szczegółowych danych radiowych, takich jak RSRP (ang. *Reference Signal Received Power*), RSRQ (ang. *Reference Signal Received Quality*), RSSI (ang. *Received Signal Strength Indicator*), SINR (ang. *Signal-to-Interference-plus-Noise Ratio*) [25]. Parametr RSRP określa poziom mocy sygnału referencyjnego odbieranego przez urządzenie i jest podstawowym wskaźnikiem zasięgu sieci komórkowej [26]. Niskie wartości RSRP mogą prowadzić do obniżenia

prędkości transmisji danych oraz niestabilności połączenia. RSRQ opisuje jakość sygnału, uwzględniając wpływ zakłóceń oraz obciążenie komórki, dzięki czemu pozwala na ocenę efektywności wykorzystania zasobów radiowych [26]. RSSI informuje o całkowitej mocy sygnału odbieranego w kanale radiowym [27], natomiast parametr SINR [26] określa stosunek sygnału użytecznego do zakłóceń i szumu, będąc kluczowym czynnikiem wpływającym na osiągane prędkości transmisji oraz stabilność połączenia.

Dane te pozyskiwane są bezpośrednio z warstwy radiowej urządzenia mobilnego i pochodzą z modemu komórkowego odpowiedzialnego za komunikację ze stacją bazową. Odczyty te odzwierciedlają aktualne warunki propagacyjne sygnału radiowego oraz stopień obciążenia sieci w danej lokalizacji i czasie, co czyni je wartościowym źródłem informacji zarówno do analiz technicznych, jak i do oceny jakości usług z perspektywy użytkownika końcowego [28].

Dostęp do opisanych parametrów pomiarowych jest jednak w dużym stopniu uzależniony od systemu operacyjnego urządzenia mobilnego oraz przyjętej przez jego producenta polityki bezpieczeństwa i prywatności. W praktyce oznacza to, że pełna funkcjonalność narzędzi do analizy sieci komórkowych dostępna jest przede wszystkim na platformie Android, która – będąc dominującym systemem mobilnym z udziałem około 70% rynku - stanowi główny ekosystem dla tego typu rozwiązań [29]. System **iOS** (ang. *iPhone Operating System*), ze względu na bardziej restrykcyjną politykę bezpieczeństwa i prywatności, ogranicza możliwość bezpośredniego odczytu tych parametrów [30]. Z tego powodu część aplikacji na iOS ma charakter uproszczony i koncentruje się głównie na pomiarach przepustowości lub ogólnej jakości połączenia. Jednocześnie, niezależnie od ograniczeń platformowych, podstawą technicznej analizy jakości połączenia w sieciach komórkowych pozostają parametry opisujące poziom i jakość sygnału radiowego.

W dalszej części podrozdziału dokonano analizy i porównania najpopularniejszych aplikacji mobilnych służących do zbierania i wizualizacji danych sieciowych, takich jak *OpenSignal* [31], *nPerf* [32] oraz *Network Cell Info* [33], z opracowanym przez autorów rozwiązaniem SignalMap. Ich popularność oszacowano na podstawie liczby pobrań w sklepie Google Play oraz ogólnodostępnych informacji producentów (tabela 2.1) [34].

Tabela 2.1: Popularność wybranych aplikacji w Google Play (stan na 3 listopada 2025 r.) [35, 36, 33].

Aplikacja	**Pobrania (Google Play)**
OpenSignal [35]	10 000 000+
nPerf [36]	10 000 000+
Network Cell Info Lite & Wifi [33]	10 000 000+

Jak wynika z zestawienia, wszystkie analizowane aplikacje charakteryzują się bardzo dużą popularnością, przekraczającą 10 milionów pobrań. Wskazuje to na duże zapotrzebowanie użytkowników na narzędzia do monitorowania jakości połączeń sieciowych.

W związku z tym, autorzy, w opozycji do istniejących rozwiązań przedstawili również projekt systemu **SignalMap**, bazującego na idei pomiarów crowdsourcingowych (ang. *crowdsourcing* – model pozyskiwania danych od wielu użytkowników). Podejście to pozwala na gromadzenie dużych zbiorów danych pomiarowych w rzeczywistych warunkach użytkowania sieci, co zwiększa wiarygodność uzyskiwanych wyników. Narzędzie będzie rozszerzone o nowe funkcjonalności, skuteczniejsze zarządzanie danymi oraz bardziej przejrzysty interfejs użytkownika. Celem projektu jest wykorzystanie mocnych stron istniejących narzędzi przy jednoczesnym wyeliminowaniu ich ograniczeń.

Analizę porównawczą przeprowadzono w pięciu kategoriach:

1. Podstawowe informacje o systemach,
2. Zakres i parametry danych technicznych,
3. Funkcjonalność i wizualizacja danych,
4. Aspekty prywatności i bezpieczeństwa,
5. Architektura systemu.

Porównania dokonano na podstawie ogólnodostępnych informacji zamieszczonych na stronach producentów, w sklepach z aplikacjami mobilnymi (Google Play), dokumentacjach użytkowych, politykach prywatności oraz na podstawie obserwacji działania aplikacji testowych uruchomionych na urządzeniach z systemem Android.

2.4.1 Podstawowe informacje o systemach

W celu wstępnego porównania analizowanych aplikacji, autorzy zdecydowali się na określenie kryteriów porównawczych, na podstawie zestawienia ich podstawowych cech funkcjonalnych i użytkowych (tabela 2.2). Przyjętymi kryteriami oceny były:

- **Platforma** – określa, na jakich systemach operacyjnych dana aplikacja jest dostępna (Android, iOS).
- **Grupa docelowa** – wskazuje głównych użytkowników aplikacji. Jeśli aplikacja oferuje proste pomiary oraz prezentuje wyniki w przystępny sposób, przeznaczona jest głównie dla *konsumentów*. W przypadku bardziej zaawansowanych narzędzi pomiarowych, które prezentują szczegółowe parametry sygnału i wymagają specjalistycznej interpretacji, adresatami są *technicy* lub osoby zajmujące się analizą sieci. Natomiast aplikacje umożliwiające wgląd w mapy pokrycia, analizę sektorów stacji bazowych i porównania pomiędzy operatorami są kierowane przede wszystkim do *operatorów sieci komórkowych*.
- **Dostępność mapy** – informuje, czy aplikacja udostępnia wizualizację danych pomiarowych w formie mapy oraz w jakiej postaci (np. tylko w aplikacji mobilnej lub również w wersji webowej).
- **Licencja** – kryterium to określa, czy aplikacja jest całkowicie darmowa, oferuje dodatkowe funkcje w modelu *freemium*, czy też finansowana jest poprzez wyświetlanie *reklam*. Obecność reklam ma bezpośredni wpływ na komfort użytkowania, dlatego w analizie zwraca się uwagę, czy aplikacja umożliwia pracę w trybie bezreklamowym, czy wymaga ich akceptacji w zamian za bezpłatny dostęp [37].

Tabela 2.2: Porównanie kluczowych cech aplikacji mobilnych do analizy sieci komórkowych.

Kryterium	OpenSignal [31]	nPerf [32]	NetworkCellInfo [33]	SignalMap
Platforma	Android, iOS	Android, iOS	Android	Android
Grupa docelowa	Konsumenci Operatorzy	Konsumenci Operatorzy	Technicy	Konsumenci Technicy Operatorzy
Dostępność mapy	App	Web + app	—	Web + app
Licencja	Darmowa	Freemium	Freemium	Darmowa

Zestawienie wskazywało, że analizowane aplikacje różniły się przeznaczeniem oraz zakresem funkcjonalności. Jednocześnie w ramach pracy przyjęto, że projektowane rozwiązanie miało łączyć prostotę narzędzi konsumenckich z możliwością wglądu w bardziej szczegółowe parametry techniczne, tak aby zwiększyć użyteczność systemu w scenariuszach porównawczych i diagnostycznych.

Jak można zauważyć, żadna z analizowanych aplikacji komercyjnych nie łączy w sobie kilku podejść jednocześnie – nie stanowi kompleksowego narzędzia, które byłoby użyteczne zarówno dla zwykłych użytkowników, jak i dla specjalistów zajmujących się analizą sieci komórkowych. Jednocześnie w ramach pracy przyjęto, że projektowane rozwiązanie miało łączyć prostotę narzędzi konsumenckich z możliwością wglądu w bardziej szczegółowe parametry techniczne, tak aby zwiększyć użyteczność systemu w scenariuszach porównawczych i diagnostycznych.

Warto również zauważyć, że spośród istniejących rozwiązań jedynie nPerf oferuje mapę pokrycia dostępną w wersji webowej. Takie podejście zostanie również zastosowane w systemie SignalMap, gdyż analiza i przegląd danych na większym ekranie pozwala na uzyskanie większej czytelności oraz zapewnia szersze możliwości interpretacji wyników.

Pod względem licencjonowania większość aplikacji komercyjnych finansowana jest z reklam lub oferuje część funkcji w modelu freemium, co ogranicza pełną funkcjonalność użytkownika. Projekt SignalMap stanowi przeciwieństwo tego podejścia – jest w pełni darmowy, pozbawiony reklam oraz nie wprowadza ograniczeń w zakresie dostępnych funkcji.

2.4.2 Zakres i parametry danych technicznych

W celu porównania możliwości analitycznych badanych aplikacji dokonano zestawienia ich wybranych parametrów technicznych (tabela 2.3) oraz zakresu gromadzonych danych pomiarowych. Kryteriami oceny były:

- **Typ pomiarów** – określa rodzaj danych zbieranych przez aplikację, takich jak parametry sygnału radiowego (np. RSRP, RSRQ, RSSI, SINR), identyfikatory komórek sieciowych, w tym PCI (ang. *Physical Cell Identity* – lokalny identyfikator komórki na interfejsie radiowym) oraz ECI (ang. *E-UTRAN Cell Identifier* – unikatowy identyfikator komórki w sieci LTE) oraz Cell ID (unikalny identyfikator przypisywany komórce przez operatora), a także wyniki testów jakości połączenia, takich jak prędkość pobierania, wysyłania czy opóźnienie transmisji [38, 39, 40].
- **Częstotliwość pomiaru** – wskazuje sposób realizacji odczytów, tj. czy są one wykonywane automatycznie w określonych odstępach czasu lub dystansu, czy też uruchamiane ręcznie przez użytkownika.
- **Obsługa BTS** – informuje, czy aplikacja potrafi rozpoznać i wyświetlić dane dotyczące stacji bazowej, z którą połączone jest urządzenie.
- **Dostępność mapy** – określa, czy aplikacja udostępnia wizualizację zebranych danych w postaci mapy.
- **Informacje na mapie** – opisuje, jakie dane są prezentowane na mapie.
- **Dane w czasie rzeczywistym** – określa, czy aplikacja umożliwia bieżące śledzenie parametrów połączenia i aktualizację wyników w czasie rzeczywistym podczas pomiaru.

Tabela 2.3: Porównanie parametrów technicznych aplikacji do analizy sieci komórkowych.

Kryterium	OpenSignal [31]	nPerf [32]	NetworkCellInfo [33]	SignalMap
Typ pomiarów	Speed test QoE (video/gaming/voice) dostępność 4G/5G	Speed test ping jitter zasięg 2G–5G	RSRP RSRQ RSSI SINR PCI Band (NR)ARFCN TAC/LAC eNB/ECI/NCI	Speed test ping jitter RSRP RSRQ SINR RSSI PCI Cell ID Band (NR)ARFCN
Częstotliwość pomiaru	100 pomiarów dziennie	Manualna	Manualna	Dynamiczna
Czy wspiera BTS	Tak	Nie	Tak	Tak
Dostępność mapy	Tak	Tak	Tak	Tak
Informacje na mapie	Pokrycie sygnału	Zasięg sieci Transfer odbioru	Parametry sygnału	Dane pomiarowe BTS
Dane na mapie w czasie rzeczywistym	Brak informacji	Nie	Tak	Tak

Analiza przedstawionych aplikacji wykazała wyraźne różnice w zakresie gromadzonych danych oraz sposobu ich przetwarzania. W przypadku prostszych rozwiązań, takich jak OpenSignal czy nPerf, pomiar koncentruje się głównie na ocenie jakości połączenia i prędkości transmisji danych (testy QoS i QoE), co wynika z ich powszechnej dostępności w systemach operacyjnych Android i iOS. Z kolei aplikacje o charakterze bardziej technicznym, takie jak Network Cell Info, skupiają się wyłącznie na analizie parametrów radiowych sieci, najczęściej w trybie offline, bez integracji z szerszym systemem wizualizacji czy przetwarzania danych w czasie rzeczywistym.

Projektowana aplikacja SignalMap łączy oba te podejścia – umożliwia zarówno analizę jakości usług i zasięgu sieci, jak i szczegółowe śledzenie parametrów technicznych (RSRP, RSRQ, SINR) w sposób ciągły i zautomatyzowany. Częstotliwość pomiarów w większości analizowanych aplikacji jest określana manualnie przez użytkownika, natomiast w SignalMap proces ten został zaprojektowany jako dynamiczny. Oznacza to, że dane są zbierane w sposób adaptacyjny – z większą częstotliwością podczas ruchu użytkownika, a rzadziej w stanie spoczynku. Takie rozwiązanie pozwala ograniczyć redundancję danych i zoptymalizować obciążenie systemu.

W przeciwieństwie do innych aplikacji, SignalMap prezentuje na mapie rzeczywiste lokalizacje stacji bazowych (BTS). W pozostałych aplikacjach lokalizacja BTS jest zwykle jedynie przybliżona – użytkownik otrzymuje orientacyjną informację o kierunku lub najbliższym nadajniku. W przypadku Network Cell Info funkcja ta działała niejednoznacznie — podczas testów w niektórych lokalizacjach nie udało się potwierdzić prawidłowego wskazania połączonej stacji, mimo że zrzuty ekranu z Google Play sugerowały jej dostępność, co wskazuje na możliwe ograniczenia regionalne.

Ponadto SignalMap umożliwia wizualizację danych w czasie rzeczywistym, co nie jest standardem w analizowanych rozwiązaniach. Większość istniejących aplikacji gromadzi dane lokalnie, a ich przetwarzanie następuje dopiero po zakończeniu sesji pomiarowej. Zastosowanie w SignalMap architektury opartej na aktualizacji danych w trybie online pozwala użytkownikowi natychmiast obserwować zmiany jakości sygnału oraz identyfikować obszary problemowe w sposób ciągły. Jak zostało wskazane w pracy [41], monitorowanie jakości sieci w czasie rzeczywistym staje się dziś niezbędnym elementem nowoczesnych systemów komunikacyjnych. Dlatego takie podejście zostało zastosowane w opracowanym rozwiązaniu.

2.4.3 Funkcjonalność i wizualizacja danych

W celu porównania funkcjonalności oraz sposobu wizualizacji danych w analizowanych aplikacjach dokonano zestawienia ich kluczowych cech (tabela 2.4) związanych z prezentacją, przetwarzaniem i dostępnością informacji pomiarowych. Kryteriami oceny były:

- **Mapa wizualizacji** – określa sposób prezentacji danych pomiarowych na mapie. Może to być mapa cieplna (ang. *heatmapa*) pokazująca poziom sygnału w uśrednionej formie, mapa punktowa prezentująca rzeczywiste trasy i lokalizacje pomiarów lub mapa klastrowa grupująca pomiary w zależności od poziomu przybliżenia.
- **Filtry danych** – opisują możliwość wyboru zakresu prezentowanych informacji.
- **Agregacja punktów** – wskazuje, czy aplikacja stosuje mechanizmy łączenia lub uśredniania wielu pomiarów w jeden punkt lub obszar, co ułatwia interpretację dużych zbiorów danych.
- **Odtwarzanie trasy** – informuje o możliwości wizualnego odtworzenia przebiegu pomiarów użytkownika na mapie, co pozwala na analizę zmian jakości sygnału w czasie ruchu.
- **Dane historyczne** – określają, czy aplikacja umożliwia dostęp do zapisanych wcześniej pomiarów oraz analizę zmian parametrów w czasie.
- **Surowe dane** – opisują, czy użytkownik ma możliwość eksportu wyników pomiarów, co umożliwia dalszą, zewnętrzną analizę danych.

Tabela 2.4: Porównanie metod wizualizacji danych w aplikacjach do analizy sieci komórkowych.

Kryterium	OpenSignal [31]	nPerf [32]	NetworkCellInfo [33]	SignalMap
Mapa wizualizacji	Heatmapa	Punktowa	Punktowa	Heatmapa Punktowa Klastrowa
Filtry danych	Operator sieć	Operator	brak	Operator typ sieci czas i inne
Agregacja punktów	Tak	Częściowo	Nie	Tak
Odtwarzanie trasy	Nie	Tak	Tak	Tak
Dane historyczne	Brak informacji	Brak informacji	Tak	Tak
Surowe dane dane	Brak informacji	Tak	Tak	Tak

Analiza przedstawionych aplikacji pod względem funkcjonalności i wizualizacji danych wskazuje na znaczące różnice w sposobie prezentacji oraz przetwarzania informacji pomiarowych. Aplikacja OpenSignal wykorzystuje mapę cieplną, która pozwala na ogólną ocenę jakości sygnału w danym obszarze, jednak nie umożliwia szczegółowej analizy poszczególnych punktów pomiarowych. Z kolei nPerf i Network Cell Info stosują mapy punktowe, które oferują większą precyzję w odwzorowaniu lokalnych pomiarów, ale przy dużej liczbie danych stają się mniej czytelne. Projektowana aplikacja SignalMap wprowadza elastyczne podejście do prezentacji danych, oferując użytkownikowi wybór między trzema trybami wizualizacji: mapą punktową, prezentującą każdą lokalizację pomiarową z osobna, heatmapą, obrazującą zagęszczenie sygnału za pomocą gradientów kolorystycznych, oraz rozwiązaniem pośrednim w postaci mapy klastrowej.

Pod względem filtrowania danych, jedynie SignalMap oferuje możliwość wyboru według wielu kryteriów, takich jak operator, typ sieci czy przedział czasowy. W pozostałych aplikacjach opcje te są ograniczone wyłącznie do wyboru operatora lub całkowicie niedostępne, co utrudnia analizę porównawczą i historyczną.

Mechanizmy agregacji punktów występują w OpenSignal (uśrednianie pomiarów w siatce) oraz w ograniczonym zakresie w nPerf, gdzie uśredniane są głównie wyniki testów QoS w kaflach mapy, bez pełnego klastrowania surowych punktów. Network Cell Info prezentuje natomiast surowe punkty pomiarowe bez grupowania, co utrudnia interpretację większych zbiorów danych. SignalMap, poprzez zastosowanie klastrowania, umożliwia analizę zarówno globalną, jak i lokalną.

Funkcja odtwarzania trasy pomiarowej jest obecna we wszystkich aplikacjach poza OpenSignal, w którym dane mają charakter statystyczny. SignalMap, podobnie jak nPerf i Network Cell Info, pozwala śledzić przebieg trasy użytkownika wraz ze zmianami jakości sygnału.

W kontekście danych historycznych oraz dostępu do surowych danych, rozwiązania komercyjne nie oferują pełnej transparentności — brak jest publicznych informacji o możliwości eksportu lub analizy archiwalnych pomiarów. W ramach opracowanej aplikacji SignalMap możliwy jest pełny dostęp do historii pomiarów, wraz z ich eksportem, co zdaniem autorów, znacząco zwiększa jego potencjał badawczy i użytkowy.

2.4.4 Aspekty prywatności i bezpieczeństwa

W celu porównania analizowanych aplikacji pod względem prywatności i bezpieczeństwa danych (tabela 2.5) użytkowników dokonano zestawienia ich najważniejszych cech związanych ze sposobem gromadzenia, przechowywania oraz przetwarzania informacji. Kryteriami oceny były:

- **Anonimizacja** – określa, czy dane gromadzone przez aplikację są przetwarzane w sposób uniemożliwiający identyfikację użytkownika. Dotyczy to zarówno usuwania danych osobowych, jak i pseudonimizacji identyfikatorów urządzeń oraz lokalizacji [42].
- **Wymaganie logowania** – wskazuje, czy korzystanie z aplikacji wymaga utworzenia konta użytkownika lub uwierzytelnienia się w systemie. Kryterium to ma istotne znaczenie dla oceny poziomu prywatności, ponieważ logowanie wiąże się z przetwarzaniem dodatkowych danych osobowych [43].
- **Przechowywanie danych** – opisuje miejsce i sposób utrwalania danych pomiarowych, tj. czy są one zapisywane lokalnie na urządzeniu użytkownika, czy przesyłane do infrastruktury chmurowej dostawcy. Lokalne przetwarzanie ogranicza ryzyko naruszenia prywatności, natomiast przechowywanie w chmurze pozwala na agregację i analizę danych w skali globalnej. Jest tak, ponieważ dane przechowywane centralnie przez dostawcę usług chmurowych mogą być łączone i analizowane z wielu różnych źródeł jednocześnie, co umożliwia tworzenie zestawień, raportów i statystyk w skali całej bazy użytkowników [44].
- **Usunięcie danych** – określa, czy użytkownik ma możliwość trwałego usunięcia swoich danych z systemu oraz w jaki sposób może tego dokonać. Kryterium to jest istotne z punktu widzenia zgodności z przepisami o ochronie danych osobowych, w szczególności **RODO** (*Rozporządzenie o Ochronie Danych Osobowych* ang. *General Data Protection Regulation*, **GDPR**), oraz z zasadą transparentności polityki prywatności aplikacji [45, 46, 47] .

Tabela 2.5: Porównanie aplikacji pod względem prywatności i bezpieczeństwa danych.

Kryterium	OpenSignal [31]	nPerf [32]	NetworkCellInfo [33]	SignalMap
Anonimizacja	Tak	Tak	Brak danych	Tak
Wymaganie logowania	Nie	Opcjonalne	Opcjonalne	Nie
Przechowywanie danych	Chmura	Lokalnie / Chmura	Lokalnie	Chmura
Usunięcie danych	Deklarowane w polityce prywatności	Brak danych	N/D	Formularz żądania administratorowi

Analiza aspektów prywatności i bezpieczeństwa danych w dostępnych aplikacjach wskazuje, że większość z nich stosuje podstawowe mechanizmy ochrony informacji, takie jak anonimizacja odczytów i przechowywanie danych w chmurze. Zarówno OpenSignal [48], jak i nPerf [49] deklarują stosowanie anonimizacji danych użytkowników, jednak szczegółowy opis sposobu realizacji tego procesu nie jest publicznie dostępny. Projekt SignalMap zapewnia anonimizację danych już na etapie ich pozyskiwania, co minimalizuje ryzyko identyfikacji użytkownika.

Pod względem autoryzacji i logowania, aplikacje komercyjne często stosują logowanie opcjonalne, umożliwiające synchronizację danych z kontem w chmurze. W SignalMap przyjęto podejście odwrotne – system nie wymaga logowania, w celu ograniczenia ilość przetwarzanych danych osobowych.

Dane w większości analizowanych aplikacji są przechowywane w chmurze, co zwiększa skalowalność. W ramach aplikacji SignalMap, również stosowane jest podejście chmurowe, jednak dane są okresowo usuwane. Kwestia usunięcia danych pozostaje niejednoznaczna w aplikacjach komercyjnych — jedynie OpenSignal [48] formalnie deklaruje taką możliwość w polityce prywatności. W nPerf i Network Cell Info brak jest jednoznacznych informacji o sposobie realizacji tego prawa. W SignalMap procedura usunięcia danych jest jasno określona — użytkownik może zgłosić żądanie usunięcia danych poprzez formularz kontaktowy skierowany bezpośrednio do administratora systemu.

2.4.5 Architektura systemu

Z uwagi na brak publicznie dostępnej dokumentacji technicznej, dokładne informacje dotyczące architektury analizowanych aplikacji nie są znane. Na podstawie analizy funkcjonalnej, obserwacji sposobu komunikacji z serwerami oraz zapisów zawartych w politykach prywatności można jednak stwierdzić, że większość z nich wykorzystuje architekturę chmurową typu klient–serwer.

Taki model jest charakterystyczny dla systemów przetwarzających duże ilości danych pochodzących od użytkowników końcowych, w tym pomiarów parametrów sieci komórkowej i danych geolokalizacyjnych [50, 51]. Architektura chmurowa umożliwia skalowanie zasobów obliczeniowych w zależności od obciążenia, zapewnia wysoką dostępność (ang. *High Availability, HA*) — czyli zdolność systemu do ciągłego działania przez dłuższy czas nawet przy awarii części komponentów, dzięki redundancji i automatycznemu przełączaniu na zasoby zapasowe [52] — oraz pozwala na centralne zarządzanie i analizę danych w środowisku serwerowym dostawcy.

W przypadku aplikacji OpenSignal dominująca architektura chmurowa oraz analiza nagłówków sieciowych sugerują korzystanie z infrastruktury usługi Amazon Web Services (AWS), typowej dla aplikacji o globalnym zasięgu i wysokich wymaganiach skalowalności [53]. Informacja ta została potwierdzona na podstawie nagłówków odpowiedzi serwera domeny *opensignal.com* oraz danych z serwisu SecurityTrails [54]. Zastosowanie infrastruktury chmurowej AWS jest spójne z globalnym

charakterem aplikacji i pozwala zapewnić wysoką dostępność usług, niskie opóźnienia w dostępie do danych oraz większą odporność systemu na przeciążenia i awarie.

Z kolei w przypadku projektu SignalMap, będącego częścią niniejszej pracy inżynierskiej, planowana architektura systemu oparta jest o środowisko chmurowe Microsoft Azure. Microsoft Azure to kompleksowa platforma chmury obliczeniowej oferująca szeroki zestaw usług przetwarzania, przechowywania danych, analizy, API oraz narzędzi umożliwiających budowę nowoczesnych aplikacji w modelu rozproszonym, co ułatwia implementację skalowalnych i elastycznych rozwiązań informatycznych [20]. Wykorzystanie tej platformy umożliwia łatwe skalowanie zasobów w zależności od obciążenia, utrzymanie wysokiej dostępności usług, a także integrację z nowoczesnymi narzędziami analitycznymi oraz komunikacyjnymi dzięki bogatemu zestawowi API dostępnych usług [20, 55]. System będzie korzystał z warstwy API, która zapewni bezpieczną i efektywną komunikację między aplikacją mobilną a usługami chmurowymi, odpowiedzialnymi za przetwarzanie i wizualizację danych. Dzięki temu architektura systemu jest zgodna z zasadami projektowania systemów rozproszonych, w których warstwa API oddziela logikę biznesową od klientów i umożliwia standaryzowany dostęp do zasobów chmurowych [20].

Podsumowując, choć szczegółowa architektura systemów takich jak OpenSignal czy nPerf nie jest jawna, obserwacja ich działania jednoznacznie wskazuje na model rozproszonego przetwarzania w chmurze. Projekt SignalMap wykorzystuje tę samą ideę. W rozwiązaniu zaimplementowane są autorskie rozwiązania oparte na chmurze Azure, które zapewniają elastyczność, bezpieczeństwo i wysoką dostępność infrastruktury.

2.4.6 Podsumowanie

Przeprowadzona analiza porównawcza wykazała, że dostępne na rynku aplikacje do analizy sieci komórkowych różnią się znacznie pod względem zakresu funkcjonalności, przeznaczenia oraz poziomu zaawansowania technicznego. Aplikacje takie jak OpenSignal i nPerf skupiają się głównie na aspektach użytkowych, oferując testy prędkości oraz ogólną ocenę jakości usług (QoS, QoE), podczas gdy Network Cell Info koncentruje się na szczegółowej analizie parametrów technicznych sygnału. Żadne z tych rozwiązań nie łączy jednak wszystkich podejść w jednym narzędziu. W związku z tym autorzy, w ramach opracowanej aplikacji SignalMap postanowili, zintegrować te funkcje charakterystyczne zarówno dla narzędzi konsumenckich, jak i profesjonalnych. Dzięki temu aplikacja umożliwia dynamiczny tryb pomiaru, prezentację danych w czasie rzeczywistym oraz klastrową wizualizacją map. Zdaniem autorów, poprawia to zarówno czytelność, jak i ma potencjalny wpływ na wartość analityczną danych. Pod względem prywatności i bezpieczeństwa SignalMap stosuje pełną anonimizację, nie wymaga logowania oraz zapewnia przejrzysty proces usuwania danych. W warstwie technicznej wszystkie analizowane systemy bazują na architekturze chmurowej. SignalMap wykorzystuje rozwiązania oparte na Microsoft Azure, umożliwiające skalowalność i wysoką dostępność usług.

Rozdział 3

Projekt systemu

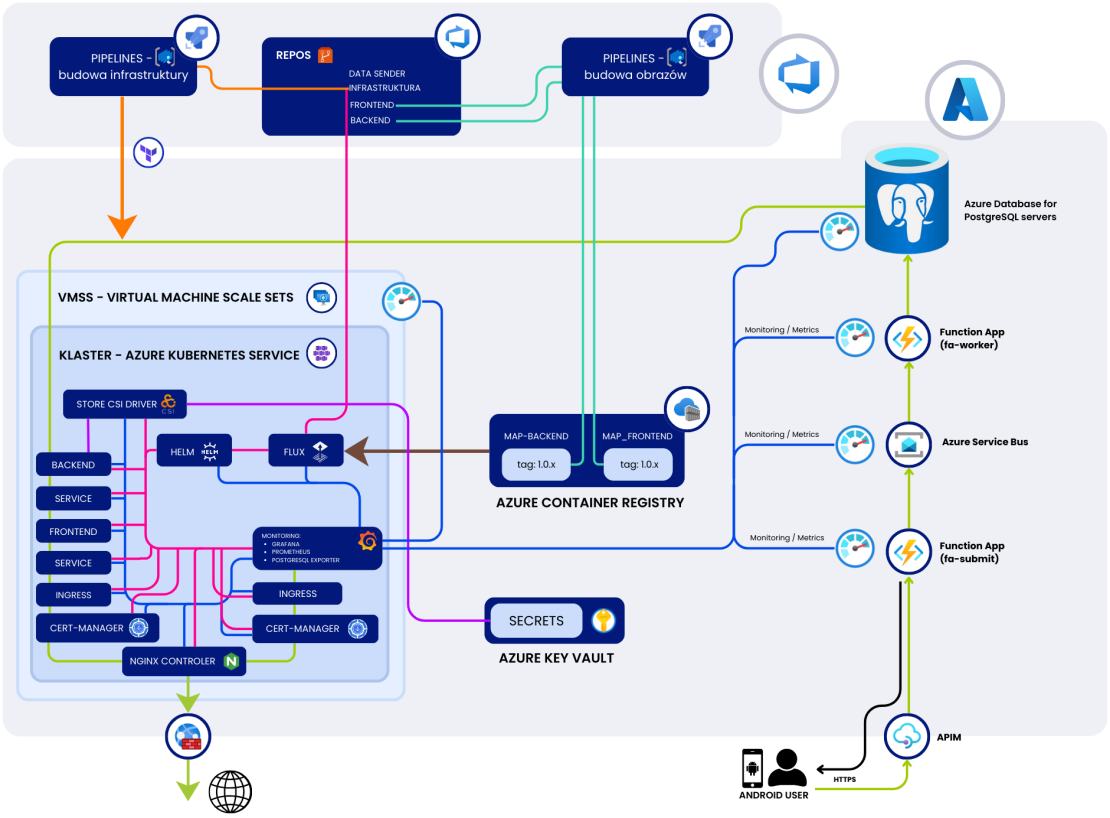
Rozdział poświęcony jest szczegółowemu opisowi architektury projektowanego systemu, obejmującemu zarówno warstwę infrastrukturalną, jak i logiczny podział komponentów aplikacyjnych oraz przepływ danych pomiędzy nimi. Celem rozdziału jest przedstawienie przyjętych decyzji projektowych oraz uzasadnienie wyboru zastosowanych technologii w kontekście wymagań funkcjonalnych i niefunkcjonalnych systemu.

W pierwszej części zaprezentowano ogólną koncepcję architektury rozwiązania, uwzględniającą wykorzystanie chmurowej platformy Microsoft Azure [20] oraz klastra Kubernetes [19] jako podstawy do uruchamiania i skalowania aplikacji. Następnie omówiono architekturę sieciową systemu, ze szczególnym naciskiem na rozdzielenie komponentów publicznych i prywatnych, mechanizmy bezpiecznego przesyłu danych telemetrycznych oraz integrację usług w obrębie wirtualnej sieci Azure.

Kolejne sekcje rozdziału zawierają uzasadnienie wyboru kluczowych technologii i narzędzi, opis projektu klastra Kubernetes, a także model danych wykorzystywany w systemie. Przedstawiono również szczegółowy projekt przepływu danych telemetrycznych – od momentu ich wygenerowania w aplikacji mobilnej, poprzez warstwę integracyjną i mechanizmy asynchronicznego przetwarzania, aż do trwałego zapisu w relacyjnej bazie danych.

Rozdział stanowi podstawę do dalszych części pracy, w których zaprezentowane zostaną szczegóły implementacyjne poszczególnych komponentów systemu oraz sposób ich wdrożenia i testowania w środowisku chmurowym.

3.1 Ogólna koncepcja architektury



Rysunek 3.1: Koncept architektury

Na rysunku 3.1 przedstawiono ogólną koncepcję architektury projektowanego systemu, bazującego na chmurowej infrastrukturze platformy Microsoft Azure oraz klastrze Kubernetes (Azure Kubernetes Service). Architektura została zaprojektowana w sposób modułowy, z wyraźnym podziałem na warstwy odpowiedzialne za budowę infrastruktury, wdrażanie aplikacji, przetwarzanie danych oraz ich wizualizację.

Kluczowe elementy infrastruktury systemu zostały zdefiniowane i wdrożone z wykorzystaniem narzędzia Terraform, zgodnie z podejściem Infrastructure as Code [56]. Obejmują one w szczególności:

- klaster *Azure Kubernetes Service (AKS)*,
- instancje *Azure Function App*,
- usługę *Azure Database for PostgreSQL Flexible Server*,
- zasoby wspierające, takie jak sieci wirtualne, konta *Storage Account* oraz konfiguracje uprawnień i ról dostępu.

Mechanizm Pipeline jest także używany do wykonywania implementacji przez narzędzie Terraform oraz podczas wdrażania kodu do instancji Azure FunctionApp.

Centralnym elementem systemu jest klaster Kubernetes uruchomiony na zestawach skalujących maszyn wirtualnych (VMSS), który odpowiada za uruchamianie serwerów, na których implementowane są Pody z kontenerami. Aplikacja webowa została podzielona na komponent frontendowy

oraz backendowy, komunikujące się za pośrednictwem usług sieciowych. Użyto mechanizmów Ingress oraz Nginx Controller, aby wystawić aplikacje do Internetu.

W ramach systemu zaimplementowano proces przesyłania i przetwarzania danych telemetrycznych pochodzących z aplikacji mobilnej. Dane są przekazywane do infrastruktury chmurowej poprzez warstwę brzegową opartą o usługę *Azure API Management (APIM)*, która pełni rolę centralnej bramy API. Następnie dane podlegają asynchronicznemu przetwarzaniu z wykorzystaniem mechanizmu kolejkowania, a końcowym etapem procesu jest ich zapis w relacyjnej bazie danych *Azure Database for PostgreSQL*.

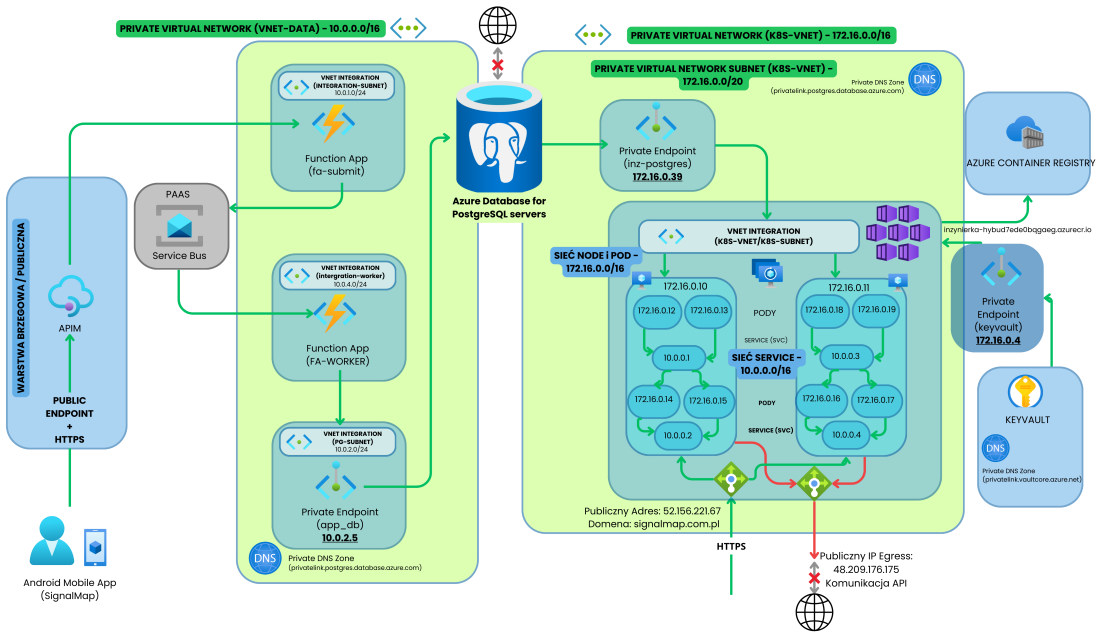
Proces budowy i wdrażania infrastruktury oraz aplikacji realizowany jest z wykorzystaniem uproszczonych mechanizmów CI/CD platformy Azure DevOps. Obrazy kontenerów są budowane automatycznie i przechowywane w rejestrze Azure Container Registry, skąd następnie pobierane są przez klaster Kubernetes. Konfiguracja aplikacji oraz zarządzanie wdrożeniami odbywa się z użyciem narzędzi Helm. Zmiany wprowadzone do plików konfiguracyjnych obsługujących wdrożenie klastra są monitorowane przez Flux, a wykryte zmiany są przez niego automatycznie wdrażane.

System wykorzystuje usługi Azure Key Vault do bezpiecznego przechowywania sekretów oraz danych wrażliwych. W przypadku tej infrastruktury są to dane logowania do bazy dla backenowych aplikacji. Dane z Azure Key Vault są pobierane poprzez pody STORAGE CSI DRIVER. Monitorowanie działania klastra, aplikacji oraz komponentów Azure jest realizowane przez Azure Monitor oraz przy użyciu narzędzi takich jak Prometheus oraz Grafana.

Wszystkie wykorzystane rozwiązania oraz technologie zostały opisane w kolejnych podrozdziałach pracy.

3.2 Architektura sieciowa

3.2.1 Ogólny model sieciowy systemu



Rysunek 3.2: Koncept architektury sieciowej systemu

Ogólny model sieciowy systemu został zaprojektowany w oparciu o architekturę chmurową, której głównym celem jest zapewnienie skalowalności, wysokiej dostępności oraz bezpieczeństwa

przetwarzanych danych. Współczesne systemy informatyczne, szczególnie te gromadzące dane pochodzące od wielu użytkowników końcowych, coraz częściej wykorzystują architekturę chmurową jako podstawę organizacji infrastruktury sieciowej [50]. Koncept architektury został przedstawiony na rysunku 3.2.

Przyjęty model bazuje się na logicznym podziale systemu na kilka warstw funkcjonalnych, z których każda pełni odrębną rolę w procesie obsługi danych i komunikacji sieciowej. Wyróżnić można warstwę dostępu publicznego, warstwę przetwarzania oraz warstwę danych. Taki podział umożliwia separację odpowiedzialności poszczególnych komponentów systemu oraz ułatwia zarządzanie bezpieczeństwem i rozbudową infrastruktury [57].

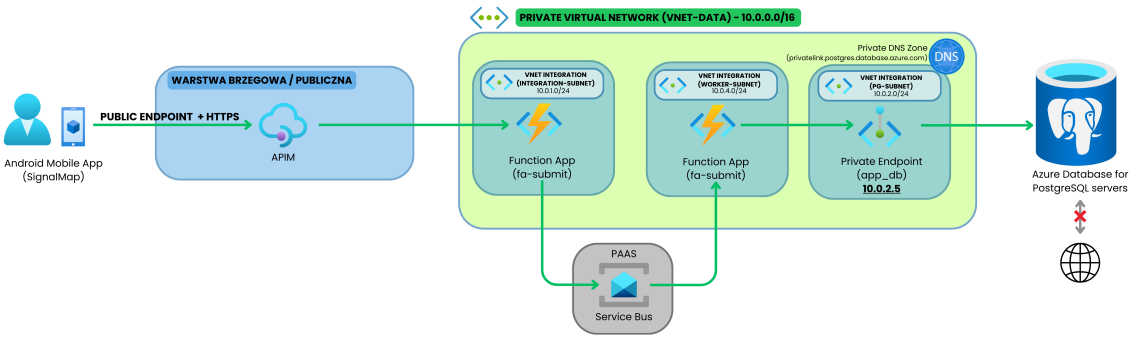
Warstwa brzegowa, oparta na usłudze *Azure API Management (APIM)*, pełni rolę bramy API zarządzającej ruchem między użytkownikami zewnętrznymi a wewnętrznymi zasobami systemu. Rozwiązanie to zapewnia centralny punkt autoryzacji dla aplikacji mobilnej oraz warstwy wizualizacji, izolując logikę przetwarzania i składowania danych od publicznego internetu [58].

Właściwe przetwarzanie danych oraz realizacja logiki aplikacyjnej odbywają się w warstwie prywatnej, funkcjonującej w obrębie izolowanych sieci wirtualnych. Komponenty tej warstwy nie są bezpośrednio dostępne z Internetu i komunikują się ze sobą wyłącznie w ramach infrastruktury chmurowej. Zastosowanie wirtualnych sieci umożliwia logiczną izolację zasobów oraz kontrolę przepływu ruchu pomiędzy poszczególnymi elementami systemu [59].

Warstwa danych obejmuje usługi odpowiedzialne za trwałe przechowywanie informacji oraz zarządzanie danymi systemu. Dostęp do tych zasobów realizowany jest z wykorzystaniem mechanizmów sieci prywatnej, co eliminuje konieczność ich udostępniania w przestrzeni publicznej. Takie podejście znacząco zwiększa poziom ochrony danych oraz ogranicza ryzyko nieautoryzowanego dostępu [60].

Przedstawiony ogólny model sieciowy ma charakter koncepcyjny i stanowi punkt wyjścia do dalszego, bardziej szczegółowego opisu architektury systemu. W kolejnych podrozdziałach omówiono szczegółowo ścieżki przepływu danych oraz organizację infrastruktury obliczeniowej.

3.2.2 Model sieciowy przesyłu danych telemetrycznych



Rysunek 3.3: Architektura sieciowa ścieżki przesyłu danych telemetrycznych

Architektura sieciowa przesyłu danych telemetrycznych została zaprojektowana w oparciu o logiczny podział na komponenty udostępnione publicznie oraz zasoby funkcjonujące w obrębie prywatnej sieci wirtualnej Azure (*Azure Virtual Network, VNet*) [59]. Izolacja zasobów backendowych od ruchu publicznego realizowana jest poprzez mechanizmy integracji z siecią wirtualną (*VNet Integration*) oraz wykorzystanie prywatnych punktów końcowych (*Private Endpoint*). Ruch sieciowy inicjowany przez aplikację mobilną kierowany jest do infrastruktury chmurowej za pośrednictwem

protokołu HTTPS, co zapewnia poufność i integralność przesyłanych danych dzięki kryptografii warstwy transportowej TLS [61, 12].

Pierwszym elementem infrastruktury jest warstwa brzegowa oparta na usłudze *Azure API Management (APIM)*, która stanowi publiczny punkt styku systemu z otoczeniem zewnętrznym. Usługa ta zarządza ruchem przychodzącym z urządzeń mobilnych izolując wewnętrzne komponenty systemu przed bezpośrednimi połączeniami z sieci Internet. Dane z bramy API przekazywane są do aplikacji funkcyjnej *fa-submit* przy wykorzystaniu jej domyślnej nazwy domeny (`azurewebsites.net`). Wybór ten wynika z charakterystyki usługi Azure Functions, w której adresy IP są przypisane do całej aplikacji (ang. *Function App*), a mechanizmy routingu platformy Azure optymalizują komunikację właśnie poprzez wykorzystanie nazw domenowych usług [62].

Mimo że funkcja *fa-submit* komunikuje się z kolejką *Azure Service Bus* w trybie dostępu publicznego, została ona zintegrowana z siecią wirtualną (*VNet Integration*). Decyzja ta podyktowana jest dbałością o spójność architektury bezpieczeństwa oraz przygotowaniem pod przyszłą, całkowitą izolację logiki biznesowej. Podobny mechanizm integracji z siecią VNet zastosowano w przypadku funkcji *fa-worker*. Wykorzystany w projekcie *Azure Service Bus* jest usługą typu *Platform as a Service* (PaaS) [63, 64]. Oznacza to, że dostawca chmurowy odpowiada za utrzymanie infrastruktury bazowej, takiej jak serwery czy systemy operacyjne, podczas gdy użytkownik zarządza jedynie logiką systemu. Z perspektywy architektury sieciowej, kolejka ta funkcjonuje jako wewnętrzny mechanizm wymiany informacji, z którego korzystają wyłącznie autoryzowane komponenty systemu (*fa-submit* i *fa-worker*). Brak bezpośredniego wystawienia kolejki jako otwartego punktu końcowego dla użytkowników zewnętrznych ogranicza możliwość inicjowania połączeń spoza zaufanego środowiska chmurowego [65].

Baza danych *Azure Database for PostgreSQL* została całkowicie odcięta od dostępu z sieci Internet. Dostęp do niej realizowany jest wyłącznie poprzez mechanizm *Private Endpoint*, który przypisuje usłudze stały, prywatny adres IP `10.0.2.5` w obrębie dedykowanej podsieci [60]. Takie rozwiązanie gwarantuje, że baza danych pozostaje niewidoczna dla użytkowników zewnętrznych, a cała komunikacja między warstwą przetwarzania a warstwą składowania odbywa się wyłącznie wewnątrz izolowanego środowiska sieciowego przy użyciu bezpiecznego łącza *Azure Private Link*.

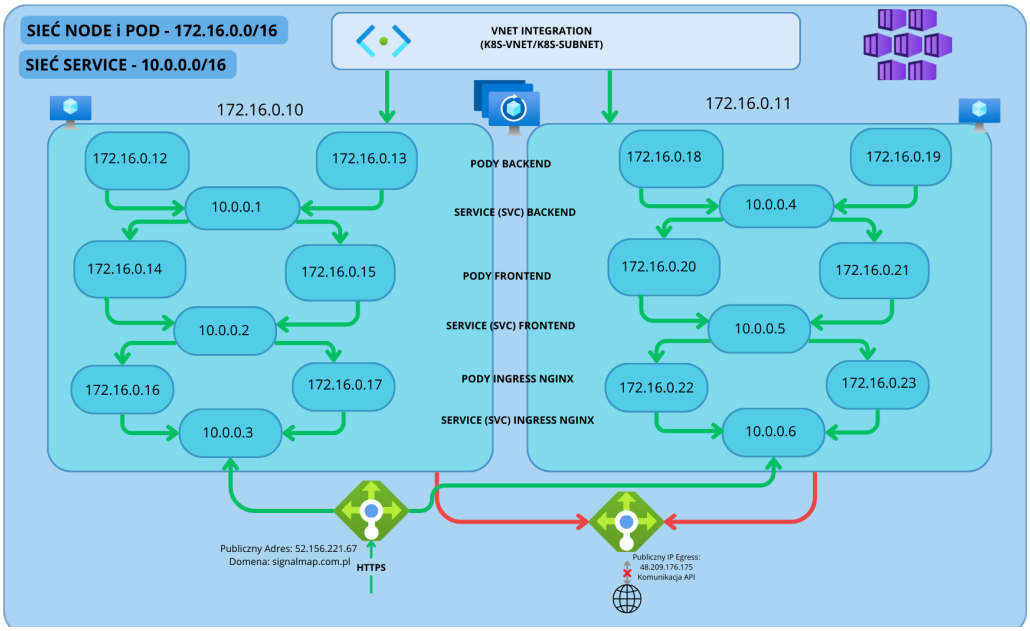
Uzupełnieniem tej architektury są prywatne strefy DNS Azure, które pozwalają na rozwiązywanie nazw domenowych usług na ich adresy prywatne [66]. W celu zapewnienia wysokiej szczegółowości w zarządzaniu przepływem informacji oraz możliwości monitorowania ruchu między usługami, w obrębie sieci `VNET-DATA` zaimplementowano dedykowane podsieci, których zestawienie przedstawiono w tabeli 3.1.

Tabela 3.1: Zestawienie adresacji podsieci w warstwie przetwarzania i danych

Komponent	**Nazwa podsieci**	**Adres sieci**
Integracja *fa-submit*	INTEGRATION-SUBNET	10.0.1.0/24
Integracja *FA-WORKER*	WORKER-SUBNET	10.0.4.0/24
Punkt końcowy bazy danych (`10.0.2.5`)	PG-SUBNET	10.0.2.0/24

Podsumowując, przedstawiona architektura realizuje zasadę minimalnej ekspozycji usług. Komunikacja wejściowa jest ściśle kontrolowana przez bramę APIM, natomiast warstwa danych jest fizycznie nieosiągalna z sieci publicznej, co minimalizuje powierzchnię ataku i zwiększa odporność systemu na próby nieautoryzowanego dostępu.

3.2.3 Sieć klastra Kubernetes



Rysunek 3.4: Sieć klastra Kubernetes

Dla sieci klastra wykorzystywana jest podsieć *k8s-subnet 172.16.0.0/20*, która została stworzona jako zasób w strukturze Azure. Komponentem kluczowym dla poprawnego działania sieci w klastrze Kubernetes jest implementacja narzędzia CNI. Mechanizm ten tworzy interfejs między działającymi kontenerami a wtyczkami sieciowymi. W przypadku tej pracy wykorzystano Azure CNI, domyślną wtyczkę przy tworzeniu zasobu AKS. Wtyczka ta odpowiada za klika funkcji: konfiguracje interfejsów sieciowych w kontenerach, zarządzanie pulą dostępnych adresów IP, konfiguruję i zarządza tablicą routingu. W przypadku tej pracy zdecydowano się na taki przydział adresacji, który powoduje, że Node jak i Pod otrzymują adresy z tej samej puli. Jest to jeden z powodów dla którego wybrano Azure CNI zamiast innych wtyczek np.: Flannel czy Calico. Inne wtyczki wprowadzają osobną adresacje dla Node, Pod i Service - w takim przypadku komunikacja Pod-Pod musi zostać obsłużona przez mechanizm VxLAN co powoduje większe zużycie zasobów CPU. Kluczowym elementem połączeń sieciowych jest obiekt Service. Jest to wirtualny, niezmienny punkt dostępowy, którego zadaniem jest przekierowywanie ruchu do Podów uwzględnionych w konfiguracji pod parametrem *labels*. Kolejnym zadaniem Service jest zapewnianie obsługi LoadBalancingu, czyli równoważenie zapytań do Podów, których ruchem zarządzają. W pracy Service kluczowy jest Pod o nazwie *Kube-Proxy*. Jest on implementowany na każdym Node w celu zarządzanie i manipulacja tablic routingu. Konfiguruje on wpisy, które pozwalają na routing między Podami w sieci (172.16.0.0/20) a siecią Service (10.0.0.0/16). Jeśli chodzi o komunikacje wynikająca z użytkowania aplikacją webową to rozpoczyna się ona poprzez rozwiązanie nazwy *signalmap.com.pl* na adres *52.156.221.67* przypisany jako Service o typie LoadBalancer. Następnie Service przypisany do obsługi Podów Nginx Controller analizuje z jakiej strony pochodzi ruch (czy jest to signalmap.com.pl czy tez grafana.signalmap.com.pl) i przekazuje ruch za pomocą odpowiedniego obiektu Ingress. Kolejnym krokiem jest otrzymanie ruch przez Service aplikacji frontendowej. Tam następuje mechanizm routingu przekierowujący ruch na dołączony do Service Pod aplikacji frontedowj. Przy komunikacji frontend-backend mechanizm jest taki sam. Ważnym aspektem w zarządzaniu siecią jest funkcjonalność Podów *CoreDNS*. Narzędzie to przygotowuje i zarządza wewnętrznym serwerem DNS klastra. W tej sposób aplikacja frontedowa może komunikować się z aplikacją backednową

za pomocą rozwiązywanej nazwie Service przypisanej do aplikacji backendowej.

3.3 Omówienie technologii i narzędzi wykorzystanych w implementacji

W niniejszym rozdziale zaprezentowano szczegółowe zestawienie technologii oraz narzędzi programistycznych i infrastrukturalnych, które posłużyły do zaprojektowania oraz implementacji systemu SignalMap. Wybór poszczególnych rozwiązań został podyktowany koniecznością zapewnienia wysokiej skalowalności środowiska, automatyzacji procesów wdrażania w modelu GitOps oraz bezpieczeństwa danych w architekturze chmurowej. Opisane narzędzia zostały pogrupowane według ich roli w systemie – od fundamentów platformy chmurowej, przez mechanizmy automatyzacji i zabezpieczeń, aż po konkretne biblioteki oraz frameworki wykorzystane w warstwie aplikacyjnej.

3.3.1 Platforma chmurowa i model wdrożenia

Microsoft Azure jako środowisko uruchomieniowe

Microsoft Azure [20]- platforma chmurowa zarządzana i utrzymywana przez firmę Microsoft. Stworzono konto Azure for Students, na którym wykonano implementacje systemu. Czynnikiem istotnym przy wyborze tego narzędzia jest wysoka skalowalność oraz dostępność. Skalowalność polega na dynamicznym dostosowywaniu ilości zasobów do potrzeb generowanych przez użytkowników końcowych lub przez obciążenie samego systemu. Dostępność natomiast objawia się parametrem SLA. W przypadku tej pracy SLA dla większości usług zapewnianych przez Azure wynosi ponad 99%. Kluczowa jest także łatwość w wdrażaniu poszczególnych serwerów na platformie. Wykorzystywany jest model PaaS [63] (ang. Platfrom as a Service). Pozwala to na uruchomienie usług Kuberentes lub baz danych bez konieczności utrzymywania i konfiguracji samych serwerów. Dodatkowo istotnym czynnikiem przy wyborze dostawcy usług chmurowych była naturalna integracja Microsoft Azure z Azure DevOps.

Warstwa obliczeniowa: AKS i Azure Functions

Azure Kubernetes Service (AKS) [67] - zarządzana usługa orkiestracji kontenerów Kubernetes. Pozwala na uruchamianie aplikacji kontenerowych opartych na silniku Containerd. Głównym czynnikiem wyboru tego narzędzia jest przekazanie odpowiedzialności za warstwę sterującą (control plane) firmie Microsoft. Pozwala to ominąć trudną konfiguracje warstwy, zarządzane certyfikatami przy dodawaniu nowego node oraz implementacje podstawowych Podów do obsługi środowiska. W pracy wykorzystano wersje 1.32.6 AKS.

Kolejnym istotnym elementem architektury obliczeniowej jest usługa Azure Function App. Jest to rozwiązanie typu Serverless, które umożliwia uruchamianie izolowanych fragmentów kodu w odpowiedzi na konkretne, zdefiniowane zdarzenia [68]. Kluczową cechą tego modelu jest uproszczenie zarządzania warstwą serwerową – deweloper skupia się wyłącznie na logice biznesowej, podczas gdy dostawca chmury dynamicznie zarządza alokacją zasobów niezbędnych do wykonania zadania.

W kontekście systemu SignalMap wybór ten był podyktowany przede wszystkim optymalizacją zasobów i kosztów operacyjnych. Model Serverless pozwala na unikanie opłat za niewykorzystaną moc obliczeniową, ponieważ zasoby są przydzielane wyłącznie w momencie wystąpienia zdarzenia wyzwalającego [69]. W niniejszej pracy funkcje te pełnią krytyczną rolę w procesie przetwarzania danych telemetrycznych przesyłanych z aplikacji mobilnych. Dzięki takiemu podejściu, system aktywuje moce obliczeniowe tylko w krótkich interwałach czasowych, gdy dane są faktycznie wysy-

łane do chmury, co zapewnia wysoką efektywność kosztową przy zachowaniu zdolności do obsługi nagłych skoków natężenia ruchu.

Warstwa danych i kolejkowania

Rolę centralnego punktu systemu, odpowiedzialnego za trwałe przechowywanie informacji, pełni **Azure Database for PostgreSQL Flexible Server**. Jest to w pełni zarządzana usługa, umożliwiająca wykorzystanie silnika PostgreSQL bez konieczności angażowania zasobów w administrację serwerem czy bieżące utrzymanie infrastruktury [70]. Takie podejście pozwoliło na pełne skupienie się na projektowaniu schematu danych oraz logice aplikacyjnej.

Wybór relacyjnego modelu danych wynikał bezpośrednio z charakteru przetwarzanych informacji. Dane dotyczące powiązań między urządzeniami, pomiarami telemetrycznymi a wynikami testów jakości połączenia cechują się ściśle zdefiniowanym schematem. Zastosowanie relacyjnej bazy danych gwarantuje zachowanie spójności danych oraz pozwala na wykonywanie złożonych zapytań, co jest niezbędne na etapie analizy i wizualizacji zgromadzonych wyników [71].

Mimo dostępności alternatywnych rozwiązań w chmurze Azure (takich jak Azure SQL czy MySQL), zdecydowano się na silnik PostgreSQL ze względu na wsparcie dla rozszerzenia **PostGIS**. Rozszerzenie to przekształca relacyjną bazę danych w zaawansowane narzędzie geoprzestrzenne, oferując natywne mechanizmy przechowywania i przetwarzania współrzędnych GPS [72]. W systemie, w którym każdy pomiar jest ściśle przypisany do lokalizacji geograficznej, priorytetem było uzyskanie wysokiej wydajności operacji przestrzennych. Dodatkowym argumentem przemawiającym za tym wyborem była wcześniejsza znajomość tego systemu, co przełożyło się na sprawną implementację modelu danych.

W opracowanej architekturze wykorzystano PostgreSQL w wersji 16 wraz z rozszerzeniem PostGIS 3.4. Wybór ten zapewnił dostęp do zaawansowanych mechanizmów przetwarzania danych przestrzennych:

- **Specjalistyczne typy danych** – zastosowano typ `GEOGRAPHY` dla punktów GPS w układzie WGS84, co umożliwia precyzyjne obliczenia na sferze ziemskiej z uwzględnieniem krzywizny planety, oraz typ `GEOMETRY` dla reprezentacji kartezjańskiej, przyspieszający operacje na mniejszych, lokalnych obszarach [73].
- **Funkcje przestrzenne** – wykorzystano standardowe funkcje zgodne z normą OGC (ang. Open Geospatial Consortium), takie jak `ST_X()` oraz `ST_Y()`, pozwalające na efektywną ekstrakcję współrzędnych bezpośrednio w zapytaniach SQL bez konieczności rzutowania typów w warstwie aplikacyjnej [74].

Komunikacja pomiędzy warstwą aplikacji a bazą danych realizowana jest poprzez driver **psycopg2-binary** w wersji 2.9. Jest to najpopularniejszy adapter standardu DB API 2.0 dla języka Python, który zapewnia stabilne i bezpieczne (poprzez parametryzację zapytań) połączenie z silnikiem bazy danych [75].

W celu niezawodnego przesyłu danych, zdecydowano się na wykorzystanie usługi **Azure Service Bus**. Jest to w pełni zarządzany broker komunikatów (ang. message broker), który w architekturze SignalMap pełni rolę pośredniej warstwy wymiany danych oraz bufora zdarzeń. Rozwiązanie to pozwoliło na skuteczne odseparowanie etapu przyjmowania danych od ich właściwego przetwarzania i zapisu, co określa się mianem luźnego powiązania komponentów (ang. loose coupling) [76]. Takie podejście znacząco podnosi odporność systemu na nagłe skoki obciążenia oraz umożliwia niezależne skalowanie warstwy brzegowej i warstwy bazodanowej.

Zastosowanie mechanizmu kolejek umożliwiło asynchroniczne przetwarzanie napływającej telemetrii. Dzięki temu komponent odpowiedzialny za odbieranie pakietów od aplikacji mobilnych nie jest bezpośrednio obciążony czasem trwania operacji zapisu w bazie danych, co minimalizuje opóźnienia z perspektywy użytkownika końcowego. Azure Service Bus udostępnia zaawansowane mechanizmy kontroli przepływu, wśród których kluczowe znaczenie ma tryb *Peek-Lock*. Pozwala on na tymczasowe zablokowanie wiadomości przez proces konsumujący, co gwarantuje, że w przypadku wystąpienia awarii podczas przetwarzania, komunikat nie zostanie utracony i powróci do kolejki, stając się ponownie dostępnym do pobrania [77]. Dodatkowo usługa zapewnia obsługę sesji oraz gwarancję zachowania kolejności komunikatów (FIFO), co jest istotne przy analizie chronologicznych zdarzeń sieciowych.

Podczas projektowania systemu rozważano również alternatywne technologie, takie jak *Azure Storage Queues*. Choć jest to rozwiązanie prostsze, wybór padł na Azure Service Bus ze względu na znacznie szerszy wachlarz funkcjonalności, w tym precyzyjne zarządzanie czasem blokady (ang. *Lock duration*) oraz obsługę operacji atomowych [78]. Podjęto również decyzję o rezygnacji z platform strumieniowych, takich jak Apache Kafka. Mimo ich wysokiej wydajności, przy zakładanej skali projektu wdrożenie i utrzymanie klastra Kafka wiązałoby się z nadmierną złożonością operacyjną [79]. Azure Service Bus oferuje optymalny balans między łatwością konfiguracji a dostępnością funkcji niezbędnych do obsługi danych telemetrycznych w środowisku chmurowym.

Rejestr obrazów i artefakty uruchomieniowe

Azure Container Registry (ACR) [18]- to usługa zapewniająca bezpieczne i wersjonowane przechowywanie obrazów dockerowych. W przypadku niniejszej pracy obrazy frontendowe i backendowe są przechowywane w osobnych repozytoriach w ACR. Ze względu na ograniczenia konta możliwe było stworzenie tylko ACR z publicznym adresem, jednak sam dostęp jest możliwy tylko przy odpowiedniej roli RBAC [80] lub z poziomu Azure Portal.

3.3.2 Automatyzacja wdrożeń i zarządzanie konfiguracją

Zarządzanie projektem i kodem źródłowym

Microsoft Azure DevOps - chmurowa platforma udostępniająca narzędzia i zasoby do pracy zespołowej. Pozwala na łatwe planowani, zarządzenie i konfigurowanie zadań podczas tworzenia projektu. Składa się z pięciu podstawowych elementów: Boards, Repos, Pipelines, Test Plans oraz Artifacts. W niniejszej pracy używano głównie trzech pierwszych elementów.

a) **Boards** [81] - wykorzystywane do planowania zadań związanych z kolejnymi etapami wdrożenia systemu. Zadania opisywane były jako Task. Podział prac na mniejsze kroki pozwolił na optymalną delegację zadań, co przełożyło się na lepszą organizację pracy grupowej.

b) **Repos** [82] - najistotniejszy element podczas pracy z Azure DevOps. Pozwala na ustrukturyzowane zarządzanie kodem, opisz poszczególnych zmian. W projekcie używano czterech odseparowanych repozytoriów:

   - Data Sender - repozytorium zawierające kod aplikacji mobilnej
   - Infrastruktura - repozytorium zawierające pliki opisujące środowisko oraz pliki konfiguracyjne Kubernetes.
   - Frontend - repozytorium zawierające kod aplikacji frontendowej.
   - Backend - repozytorium zawierające kod aplikacji backendnowej.

c) **Pipelines** [83] - służy do stworzenia i zarządzania pipelinami z poziomu Azure DevOps.

Pipelines CI/CD i Infrastructure as Code

c) **Pipelines** - wykorzystywane do tworzenie zarządzanych pipelinów. W niniejszej pracy używano pipelinów do tworzenia infrastruktury za pomocą Terraform, implementacji kodów dla usług Function App oraz do tworzenia i publikowania obrazów aplikacji w Azure Container Registry.

Terraform![84] - narzędzie typu IaC (ang. Infrastructure as Code). Pozwala na definiowanie, tworzenie i zarządzanie architektura chmurową za pomocą deklaratywnych plików. Posiada on pliki stanu (tfstate) co pozwala na zapamiętywanie stanu utworzonej infrastruktury. Przy zmianach plików Terraform porównuje stan z dodanymi wdrażaniami i stworzy te usługi, których nie ma. W sekcji Pipelines w Azure DevOps zdefiniowane są pipline, które wykonują wdrażania na platformie Azure.

Wdrożenie na Kubernetes: Helm i GitOps

Pod [85] - jest najmniejszą jednostką uruchomieniową w Kubernetes. Reprezentuje jedną lub więcej ściśle powiązanych kontenerów, które współdzielą tę samą przestrzeń sieciową (adres IP, porty), wolumeny danych oraz cykl życia. Kontenery w obrębie jednego Poda są zawsze uruchamiane na tym samym węźle i komunikują się ze sobą za pomocą interfejsu localhost.

Node [86] - jest maszyną roboczą w klastrze Kubernetes, fizyczną lub wirtualną, odpowiedzialną za uruchamianie Podów. Każdy węzeł zawiera komponenty niezbędne do działania kontenerów, takie jak kubelet, środowisko uruchomieniowe kontenerów oraz kube-proxy, a także komunikuje się z płaszczyzną sterowania klastra w celu realizacji przydzielonych zadań.

Helm [15] - menedżer pakietów w ekosystemie Kubernetes. Pozwala na definiowanie, konfigurowaniu poszczególnych usług za pomocą *Chartów*![87]. W pracy wykorzystano narzędzie do instalacji gotowych chartów konfiguracyjnych oraz do utworzenia i zarządzania Deploymentem aplikacji frontedowych oraz backendnowych. Posiada on mechanizmy związane z zarządzaniem cyklem życia (Releases) co pomaga we wdrażaniu nowych wersji obrazów lub przywracania poprzednich działających wersji.

Flux [16]- narzędzie z rodziny GitOps, które dba o to, aby stan klastra AKS był adekwatny do plików konfiguracyjny przechowywanych w Repos. Flux na bieżąco monitoruje stan repozytorium i w momencie wykrycia zmian wprowadza je do klastra. Działa on jako pod w samym AKS stąd potrzeba dodania odpowiedniego klucza dostępu do repozytorium w Repos. W przypadku tej pracy nasłuchuje on na folder Infrastruktura/k8s/k8s_config.

Zarządzanie konfiguracją aplikacji

Wszystkie parametry konfiguracyjne aplikacji, takie jak adresy URL usług czy limity zasobów, są zarządzane poprzez zmienne środowiskowe i pliki YAML w repozytorium infrastrukturalnym, co zapewnia spójność środowisk.

3.3.3 Bezpieczeństwo i segmentacja sieciowa

Warstwa brzegowa: WAF i API Management

Bezpieczeństwo oraz kontrola ruchu przychodzącego do systemu są realizowane w dedykowanej warstwie brzegowej, która stanowi pierwszą linię obrony przed zagrożeniami z sieci publicznej.

Kluczowym elementem chroniącym aplikację webową (frontend) oraz jej punkty końcowe jest Web Application Firewall (WAF), działający w ramach usługi *Azure Application Gateway*. Rozwiązanie to służy do filtrowania ruchu HTTP/HTTPS i ochrony przed powszechnymi atakami celowanymi w interfejsy przeglądarkowe, takimi jak *SQL Injection* oraz *Cross-Site Scripting* (XSS):

- **SQL Injection** – polega na próbie przemycenia złośliwych fragmentów kodu SQL w parametrach zapytań (np. w polach formularzy). Brak walidacji danych wejściowych może pozwolić intruzowi na modyfikację struktury zapytania, prowadząc do wycieku lub usunięcia danych [88].
- **Cross-Site Scripting (XSS)** – atak polegający na wstrzyknięciu złośliwych skryptów wykonywanych w przeglądarce innego użytkownika, co może służyć do kradzieży ciasteczek sesyjnych lub przejęcia konta [89].

Stosowanie warstwy WAF pozwala na blokowanie takich wzorców ataku przed dotarciem do logiki systemu, co stanowi krytyczny element strategii *Defense in Depth*. Wykorzystanie ustandaryzowanych baz sygnatur (ang. *Managed Rule Sets*) odciąża wewnętrzne zasoby od konieczności samodzielnej inspekcji podejrzanych pakietów [90].

Z kolei na drodze komunikacji z aplikacją mobilną (Android) znajduje się Azure API Management (APIM). Pełni on rolę wyspecjalizowanej bramy API (ang. *API Gateway*), która zarządza ruchem generowanym przez urządzenia końcowe przesyłające dane telemetryczne. Dzięki zastosowaniu polityk dostępowych (ang. *policies*) zaimplementowano mechanizmy specyficzne dla ruchu ruchu zautomatyzowanego:

- **Limitowanie liczby żądań (ang. *throttling*)** – chroni backend przed przeciążeniem i atakami DoS, odrzucając nadmiarowe żądania z kodem HTTP 429 (*Too Many Requests*) po przekroczeniu zdefiniowanego limitu w oknie czasowym [91].
- **Kontrola wersji interfejsów** – pozwala na bezpieczne współistnienie wielu wersji API, co jest niezbędne w cyklu życia aplikacji mobilnych, gdzie użytkownicy nie zawsze aktualizują oprogramowanie w tym samym czasie [92].

Wykorzystanie obu tych mechanizmów sprawia, że system jest zabezpieczony wielowarstwowo: WAF zabezpiecza interfejs użytkownika przed atakami typu *web-based*, natomiast APIM dba o stabilność i uporządkowanie ruchu telemetrycznego z urządzeń mobilnych. Zapewnia to spójną politykę bezpieczeństwa w całym systemie SignalMap [93].

Ruch przychodzący do klastra i TLS

Nginx Controller [94] - narzędzie służące do zarządzania ruchem przychodzącym do klastra AKS na adres publiczny. Działa jako pod w samym AKS. Za pomocą definiowanych reguły routinguje ruch na odpowiednie aplikacje oraz serwuje certyfikaty TLS/SSL z współpracy z Cert-Manager.

Cert-Manager [95] - to natywne narzędzie dla Kubernetesa do automatyzacji zarządzania certyfikatami TLS/SSL. Jest on zintegrowanych z darmowym urzędem certyfikacji Let's Encrypt [96]

poprzez protokół ACME [97]. Jest on implementowany jako pod w AKS i ściśle współpracuje z Podami Nginx Controller oraz z obiektami Ingress.

Sekrety i dane wrażliwe

Azure Key Vault [98] - usługa służąca do przechowywania i zarządzania wrażliwymi danymi takimi jak: klucze, hasła, certyfikaty. Został on użyty ze względu na eliminacje problemy związanego z przechowywaniem danych wrażliwych w formacie base64 w Kubernetes. Base64 nie jest odpowiednim sposobem kodowania danych związanych z logowaniem. Jedynymi Podami mogącymi komunikować się z Key Vault są Pody Secrets Storage CSI Driver. Pody te odpowiadają za odczyt i montowanie w pamięci RAM kontenerów (tmpfs) danych logowania do bazy w Azure PostgreSQL.

Store CSI Driver, Secrets Store CSI Driver, CSI Driver [99] - mechanizm implementowany jako pod w AKS służący do pobierania danych logowania, kluczy z Azure Key Vault. Pozwala o na bezpieczne monitowania danych wrażliwych w Podach backendowych.

Dostęp prywatny do usług: Private Link, Private Endpoint i DNS

Istotnym aspektem bezpieczeństwa infrastruktury systemu SignalMap jest całkowita izolacja ruchu sieciowego pomiędzy kluczowymi usługami. W celu wyeliminowania konieczności przesyłania danych przez publiczną sieć Internet, wdrożono mechanizmy oparte na rozwiązaniach **Azure Private Link** oraz **Azure Private Endpoint**.

Azure Private Link stanowi fundament bezpiecznej architektury, umożliwiając mapowanie usług platformy Azure (takich jak bazy danych czy magazyny kluczy) na prywatną przestrzeń adresową wewnątrz konkretnej sieci wirtualnej (VNet) [100]. Dzięki temu usługi te stają się dostępne wyłącznie z poziomu zdefiniowanej sieci wewnętrznej, co zmniejsza powierzchnię ataku poprzez całkowite odcięcie ich od publicznych adresów IP.

Techniczną realizacją tego połączenia jest **Azure Private Endpoint**. Można go zdefiniować jako wirtualną emulację fizycznego interfejsu sieciowego (NIC), który zostaje „wpięty" do wybranej podsieci wewnątrz VNet [101]. W ten sposób usługa Azure (np. PostgreSQL), która naturalnie funkcjonuje jako zasób publiczny, otrzymuje swój unikalny adres prywatny i jest traktowana przez inne komponenty (np. klastry Kubernetes) jako lokalny zasób sieciowy. Takie podejście zapewnia, że proces przetwarzania krytycznych informacji oraz danych telemetrycznych odbywa się wyłącznie wewnątrz odizolowanej infrastruktury chmurowej, bez ekspozycji na ruch z publicznej sieci internet.

Dopełnieniem tej architektury jest usługa **Azure Private DNS Zones**. W środowisku chmurowym poprawne rozwiązywanie nazw domenowych (DNS) jest niezbędne dla zachowania ciągłości działania aplikacji. Private DNS Zones pozwala na utrzymanie niezmiennych nazw FQDN (ang. *Fully Qualified Domain Name*) poszczególnych usług, przy jednoczesnym automatycznym kierowaniu ruchu na ich wewnętrzne adresy prywatne zamiast na adresy publiczne [102]. Przykładowo, backend aplikacji może komunikować się z bazą danych przy użyciu jej standardowej nazwy domenowej, a system DNS wewnątrz sieci VNet zadba o to, aby zapytanie trafiło bezpośrednio do prywatnego punktu końcowego (Private Endpoint), co zapewnia pełną transparentność dla warstwy aplikacyjnej przy zachowaniu najwyższych standardów bezpieczeństwa [103].

Monitoring i obserwowalność

Azure Monitor [104] - stanowi nadrzędną warstwę analityczną, która gromadzi dane telemetryczne bezpośrednio z poziomu platformy chmurowej, monitorując zasoby niedostępne dla we-

wnętrznych narzędzi klastra, takie jak sieć wirtualna, baza danych *PostgreSQL* czy magazyn kluczy.W systemie pełni ona rolę jednego z głównych źródeł danych dla platformy *Grafana*

Monitoring - jest to zdefiniowany chart o nazwie kube-prometheus-stack! [105]. Pozwala na wdrożenie gotowego ekosystemu monitoringu klastra oraz innych usług.

- Prometheus Operator [106] - Pod zarządzający ustawieniami Prometheusa. Zarządza on też działaniem pozostałych elementów ekosystem.
- Prometheus [107] - baza typu szeregów czasowych (ang. Time Series Database - TSDB [108]). Służy do przechowywania metryk.
- Grafana [109] - narzędzie służące do wizualizacji danych. Dane są pobierane z Prometheusa oraz Azure Monitor.
- Alertmanager - pod służy do konfigurowana powiadomień. W przypadku tej pracy nieużywany.
- Node Exporter [110] - Pody implementowane na każdym Nodzie. Służą do zbierania danych odnośnie zasobów Nodów.
- Kube-State-Metrics [111] - pod służący do translacji stanów obiektów Kubernetes do formatu rozumianego przez Prometheus.

Dodatkowo do ekosystemu dodano PostgreSQL Exporter [112]. Służy on to wyciągania metryk specyficznych dla silnika bazodanowego, aby otrzymać dokładniejsze dane odnośnie pracy usługi.

3.3.4 Technologie warstwy aplikacyjnej

Aplikacja mobilna Android

Kluczowym komponentem systemu, pełniącym funkcję źródła danych telemetrycznych, jest natywna aplikacja mobilna opracowana dla systemu operacyjnego Android. Wybór tej platformy był podyktowany jej szerokim rozpowszechnieniem na rynku oraz, co najistotniejsze z punktu widzenia celów pracy, otwartym charakterem architektury umożliwiającym głęboki dostęp do parametrów sprzętowych urządzenia [113]. W przeciwieństwie do konkurencyjnych systemów mobilnych, Android udostępnia programistom rozbudowane interfejsy API, które pozwalają na precyzyjne odczytywanie stanu modułów komunikacyjnych.

Zastosowanie podejścia natywnego umożliwiło bezpośrednie wykorzystanie mechanizmów systemowych do zbierania precyzyjnych danych lokalizacyjnych (GPS) oraz niskopoziomowych parametrów sieci komórkowej, takich jak siła sygnału (RSRP, RSRQ) czy identyfikatory stacji bazowych (Cell ID). Wykorzystanie dedykowanych interfejsów API systemu Android pozwoliło na pozyskanie szczegółowych informacji o technologii transmisji danych (LTE/5G), które w systemach konkurencyjnych są zazwyczaj niedostępne dla aplikacji firm trzecich [114].

Wybór aplikacji natywnej zamiast rozwiązań uniwersalnych był podyktowany przede wszystkim troską o wydajność urządzenia i komfort użytkownika. Podejście to pozwoliło na znacznie lepsze zarządzanie zużyciem energii, co jest kluczowe, ponieważ ciągłe korzystanie z modułu GPS oraz monitorowanie sieci komórkowej mocno obciąża baterię telefonu [115]. Dzięki bezpośredniemu dostępowi do funkcji systemowych, aplikacja może pracować w sposób zoptymalizowany, pobierając dane tylko wtedy, gdy jest to niezbędne.

Backend: Python i Flask

Backend aplikacji SignalMap został zaimplementowany w języku Python w wersji 3.12 jako RESTful API [116]. Python wybrano ze względu na szybkie prototypowanie, bogaty ekosystem bibliotek oraz prostote wykonywania operacji takich jak zapytania do bazy danych lub konwersja danych.

Flask [117] - framework webowy w wersji 3.0, odpowiedzialny za obsługę żądań HTTP i routing endpointów API. Konfiguracja **threaded=True** umożliwia obsługę wielowątkową, gdzie każde żądanie HTTP jest przetwarzane w osobnym wątku.

Backend wykorzystuje bezpośrednie zapytania SQL. Takie podejście zostało wybrane w celu zachowania pełnej kontroli nad generowanymi zapytaniami oraz ułatwienia debugowania, dzięki ich jawnej obecności w kodzie. Bezpieczeństwo operacji na bazie danych zapewnia parametryzacja dostępna w bibliotece psycopg2, która zapobiega interpretacji danych użytkownika jako kodu SQL.

Konteneryzacja - aplikacja jest pakowana w obraz Docker oparty na **python:3.12-alpine**, zapewniając niewielki rozmiar obrazu (około 200MB).

Frontend: React i deck.gl

Frontend aplikacji został zaimplementowany jako Single Page Application (SPA) [118] w bibliotece React [119] w wersji 18. React to deklaratywna biblioteka JavaScript do budowy interfejsów użytkownika, oparta na architekturze, która pozwala na podział interfejsu na niezależne komponenty (`Header`, `Sidebar`, `Map`, `Stats`). React został wybrany ze względu na duży ekosystem bibliotek dla wizualizacji danych geograficznych oraz dojrzałość technologii.

deck.gl [120] - framework WebGL w wersji 8.9 do wizualizacji dużych zbiorów danych geograficznych, wykorzystujący GPU do renderowania. Deck.gl został wybrany ze względu na dużą ilość dostępnych funkcjonalności zwłaszcza w kontekście tworzenia warstw punktów na mapie. W projekcie zastosowano następujące warstwy:

- `ScatterplotLayer` - renderowanie punktów pomiarów telemetrycznych z kolorowaniem według siły sygnału, obsługa najechania na punkt, czy kliknięcia
- `HeatmapLayer` - mapa gęstości pomiarów, z jednolitą wagą punktów
- `IconLayer` - wyświetlanie własnych ikon stacji bazowych BTS na mapie

Jako dostawcę danych mapowych dla aplikacji wykorzystano usługę MapTiler [121] poprzez jej API w darmowej wersji, co umożliwia wyświetlanie interaktywnych map w interfejsie użytkownika bez konieczności zarządzania własnymi serwerami mapowymi.

Komunikacja z backendem - jest realizowana przez wbudowany w przeglądarki mechanizm **fetch** API z parametryzowanymi zapytaniami. URL backendu jest pobierany ze zmiennej środowiskowej. W Kubernetes frontend łączy się z backendem przez DNS wewnętrzny klastra. Zapytania zawierają parametry takie jak **minutes** (zakres czasowy), **limit** (maksymalna liczba rekordów),

operator (filtr po operatorze sieci). Obsługa błędów realizowana przez try-catch z wyświetleniem komunikatu użytkownikowi.

Zarządzanie stanem aplikacji - zarządzanie odbywa się wyłącznie przez React Hooks:

- `useState` - lokalny stan komponentów (dane telemetryczne, dane o stacjach bazowych (BTS), wybrane filtry, stan UI jak otwarty/zamknięty sidebar)
- `useEffect` - wykonywanie operacji reagujących na zmiany w aplikacji takie jak automatyczne pobieranie danych z API przy zmianie filtrów daty
- `useMemo` - memoizacja kosztownych obliczeń (filtrowanie danych, tworzenie warstw deck.gl) aby uniknąć niepotrzebnych i powtarzających się obliczeń

Stylowanie - oparte na CSS [122], gdzie każdy komponent posiada dedykowany plik CSS.

Build i deployment - aplikacja jest budowana do statycznych plików (HTML, CSS, JavaScript) przez Create React App [123] i serwowana przez Nginx w kontenerze Docker. Wykorzystano multi-stage build, gdzie pierwszy stage (Node.js 18 Alpine) wykonuje `npm install` i `npm run build`, a drugi stage (Nginx Alpine) kopiuje tylko folder z wynikiem buildu. Finalny obraz ma około 25 MB.

3.4 Projekt klastra Kubernetes

Warstwę uruchomieniową aplikacji zrealizowano w oparciu o **Azure Kubernetes Service (AKS)**, tj. zarządzaną usługę orkiestracji kontenerów Kubernetes udostępnianą przez platformę Microsoft Azure. W modelu AKS dostawca chmury odpowiada za utrzymanie płaszczyzny sterującej (*control plane*), natomiast użytkownik zarządza pulami węzłów roboczych (*node pools*) oraz zasobami uruchamianymi w klastrze. Takie podejście upraszcza administrację i ogranicza nakład prac utrzymaniowych w porównaniu do klastra instalowanego samodzielnie na infrastrukturze *bare-metal* lub na maszynach wirtualnych [124]. Węzły robocze klastra uruchomiono na **Virtual Machine Scale Sets (VMSS)**, które stanowią mechanizm automatycznego skalowania liczby maszyn wirtualnych w zależności od zapotrzebowania na zasoby. Konfiguracja infrastruktury została zdefiniowana w Terraform, gdzie określono parametry puli węzłów oraz dopuszczalne granice autoskalowania [125].

W projekcie przyjęto następujące parametry maszyn wirtualnych w puli węzłów:

- lokalizacja - North Europe
- SKU - B2ls_v2
- CPU - 2
- RAM - 4 GB
- ilość dysków - 4
- operacje IOPS - 3750
- minimalna ilość VM - 1
- maksymalna ilość VM - 3

Mechanizm VMSS pozwala skalować pulę w zakresie od jednej do trzech maszyn wirtualnych. Maksymalna liczba podów na pojedynczym węźle jest parametrem konfiguracyjnym ustawianym podczas tworzenia puli węzłów; w niniejszym projekcie pozostawiono wartość domyślną wynoszącą 30 podów na węzeł. [126] VMSS posiada także swój adres publiczny oraz jest przypisany do sieci *k8s-vnet/k8s-subnet*. Odpowiednie przypisanie sieci do VMSS jest kluczowe do późniejszej obsługi monitoringu oraz połączenia z bazą.

Sieć *k8s-vnet* o adresie 172.16.0.0/16 posiada wydzieloną podsieć *k8s-subnet* z adresacją 172.16.0 .0/20, która stanowi fundament komunikacji w klastrze. AKS wykorzystuje natywną opcję zarządzania siecią poprzez *Azure CNI*, co sprawia, że adresy przypisywane zarówno do Nodów, jak i do Podów, pochodzą bezpośrednio z tej podsieci. Dzięki temu Pody są pełnoprawnymi uczestnikami sieci VNet i mogą komunikować się z innymi zasobami Azure bez dodatkowej translacji adresów. Kluczowe jest to, że obiekty *Service* posiadają całkowicie odrębną adresację z wirtualnej sieci 10.0.0.0/16, a wewnętrzny system DNS obsługujący rozwiązywanie nazw w klastrze posiada stały adres 10.0.0.10. Komunikacja między Podami a serwisami nie opiera się na klasycznym routingu, lecz wykorzystuje mechanizm DNAT realizowany przez komponent *kube-proxy* na każdym Nodzie, który tłumaczy wirtualny adres serwisu na realny adres IP konkretnego poda docelowego.

Podczas konfiguracji AKS platforma Azure automatycznie wdraża, a następnie zarządza kilkoma Podami w Namespace *kube-system* [127]:

- azure-cns - odpowiada za przydzielanie adresów IP dla podów, działa jako DaemonSet
- azure-ip-masq-agent - odpowiada za odpowiednią translacje NAT, działa jako DaemonSet
- cloud-node-manger - odpowiada za odpowiednie zarządzanie nodem, działa jako DaemonSet
- coredns - rola DNS w klastrze, pozwala na rozwiązywanie nazw. Domyślnie klaster obsługuje domenę *cluster.local*, usługa ta będzie kluczowa w komunikacji backend-frontend
- coredns-autoscaler - odpowiada za skalowanie komponentów coredns
- kube-proxy - odpowiada za komunikacje wewnątrz noda
- metrics-server - zapewnia funkcje zbierania podstawowych metryk, kluczowy przy politykach auto-skalowalności

Kluczowym aspektem podczas tworzenia AKS jest także odpowiednie ustawienie uprawnień do pobierania obrazów z ACR (Azure Container Registry), komunikacji z Azure Key Vault, możliwość wykorzystania adresu publicznego z puli Public IP oraz utworzenie obiektu App Registration wykorzystywanego do pobierania danych z usług Azure do Grafany. Ustawienie tych uprawnień zostało opisane poniżej:

a) **ACR** - klaster AKS potrzebuje uprawnień `AcrPull`. Uprawnienia te przypisano komendą:

```
az aks update -n inzynierkaaks -g Inzynierka --attach-acr
inzynierka
```

b) **Dostęp do Key Vault**:

- Kluczowe jest utworzenie *Private Endpoint* w tej samej podsieci co AKS.
- Konieczna jest konfiguracja *Private DNS Zone*, aby klaster mógł rozwiązać nazwę usługi na adres prywatny.
- Dodanie dodatku (*addon*) do komunikacji z Key Vault realizuje się komendą:

```
az aks enable-addons --addons azure-keyvault-secrets-provider
-g Inzynierka -n inzynierkaaks
```

c) **Dostęp do publicznego IP w AKS** - ze względu na umiejscowienie adresu IP w innej grupie zasobów niż ta, w której jest klaster AKS, należy nadać uprawnienia do zarządzania adresami IP. Rolę, którą trzeba przypisać to *Network Contributor*:

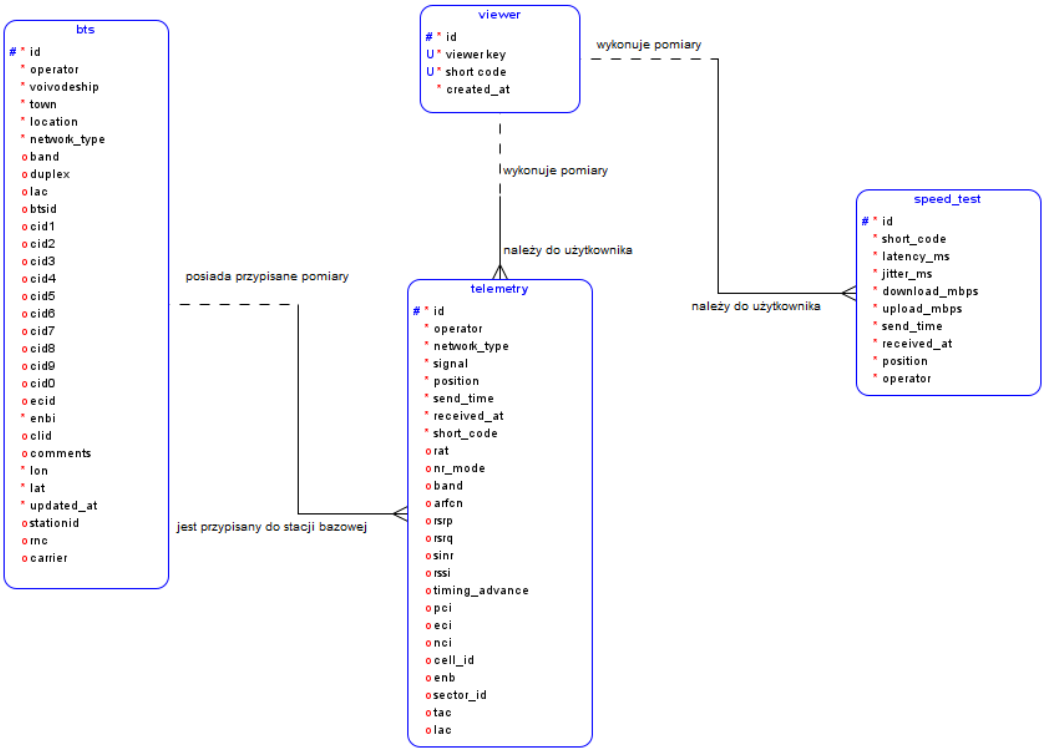
```
az role assignment create --assignee $(az aks show -g Inzynierka -n
    inzynierkaaks --query identityProfile.kubeletidentity.objectId -o tsv)
    --role "Network Contributor" --scope $(az group show --name Inzynierka
    --query id -o tsv)
```

d) **Dostęp do metryk usług Azure za pomocą AppRegistration**:

- Skonfigurowanie odpowiedniego *App Registration* oraz dodanie uprawnień *Monitoring Reader* na poziomie Microsoft Entra ID.
- Autoryzacja połączenia w Grafanie do usług Azure Monitor realizowane jest poprzez *Data Source* typu *Azure Monitor*. Do poprawnej komunikacji wykorzystywane są trzy parametry: `TENANT_ID`, `CLIENT_ID` oraz `CLIENT_SECRET`.

3.5 Model danych

W podrozdziale przedstawiono logiczny model danych projektowanego systemu, odpowiedzialnego za gromadzenie i analizę pomiarów jakości sieci komórkowej. Celem modelu jest zapewnienie spójnej i elastycznej struktury umożliwiającej przechowywanie danych pomiarowych, informacji o użytkownikach oraz elementach infrastruktury sieciowej.



Rysunek 3.5: Diagram związków encji systemu

Rysunek 3.5 przedstawia logiczny model danych systemu w postaci diagramu związków encji (ERD), opisujący strukturę przechowywanych informacji oraz relacje pomiędzy poszczególnymi encjami.

Centralnym elementem modelu jest encja *telemetry*, reprezentująca pojedyncze pomiary parametrów sieci komórkowej zbierane przez aplikację mobilną. Rekordy tej tabeli zawierają informacje o jakości sygnału, typie technologii radiowej, wybranych parametrach połączenia oraz pozycji geograficznej punktu pomiarowego. Każdy pomiar jest powiązany z konkretnym użytkownikiem systemu oraz — jeżeli to możliwe — przypisany do odpowiedniej stacji bazowej.

Encja *viewer* reprezentuje użytkownika wykonującego pomiary. Każdy użytkownik identyfikowany jest za pomocą unikalnego klucza oraz skróconego identyfikatora `short_code`, który wykorzystywany jest do powiązania pomiarów telemetrycznych oraz testów jakości połączenia z danym użytkownikiem. Relacja pomiędzy encjami *viewer* oraz *telemetry* ma charakter jeden-do-wielu — jeden użytkownik może wykonać wiele pomiarów.

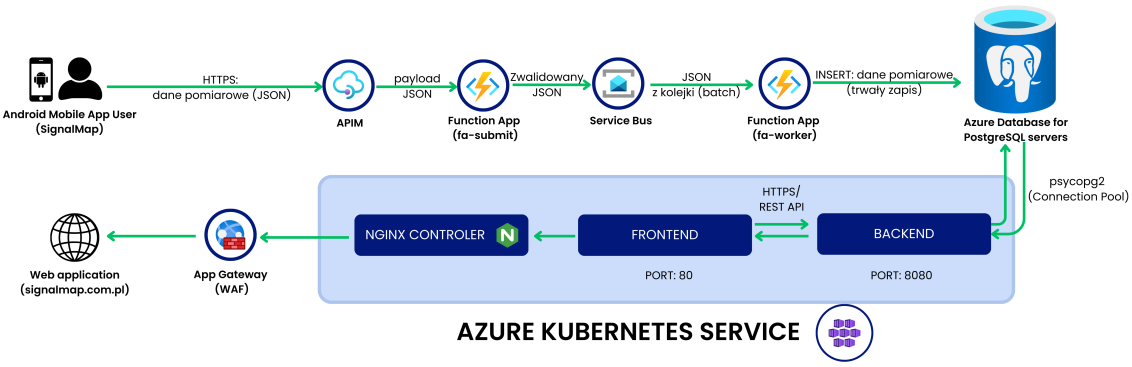
Dodatkowym typem danych gromadzonych w systemie są wyniki testów jakości połączenia, reprezentowane przez encję *speed_test*. Zawiera ona informacje o opóźnieniach, przepustowości pobierania i wysyłania danych oraz czasie wykonania testu. Każdy test prędkości, podobnie jak pomiary telemetryczne, przypisany jest do konkretnego użytkownika poprzez relację z encją *viewer*.

Encja *bts* opisuje stacje bazowe sieci komórkowej wraz z ich lokalizacją, operatorem oraz wybranymi parametrami technicznymi. Dane dotyczące stacji bazowych pochodzą z zewnętrznego serwisu BTSearch, który udostępnia zbiór obiektów infrastruktury sieciowej na terenie Polski [128]. Pomiary telemetryczne mogą być przypisywane do stacji bazowych na podstawie identyfikatorów sieciowych, takich jak `eNB`, `CID` czy `LAC`. Relacja pomiędzy encjami *telemetry* oraz *bts* umożliwia analizę jakości sygnału w kontekście konkretnej infrastruktury sieciowej.

Zaprojektowany model danych zapewnia spójne przechowywanie informacji o użytkownikach, pomiarach telemetrycznych, testach jakości połączenia oraz stacjach bazowych. Taka struktura umożliwia wykonywanie analiz przestrzennych i czasowych, a także korelowanie jakości sieci z lokalizacją użytkownika oraz parametrami infrastruktury operatorów.

3.6 Projekt przepływu danych

Celem zaprojektowanego przepływu danych jest zapewnienie niezawodnego i skalowalnego przetwarzania danych pomiarowych pochodzących z rozproszonych urządzeń mobilnych. Architektura została oparta na podejściu asynchronicznym, które umożliwia separację etapu przyjmowania danych od ich dalszego przetwarzania i zapisu. W niniejszym rozdziale przedstawiono ogólną koncepcję przepływu danych w systemie oraz rolę poszczególnych komponentów w procesie ich obsługi.



Rysunek 3.6: Schemat przepływu danych telemetrycznych w systemie

Na rysunku 3.6 przedstawiono szczegółowy przebieg danych telemetrycznych w systemie. Proces rozpoczyna się w aplikacji mobilnej, w której generowane są dane pomiarowe, a następnie są one przesyłane do infrastruktury chmurowej w sposób asynchroniczny. Każdy etap przepływu odpowiada za realizację ściśle określonej funkcji, począwszy od przyjęcia danych, poprzez ich buforowanie i przetwarzanie, aż po trwały zapis w warstwie danych.

3.6.1 Generowanie i przesył danych z aplikacji mobilnej

Dane pomiarowe są generowane w aplikacji mobilnej działającej na systemie Android. Obejmują one informacje telemetryczne dotyczące parametrów sieci komórkowej, takie jak poziom sygnału, identyfikatory komórki, pasmo częstotliwości oraz typ technologii radiowej, a także dane lokalizacyjne. W przypadku testów jakości połączenia przesyłane są wyniki testów prędkości transmisji danych.

Zebrane dane są pakowane w postać obiektu JSON i przesyłane z aplikacji mobilnej do systemu backendowego z wykorzystaniem protokołu HTTPS. Zastosowanie formatu JSON jako reprezentacji danych umożliwia ich łatwą serializację i walidację oraz zapewnia pełną spójność wymiany informacji pomiędzy aplikacją mobilną a komponentami backendowymi, niezależnie od użytych technologii [129].

3.6.2 Warstwa brzegowa i integracyjna API

Pierwszym elementem infrastruktury chmurowej odbierającym dane jest usługa *Azure API Management (APIM)*, pełniącą rolę centralnej bramy API (*API Gateway*). Odpowiada ona za obsługę bezpiecznego ruchu HTTPS, stanowiąc jednolity punkt styku systemu z aplikacją mobilną oraz zapewniając kontrolę nad dostępem do zasobów backendowych.

Usługa API Management pełni funkcję pośrednika odpowiedzialnego za uwierzytelnianie żądań oraz ich kierowanie do właściwych punktów końcowych aplikacji funkcyjnych. Na tym etapie dane nie są jeszcze poddawane analizie biznesowej; warstwa ta koncentruje się na bezpiecznym przyjęciu pakietów, egzekwowaniu limitów wywołań oraz izolacji wewnętrznej logiki systemu przed bezpośrednim dostępem z sieci Internet.

3.6.3 Walidacja i normalizacja danych w Function App

Kolejnym etapem przepływu danych jest funkcja backendowa realizowana w usłudze Azure Function App (FA-SUBMIT). Jej zadaniem jest przyjęcie danych pomiarowych w formacie JSON oraz wykonanie ich wstępnej walidacji i normalizacji. Przeniesienie walidacji danych na wczesny etap przepływu pozwala na szybkie odrzucenie niepoprawnych lub niekompletnych komunikatów, ograniczając propagację błędów w dalszych etapach przetwarzania oraz zmniejszając obciążenie warstwy danych [130].

Na tym etapie sprawdzana jest poprawność kluczowych pól, w tym typy danych, zakresy wartości oraz poprawność współrzędnych geograficznych. W wyniku tego procesu tworzony jest kanoniczny komunikat danych pomiarowych, przygotowany do dalszego przetwarzania.

Tak zwalidowany i znormalizowany komunikat jest następnie przekazywany asynchronicznie do kolejki wiadomości.

3.6.4 Asynchroniczne buforowanie danych w kolejce

Do buforowania danych pomiarowych zastosowano usługę Azure Service Bus w trybie kolejki. Wykorzystanie kolejki umożliwia rozdzielenie etapu przyjmowania danych od ich dalszego prze-

twarzania oraz zapisu w bazie danych.

Takie podejście zwiększa odporność systemu na chwilowe przeciążenia oraz pozwala na skalowanie poszczególnych komponentów niezależnie od siebie. Dane w kolejce są przechowywane w postaci komunikatów JSON, które mogą być pobierane i przetwarzane partiami. Zastosowanie asynchronicznego mechanizmu kolejkowania jest charakterystyczne dla systemów przetwarzających dane pochodzące z wielu rozproszonych źródeł, ponieważ pozwala na wygładzenie obciążenia oraz zwiększenie odporności systemu na skoki natężenia ruchu [131].

3.6.5 Przetwarzanie danych i trwały zapis w bazie danych

Ostatnim etapem przepływu informacji jest funkcja wykonawcza (*fa-worker*), działająca w modelu sterowanym zdarzeniami, która aktywuje się w odpowiedzi na nowe komunikaty w kolejce. Komponent ten wykorzystuje mechanizm przetwarzania wsadowego (ang. *batch processing*), definiowany jako sposób realizacji zadań (*batch jobs*), które mogą być wykonywane bez interakcji z użytkownikiem końcowym przy minimalnym udziale człowieka [132]. Rozwiązanie to pozwala na efektywną normalizację zgromadzonych w paczkach danych oraz ich mapowanie na strukturę relacyjną, co optymalizuje proces zapisu w bazie danych poprzez jednoczesną obsługę dużych zbiorów informacji.

Po przetworzeniu dane są zapisywane w relacyjnej bazie danych PostgreSQL, gdzie stanowią trwały zapis danych pomiarowych. W zależności od typu danych trafiają one do odpowiednich tabel, takich jak dane telemetryczne lub wyniki testów jakości połączenia.

3.6.6 Podsumowanie

Takie rozdzielenie ról pomiędzy komponenty odpowiedzialne za przyjmowanie danych a warstwę realizującą zapis trwały umożliwia niezależne skalowanie zasobów obliczeniowych oraz bazy danych. Zastosowanie kolejki komunikatów zapewnia odporność na błędy i pozwala na skuteczne oddzielenie procesów wejściowych od składowania informacji. Stanowi to realizację kluczowych zasad projektowania skalowalnych systemów rozproszonych, w których asynchroniczny przepływ danych pozwala na optymalne zarządzanie obciążeniem w środowisku chmurowym [133].

Rozdział 4

Implementacja

W rozdziale przedstawiono sposób implementacji systemu SignalMap – od konfiguracji środowiska uruchomieniowego w chmurze Microsoft Azure, przez wdrożenie klastra Kubernetes i mechanizmów GitOps, aż po implementację warstw aplikacyjnych (backend, frontend) oraz elementów wspierających, takich jak monitoring i zarządzanie sekretami. Opis skupia się na praktycznych aspektach realizacji projektu: organizacji repozytoriów, strukturze plików konfiguracyjnych, sposobie budowania i publikowania obrazów kontenerów, a także na sposobie wdrażania zmian do klastra z użyciem Flux i Helm. Ważnym elementem jest również szczegółowe przedstawienie implementacji komponentów przetwarzania danych, obejmujące pełną ścieżkę telemetrii: od funkcjonalności aplikacji mobilnej SignalMap, poprzez asynchroniczną warstwę odbioru, aż po trwały zapis informacji w relacyjnej bazie danych.

Rozdział został zorganizowany tak, aby prowadzić czytelnika od elementów infrastrukturalnych do komponentów aplikacyjnych. Najpierw omówiono strukturę konfiguracji klastra oraz podział na katalogi *apps*, *flux-system* i *infra*, następnie przedstawiono wdrożenia aplikacji oraz mechanizmy bezpieczeństwa (m.in. integrację z Azure Key Vault). W dalszej części opisano implementację backendu i frontendu, w tym sposób konfiguracji, komunikacji z bazą danych i integracji z Kubernetes. Równolegle szczegółowo przeanalizowano proces gromadzenia danych przez aplikację mobilną oraz architekturę samej bazy danych wraz z jej implementacją, co pozwala na pełne zrozumienie cyklu życia danych w systemie. Rozdział zamyka opis procesu CI/CD, działania narzędzia Terraform, zestawienie napotkanych problemów wraz z ich rozwiązaniami oraz prezentacja wyników przeprowadzonych testów implementacji.

4.1 Konfiguracja klastra Kubernetes

4.1.1 Podział plików konfiguracyjnych

Struktura podziału folderów z zawartością poszczególnych konfiguracji został przedstawiony poniżej:

```
├──apps
│   │   kustomization.yaml
│   │
│   ├──backend
│   │   │   helmrelease.yaml
│   │   │   namespace.yaml
│   │   │
```

```
│   │   └──charts
│   │       │   Chart.yaml
│   │       │   values.yaml
│   │       │
│   │       └──templates
│   │               deployment.yaml
│   │               hpa.yaml
│   │               service.yaml
│   │               _helpers.tpl
│   │
│   └──frontend
│       │   helmrelease.yaml
│       │   namespace.yaml
│       └──charts
│           │   Chart.yaml
│           │   values.yaml
│           │
│           └──templates
│                   deployment.yaml
│                   hpa.yaml
│                   ingress.yaml
│                   networkpolicy.yaml
│                   service.yaml
│                   _helpers.tpl
│
├──flux-system
│   │   flux-namespace.yaml
│   │   gitrepo.yaml
│   │
│   └──kustomizations
│           kustomize-apps.yaml
│           kustomize-infra.yaml
└──infra
    │   kustomization.yaml
    │
    ├──cert-manager
    │       clusterissuer.yaml
    │       helmrelease.yaml
    │       helmrepo.yaml
    │       namespace.yaml
    │
    ├──csi
    │       csi-config.yaml
    │       csi-postsql-config.yaml
    │
    ├──ingress-controller-nginx
    │       helmrelease.yaml
    │       helmrepo.yaml
    │       ingress-namespace.yaml
    │
    └──monitoring
```

```
        │   grafana-ingress.yaml
        │   helm-release.yaml
        │   helm-repo.yaml
        │   monitoring-namespace.yaml
        │   storage-class-grafana.yaml
        │   storage-class-prometheus.yaml
        │   nginx-monitoring.yaml
        │
        └───postgres-exporter
                configmap.yaml
                deployment.yaml
                serivce.yaml
                service-monitor.yaml
```

4.1.2 Opis działania Flux i podstawowe pliki

Struktura katalogów dąży do podejścia GitOps, w którym stan klastra jest w pełni deklaratywny i zarządzany przez zmiany dokonywane w repozytorium. Narzędzie Flux odpowiada za cykliczne monitorowanie zmian w plikach konfiguracyjnych oraz informowanie o synchronizacji tych zmian z klastrem AKS. W projekcie zdecydowano się na podział konfiguracji na trzy główne katalogi: `apps`, `flux-system` oraz *infra*. Ma to na celu rozdzielenie plików odpowiadających za wdrożenia aplikacji, konfigurację narzędzi zarządzających stanem klastra oraz elementy infrastrukturalne. Taki podział zwiększa czytelność repozytorium, ułatwia nawigację po jego strukturze oraz upraszcza proces wprowadzania nowych zmian i modyfikacji.

4.1.3 Aplikacja - konfiguracja

Katalog `apps` zawiera pliki definiujące wdrożenie komponentów aplikacji backendowej oraz frontendnowej. Aplikacje posiadają podobne struktury plików. Podstawowe dwa to: `helmrelease.yaml` oraz `namespace.yaml`. Pierwszy służy do zarządzania najważniejszymi wartościami konfiguracji chartów, drugi natomiast do stworzenia namespace dla konkretnej aplikacji. W podkatalogu *charts* znajdują się lokalne definicje i poszczególne szablony konfiguracyjne. Plik *Chart.yaml* pomaga określić wersje chartu, plik *values.yaml* służy natomiast do nadawania parametrów dla konkretnego charta. Kluczowe jest to, że zmiana wartości w pliku *helmrelease.yaml* nadpisuje wartości jeśli istnieją one także w `vaules.yaml`. Taki podział plików pozwala na nadanie podstawowych parametrów dla aplikacji w pliku *helmrelease.yaml*, a konfiguracja dla konkretnych wersji jest nadawana w `vaules.yaml`. Kolejnym istotnym faktem jest to, że przy ustawieniu *reconcileStrategy: ChartVersion* w *helmrelease.yaml* zmiany będą sychronizowane do klastra tylko przy zmianie w *helmrelease.yaml* lub *Chart.yaml*. W pracy użyto ustawienia *reconcileStrategy: Revision* dlatego zmiany są wdrażane przy każdym commit w repozytorium. W folderze *templates* znajdują się konfiguracje specyficzne dla danej aplikacji.

Aplikacja backendowa - konfiguracja

Dla aplikacji backendowej są to pliki:

- `deployment.yaml` - tworzący obiekt Deployment, w Kubernetesie. Odpowiada za wdrożenie i utrzymanie właściwej ilości kopii aplikacji (ang. Replica Set). W przypadku tej aplikacji uruchamiane są dwa Pody instancji oparte na obrazie pobranym z ACR z repozytorium

backend. Kontenery umieszczone w Podach nasłuchują na port 8080. Konfiguracja jest tak stworzona, aby konkretne parametry pobierać z plików *helmrelease.yaml* lub *vaules.yaml*

- `hpa.yaml` - służy do zapewniania HPA - Horizontal Pod Autoscaler. Mechanizm ten służy do automatycznego skalowania aplikacji w "poziomie". Oznacza to dodawania kolejnych kopii (Podów) aplikacji w momencie wzrostu zużycia pamięci RAM lub CPU przez daną aplikacje. W tym przypadku skalowanie rozpocznie się w momencie przekroczenie 70% zasobów. Koniecznym warunkiem do działania mechanizmu HPA jest posiadanie w klastrze AKS *metrics server*. Jest on kluczowy do tego, aby klaster informował HPA o tym ile CPU oraz pamięci RAM zużywają Pody.
- `service.yaml` - pełni rolę stałego i niezmiennego punktu dostępowego do aplikacji, która działa wewnątrz klastra jako grupa Podów. Głównym zadaniem jest nadanie backendowi wewnętrznej nazwy DNS oraz adresu IP (ClusterIP), dzięki czemu inne mikroserwisy mogą się z nim łączyć zawsze w ten sam sposób. Za pomocą sekcji *selector* usługa automatycznie odnajduje wszystkie kontenery z etykietą backend, bez względu na to, ile instancji aplikacji jest. Service działa jak wewnętrzny Load Balancer, rozdzielając ruch przychodzący na port 80 i przekazując go bezpośrednio do aplikacji nasłuchującej na porcie 8080. Takie podejście gwarantuje ciągłość działania – nawet jeśli konkretny Pod ulegnie uszkodzeniu.
- `_helpers.tpl` - służy do definiowania stałych nazw, które później są używane w różnych szablonach klastra. Plik ten pomaga zachować spójność nazw oraz etykiet (labels) przez co poszczególne obiekty klastra potrafią się ze sobą komunikować.

Kluczowe dla bezpieczeństwa działania aplikacji backend jest sposób uzyskiwania danych do połączenia z baza danych. Standardowo Kubernetes pozwala na przechowywanie danych w bazie etcd w formacie Base64 [134]. Z tego powodu zaimplementowano mechanizm, który pobiera dane logowania w momencie samego powstawania Podów backendu co powoduje, że dane te są przechowywane tylko w pamięci RAM aplikacji, a nie w bazie etcd. W pliku *helmrelease.yaml* skonfigurowano obiekt extraVolumes, który korzysta z Podów *CSI-Driver-Storgae*. Ten wolumen odwołuje się do secretProviderClass: csi-driver, który zawiera instrukcję dla klastra, jakie konkretnie dane ma pobrać z Azure Key Vault. Następnie w pliku *deployment.yaml* wolumen jest montowany w kontenerze w ścieżce /appuser/kv. Podczas uruchamiania podów CSI Driver komunikuje się z Azure Key Vault, pobiera konkretne wartości do pamięci RAM (tmpfs) do folderu /appuser/kv. Wstępnie w Podzie uruchamia się skrypt *endpoint.sh*, który odczytuje dane z folderu i przypisuje je jako systemowe zmienne środowiskowe. Dopiero po tym jak pod pobierze te dane uruchamiana jest właściwa aplikacja backendowa.

Aplikacja frontendowa - konfiguracja

Przechodząc do struktury aplikacji frontendowej posiada ona takie pliki konfiguracyjne:

- `deployment.yaml` - odpowiada za utrzymanie określonej liczby replik (instancji) aplikacji, korzystając z obrazu przechowywanego w zewnętrznym rejestrze obrazów. Kluczowym elementem jest sekcja env, która automatycznie konfiguruje połączenie z backendem poprzez wstrzyknięcie zmiennych środowiskowych, takich jak BACKEND_BASE_URL, wykorzystując przy tym wewnętrzny system DNS klastra (backend.namespace.svc). Deployment określa również port, na którym nasłuchuje kontener (domyślnie 8080), oraz definiuje limity zasobów CPU i pamięci RAM.

- `hpa.yaml` - służy do zapewniania HPA - Horizontal Pod Autoscaler. Mechanizm ten służy do automatycznego skalowania aplikacji w "poziomie". Oznacza to dodawania kolejnych kopii (podów) aplikacji w momencie wzrostu zużycia pamięci RAM lub CPU przez daną aplikacje. W tym przypadku skalowanie rozpocznie się w momencie przekroczenie 70% zasobów. Koniecznym warunkiem do działania mechanizmu HPA jest posiadanie w klastrze AKS *metrics server*. Jest on konieczny do tego, aby klaster informował HPA o tym ile CPU oraz pamięci RAM zużywają pody.
- `networkpolicy.yaml` - pełni rolę wewnętrznej zapory ogniowej, która precyzyjnie kontroluje ruch wychodzący (Egress) z aplikacji frontendowej. Mechanizm ten ogranicza możliwości komunikacji frontendu, zezwalając mu jedynie na połączenie z usługą DNS w celu rozpoznawania nazw oraz na przesyłanie żądań bezpośrednio do backendu na porcie 80.
- `service.yaml` - pełni rolę stałego i niezmiennego punktu dostępowego aplikacji, która działa wewnątrz klastra jako grupa podow. Głównym zadaniem jest nadanie backendowi wewnętrznej nazwy DNS oraz adresu IP (ClusterIP), dzięki czemu inne mikroserwisy mogą się z nim łączyć zawsze w ten sam sposób. Za pomocą sekcji *selector* usługa automatycznie odnajduje wszystkie kontenery z etykietą backend, bez względu na to, ile instancji aplikacji jest. Service działa jak wewnętrzny Load Balancer, rozdzielając ruch przychodzący na port 80 i przekazując go bezpośrednio do aplikacji nasłuchującej na porcie 8080. Takie podejście gwarantuje ciągłość działania – nawet jeśli konkretny pod ulegnie uszkodzeniu.
- `ingress.yaml` - pełni funkcje bramy wejściowej, która pozwala na bezpiecznie udostępnianie aplikacji frontendowej w sieci publicznej pod domeną *signalmap.com.pl*. Zadaniem tej konfiguracji jest zarządzanie ruchem przychodzącym i kierowanie go do odpowiedniego obiektu Service aplikacji frontendowej. Obiekt Ingress ściśle współpracuje z Podami Nginx Controller, które przekierowują ruch HTTP na HTTPS. Kluczowe do tego jest adnotacja tls, która działa wraz z podami Cert-Manager. Zarządzają one certyfikatami SSL z Let's Encrypt. Ingress sprawdza hosta w nagłówku zapytania, a następnie na podstawie zdefiniowanych ścieżek (paths) przekazuje ruch do właściwego portu usługi frontendowej zapewniając przy tym szyfrowanie.
- *_helpers.tpl* - służy do definiowania stałych nazw, które później są używane w różnych szablonach klastra. Plik ten pomaga zachować spójność nazw oraz etykiet (labels) przez co poszczególne obiekty klastra potrafią się ze sobą komunikować.

4.1.4 Flux - konfiguracja

W folderze `flux-system` znajdują się konfiguracje narzędzia Flux odpowiedzialnego za obsługę synchronziację zawartości klastra z definicjami z repozytorium. Do poprawnego połączenia z repozytorium w Azure DevOps wykorzystano klucz wygenerowany na tej platformie, a następnie zapisano go jako Secret w AKS pod nazwą *azdo*. Narzędzie skonfigurowano tak, aby pobierał najnowszą wersję kodu (najnowszy commit) z gałęzi *main*. Sprawdzanie zmian następuje w wyznaczonym interwale czasowym tj. 1 minuta. Opis poszczególnych plików:

- `gitrepo.yaml` - tworzy obiekt *GitRepository* definiujący dla narzędzia Flux zewnętrzne źródło prawdy, wskazujący na prywatne repozytorium na platformie Azure DevOps. Co minutę automatycznie sprawdza on gałąź main, pobierając najnowszą wersję kodu i konfiguracji klastra. Dzięki wykorzystaniu sekretu *azdo*, proces ten odbywa się w sposób bezpieczny i uwierzytelniony.

- `flux-namespace.yaml` - tworzy *namespace* dla podów związanych z narzędziem Flux.
- `podkatalog kustomizations` - znajdują się w nim pliki związane z poszczególnymi obiektami Kustomizations:
  - `kustomize-apps.yaml` - zawiera obiekt, który wdraża zmiany dla aplikacji frontendowej oraz backendowej. Posiada konfiguracje, której zadaniem jest rozpoczęcie implementacja wdrożeń na ścieżkę *./k8s/k8s_config/apps*. Posiada on także kluczowy mechanizm *dependOn*, który zapewnia, że Flux przed wrażaniem aplikacji upewnia się o poprawnej inicjalizacji infrastruktury z *kustomize-infra*. Dopiero po otrzymaniu informacji o poprawnej implementacji rozpoczyna zmiany w aplikacjach.
  - `kustomize-infra.yaml` - zawiera obiekt, który wdraża Pody związane z infrastrukturą klastra. Posiada konfiguracje, której zadaniem jest rozpoczęcie implementacja wdrożeń na ścieżkę *./k8s/k8s_config/infra*. Są tam zawarte pliki konfiguracyjne podów: Nginx Controller, Grafana, Prometheus, CSI-Driver czy też Cert-Manager.

Dodatkowo aby Flux mógł poprawnie działać w folderach *apps* oraz *infra* każdy z nich musi zawierać pliki *kustomization.yaml*, w którym definiuje się ścieżki określające pliki konfiguracji poszczególnych narzędzi.

4.1.5 Infrastruktura - konfiguracja

W folderze *infra* zdefiniowane są konfiguracje wszystkim najważniejszych narzędzi działających we warstwie infrastrukturalnej. Znajdują się tam definicje obiektów obsługujące ruch sieciowy oraz wystawianie certyfikatów TLS/SSL. Katalog obejmuje również konfiguracje narzędzia obsługującego integracje z zewnętrznym magazynem sekretów Azure Key Vualt. Dodatkowo folder zawiera implementacje stosu do monitorowania klastra oraz narzędzie do eksportowania metryk z bazy danych PostreSQL. Opis definicji poszczególnych narzędzi:

- podfolder `cert-manager` - znajduje się tu definicja Cert-Manager:
  - `helmrepo.yaml` - definiuje oficjalne źródło, z którego FLux pobiera pliki konfiguracyjne. W tym przypadku jest to *jetstack*.
  - `helmrelease.yaml` - informacja dla Fluxa, aby wdrożył wersje 1.19.1 Cert-Manager. Dodatkowo ustawiony jest parametr *installCRDs: true*, który dodaje do klastra takie obiekty jak Certificate lub Issuer. CRD (Custom Resource Definition) [135] jest kluczowe do zrozumienia działania Cert-Manager oraz jego implementacje w AKS. Standardowo Kubernetes ma dostęp do kilkunastu obiektów takich jak: Pod, Service, Deployment. CRD pozwala na wprowadzenie do klastra dodatkowych typów zasobów. Dzięki ustawieniu parametru *installCRDs: true* Kubernetes będzie rozumiał nowe typu wdrożeń w plikach YAML (definiowane przez na przykład *kind: Certificate*). CRD związany z Cert-Manager wprowadza takie nowe obiekty jak:
    * `Issuer lub CluserIssuer` [136] - określa źródło certyfikatu w przypadku tej pracy jest to Let's Encrypt. Określa też w jaki sposób Cert-Manager ma się komunikować z serwer podczas weryfikacji. W pracy użyto HTTP-01. Issuer działa na poziomie jednego Namespace natomiast ClusterIssuer na poziomie całego klastra.
    * `Certificate` [137]- jest to deklaracja chęci wygenerowania certyfikatu pod konkretną domenę. W przypadku tej pracy jest to domena *signalmap.com.pl*. Subdomeny mogą wykorzystywać ten sam Certificate do weryfikacji certyfikatów. Poprzez

obiekt Certificate narzędzie Cert-Manager wie o zbliżającym się końcu ważności certyfikatu i inicjuje automatycznie proces odnawiania go.

  - *CertificateRequest* [138] - obiekt tworzony tymczasowo podczas inicjalizacji weryfikacji z podmiotem weryfikacyjnym. Zawiera CSR (Certificate Signing Request) przesyłany do Let's Encrypt.

- `namespace.yaml` - tworzy Namespace dla narzędzia Cert-Manager.
- `clusterissue.yaml` - tworzy obiekt ClusterIssue, którego zadaniem jest komunikacja z serwerem ACME podmiotu Let's Encrypt. Zdefiniowany jest tam sposób weryfikacji tj. HTTP-01. Sposób ten oznacza, że Cert-Manager wystawi specjalny plik określonej domeny. Następnie klaster informuje urząd certyfikujący o tym, że faktycznie kontroluje dany serwer. Końcowym wynikiem komunikacji między serwerami Let's Encrypt a AKS jest otrzymanie ceryfiaktów TLS/SSL

- podfolder `csi` - znajduje się tu definicja obiektów CSI Driver:
  - `csi-config.yaml` - definiuje obiekt SecretProviderClass, który pozwala na zdefiniowanie konfiguracji połączenia do Azure Key Vault w celu pobrania danych logowania do bazy danych. Utworzony jest w Namespace backend stąd aplikacja backendowa swobodnie może korzystać z obiektu. W konfiguracji ustawione są parametry oznaczające źródło danych (inzynierka) w konkretnym Tenent ID w Azure, sposób uwierzytelnienia na User Assigned Managed Identity oraz wartości, które zostaną pobrane: PGHOST (adres), PGPORT (port), PGDATABASE (nazwa bazy), PGUSER (użytkownik), PGPASSWORD (hasło) oraz PGSSLMODE (tryb SSL).
  - `csi-postsql-config.yaml` - definiuje obiekt SecretProviderClass, który pozwala na zdefiniowanie konfiguracji połączenia do Azure Key Vault w celu pobrania danych logowania do bazy danych poprzez dedykowanego użytkownika, który ma uprawnienia do odczytu metryk bazy danych. Zdefiniowany jest w Namespace monitoring stąd inne Pody zdefiniowane w samym Namespace swobodnie mogą korzystać z obiektu. W konfiguracji ustawione są parametry oznaczające źródło danych (inzynierka) w konkretnym Tenent ID w Azure, sposób uwierzytelnienia na User Assigned Managed Identity oraz wartości, które zostaną pobrane: PGHOST (adres), PGPORT (port), PGDATABASE (nazwa bazy), PGUSERMERICS (użytkownik z uprawnieniami do odczytu metryk), PGMETRICSPASSWORD (hasło) oraz PGSSLMODE (tryb SSL).

- podfolder `ingress-contoller-nginx` - znajduje się tu definicja obiektów Nginx-Controller:
  - `helmrelease.yaml` - definiuje źródło, z którego Flux pobiera kontroler Nginx. W pracy używana jest wersja 4.13.3. Określona jest także ilość Podów narzędzia na dwie kopie. Kluczowe jest także dodanie parametru *metrics: enabled*, który powoduje, że Pody kontrolera Nginx generują i udostępniają odpowiednie metryki. Konfiguracja posiada integracje z Azure Load Balancer z adnotacją wskazująca na wykorzystanie publicznego adresu IP o nazwie: *ingress-frontned-ip* w grupy zasobów *Inzynierka*. W ten sposób wykorzystano publiczny adres dostarczany przez Azure do skonfigurowania dostępu do aplikacji webowej z poziomu przeglądarek. Dodatkowo, aby adres był rozwiązywany jako domena *signalmap.com.pl* skonfigurowano odpowiedni rekord A z poziomu panelu użytkownika dostarczanego przez dostawcę domenowego *lh.pl*.
  - *helmrepo.yaml* - wskazuje repozytorium, z którego pobierany jest chart do wdrożenia kontrolera Nginx.

  - `ingress-namespace.yaml` - definiuje Namespace o nazwie *ingress-nginx*.

- podfolder `monitoring` - znajdują się tu definicje całego systemu do monitoringu infrastruktury:

- `namespace.yaml` - tworzy osobny Namespace o nazwie monitoring, pozwala na izolacje narzędzi monitoringu od reszty klastra.

- `helmrepo.yaml` - definiuje źródło charta dla stack kube-prometheus-stack z repozytorium prometheus-community. Wykorzystywana wersja to 65.5.0.

- `helmrelease.yaml` - wdraża stack do klastra, narzędzia, które instaluje to: Prometehus wraz z konfiguracją scraperów do zbierania danych, AlertManager, Grafana oraz obiekty SeviceMonitor. Obiekty ServiceMonitor są kluczowe do poprawnego zbierania danych z aplikacji w klastrze. Służą one do wskazywania z jakich Podów mają być pobierane metryki. Mechanizm ten jest konfigurowany za pomocą parametrów *labelSelector*. W ten sposób Prometheus wie, że pod jakim portem oraz ścieżką dane narzędzia udostępniają dane i w ten sposób może je pobierać i zapisywać. Po dodaniu nowych SeviceMonitor klaster przeładowuje Prometheusa dodają w jego konfiguracji *prometheus.yml* informacje o nowym ServiceMonitor (o ile taki zostanie dodany) i rozpoczyna zbieranie nowych metryk. Kolejnym kluczowym elementem konfiguracji monitoringu jest dodanie StorageClass dla Grafany oraz Prometheusa. Wynika to z problemu, który napotkano. Mianowicie przy wykorzystaniu implementacji Grafany z gotowego chartu narzędzie posiada gotowe dashboardy wdrożone przez Helm . W momencie dodania nowego dashboardu nie ma możliwości zapisania go na stałe. Przez to przy wyłączeniu, a następnie włączeniu Podów lub całego klastra, nowe dashboardy znikają. Taka sama sytuacja jest z danymi zapisanymi w Prometheusu. Po wyłączeniu i włączeniu klastra historyczne dane znikają z Prometheusa. Rozwiązaniem jest dodanie StorageClass wraz z PVC (Persistance Volume Claim) osobnych dla dashboardów i danych dla Grafany oraz osobnego dla metryk dla Prometehusa. PVC to żądanie przydzielenia konkretnego PV (Persistence Volume) do aplikacji. W pliku znajduje się kod definiujący jakie PVC żąda Grafana (PV o wielkości 5G) oraz Prometheus (PV o wielkości 20G). W ten sposób dane zostają trwale zapisane, a Grafana pozwala na tworzenie własnych, nowych dashboardów. W pliku są także zdefiniowane podstawowe ustawienia dla działania Grafany.

- `storage-class-grafana.yaml` - definiuje jaki StorageClass ma zostać stworzony dla Grafany. Wykorzystywany jest do tego Azure Disk CSI. Pozwala to na platformie Azure stworzyć dyski o określonym SKU (w tym przypadku LRS), określić polityki usuwania (w przypadku tej pracy dysk jest usuwany wraz z usunięciem PVC oraz tworzy się dopiero gdy Flux wykryje, że dane PVC jest potrzebne).

- `storage-class-prometheus` - definiuje jaki StorageClass ma zostać stworzony dla Prometheus. Wykorzystywany jest do tego Azure Disk CSI. Pozwala to na platformie Azure stworzyć dyski o określonym SKU (w tym przypadku LRS), określić polityki usuwania (w przypadku tej pracy dysk jest usuwany wraz z usunięciem PVC oraz tworzy się dopiero gdy Flux wykryje, że dane PVC jest potrzebne).

- `nginx-monitoring.yaml` - to definicja obiektu ServiceMonitor, dzięki ustawieniu etykiety *matchLabels* Prometheus może skorzystać z *endpoints* usługi Nginx Controller i pobrać metryki związane z tym narzędziem. Stworzenie takiego SeriveMonitor ma na celu zrealizowanie dashboardu w Grafanie, który będzie przedstwiał statystki odnośnie wejść na domenę.

- podfolder `postgres-exporter` - definuje konfiguracje dla narzędzia Postgres-Exporter, który zbiera dodatkowe, wewnętrzne metryki z usługi Azure Database for PostgreSQL flexible server:
  - `deployment.yaml` – plik definiujący zasób typu *Deployment*, odpowiedzialny za wdrożenie komponentu *postgresql-exporter* w przestrzeni nazw *monitoring*. w konfiguracji określono uruchomienie pojedynczej repliki eksportera, co jest wystarczające ze względu na jego bezstanowy charakter. Wykorzystywany jest obraz kontenera *postgres-exporter* w wersji *v0.15.0*, pochodzący z repozytorium *prometheus-community*. Zdefiniowano ustawienia bezpieczeństwa, które wymuszają uruchomienie kontenera z uprawnieniami użytkownika *nobody*, co wyklucza możliwość działania procesu z uprawnieniami administratora (*root*). Eksporter wystawia metryki na porcie *9187*, który został nazwany logicznie jako *metrics*, co ułatwia jego dalsze mapowanie w zasobach typu *Service* oraz *ServiceMonitor*. W ramach konfiguracji wykorzystano mechanizm *Secrets Store CSI* z użyciem *secretProviderClass: postgresql-metrics-secrets*, umożliwiający dynamiczne pobieranie danych uwierzytelniających z zewnętrznego magazynu sekretów. Dodatkowo zastosowano zasób ConfigMap, który zawiera skrypt startowy odpowiedzialny za odczyt danych logowania z zamontowanych wolumenów oraz zbudowanie zmiennej środowiskowej wymaganej do uruchomienia eksportera. Zastosowany mechanizm pobierania danych dostępowych jest zbliżony do rozwiązania użytego w aplikacji backendowej, jednak w tym przypadku inicjalizacja zmiennych środowiskowych nie odbywa się poprzez domyślny skrypt startowy kontenera, lecz poprzez skrypt zdefiniowany w obiekcie *ConfigMap* i zamontowany w dedykowanym wolumenie. Zasób *ConfigMap* umożliwia przechowywanie niewrażliwych danych konfiguracyjnych oraz logiki początkowej aplikacji, które mogą być wykorzystywane przez aplikacje i mechanizmy działające w klastrze Kubernetes.
  - `configmap.yaml` - definicja obiektu ConfigMap. W pliku został umieszczony kod powłoki bash, który służy do pobrania z wolumenu */app/kv* i przypisaniu poszczególnych danych do zmiennych środowiskowych systemu. Następnie tworzony jest DATA_SOURCE_NAME do połączenia z bazą. Na końcu znajduje się komenda uruchamiająca postgres_exporter.
  - `service.yaml` - plik definiujący zasób typu *Service*, który udostępnia endpoint sieciowy dla komponentu *postgresql-exporter* klastra. Usługa działa w przestrzeni nazw *monitoring* i wybiera pody na podstawie etykiety *app=postgresql-exporter*. Zdefiniowany port usługi *9187* o nazwie logicznej *http* przekierowuje ruch na port kontenera oznaczony nazwą *metrics*, na którym eksporter wystawia metryki w formacie zgodnym z Prometheusem.
  - *service-monitor.yaml* – plik definiujący zasób typu `ServiceMonitor`, wykorzystywany do automatycznej konfiguracji procesu zbierania metryk. Zasób ten identyfikuje usługi Kubernetes na podstawie etykiety *app=postgresql-exporter* oraz inicjuje metryk z portu *http* usługi. Interwał zbierania danych został ustawiony na 30 sekund. Dodatkowa etykieta *release=monitoring* umożliwia powiązanie zasobu z instancją Prometheus wdrażaną w ramach stosu *kube-prometheus-stack*.

4.2 Implementacja warstwy gromadzenia i przetwarzania danych

W niniejszym rozdziale szczegółowo opisano implementację pełnego cyklu przetwarzania danych pomiarowych — od momentu ich rejestracji na urządzeniu mobilnym, aż po trwały zapis

w relacyjnej bazie danych. Rozwiązania te stanowią techniczną realizację założeń projektowych przedstawionych w rozdziale 3.6 (*Projekt przepływu danych*). Podstawę strukturalną opisywanego systemu stanowi architektura przesyłu danych zaprezentowana na rysunku 3.6.

W dalszej części rozdziału, zgodnie z sekwencyjnym charakterem potoku przetwarzania danych, opisano sposób wdrożenia poszczególnych komponentów oraz przedstawiono konkretne mechanizmy i technologie zastosowane w implementacji.

4.2.1 Implementacja aplikacji mobilnej SignalMap

Aplikacja mobilna stanowi istotny element w całym procesie gromadzenia danych telemetrycznych, gdyż to właśnie na dostarczanych przez nią surowych odczytach opierają się wszystkie dalsze etapy przetwarzania oraz końcowa wizualizacja pomiarów na mapie. Projektując ten komponent, przyjęto założenie, że aplikacja ma stanowić atrakcyjny punkt wejścia do systemu, który poprzez profesjonalne wrażenie i intuicyjność skłoni użytkownika do aktywnego zaangażowania w projekt crowdsourcingowy. Aby zmotywować uczestników projektu do aktywnego zbierania pomiarów telemetrycznych, wdrożono szereg mechanizmów zwiększających komfort użytkowania (ang. *User Experience*) co zostanie szczegółowo przedstawione w podrozdziale 4.2.1 zatytułowanym *Projekt interfejsu użytkownika*.

Dużym wyzwaniem podczas prac nad aplikacją było znalezienie „złotego środka" między wysoką precyzją pomiarów a oszczędnością energii. Wykorzystanie dokładnych modułów lokalizacyjnych jest niezbędne do poprawnego naniesienia danych na mapę, jednak ich intensywna eksploatacja jest głównym czynnikiem wpływającym na nadmierne zużycie baterii, co w konsekwencji zniechęca do regularnego korzystania z oprogramowania [139]. Aby wyeliminować ten problem, zdecydowano się na architekturę adaptacyjną — system nie pracuje w sposób sztywny, lecz inteligentnie dostosowuje intensywność odpytywania danych pomiarowych do aktualnej aktywności użytkownika. Pozwala to na zachowanie wysokiej jakości danych tam, gdzie jest to konieczne, przy jednoczesnym zapewnieniu wysokiego komfortu użytkowania. Taka budowa pozwala w przyszłości łatwo ulepszać algorytmy, korzystając z danych zebranych od użytkowników. Szerzej ten aspekt implementacji opisano w podrozdziale pt. *Adaptacyjna polityka próbkowania i praca w tle*.

W poniższych sekcjach szczegółowo omówiono techniczne aspekty implementacji, koncentrując się na rozwiązaniach, które pozwoliły na realizację założonych celów projektowych:

- **Architektura i wzorce projektowe** – szczegółowo omówiono implementację struktury opartej na wzorcu *MVVM* (Model-View-ViewModel), co pozwoliło na odseparowanie procesów przetwarzania danych od interfejsu użytkownika, gwarantując łatwą skalowalność aplikacji [140].
- **Gromadzenie danych telemetrycznych** – wyjaśniono, w jaki sposób aplikacja komunikuje się bezpośrednio z podzespołami telefonu, aby pozyskać precyzyjne parametry sieci, które zazwyczaj są ukryte przed zwykłym użytkownikiem.
- **Adaptacyjne próbkowanie i praca w tle** – przedstawiono mechanizm zarządzania modułami lokalizacyjnymi i radiowymi, który optymalizuje ich tryby pracy w zależności od dynamiki przemieszczania się użytkownika.
- **Moduł testów wydajnościowych** – omówiono techniczną stronę realizacji aktywnego pomiaru przepustowości łącza oraz asynchroniczne przetwarzanie wyników.
- **Łączność z chmurą** – opisano protokoły bezpiecznego przesyłu danych do bramy API oraz metody gwarantujące integralność pomiarów.

- **Interfejs użytkownika** – przedstawiono proces projektowania warstwy wizualnej aplikacji, łączących nowoczesną estetykę z prostą i intuicyjną obsługą.

Architektura systemu

Architektura aplikacji została oparta na wzorcu **MVVM** (*Model-View-ViewModel*). Istotą tego podejścia jest wprowadzenie ścisłego rozdziału odpowiedzialności pomiędzy danymi a sposobem ich prezentacji, co pozwala na uniknięcie chaosu w kodzie źródłowym i ułatwia późniejszy rozwój aplikacji [140]. Strukturę projektu podzielono na pakiety, co odzwierciedla warstwową budowę systemu:

- **Warstwa Modelu (`com.example.datasender.model`)** – zawiera klasy danych definiujące struktury przesyłane wewnątrz systemu. Kluczowym elementem jest klasa `NetworkSnapshot`, która pełni rolę niezmiennego nośnika stanu sieci i lokalizacji. W architekturze systemu mobilnego postawiono na wykorzystanie obiektów niezmiennych (ang. *immutable objects*), których stan nie może zostać zmodyfikowany po ich utworzeniu [141]. Realizację tego założenia umożliwiły klasy danych (ang.*data classes*), czyli specjalne konstrukcje w języku Kotlin służące do przechowywania informacji [142]. Poprzez przypisanie słowa kluczowego `val` do wszystkich właściwości tych klas, zagwarantowano, że parametry pomiarowe pozostaną nienaruszone podczas ich przesyłania między warstwami aplikacji. W praktyce oznacza to, że każda zmiana parametrów sieciowych skutkuje utworzeniem nowej, spójnej instancji obiektu (zazwyczaj przy użyciu wbudowanej metody `copy`), co eliminuje ryzyko przypadkowych błędów i niespójności danych w trakcie działania programu [140].

  Zastosowanie zasady niezmienności niesie ze sobą kluczowe korzyści dla stabilności systemu, szczególnie w środowisku wielowątkowym, jakim jest aplikacja mobilna. Obiekty niezmienne są znacznie łatwiejsze w analizie, ponieważ raz utworzony obiekt pozostaje w swoim pierwotnym stanie przez cały cykl życia [141].

  Wyniki testów wydajnościowych są reprezentowane przez klasę `SpeedTestResult`, która również bazuje na opisanym mechanizmie niezmienności.

- **Warstwa Widoku i Nawigacji (`com.example.datasender.ui`)** – odpowiada za interakcję z użytkownikiem oraz prezentację informacji w sposób czytelny, zapewniając przy tym wysoką płynność działania interfejsu. Warstwa ta została zrealizowana przy użyciu zestawu narzędzi *Jetpack Compose*. W tym podejściu programista nie musi ręcznie zarządzać stanem każdego elementu na ekranie, ponieważ definiuje jedynie, jak interfejs ma wyglądać w zależności od dostarczonych danych (np. inny widok dla aktywnego pomiaru, a inny dla stanu bezczynności) [143]. Dzięki takiemu podejściu kod aplikacji jest prostszy i bardziej przejrzysty, co znacznie ogranicza ryzyko błędów. Interfejs staje się w pełni automatyczny: każda zmiana parametrów sieciowych wykryta w tle jest natychmiast i płynnie nanoszona na widok użytkownika, bez potrzeby ręcznego odświeżania ekranu.

  W ramach tej warstwy wydzielono dwa kluczowe obszary:

  - **Ekrany funkcyjne** (pakiet `screens`) – gdzie zaimplementowano dedykowane widoki, takie jak `SignalScreen`, służący do monitorowania parametrów radiowych w czasie rzeczywistym, oraz `SpeedTestScreen`, który stanowi interfejs do przeprowadzania testów wydajnościowych łącza.
  - **System nawigacji** – oparty na komponencie `AppNavHost` oraz definicjach tras w obiekcie `NavItem`. Takie podejście umożliwia płynne i dynamiczne przełączanie się między

funkcjami aplikacji, zapewniając użytkownikowi intuicyjną ścieżkę poruszania się po systemie.

- **Warstwa Logiki i Stanu (`com.example.datasender.viewmodel`)** – służy jako bezpieczny magazyn dla danych i logiki aplikacji. Gwarantuje ona, że aktualny stan pomiaru zareaguje odpowiednio na typowe zdarzenia systemowe, takie jak obracanie ekranu czy przełączanie się między oknami, dzięki czemu użytkownik widzi ciągłość wyników bez zbędnych przerw. W ramach tej warstwy zaimplementowano:
  - `MainViewModel` – zarządza cyklem życia pomiarów pasywnych. Komunikuje się on z interfejsem poprzez strumienie *StateFlow*, czyli specjalne mechanizmy pozwalające na bezpieczne i automatyczne przesyłanie aktualizacji danych [144]. Dzięki temu, gdy tylko telefon wykryje zmianę siły sygnału, informacja ta jest natychmiastowo przekazywana do widoku, który odświeża się automatycznie.
  - `SpeedTestViewModel` – odpowiada za koordynację procesu aktywnego pomiaru przepustowości łącza. Kontroluje on logikę oraz kolejność wykonywania testów (opóźnienie, pobieranie, wysyłanie) i dba o to, aby wyniki były udostępniane warstwie wizualnej w sposób spójny. Mechanizm ten gwarantuje, że użytkownik widzi finalne dane dopiero po ich całkowitym zweryfikowaniu przez podsystemy pomiarowe.
- **Warstwa Danych i Repozytoriów (`com.example.datasender.data`)** – odpowiada za implementację bezpośredniego dostępu do zasobów sprzętowych oraz interfejsów sieciowych urządzenia. Warstwa ta stanowi punkt styku między systemem operacyjnym Android a logiką aplikacyjną, zapewniając ustandaryzowany sposób pozyskiwania surowych metryk radiowych i geograficznych. W jej skład wchodzą:
  - **Repozytoria sprzętowe** (`TelephonyRepository` i `LocationRepository`) – pełnią funkcję pośredników w dostępie do modułów GPS oraz radiowych. Ich głównym zadaniem jest ukrycie złożoności interakcji z usługami systemowymi systemu Android, takimi jak *TelephonyManager* [25] czy *FusedLocationProvider* [145]. Dzięki takiej strukturze, pozostałe warstwy aplikacji otrzymują przetworzone i ustandaryzowane dane.
  - **Moduł komunikacyjny** (`AzureClient`) – odpowiada za bezpieczne przekazywanie zgromadzonych pomiarów do infrastruktury chmurowej Microsoft Azure. Realizuje on transfer danych poprzez szyfrowany protokół HTTPS, dbając o to, aby każda jednostka danych dotarła do serwera w nienaruszonym stanie, zachowując przy tym pełną integralność transmitowanych informacji.
- **Zarządzanie stanem i asynchroniczność** – stabilność działania systemu opiera się na mechanizmie *Kotlin Coroutines*, który umożliwia wykonywanie wielu zadań jednocześnie bez obciążania głównego wątku aplikacji [146]. W programowaniu aplikacji mobilnych jest to kluczowe, ponieważ wszelkie operacje długotrwałe (takie jak zapytania sieciowe czy pobieranie danych z modułów sprzętowych) wykonywane na wątku głównym mogłyby doprowadzić do zamrożenia interfejsu użytkownika [147].

  Mechanizm *Coroutines* stanowi wydajną alternatywę dla tradycyjnych wątków, umożliwiając wykonywanie zadań w tle bez przerywania głównego toku pracy aplikacji oraz z zachowaniem pełnej kontroli nad ich przebiegiem [146]. W projekcie procesy te są ściśle kontrolowane poprzez:

  - **Zakresy (*Scopes*)** – określają ramy czasowe, w których mogą działać operacje w tle. Zapewniają one strukturalną współbieżność, co oznacza, że każda operacja jest przypisana do konkretnego cyklu życia (np. ekranu aplikacji). Dzięki temu, w momencie zamknięcia danego modułu, system automatycznie anuluje powiązane z nim zadania, eliminując ryzyko wycieków pamięci oraz niepotrzebnego obciążania procesora [146].
  - **Dyspozytory (*Dispatchers*)** – pełnią rolę zarządców odpowiedzialnych za przydzielanie zadań do odpowiednich zasobów procesora. Operacje obliczeniowe oraz procesy pozyskiwania informacji w `LocationService` są kierowane do wyspecjalizowanych wątków systemowych (*Dispatchers.Default* lub *Dispatchers.IO*) [146]. Takie rozwiązanie pozwala na optymalne wykorzystanie mocy obliczeniowej oraz przepustowości łączy, nie wpływając przy tym na płynność działania interfejsu ani pozostałych modułów aplikacji.

- **Integracja z usługami systemowymi** – architektura aplikacji została zaprojektowana tak, aby ściśle współpracować z fundamentami systemu Android, w celu zagwarantowania stabilności gromadzenia danych w tle. Takie rozwiązanie stanowi wyzwanie ze względu na systemowe mechanizmy zarządzania energią, takie jak tryb *Doze*, które ograniczają dostęp aplikacji do procesora i sieci w celu oszczędzania baterii [148]. Aby zapewnić ciągłość pomiarów, aplikacja wykorzystuje mechanizmy pozwalające na pracę poza standardowymi restrykcjami systemu:

  - **Usługa pierwszego planu (ang. *Foreground Service*)** – stanowi fundament pracy aplikacji w tle. W przeciwieństwie do standardowych procesów, usługa ta informuje system operacyjny oraz użytkownika (poprzez wyświetlanie stałego powiadomienia), że aplikacja wykonuje zadanie o wysokim priorytecie. Zapewnia to nieprzerwane działanie modułów pomiarowych, chroniąc sesję użytkownika przed przypadkowym przerwaniem przez system w trakcie trwania długotrwałych pomiarów [149]. Jest to warunek konieczny do zachowania rzetelności i ciągłości monitorowania sygnału, szczególnie w sytuacjach, gdy użytkownik przemieszcza się z urządzeniem przy zablokowanym ekranie.
  - **Odbiorniki komunikatów (ang. *Broadcast Receivers*)** – realizują model komunikacji oparty na zdarzeniach, pełniąc rolę interfejsu nasłuchującego na sygnały emitowane przez system Android lub wewnętrzne komponenty aplikacji [150]. W ramach projektu mechanizm ten zapewnia natychmiastową reakcję na zdarzenia asynchroniczne, takie jak interakcja użytkownika z przyciskiem sterującym w obszarze powiadomień. Pozwala to na sterowanie procesem zbierania danych bez konieczności otwierania całej aplikacji, co ułatwia obsługę programu w trakcie ruchu.
  - **Magazyn ustawień (*Jetpack DataStore*)** – nowoczesny moduł odpowiedzialny za trwałe przechowywanie kluczowych informacji konfiguracyjnych, takich jak unikalny identyfikator *ShortCode*. Został on opracowany jako technologiczny następca mechanizmu *SharedPreferences* [151], eliminując jego ograniczenia poprzez zapewnienie pełnej spójności danych oraz odporności na błędy zapisu i odczytu [152]. Co istotne, *DataStore* domyślnie obsługuje asynchroniczny przepływ informacji, co sprawia, że odczytywanie ustawień nie blokuje pracy interfejsu i jest w pełni kompatybilne z zastosowanym mechanizmem *Coroutines*.

Mechanizmy gromadzenia danych telemetrycznych

Warstwa gromadzenia danych stanowi fundament aplikacji SignalMap, odpowiadając za bezpośrednią ekstrakcję surowych parametrów fizycznych z modułów sprzętowych urządzenia. Implementacja opiera się na dedykowanych repozytoriach [25, 153, 154], które pełnią rolę ustandaryzowanego interfejsu dla usług systemowych Androida. Ich zadaniem jest przejęcie odpowiedzialności za bezpośrednią interakcję ze sprzętem i przekazanie gotowych, uporządkowanych informacji do dalszych modułów aplikacji.

Monitorowanie parametrów radiowych (Telephony Layer)
Za interakcję z podsystemem komunikacji bezprzewodowej odpowiada klasa `TelephonyRepository`, która korzysta z usługi systemowej `TelephonyManager` [25]. Projektowanie tego mechanizmu wymagało uwzględnienia specyfiki systemu Android, charakteryzującego się ogromną różnorodnością konfiguracji sprzętowych oraz odmiennymi sposobami raportowania danych przez moduły radiowe różnych producentów. Aby zapewnić pełną kompatybilność aplikacji i zagwarantować, że zbierane parametry będą porównywalne niezależnie od modelu urządzenia, wdrożono następujące rozwiązania:

- **Analiza jakości sygnału** – do pozyskiwania metryk radiowych wykorzystano klasę `TelephonyManager`, która umożliwia dostęp do surowych danych o komórkach (`allCellInfo`). Implementacja filtruje zarejestrowane obiekty `CellInfo` (NR, LTE, WCDMA, GSM) i bezpiecznie mapuje specyficzne parametry, takie jak RSRP, RSRQ czy SINR. Ze względu na różnice w nazewnictwie metod dla technologii 5G (np. `getCsiRsrp` lub `getSsRsrp`), system próbuje sekwencyjnie odczytać najbardziej wiarygodną wartość, zapewniając spójność danych niezależnie od wersji API [25].

- **Ekstrakcja identyfikatorów i lokalizacja infrastruktury** – dane o infrastrukturze są wyodrębniane bezpośrednio z obiektów (`CellIdentity`), które dostarczają unikalne parametry sieciowe dla danej technologii (np. `CellIdentityLte` lub `CellIdentityNr`) [154]. W przypadku sieci LTE, identyfikator *ECI* (*E-UTRAN Cell Identifier*) ma długość 28 bitów i składa się z 20-bitowego numeru stacji bazowej oraz 8-bitowego identyfikatora sektora. Implementacja wykorzystuje tę strukturę do obliczenia *eNodeB ID* poprzez dzielenie całkowite wartości `CI` (reprezentującej pełny identyfikator komórki ECI) przez 256, co odpowiada przesunięciu o 8 bitów. Natomiast operacja modulo `CI % 256` pozwala wyodrębnić numer sektora (ostatnie 8 bitów) [39]. Dodatkowo zaimplementowano mapowanie parametrów *ARFCN* na konkretne numery pasm (np. B1, B3, n78), co pozwala na interpretację częstotliwości pracy nadajnika bez konieczności odwoływania się do zewnętrznych źródeł.

- **Detekcja standardów 5G w trybach SA i NSA** – obsługa sieci 5G została zrealizowana przy użyciu mechanizmu `callbackFlow`, który asynchronicznie obserwuje zmiany w systemie [155]. Aplikacja wykorzystuje `TelephonyDisplayInfo` (dla API 30+) do rozróżnienia trybu *Non-Standalone* (NSA) poprzez flagę `OVERRIDE_NETWORK_TYPE_NR_NSA`[153]. Z kolei detekcja trybu *Standalone* (SA) bazuje na bezpośrednim odczycie stanu z `ServiceState` oraz sprawdzeniu, czy aktywny typ sieci to `NETWORK_TYPE_NR`, co zapewnia rzetelną informację o charakterze połączenia [25].

- **Dynamiczny dostęp do zasobów poprzez mechanizm refleksji** – ze względu na fakt, że część kluczowych parametrów (takich jak numery pasm `getBands()` czy szczegółowy stan sieci `getNrState()`) jest ukryta wewnątrz systemu Android i niedostępna przez standardowe, publiczne interfejsy API, zastosowano technikę refleksji (ang. *Reflection*). Technika

ta pozwala aplikacji na bezpieczne uzyskanie informacji o dostępności tych metod bezpośrednio w czasie działania programu [156]. Implementacja zawiera również mechanizmy ochronne w postaci bloków `try-catch`, co gwarantuje, że w przypadku braku wsparcia danej funkcji przez konkretnego producenta telefonu, aplikacja pominie dany odczyt zamiast ulec awarii. Umożliwia to pozyskanie danych technicznych, które są zazwyczaj zarezerwowane wyłącznie dla systemowych narzędzi diagnostycznych.

Pozycjonowanie i geolokalizacja (Location Layer)

Każdy odczyt telemetryczny jest nierozerwalnie powiązany z precyzyjnymi współrzędnymi geograficznymi dzięki modułowi `LocationRepository`. Kluczowy mechanizmem tego komponentu jest usługa *Google Fused Location Provider*. Zadaniem tego silnika, jest inteligentne agregowanie danych z systemów satelitarnych (GNSS), sieci Wi-Fi oraz czujników bezwładnościowych urządzenia [157]. W celu zapewnienia wiarygodnych map zasięgu, w implementacji zastosowano następujące mechanizmy kontrolne:

- **Priorytet wysokiej dokładności** – poprzez wymuszenie trybu `PRIORITY_HIGH_ACCURACY`, system priorytetyzuje odczyty z odbiornika GPS/GNSS[157]. Jest to istotna decyzja z punktu widzenia późniejszego nakładania pomiarów na mapę, gdyż błąd rzędu kilkunastu metrów mógłby skutkować błędnym przypisaniem próbki do sąsiedniej ulicy lub innego sektora nadajnika, co zafałszowałoby końcową mapę zasięgu.
- **Automatyczne odbieranie danych (*callbackFlow*)** – zamiast wymuszać na aplikacji ciągłe, ręczne sprawdzanie lokalizacji, zastosowano mechanizm, który samodzielnie przekazuje informację o nowej pozycji, gdy tylko zostanie ona wykryta przez system. Rozwiązanie to jest niezwykle oszczędne: aktywuje funkcje GPS tylko na czas trwania pomiarów, a po ich zakończeniu natychmiastowo wyłącza nasłuch (`awaitClose`)[155]. Dzięki temu aplikacja nie zużywa energii ani zasobów urządzenia w momentach, gdy użytkownik nie wykonuje aktywnego testu sieci.
- **Adaptacyjna filtracja zdarzeń** – kluczowym elementem optymalizacji jest zastosowanie filtrów czasowo-przestrzennych: `intervalMillis` oraz `minDistanceMeters`[157, 158]. W implementacji przyjęto zasadę, że nowa próbka lokalizacji jest przesyłana do systemu tylko wtedy, gdy użytkownik przemieści się o określoną odległość lub upłynie dany czas. Takie rozwiązanie zapobiega gromadzeniu powtarzalnych danych w sytuacjach, gdy urządzenie pozostaje w bezruchu, oraz skutecznie eliminuje błędy wynikające z naturalnych wahań wskazań odbiornika GPS (tzw. dryfu lokalizacji) [159]. Szczegółowe zasady dobierania tych parametrów w zależności od trybu pracy aplikacji zostały opisane w podrozdziale 4.2.1 pt. *Adaptacyjna polityka próbkowania i praca w tle.*

Integracja i spójność danych (NetworkSnapshot)

Finalnym etapem procesu gromadzenia jest agregacja rozproszonych odczytów w spójny model danych. Wszystkie informacje pochodzące z warstwy telefonicznej oraz lokalizacyjnej są scalane w obiekty klasy `NetworkSnapshot`. Dzięki zastosowaniu opisanej wcześniej struktury niemutowalnej, system zyskuje pewność, że parametry radiowe oraz współrzędne geograficzne w obrębie jednej próbki stanowią spójną całość. Rozwiązanie to eliminuje ryzyko wystąpienia błędów synchronizacji, w których nowa pozycja GPS mogłaby zostać omyłkowo przypisana do nieaktualnych parametrów sieciowych podczas asynchronicznego przekazywania danych do chmury.

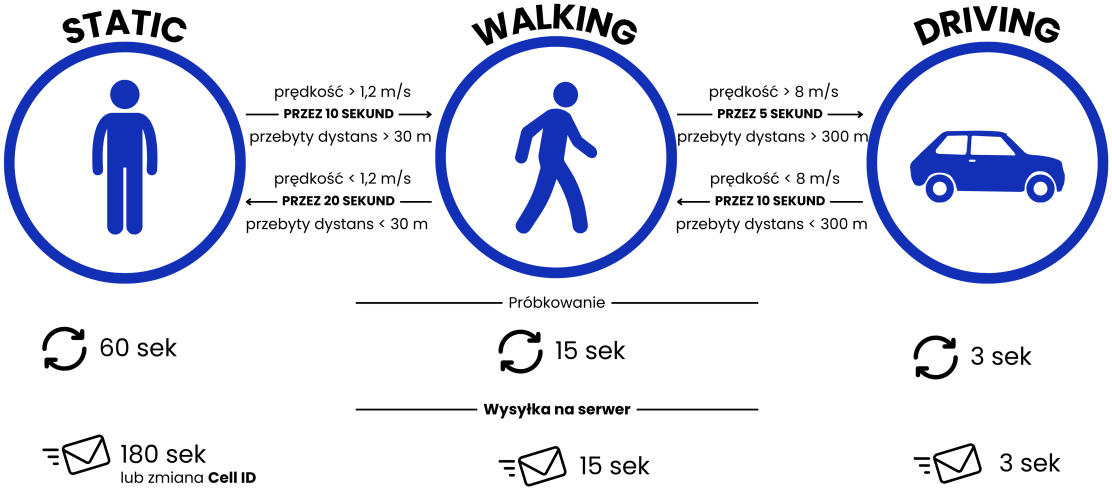
Adaptacyjna polityka próbkowania i praca w tle

Kluczowym wyzwaniem inżynierskim przy projektowaniu aplikacji SignalMap było zapewnienie ciągłości zbierania danych telemetrycznych przy jednoczesnej optymalizacji zużycia energii przez moduły radiowe i GPS. Cel ten zrealizowano poprzez mechanizm adaptacyjnego próbkowania.

Obecnie stosowane parametry częstotliwości odczytów zostały dobrane drogą empiryczną, na podstawie testów terenowych, tak aby zapewnić pokrycie mapy przy zachowaniu akceptowalnego zużycia baterii. Należy jednak zaznaczyć, że jest to proces ewolucyjny – w przyszłych wersjach aplikacji planowane jest wprowadzenie w pełni dynamicznego modelu zbierania danych, który będzie jeszcze precyzyjniej reagował na sugestie użytkowników oraz zmieniające się warunki otoczenia.

Dynamika próbkowania i mechanizm histerezy

Za inteligentne zarządzanie częstotliwością pomiarów odpowiada moduł polityki próbkowania `Sampling Policy`. Zamiast stosować sztywny interwał zapisu, algorytm analizuje w czasie rzeczywistym profil ruchu użytkownika, opierając się na prędkości raportowanej przez sensor GPS oraz dystansie przebytym między odczytami.



Rysunek 4.1: Diagram stanów adaptacyjnego algorytmu próbkowania z uwzględnieniem mechanizmu histerezy.

Na tej podstawie system klasyfikuje stan aktywności do jednego z trzech trybów pracy, różnicując częstotliwość odczytów (próbkowania) oraz przesyłu danych na serwer (Rys. 4.1):

- **Tryb stacjonarny (STATIC)** – aktywowany podczas bezruchu. System pobiera parametry sieciowe co 60 sekund, co minimalizuje obciążenie procesora. Wysyłka danych na serwer jest dodatkowo optymalizowana: odbywa się co 180 sekund, chyba że nastąpi zmiana stacji bazowej (*Cell ID*), co wymusza natychmiastową wysyłkę.
- **Tryb pieszy (WALKING)** – uruchamiany przy wykryciu przemieszczania się (prędkość $> 1,2$ m/s lub dystans > 30 m). Interwał pomiarowy i transmisyjny zostaje skrócony do 15 sekund, co pozwala na dokładne odwzorowanie zmian zasięgu w terenie zurbanizowanym.
- **Tryb mobilny (DRIVING)** – tryb najwyższej gęstości, aktywowany przy prędkościach > 8 m/s lub dystansie > 300 m. Zarówno próbkowanie, jak i wysyłka odbywają się co 3 sekundy, co zapobiega powstawaniu luk w danych przy szybkim przemieszczaniu się wzdłuż szlaków komunikacyjnych.

Kluczowym elementem stabilizującym pracę systemu jest **mechanizm histerezy czasowej**. Histereza definiowana jest jako zjawisko zależności aktualnego stanu układu od stanów występujących w poprzedzających go chwilach, co w praktyce oznacza celowe opóźnienie reakcji systemu na zmieniający się czynnik zewnętrzny [160]. Zapobiega on ciągłemu przełączaniu trybów przy prędkościach oscylujących wokół progu decyzyjnego. Zmiana trybu na wyższy lub niższy następuje dopiero po utrzymaniu się nowych warunków ruchu przez określony czas:

- **5 sekund** dla przejścia w tryb *DRIVING*, co zapobiega aktywacji gęstego próbkowania przy chwilowym przyspieszeniu,
- **10 sekund** dla przejścia w tryb *WALKING* lub powrotu z trybu mobilnego,
- **20 sekund** dla powrotu do trybu *STATIC*. Najdłuższy czas oczekiwania na uśpienie modułów zapobiega przerwaniu pomiarów podczas postojów, przykładowo na sygnalizacji świetlnej, gwarantując ciągłość zbieranej trasy.

Moduł testów wydajnościowych

Moduł ten odpowiada za aktywne sprawdzanie jakości łącza (QoS) poprzez wymuszenie przesyłu danych. Pozwala to zweryfikować realną przepustowość sieci, wykraczając poza same informacje o zasięgu. Cały proces pomiarowy został zaprojektowany tak, aby działać niezależnie od widoku aplikacji. Rozdzielenie tych warstw sprawia, że że intensywne operacje matematyczne nie wpływają na płynność interfejsu użytkownika.

Metodyka pomiaru opóźnień i stabilności (Ping oraz Jitter)

Pierwszym etapem weryfikacji łącza jest określenie czasu odpowiedzi serwera (ang. *latency*). Zgodnie z zaleceniami *IP Performance Metrics* (IPPM), system wyznacza medianę wyników RTT (ang. *Round Trip Time*) [161]. Takie podejście pozwala na uzyskanie miarodajnego wyniku nawet w niestabilnych warunkach sieci mobilnej, gdzie chwilowe piki opóźnień mogłyby sztucznie zawyżyć średnią [162].

Równie istotnym parametrem jest *Jitter*, definiowany jako zmienność opóźnień pakietów w czasie [162]. W aplikacji *SignalMap* do jego wyznaczenia zastosowano różnicę między 90. percentylem (P90) a medianą (P50). Wybór tej metody podyktowany jest chęcią uchwycenia rzeczywistej zmienności opóźnień przy jednoczesnym odfiltrowaniu skrajnych wartości. Wykorzystanie 90. percentyla (P90) pozwala na wyznaczenie górnej granicy opóźnień operacyjnych, której nie przekracza zdecydowana większość pakietów [163]. Dzięki temu możliwe jest pominięcie 10% najbardziej nietypowych wartości odstających, które w środowisku sieci mobilnych wynikają zazwyczaj z chwilowych fluktuacji warunków radiowych, a nie z realnej wydajności łącza. Obliczenie różnicy $P90 - Mediana$ służy jako ilościowy wskaźnik stabilności łącza, obrazujący zakres wahań opóźnień w warunkach typowej eksploatacji sieci [163].

Ze względu na to, że testy wydajnościowe generują skończoną liczbę próbek, konkretna wartość percentyla może wypadać pomiędzy dwoma faktycznymi pomiarami. Aby uniknąć niedokładności wynikających z przyjmowania wartości dyskretnych najbliższych sąsiednich próbek, zastosowano **interpolację liniową**. Jest to proces matematyczny polegający na wyznaczeniu wartości pośredniej na wirtualnej linii łączącej dwa sąsiednie punkty danych [164]. Zamiast polegać na schodkowej charakterystyce wynikającej z ograniczonej wielkości próby, system dokonuje **estymacji ciągłej**. W przeciwieństwie do innych metod, które wybierają najbliższy istniejący pomiar, estymacja ciągła zakłada, że między zarejestrowanymi punktami danych istnieje funkcja przejścia (w tym przypadku liniowa).

Pozwala to na wyznaczenie wartości, które nie wystąpiły bezpośrednio w próbie, ale są statystycznie najbardziej prawdopodobne dla danego rozkładu [165]. Dzięki temu uzyskuje się precyzyjny i powtarzalny wynik parametru *Jitter* oraz minimalizację błędów wynikających z kwantyzacji wyników, co jest kluczowe przy analizie stabilności łącza na podstawie niewielkiej liczby pakietów kontrolnych [165].

Algorytmy pomiaru przepustowości i infrastruktura

Pomiar przepustowości (*download* oraz *upload*) to najbardziej wymagający energetycznie i procesowo etap testu. Podczas pobierania danych system inicjuje równoległe połączenia, wykorzystując fazę „rozgrzewki" (ang. *warm-up*), która pozwala na ustabilizowanie okna przesyłania danych protokołu TCP [166]. W fazie wysyłania (*upload*) prędkość obliczana jest wyłącznie na podstawie bajtów, których odebranie zostało potwierdzone przez serwer, co eliminuje błąd wynikający z buforowania danych w pamięci telefonu.

Sercem tego modułu jest klasa `SpeedTestRepository`, która realizuje powyższą logikę w oparciu o bibliotekę *OkHttp* oraz mechanizm *Kotlin Coroutines* [167]. W celu zapewnienia rzetelności pomiaru, algorytm został sparametryzowany w sposób eliminujący typowe błędy transmisji mobilnej:

- **Wielowątkowość (`parallel = 2`):** Inicjowanie równoległych strumieni pozwala na pełne nasycenie pasma TCP, co jest niezbędne w sieciach o wysokiej przepustowości (np. 5G), gdzie pojedyncze połączenie mogłoby zostać ograniczone przez czas oczekiwania na potwierdzenia pakietów.
- **Faza stabilizacji (`warmupSeconds = 1`):** Pierwsza sekunda transmisji nie jest wliczana do wyniku, co pozwala na ominięcie fazy *Slow Start* protokołu TCP.
- **Czas trwania i powtórzenia (`measureSeconds = 7`, `rounds = 2`):** Siedmiosekundowy interwał właściwego testu zapewnia odpowiednią bazę statystyczną, a wykonanie dwóch rund pomiarowych pozwala na wyciągnięcie mediany i odrzucenie błędów grubych wynikających np. z chwilowego przełączenia stacji bazowej (ang. handover).
- **Strumieniowanie segmentowe (`rangeChunkBytes = 5MB`):** Użycie nagłówka *Range* oraz funkcji `blackholeSink()` umożliwia przetwarzanie dużej ilości danych bez obciążania pamięci RAM i nadmiarowego zapisu dyskowego smartfona.

Kluczowym czynnikiem wpływającym na uzyskiwane wyniki jest fizyczna lokalizacja infrastruktury. Serwer pomiarowy oraz pliki testowe zlokalizowane są w regionie *North Europe*, którego główne centra danych znajdują się w Dublinie [168].

Odległość ta ma kluczowe znaczenie, ponieważ nawet przy wysokiej przepustowości łącza, sygnał potrzebuje określonego czasu na pokonanie drogi do serwera. W celu zapewnienia pełnej przejrzystości procesu pomiarowego, w interfejsie użytkownika umieszczono informację o lokalizacji serwera docelowego, co przedstawiono na rysunku 4.5. .

Raportowanie i spójność danych

Logika aplikacji opiera się na maszynie stanów zaimplementowanej za pomocą *sealed classes*, co pozwala na sztywne zdefiniowanie wszystkich dopuszczalnych etapów pomiaru. Dzięki temu system automatycznie blokuje możliwość wystąpienia błędnych przejść między procesami (np. startu uploadu przed zakończeniem downloadu), co zapewnia sekwencyjną kolejność kolejnych etapów testu [169].

Dzięki reaktywnemu przesyłaniu stanów do warstwy prezentacji, system generuje dynamiczne wykresy liniowe (*sparklines*) w czasie rzeczywistym. Pełnią one rolę wskaźników trendu, umoż-

liwiając przybliżoną ocenę stabilności połączenia [170]. Po sfinalizowaniu pomiarów, kompletny obiekt jest serializowany do formatu JSON i przesyłany asynchronicznie do chmury obliczeniowej. Zapewnia to, że baza danych systemu *SignalMap* jest stale zasilana świeżymi danymi o realnej wydajności sieci w różnych lokalizacjach, budując rzetelną mapę jakości usług.

Komunikacja z infrastrukturą chmurową

Warstwa komunikacyjna pełni rolę punktu styku między środowiskiem aplikacji mobilnej a infrastrukturą serwerową platformy Azure. Została ona zaprojektowana jako ośrodek odpowiedzialny za koordynację bezpiecznego transferu zgromadzonych danych telemetrycznych oraz wyników testów wydajnościowych do bramy API. Za realizację tych zadań odpowiada obiekt `AzureClient`, który został zaimplementowany jako **Singleton**.

Wzorzec ten gwarantuje, że w całym cyklu życia aplikacji istnieje tylko jedna, unikalna instancja klasy, zapewniająca globalny punkt dostępu do swoich zasobów [171]. W architekturze systemu rozwiązanie to pełni funkcję centralnego kontrolera ruchu sieciowego, co zapobiega powstawaniu nadmiarowych połączeń i pozwala na efektywne zarządzanie wspólnymi zasobami, takimi jak mechanizmy autoryzacji czy kolejka wysyłkowa. Dzięki enkapsulacji logiki komunikacyjnej w pojedynczym obiekcie, `AzureClient` stanowi kontrolowany punkt wyjścia danych z urządzenia. Ułatwia to monitorowanie ruchu, jednolitą obsługę błędów oraz zapewnia spójność sesji bez ryzyka konfliktów zasobów, które mogłyby wystąpić przy próbie równoległego dostępu z różnych modułów aplikacji [171].

Identyfikacja urządzenia i uwierzytelnianie sesji

Bezpieczeństwo dostępu do zasobów chmurowych oraz zapewnienie integralności przesyłanych informacji oparto na szyfrowanym protokole HTTPS i mechanizmie autoryzacji. Architektura systemu wprowadza wyraźny rozdział między procesem rozpoznawania urządzenia a faktycznym uwierzytelnianiem zapytań API, co pozwala na zachowanie balansu między funkcjonalnością systemu a prywatnością użytkownika.

Kluczowym założeniem projektowym jest model rejestracji odroczonej. Procedura ta nie jest inicjowana automatycznie po instalacji, lecz następuje dopiero w momencie świadomej interakcji użytkownika z systemem, na przykład poprzez uruchomienie pierwszego pomiaru sygnału lub testu wydajnościowego. W odpowiedzi na to zdarzenie, system wykorzystuje identyfikator sprzętowy (`ANDROID_ID`) do wygenerowania unikalnego kodu *ShortCode*. Pełni on funkcję trwałego łącznika między urządzeniem a gromadzonymi w chmurze danymi telemetrycznymi. Dzięki temu rozwiązaniu, użytkownik zyskuje dostęp do spersonalizowanej historii swoich wyników oraz statystyk, zachowując przy tym anonimowość – system nie wymaga bowiem zakładania tradycyjnego konta ani podawania danych osobowych. Szczegółowy opis algorytmu generowania tych kodów po stronie serwera oraz metody ich walidacji został przedstawiony w sekcji 5.4 poświęconej bezpieczeństwu.

Właściwy proces uwierzytelniania (ang. *authentication*) odbywa się dynamicznie przy użyciu tokenów dostępu [172], które stanowią cyfrowy klucz uprawniający do zapisu danych w infrastrukturze Azure. Aby zminimalizować obciążenie akumulatora, zaimplementowano mechanizm buforowania tokenu w pamięci ulotnej (*cachedToken*)[173]. Odświeżenie tokena następuje jedynie po wygaśnięciu jego ważności lub w przypadku błędu transmisji. Pełna specyfikacja techniczna procesu wystawiania i odświeżania tokenów przez brokera uwierzytelniającego została szczegółowo opisana w sekcji 5.3 dotyczącej bezpieczeństwa systemu.

Strukturyzacja i technologia przesyłu danych

Proces transferu informacji telemetrycznych opiera się na formacie JSON, który stanowi standard wymiany danych w nowoczesnych systemach rozproszonych. Takie podejście zapewnia pełną kom-

patybilność z architekturą *Serverless* po stronie backendu, ułatwiając automatyczne przetwarzanie rekordów.

Kluczowym wyzwaniem w fazie przygotowania danych jest zapewnienie ich spójności, niezależnie od różnic technicznych między modelami smartfonów. W tym celu zaimplementowano mechanizm normalizacji komunikatów – jeżeli dane urządzenie nie obsługuje konkretnej technologii (np. brak odczytów parametrów 5G), klasa `AzureClient` automatycznie mapuje te pola na wartość `JSONObject.NULL`. Zapewnia to stałą strukturę komunikatów trafiających do systemu, dzięki czemu brak określonych parametrów technicznych nie jest interpretowany jako błąd systemu, lecz jako naturalne ograniczenie sprzętowe urządzenia.

Realizacja procesów sieciowych wewnątrz modułu `AzureClient` została zaprojektowana tak, aby uwzględnić rygorystyczne wymagania wydajnościowe środowiska Android, gdzie kluczowe jest zachowanie responsywności głównego wątku. Wszystkie operacje związane z komunikacją z infrastrukturą chmurową są asynchronicznie delegowane do `Dispatchers.IO` przy wykorzystaniu mechanizmu korutyn języka Kotlin [146].

Takie podejście architekturalne pozwala na odizolowanie kosztownych czasowo operacji wejścia-wyjścia (*I/O*) od warstwy prezentacji. Dzięki przeniesieniu obciążenia związanego z nawiązywaniem połączeń HTTPS oraz transmisją payloadów JSON na wątki zoptymalizowane pod kątem blokujących operacji sieciowych, system eliminuje ryzyko wystąpienia błędów typu *Application Not Responding* (ANR).

Zjawisko to występuje, gdy główny wątek aplikacji zostaje zablokowany na zbyt długi czas, uniemożliwiając systemowi obsługę zdarzeń wejściowych od użytkownika. Stosowanie asynchronicznego modelu komunikacji pozwala na zachowanie pełnej dostępności interfejsu graficznego, nawet podczas intensywnej wymiany danych z infrastrukturą Azure [174].

Zapewnienie spójności przesyłanych danych zrealizowano poprzez wdrożenie mechanizmu idempotencji. Przed każdą próbą wysłania nowego pakietu, aplikacja generuje unikalny klucz identyfikacyjny przy użyciu klasy `java.util.UUID.randomUUID().toString()`, który jest przesyłany w metadanych komunikatu jako `idempotencyKey`. Rozwiązanie to pozwala usłudze *Azure Service Bus* na skuteczną weryfikację, czy dany pomiar nie został już wcześniej poprawnie odebrany i przetworzony. Dzięki temu, mimo wielokrotnego ponawiania zapytań przez klienta HTTP w warunkach niestabilnego łącza, system eliminuje ryzyko powielania tych samych rekordów w bazie danych.

Integracja z procesami zbierania danych i raportowanie statusu

Komunikacja z infrastrukturą chmurową jest ściśle zsynchronizowana z cyklem życia aplikacji i procesami działającymi w tle. W ramach usługi `LocationService` zaimplementowano mechanizm zwrotnego raportowania sukcesu transmisji. Każda pomyślna operacja zapisu danych telemetrycznych w chmurze Azure jest komunikowana do modułu powiadomień systemowych. Pozwala to na informowanie użytkownika o statusie synchronizacji w czasie rzeczywistym.

W przypadku modułu *Speedtest*, priorytetem jest zachowanie aktualności danych analitycznych. Z tego względu wynik pomiaru przesyłany jest asynchronicznie, w sposób priorytetowy, natychmiast po zakończeniu ostatniej fazy testu. Dzięki tak ścisłemu powiązaniu warstwy pomiarowej z klientem `AzureClient`, system minimalizuje opóźnienie między wystąpieniem zjawiska sieciowego a jego utrwaleniem w bazie danych. Stanowi to kluczową wartość dla późniejszych procesów wizualizacji danych na mapach zasięgu.

Projekt interfejsu użytkownika

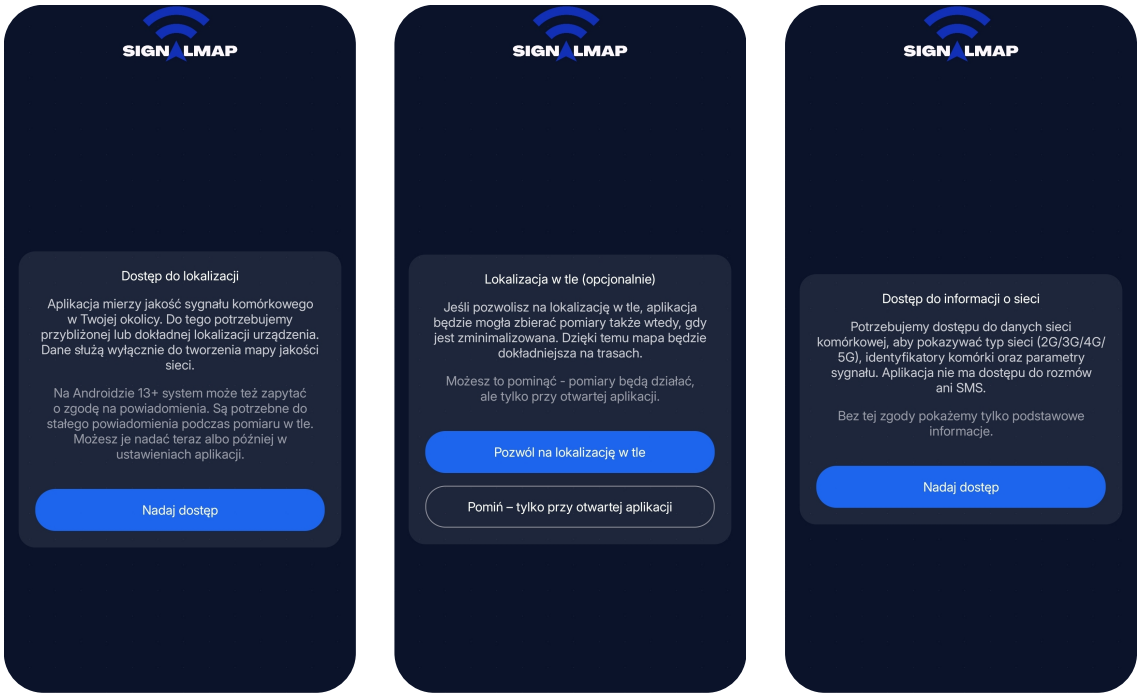
Warstwa prezentacji aplikacji SignalMap została stworzona przy użyciu biblioteki Jetpack Compose [143]. Jest to nowoczesne rozwiązanie, które pozwala budować interfejs użytkownika bezpośrednio w języku Kotlin, co zapewnia płynne działanie ekranów oraz ułatwia zarządzanie ich wyglądem.

Struktura interfejsu i proces Onboardingu

Warstwa wizualna aplikacji *SignalMap* została zbudowana zgodnie z wzorcem **Single Activity**. W tej architekturze cała aplikacja bazuje na jednym głównym oknie (aktywności), wewnątrz którego wymieniane są poszczególne ekrany. Takie podejście nie tylko upraszcza nawigację i zarządzanie danymi, ale także sprawia, że interfejs działa szybciej i bardziej naturalnie dla użytkownika [175].

Kluczowym etapem pierwszego kontaktu z programem jest proces **onboardingu** (wprowadzenia użytkownika). W tym celu zaprojektowano sekwencję ekranów edukacyjnych. Ich zadaniem jest wyjaśnienie użytkownikowi, dlaczego aplikacja potrzebuje konkretnych uprawnień (np. do lokalizacji czy stanu telefonu) jeszcze przed pojawieniem się systemowych komunikatów.

Ze względu na rygorystyczne zasady bezpieczeństwa systemów Android 11+ oraz 13+, zaimplementowano kaskadowy system pozyskiwania zgód, reprezentowany przez klasę `PermissionsOnboardingScreen`. Proces ten, zarejestrowany na urządzeniu Samsung Galaxy A36, przedstawiono na rysunku 4.2.



(A) Prośba o dostęp do lokalizacji (B) Prośba o lokalizację w tle (C) Dostęp do informacji o sieci

Rysunek 4.2: Proces onboardingu uprawnień na urządzeniu Samsung Galaxy A36: ekrany informacyjne aplikacji przygotowujące do nadania uprawnień systemowych.

Pierwszy etap (Rys. 4.2a) koncentruje się na podstawowym dostępie do lokalizacji, niezbędnym do osadzenia pomiarów w przestrzeni. Aplikacja wyjaśnia cel zbierania danych, zanim użytkownik zostanie przekierowany do systemowego okna wyboru dokładności pomiaru. Kolejnym krokiem jest ekran prośby o dostęp do lokalizacji w tle (Rys. 4.2b). Jest to etap opcjonalny, lecz kluczowy dla ciągłości zbierania danych przy zminimalizowanej aplikacji; widok ten przygotowuje użytkownika

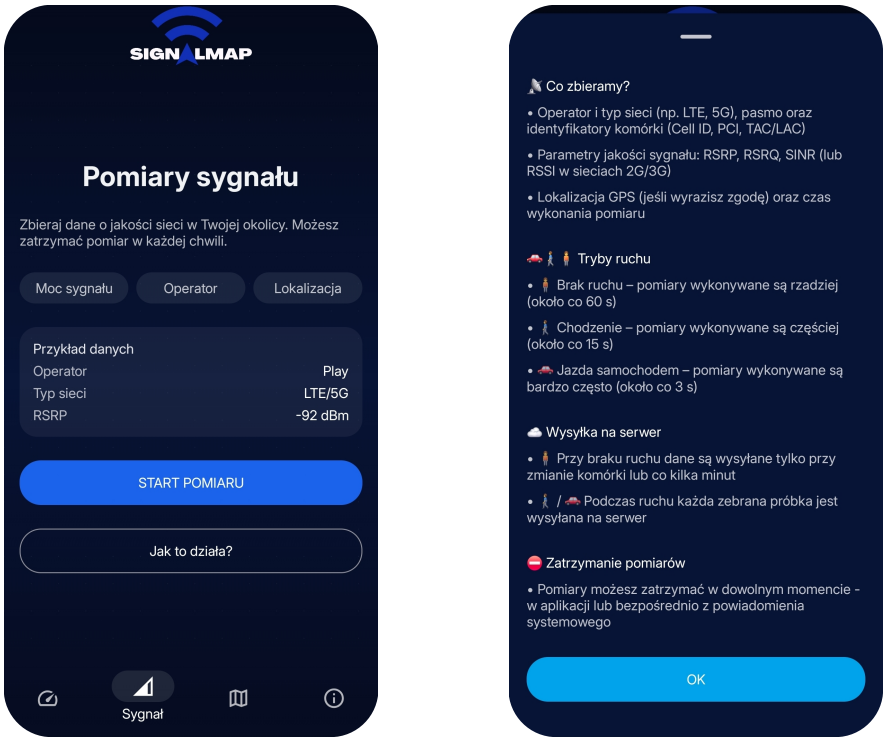
na konieczność manualnej zmiany uprawnień w ustawieniach systemowych.

Ostatnim elementem tej fazy jest prośba o dostęp do informacji o sieci (Rys. 4.2c). Widok ten tłumaczy potrzebę odczytu parametrów technicznych stacji bazowych (takich jak identyfikator komórki czy pasmo), co w systemie Android jest powiązane z uprawnieniami telefonicznymi.

Monitorowanie sygnału i testy wydajnościowe

Ekran **Signal** (`SignalScreen`) stanowi centralny moduł aplikacji, odpowiedzialny za bieżący monitoring parametrów radiowych stacji bazowej, z którą połączone jest urządzenie.

Przed rozpoczęciem sesji pomiarowej, użytkownikowi prezentowany jest widok informacyjny (Rys. 4.3a), pełniący rolę pulpitu z przykładowymi danymi. Kluczowym elementem wspierającym świadomość użytkownika jest sekcja „Jak to działa?" (Rys. 4.3b), która szczegółowo wyjaśnia mechanikę gromadzenia danych, w tym logikę częstotliwości próbkowania uzależnioną od trybu ruchu.

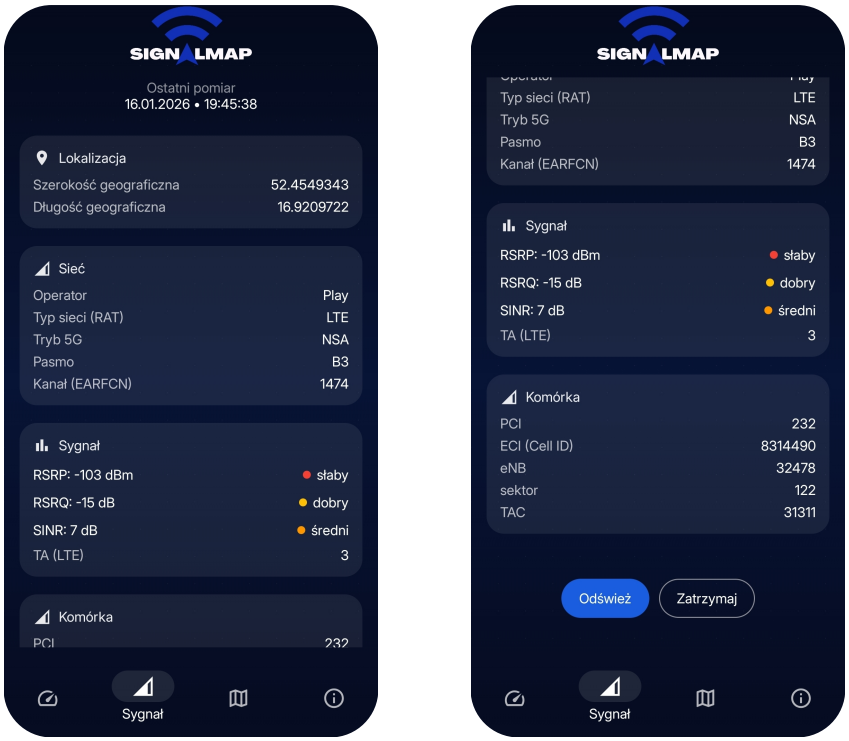


(A) Widok przed inicjalizacją pomiaru (B) Prezentacja mechanizmu zbierania

Rysunek 4.3: Ekrany informacyjne i konfiguracyjne modułu pomiarowego SignalMap.

Właściwa prezentacja wyników odbywa się w czasie rzeczywistym po inicjalizacji pomiaru. W trybach LTE oraz 5G NSA system wyświetla parametry jakościowe, takie jak RSRP, RSRQ i SINR, wzbogacone o wizualną interpretację jakości połączenia (Rys. 4.4a). Ze względu na dużą gęstość informacji, interfejs wykorzystuje pionowe przewijanie danych (*scrolling*), co pozwala na przedstawienie szczegółowych identyfikatorów komórki (*ECI, eNB, PCI*) oraz przycisków kontrolnych w dolnej sekcji widoku (Rys. 4.4b).

Ekran **Speedtest** (`SpeedTestScreen`) odpowiada za weryfikację wydajności łącza. Przed rozpoczęciem procedury pomiarowej, interfejs aplikacji prezentuje użytkownikowi m.in. lokalizację serwera docelowego (Rys. 4.5a). W fazie aktywnej (Rys. 4.5b) komponent płynnie wizualizuje postęp oraz chwilowe wartości przepustowości, podczas gdy generowane na bieżąco wykresy typu *sparklines* dostarczają przybliżonej informacji o stabilności łącza. Proces kończy się prezentacją



(A) Górna sekcja wyników pomiarowych

(B) Sekcja dolna z przyciskami kontrolnymi

Rysunek 4.4: Prezentacja wyników pomiaru sygnału w czasie rzeczywistym.

zagregowanego podsumowania (Rys. 4.5c), zawierającego nie tylko prędkości pobierania i wysyłania, ale także parametry opóźnień (*Ping*) i ich zmienności (*Jitter*), co pozwala na ocenę jakości stanu łącza.



(A) Interfejs przed startem testu

(B) W trakcie pomiaru

(C) Wynik testu

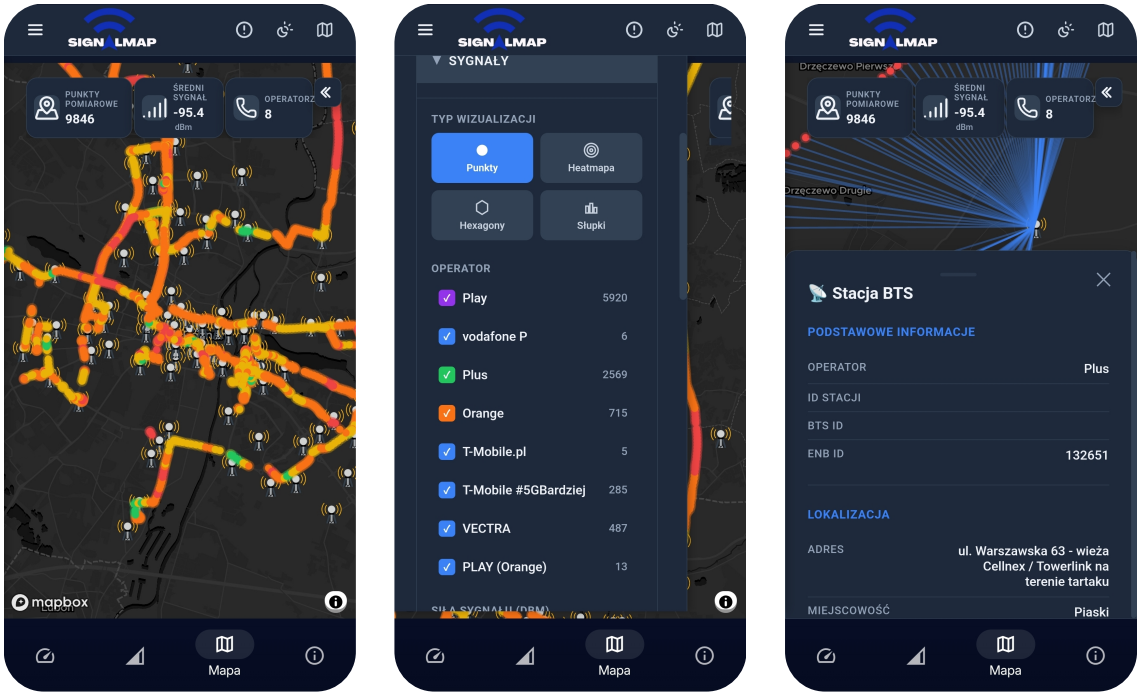
Rysunek 4.5: Przebieg testu wydajnościowego: od informacji o serwerze po analizę opóźnień i Jittera.

Hybrydowa integracja mapy (WebView)

Wybór architektury modułu mapy podyktowany był koniecznością zapewnienia pełnej spójności wizualnej oraz funkcjonalnej między platformą mobilną a systemem analitycznym dostępnym w przeglądarce. Zamiast implementacji natywnego pakietu **SDK** (ang. *Software Development Kit*), czyli zestawu gotowych narzędzi i bibliotek dostarczanych przez zewnętrznych dostawców w celu ułatwienia tworzenia oprogramowania na daną platformę [176], zdecydowano się na podejście hybrydowe. Wykorzystano autorski komponent `SignalMapWebView` w ramach ekranu `MapScreen`, którego implementację i wygląd szczegółowo opisano w rozdziale 4.5 pt. „Implementacja frontendu".

Takie rozwiązanie pozwoliło na unifikację doświadczenia użytkownika oraz scentralizowanie logiki renderowania danych przestrzennych. Wysoka wydajność renderowania, kluczowa przy operowaniu na zbiorach obejmujących tysiące punktów telemetrycznych, została osiągnięta poprzez pełne wykorzystanie akceleracji sprzętowej silnika Chromium, zintegrowanego z systemem Android [177]. Pozwala to na płynną obsługę interaktywnych warstw mapy, takich jak heksagonalne agregacje czy mapy ciepła, bez nadmiernego obciążenia procesora głównego urządzenia.

W celu zapewnienia najwyższej użyteczności, interfejs mapy został poddany pełnej optymalizacji responsywnej. Wszystkie elementy sterujące, legendy oraz panele informacyjne zostały dostosowane do mniejszych ekranów urządzeń mobilnych, zapewniając komfortową obsługę dotykową i czytelność danych nawet przy dużym zagęszczeniu informacji (Rys. 4.6). Szczegółowy opis algorytmów filtrowania, dostępnych trybów wizualizacji oraz zaawansowanych mechanizmów interakcji z mapą został zamieszczony w rozdziale poświęconym systemowi analitycznemu (Rozdział 4.5 pt. „Implementacja frontendu").



(A) Wizualizacja punktów pomiarowych na mapie

(B) Zaawansowane opcje filtrowania i wizualizacji

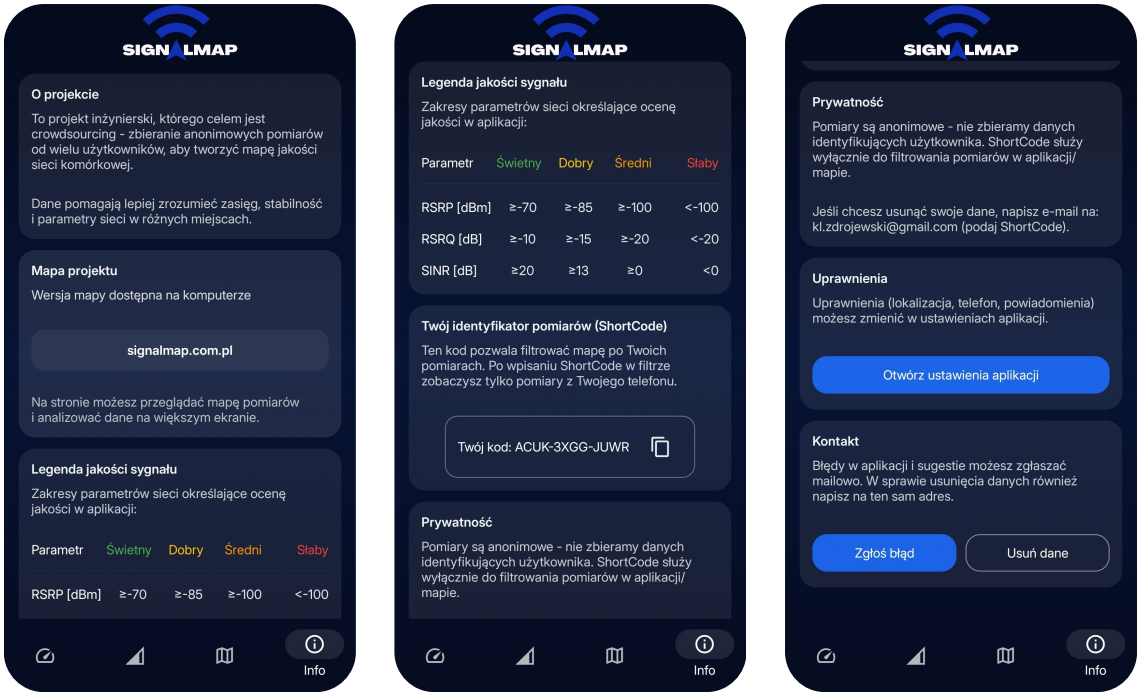
(C) Szczegóły stacji BTS i powiązane pomiary

Rysunek 4.6: Moduł analityczny mapy: od wizualizacji globalnej telemetrii, przez personalizację widoku, aż po podgląd danej stacji bazowych.

Centrum informacji i zarządzanie prywatnością

Ostatnim modułem warstwy prezentacji jest ekran **Info & Settings** (`InfoSettingsScreen`). Projekt tego widoku został podporządkowany zasadzie pełnej transparentności działań aplikacji,

co jest niezbędne w systemach opartych na modelu *crowdsourcingu*. Interfejs integruje aspekty edukacyjne z narzędziami kontroli nad danymi użytkownika, co przedstawiono na rysunku 4.7.



(A) Informacje o celach projektu (B) Parametry sieci i identyfikator (C) Prywatność i wsparcie

Rysunek 4.7: Struktura modułu informacyjnego aplikacji SignalMap.

Proces edukacji użytkownika oraz zarządzania anonimową tożsamością i uprawnieniami został zrealizowany poprzez dedykowane komponenty:

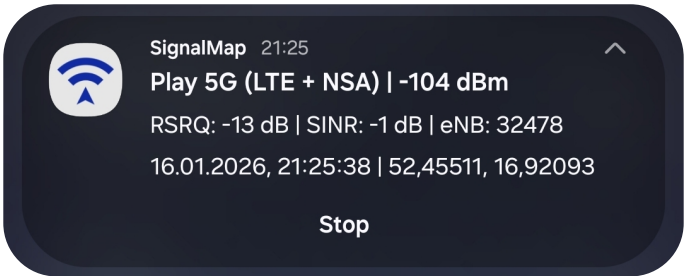
- **Legenda jakości sygnału** – w górnej części ekranu (Rys. 4.7b) zaimplementowano sekcję edukacyjną, która definiuje progi decyzyjne dla kluczowych parametrów sieciowych: RSRP, RSRQ oraz SINR. Tabela ta pozwala użytkownikowi zrozumieć przypisanie kolorystyczne (od świetnego po słaby) stosowane w pozostałych modułach aplikacji.
- **Identyfikacja użytkownika** – poniżej legendy umieszczono komponent `ShortCodeBadge`, eksponujący unikalny, anonimowy identyfikator instalacji (ShortCode). Moduł ten pozwala użytkownikowi na skopiowanie kodu do schowka, co umożliwia późniejsze filtrowanie własnych pomiarów na publicznej mapie zasięgu.
- **Zarządzanie uprawnieniami i prywatnością** – interfejs udostępnia bezpośredni skrót do systemowych ustawień Androida (Rys. 4.7c), ułatwiając rewizję zgód na dostęp do lokalizacji czy stanu telefonu. Dodatkowo, wdrożono mechanizm „prawa do zapomnienia", realizowany przez dedykowane przyciski kontaktu i usuwania danych, co zapewnia zgodność ze standardami ochrony prywatności w systemach mobilnych [43].

Monitorowanie w tle i usługa pierwszoplanowa

Kluczowym elementem architektury zapewniającym ciągłość zbierania telemetrii jest usługa pierwszoplanowa. Zgodnie z wytycznymi systemu Android, działanie takiej usługi musi być sygnalizowane użytkownikowi poprzez powiadomienie na pasku stanu (Rys. 4.8).

Zaimplementowane powiadomienie pełni funkcję informacyjnego panelu kontrolnego. Wyświetla ono w czasie rzeczywistym najważniejsze parametry aktualnego połączenia: nazwę operatora, tech-

nologię radiową (np. Play 5G LTE+NSA) oraz siłę sygnału wyrażoną w dBm. Dodatkowo, w treści powiadomienia zawarto precyzyjne współrzędne geograficzne oraz identyfikator stacji bazowej (*eNB*), co pozwala na szybką weryfikację jakości sygnału bez konieczności otwierania głównego interfejsu aplikacji. Powiadomienie wyposażono również w przycisk akcji „Stop", który umożliwia natychmiastowe przerwanie sesji pomiarowej i bezpieczne zamknięcie procesów działających w tle.



Rysunek 4.8: Interfejs powiadomienia wyświetlający parametry sieciowe w czasie rzeczywistym.

4.2.2 Zarządzanie infrastrukturą API

W celu zapewnienia bezpiecznej i wydajnej komunikacji między aplikacją mobilną a logiką bezserwerową, wdrożono usługę **Azure API Management (APIM)**. Pełni ona rolę bramy API (*ang. API Gateway*) izolującej funkcje chmurowe od bezpośredniego dostępu z sieci publicznej oraz narzuca polityki przetwarzania żądań [58].

Architektura fasady i abstrakcja backendu

Usługa APIM została skonfigurowana do obsługi interfejsu `fa-submit`, mapując przyjazne dla klienta adresy URL na wewnętrzne ścieżki usługi *Azure Functions* [68]. Dzięki zastosowaniu polityki `set-backend-service`, cała komunikacja jest kierowana pod adres bazowy instancji backendowej: `https://fa-submit-cdhvaqccepawf0hy.northeurope-01.azurewebsites.net`.

Mechanizmy bezpieczeństwa i uwierzytelniania

Zastosowanie bramy APIM pozwoliło na wdrożenie warstwowej strategii bezpieczeństwa:

- **Zarządzanie kluczami funkcji** – Aby uniknąć przesyłania kluczy dostępu do funkcji (*Function Keys*) bezpośrednio z aplikacji mobilnej, APIM wstrzykuje je po stronie serwera. Wykorzystano do tego politykę `set-header`, która dodaje nagłówek `x-functions-key` o wartości pobieranej z bezpiecznego repozytorium nazwanych wartości (`{{fa-submit-key}}`)[178].
- **Izolacja tokenów** – w przypadku endpointów takich jak *gettoken*, *ping* czy *upload*, brama APIM usuwa nagłówek `Authorization` przed przekazaniem żądania do backendu (polityka `exists-action="delete"`), co zapobiega zbędnemu procesowaniu poświadczeń tam, gdzie nie są one wymagane [178].

Kontrola przepływu i ochrona przed przeciążeniem

Kluczowym zadaniem bramy APIM jest ochrona zasobów obliczeniowych poprzez nakładanie limitów żądań (ang. *Rate Limiting*) [178]. Dla każdego punktu końcowego zdefiniowano indywidualne progi, dostosowane do jego specyfiki technicznej oraz przewidywanego natężenia ruchu, co zestawiono w tabeli 4.1.

Tabela 4.1: Konfiguracja limitów przepustowości (Rate Limit) w usłudze APIM.

Endpoint	**Limit żądań**	**Okres odnowienia**
`/register`	15	60 sekund
`/gettoken`	120	60 sekund
`/submit`	3000	60 sekund
`/speedtest`	30	60 sekund
`/speedtest-ping`	3000	60 sekund

Limity te ustalono dla grupy ok. 100 aktywnych użytkowników, opierając się na rzeczywistym zapotrzebowaniu aplikacji na przesył danych w trakcie wykonywania pomiarów. Punkty końcowe o charakterze administracyjnym oraz wynikowym, takie jak `/register` czy `/speedtest`, posiadają restrykcyjne limity dostępu. Ma to na celu ochronę przed atakami typu *DoS* oraz optymalizację zasobów, ponieważ operacje te (rejestracja konta czy przesłanie gotowego raportu z testu) są wykonywane sporadycznie. Z kolei wysokie progi dla `/submit` oraz `/speedtest-ping` wynikają bezpośrednio z charakteru pracy systemu: pierwszy obsługuje ciągły napływ danych telemetrycznych z urządzeń w ruchu, a drugi jest wykorzystywany do generowania dużej liczby zapytań niezbędnych do precyzyjnego pomiaru parametrów opóźnień.

Warto podkreślić, że parametry te mają charakter dynamiczny. Ze względów optymalizacji kosztów utrzymania infrastruktury, zdecydowano się na wykorzystanie podstawowego planu cenowego usługi APIM (pakiet *Consumption*). Wybór ten determinuje zakres dostępnych polityk bezpieczeństwa – stosowany mechanizm *rate-limit* operuje na ogólnej liczbie żądań wpływających do bramy.

Mimo braku wsparcia dla bardziej zaawansowanych limitów bazujących na adresacji IP (*rate-limit-by-key*) w tym wariancie, przyjęte rozwiązanie jest w pełni wystarczające dla celów demonstracyjnych i prototypowych projektu. W scenariuszu dalszego rozwoju aplikacji oraz jej komercjalizacji, plan zakłada migrację do wyższych warstw usługi, co pozwoliłoby na wdrożenie bardziej złożonych mechanizmów ochrony, takich jak limity per użytkownik czy zaawansowane filtrowanie geolokalizacyjne, zwiększając tym samym odporność systemu na nadużycia bez wpływu na komfort zwykłych użytkowników.

Optymalizacja odpowiedzi i zarządzanie pamięcią podręczną

Brama API odpowiada również za wymuszanie poprawnego zachowania mechanizmów buforowania. Dla operacji krytycznych czasowo, takich jak pomiary opóźnień (*Ping*), APIM nadpisuje nagłówki `Cache-Control` wartością `no-store, no-cache, must-revalidate`, co gwarantuje, że smartfon otrzymuje faktyczną odpowiedź z serwera, a nie wynik pośredni z pamięci podręcznej węzłów sieciowych [179].

Transformacja URI

Dzięki polityce `rewrite-uri`, brama APIM maskuje wewnętrzną strukturę katalogów *Azure Functions*. Żądania kierowane do publicznych ścieżek (np. `/submit`) są wewnątrz bramy mapowane na kanoniczne trasy API (np. `/api/submit`), co pozwala na zachowanie czystości interfejsu programistycznego i ułatwia ewentualną migrację backendu w przyszłości bez konieczności aktualizacji aplikacji mobilnej [180].

4.2.3 Implementacja warstwy odbioru i walidacji danych

Moduł logiki serwerowej został zaimplementowany w języku Python w ramach usługi Azure Functions (*fa-submit*). Aplikacja ta pełni rolę inteligentnego kontrolera, który przetwarza przychodzące żądania HTTP, dokonuje ich walidacji oraz zarządza sesjami użytkowników.

Mechanizmy identyfikacji i autoryzacji

Ze względu na wymóg zachowania prywatności przy jednoczesnym zabezpieczeniu infrastruktury, wdrożono dwa kluczowe mechanizmy, których szczegóły implementacji opisano w rozdziale 5 pt. *Bezpieczeństwo, autoryzacja oraz monitoring klastra*.

- **Generowanie ShortCode** – endpoint `/register` implementuje algorytm tworzenia 12-znakowych identyfikatorów na podstawie `androidId`. Przed przetworzeniem, identyfikator urządzenia poddawany jest walidacji pod kątem zgodności ze standardem systemowym Android, co eliminuje błędne lub niepełne dane. Następnie system wykorzystuje funkcję skrótu HMAC-SHA256 z unikalną wartością (*pepper*), zapewniając determinizm i anonimowość. Implementacja zawiera mechanizm sumy kontrolnej (*Mod32*) oraz pętlę obsługi kolizji, gwarantującą unikalność kodu w bazie PostgreSQL.
- **Autoryzacja JWT** – dostęp do krytycznych funkcji (*submit*, *speedtest*) jest chroniony przez autoryzację tokenami *Bearer*. Endpoint `/gettoken` wystawia krótkotrwałe tokeny podpisane algorytmem HS256, przy czym proces ten jest poprzedzony weryfikacją obecności powiązanego identyfikatora ShortCode w bazie danych. Token wydawany jest wyłącznie dla zarejestrowanych użytkowników. Tokeny są weryfikowane pod kątem ważności (*expiration time*) i autentyczności wystawcy przed każdym zapisem danych.

Przetwarzanie pomiarów sygnału i telemetrii

Głównym zadaniem funkcji `submit` jest odbiór oraz weryfikacja parametrów radiowych przesyłanych w formacie JSON. Stanowi ona pierwszy etap filtrowania danych wejściowych, którego celem jest odrzucenie błędnych lub zafałszowanych rekordów już na krawędzi systemu. W ramach walidacji system automatycznie eliminuje dane, w których parametry fizyczne sygnału (takie jak RSRP, RSRQ czy SINR) wykraczają poza ramy techniczne standardów LTE/5G, a współrzędne geograficzne nie mieszczą się w poprawnych zakresach matematycznych. Takie podejście pozwala uniknąć zbędnego kolejkowania niepoprawnych pakietów w usłudze *Azure Service Bus*, co optymalizuje koszty procesowe i zapobiega marnowaniu zasobów obliczeniowych na przetwarzanie nieużytecznych informacji w dalszych etapach potoku danych.

Obsługa testów wydajnościowych

Aplikacja udostępnia zestaw punktów końcowych dedykowanych dla modułu Speedtest, zoptymalizowanych pod kątem minimalnego narzutu obliczeniowego:

- **/speedtest/ping** – lekki endpoint zwracający status `200 OK` wraz z nagłówkami blokującymi buforowanie (*No-Cache*), co umożliwia pomiar czasu RTT przez aplikację mobilną.
- **/speedtest/upload** – punkt ten realizuje funkcję ujścia danych. Przyjmuje strumień binarny (do 20 MB) i zwraca liczbę faktycznie odebranych bajtów w nagłówku `X-Bytes-Received`, co pozwala smartfonowi na weryfikację realnej przepustowości typu *uplink*.

- **/speedtest (wyniki)** – dedykowany endpoint typu POST, który odbiera zagregowane statystyki testu i przygotowuje je do asynchronicznego zapisu.

Zgodnie z najlepszymi praktykami projektowania systemów rozproszonych, powyższe operacje powinny być odseparowane do niezależnych mikroserwerów lub dedykowanych instancji maszyn wirtualnych, aby uniknąć wzajemnego wpływu na wydajność. Niemniej jednak, w ramach optymalizacji kosztów utrzymania infrastruktury projektem, zdecydowano się na konsolidację wszystkich tych funkcji w ramach jednej usługi *Azure Function App*. Rozwiązanie to pozwoliło na współdzielenie zasobów w obrębie wspólnego planu cenowego przy jednoczesnym zachowaniu wystarczającej izolacji logicznej modułów dzięki wykorzystaniu mechanizmów routingu.

4.2.4 Asynchroniczna magistrala komunikatów (Azure Service Bus)

W celu zapewnienia wysokiej niezawodności systemu oraz odporności na nagłe skoki obciążenia, w architekturze zastosowano usługę **Azure Service Bus** w trybie *Standard*. Pełni ona rolę pośrednika (*Message Broker*), oddzielając warstwę przyjmowania danych od warstwy ich trwałego składowania [65].

Konfiguracja kolejki i mechanizmy buforujące

Centralnym elementem szyny danych jest kolejka o nazwie `submissions`, zlokalizowana w przestrzeni nazw `sb-inz-telemetry` (region *North Europe*). Zastosowanie kolejki pozwala na asynchroniczne przetwarzanie danych:

- **Skalowalność**: System może przyjąć znacznie więcej żądań na sekundę, niż jest w stanie przetworzyć baza danych w czasie rzeczywistym. Maksymalny rozmiar kolejki ustalono na 1 GB, co jest wystarczające dla małej wielkości tekstowych komunikatów JSON.
- **Odporność na awarie**: w przypadku chwilowej niedostępności bazy danych, informacje pozostają bezpiecznie zapisane w usłudze Service Bus przez okres do 14 dni [65].
- **Obsługa błędów**: Po 10 nieudanych próbach doręczenia, komunikaty trafiają do kolejki komunikatów porzuconych (*Dead-letter queue*), co umożliwia ich późniejszą analizę [65].

Struktura komunikatu i metadane

Każdy komunikat przesyłany przez kolejkę danych zawiera kompletny kontekst pomiaru oraz metadane ułatwiające procesowanie:

- **Body**: Znormalizowany obiekt JSON zawierający parametry sieciowe oraz lokalizację.
- **Deduplikacja i Idempotencja**: System wykorzystuje mechanizm *Duplicate detection* z 10-minutowym oknem czasowym. Kolejka weryfikuje unikalny klucz idempotencji (*idempotency key*) generowany po stronie aplikacji Android i przekazywany jako *MessageId*. W przypadku wykrycia komunikatu o identyfikatorze, który został już przetworzony w danym oknie czasowym, usługa automatycznie odrzuca go jako duplikat, zapobiegając wielokrotnemu zapisowi tego samego pomiaru w bazie danych.
- **Bezpieczeństwo**: Komunikacja z kolejką komunikatów wymusza stosowanie protokołu TLS w wersji minimum 1.2.

4.2.5 Implementacja warstwy procesowania i utrwalania danych

Ostatnim elementem w procesie gromadzenia danych telemetrycznych jest bezserwerowa funkcja wykonawcza *fa-worker*, zaimplementowana w języku Python w ramach usługi Azure Functions. Jej zadaniem jest asynchroniczne pobieranie komunikatów z kolejki `Azure Service Bus`, ich ostateczna walidacja oraz trwały zapis w relacyjnej bazie danych PostgreSQL.

Architektura wyzwalania i przetwarzanie wsadowe

Funkcja wykorzystuje mechanizm `service_bus_queue_trigger` z trybem dostarczania ustawioną na `many`, co pozwala na pobieranie komunikatów w paczkach (ang. *batches*). Takie podejście znacząco optymalizuje wydajność systemu, redukując liczbę transakcji bazodanowych poprzez grupowanie rekordów przed wykonaniem operacji `INSERT`. Wielkość paczki sterowana jest zmienną `BATCH_SIZE`, co pozwala na balansowanie między obciążeniem pamięci a szybkością zapisu.

Normalizacja danych i mapowanie operatorów

Ze względu na różnorodność wersji systemu Android oraz implementacji sterowników radiowych, *fa-worker* przeprowadza proces normalizacji danych. Kluczowe operacje obejmują:

- **Walidacja i typowanie danych** – wymuszenie typów numerycznych dla parametrów sygnału oraz weryfikacja poprawności współrzędnych geograficznych (zakresy lat: $\pm 90°$, lon: $\pm 180°$).
- **Normalizacja operatorów (MNO)** – proces ujednolicania nazw operatorów (w tym obsługi sieci wirtualnych MVNO) do czterech głównych identyfikatorów [181]. Zapewnia to bezpośrednią kompatybilność przesyłanej telemetrii z posiadaną bazą danych stacji bazowych (BTS), zorientowaną wokół kluczowych dostawców infrastruktury w Polsce.
- **Ujednolicenie formatu czasu** – konwersja znaczników czasu urządzenia do standardu ISO-8601 UTC oraz automatyczne generowanie pola `received_at` w momencie zapisu rekordu do bazy danych.

Integracja z silnikiem przestrzennym PostGIS

Unikalną cechą warstwy utrwalania danych jest wykorzystanie funkcji przestrzennych do zapisu lokalizacji. System nie przechowuje współrzędnych jako zwykłych liczb, lecz tworzy obiekty typu `geography`. Wykorzystywana jest funkcja `ST_SetSRID(ST_MakePoint(lon, lat), 4326)`, która rzutuje współrzędne na sferoidę WGS84. Pozwala to na wykonywanie precyzyjnych zapytań przestrzennych wewnątrz bazy danych.

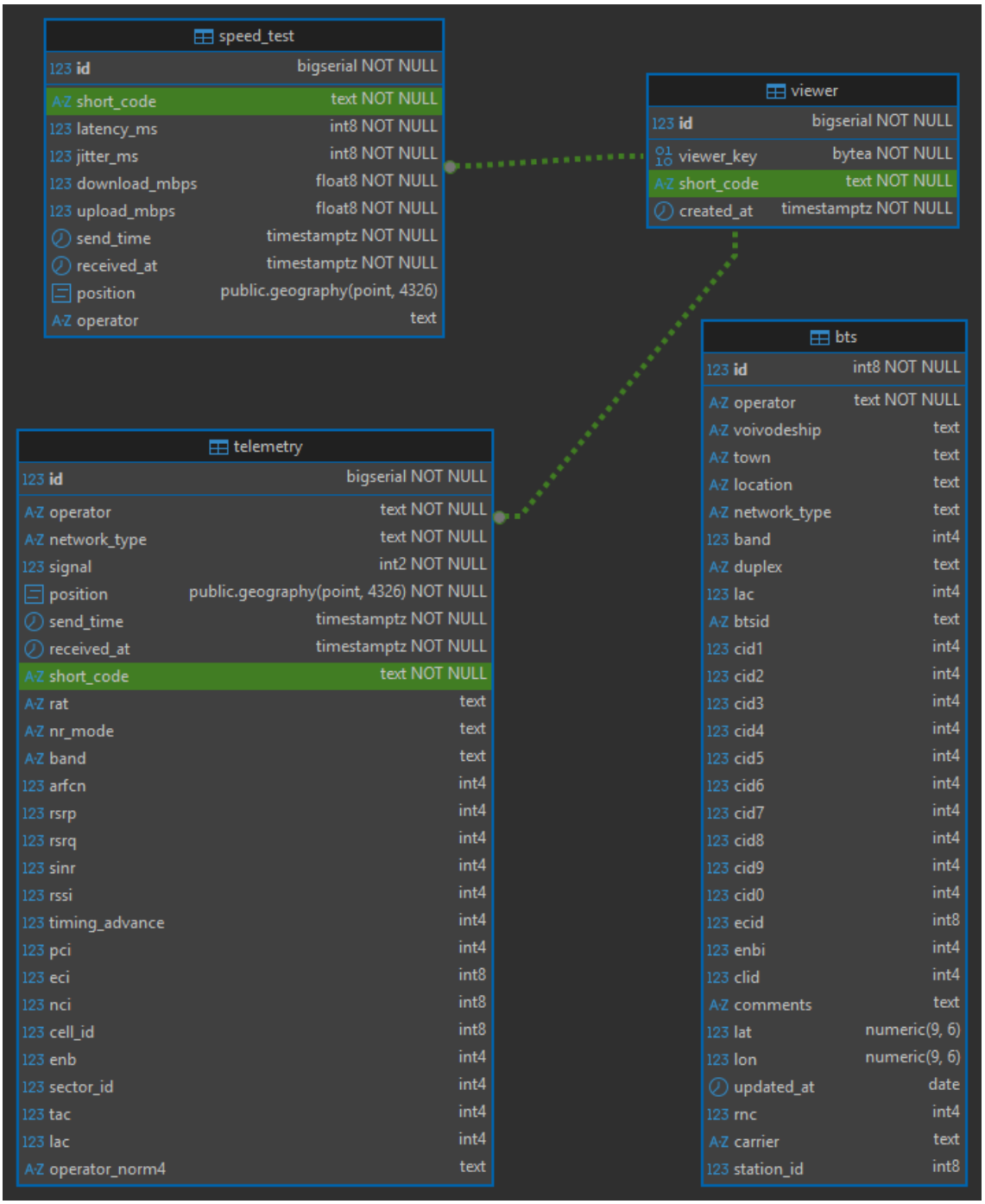
Trwałość danych i obsługa transakcji

Zapis danych do tabel `telemetry` oraz `speed_test` odbywa się w ramach transakcji bazodanowych z wykorzystaniem biblioteki `psycopg`.

- **Separacja strumieni danych** – system rozpoznaje rodzaj komunikatu (telemetria pasywna czy aktywny speedtest) na podstawie pola `kind` i kieruje je do odpowiednich tabel.
- **Odporność na błędy** – w przypadku błędów parsowania pojedynczego komunikatu, system loguje szczegóły błędu (np. błędy *diagnostics* z bazy danych), ale kontynuuje przetwarzanie pozostałej części paczki. Zapobiega to blokowaniu kolejki przez wadliwe rekordy.

4.3 Implementacja relacyjnej bazy danych

Struktura bazy danych została zaprojektowana w sposób umożliwiający efektywne składowanie dużych zbiorów danych telemetrycznych przy jednoczesnym zachowaniu relacji niezbędnych do analityki sygnału. Na poniższym diagramie (Rys. 4.9) przedstawiono fizyczny model bazy danych PostgreSQL.



Rysunek 4.9: Diagram encji systemu SignalMap z uwzględnieniem typów danych i więzów spójności.

4.3.1 Relacje fizyczne i powiązania logiczne

W architekturze bazy danych zastosowano hybrydowe podejście do relacyjności, różnicując powiązania między danymi użytkownika a danymi pomiarowymi. Relacja między dynamicznym strumieniem telemetrii a infrastrukturą operatorów ma charakter wyłącznie logiczny:

- **Logiczne powiązanie danych** – relacja między pomiarami (`telemetry`) a bazą stacji (`bts`) nie opiera się na fizycznym kluczu obcym. Pozwala to na niezależne gromadzenie telemetrii w czasie rzeczywistym względem statycznej bazy referencyjnej oraz zapobiega błędom przetwarzania w przypadku braku konkretnego rekordu BTS w bazie. Dzięki rezygnacji ze sztywnych więzów integralności, system zachowuje ciągłość zapisu danych pomiarowych nawet przy niekompletnych danych infrastrukturalnych.
- **Normalizacja operatorów** – pole `operator_norm4` jest uzupełniane przez logikę przetwarzającą dane. Standaryzuje ono nazwy operatorów do czterech głównych dostawców (zgodnie ze strukturą serwisu BTSearch), co ułatwia późniejsze filtrowanie i analizę wyników.
- **Dynamiczne mapowanie** – powiązanie pomiarów z konkretnymi stacjami bazowymi następuje dynamicznie po stronie serwera aplikacji (backendu) w momencie zapytania o dane. Takie podejście umożliwia elastyczną obsługę współdzielenia sieci (np. *RAN Sharing*). Mechanizm ten pozwala wielu operatorom na korzystanie z tej samej infrastruktury radiowej przy zachowaniu oddzielnych sieci szkieletowych, co wymaga od systemu analitycznego poprawnej identyfikacji fizycznego nadajnika niezależnie od nazwy operatora [182].

4.3.2 Typy danych i optymalizacja przestrzenna

Wszystkie tabele zostały zaprojektowane z wykorzystaniem typów danych optymalizujących wydajność operacji dyskowych oraz precyzję obliczeń:

- **Klucze i identyfikatory** – dla kluczy głównych zastosowano typ `BIGSERIAL`, oparty na 8-bajtowym formacie `BIGINT`. Zapewnia on zakres numeracji przekraczający 9×10^{18} rekordów, co gwarantuje skalowalność systemu przy masowym napływie telemetrii [183]. Do przechowywania kluczy kryptograficznych wykorzystano typ `BYTEA` dla danych binarnych, a dla unikalnych kodów użytkowników typ `TEXT` z indeksowaniem B-Tree.
- **Obsługa PostGIS** – kolumny `position` wykorzystują typ `GEOGRAPHY(Point, 4326)`. Zastosowanie rozszerzenia PostGIS oraz indeksów przestrzennych GIST umożliwia wykonywanie wydajnych zapytań geograficznych w standardzie WGS84, co pozwala na wizualizację punktów i filtrowanie pomiarów na podstawie ich lokalizacji.
- **Więzy spójności (CHECK)** – w celu zapewnienia integralności danych, tabele wyposażono w instrukcje `CHECK`. Weryfikują one, czy parametr `signal` mieści się w przedziale $[-150, 0]$ oraz czy pole `operator_norm4` zawiera wyłącznie dopuszczalne nazwy zdefiniowanych operatorów (*Orange, Play, Plus, T-Mobile, Unknown*), co eliminuje błędy na etapie zapisu transakcyjnego.

4.3.3 Optymalizacja zapytań i indeksowanie

W celu zapewnienia responsywność systemu przy rosnącej objętości danych, zaimplementowano strategię indeksowania dostosowaną do specyfiki zapytań przestrzennych:

- **Indeksy GIST (Generalized Search Tree)** – nałożone na kolumny geograficzne, umożliwiają błyskawiczne wyszukiwanie punktów w określonych obszarach geograficznych. Jest to kluczowy element optymalizacji backendu, zapobiegający kosztownemu skanowaniu całych tabel (ang. *Full Table Scan*).
- **Indeksy złożone i unikalne** – utworzono indeksy na polach `short_code` oraz `send_time`, co przyspiesza generowanie historii pomiarów dla użytkownika.

4.3.4 Rozszerzenia i funkcje systemowe

Wykorzystanie typu danych `TIMESTAMPTZ` zapewnia pełną spójność czasu między różnymi strefami czasowymi, co jest kluczowe w systemach rozproszonych zbierających dane z wielu urządzeń mobilnych. Implementacja ta stanowi fundament pod przyszłą skalowalność systemu na rynki zagraniczne, gwarantując poprawną chronologię i porównywalność pomiarów niezależnie od lokalizacji geograficznej użytkownika. Dodatkowo, dzięki zastosowaniu funkcji `now()` jako wartości domyślnej dla kolumny `received_at`, system automatycznie i precyzyjnie rejestruje moment dotarcia pakietu danych do chmury, co pozwala na późniejszą analizę opóźnień w transmisji poprzez porównanie czasu serwera z czasem wygenerowania pomiaru przez telefon.

4.4 Implementacja backendu

Backend aplikacji SignalMap został zaimplementowany jako RESTful API odpowiedzialne za przetwarzanie żądań HTTP, komunikację z bazą danych PostgreSQL oraz udostępnianie danych telemetrycznych dla warstwy wizualizacji.

4.4.1 Architektura aplikacji i routing

Aplikacja oparta jest na architekturze REST, gdzie każdy endpoint odpowiada konkretnej operacji na danych. Flask jako mikro-framework został wybrany celowo - brak narzuconej struktury projektu pozwolił na prostą, płaską architekturę plików, gdzie logika wszystkich endpointów znajduje się w pojedynczym pliku `app.py`. Backend również spełnia zadanie transformacji danych z bazy do formatu JSON.

Routing w Flask realizowany jest przez system dekoratorów, gdzie każda funkcja oznaczona dekoratorem `@app.route` staje się endpointem HTTP. Aplikacja udostępnia siedem głównych endpointów:

- `/healthz` - liveness probe dla Kubernetes, zwraca zawsze status 200 OK
- `/ready` - readiness probe, sprawdza dostępność bazy danych
- `/api/telemetry` - podstawowe dane telemetryczne
- `/api/bts` - lokalizacje stacji bazowych
- `/api/speedtest` - wyniki testów prędkości internetu
- `/api/telemetry-with-bts` - pomiary z automatycznym dopasowaniem stacji BTS

Wszystkie endpointy zwracają dane w formacie JSON z odpowiednimi kodami HTTP: 200 dla sukcesu, 503 dla niedostępności serwisu, 500 dla błędów wewnętrznych. Obsługa błędów jest zaimplementowana przez bloki try-except, które przechwytują wyjątki z bazy danych lub błędy parsowania i zwracają czytelne komunikaty użytkownikowi.

4.4.2 Zarządzanie konfiguracją

Cała konfiguracja aplikacji jest pobierana ze zmiennych środowiskowych. Obejmuje to parametry połączenia z bazą danych (host, port, nazwa bazy, użytkownik, hasło), tryb SSL oraz interwał monitoringu. Taki mechanizm zapewnia pełną separację między kodem a konfiguracją - ten sam obraz Docker może być uruchomiony w środowisku developerskim i produkcyjnym jedynie przez zmianę zmiennych środowiskowych.

W środowisku Kubernetes zmienne wrażliwe, takie jak hasło do bazy danych, są montowane z Azure Key Vault przez mechanizm Secrets Store CSI Driver. Sekrety są przekazywane do kontenera jako zmienne środowiskowe w momencie startu poda, co eliminuje potrzebę przechowywania haseł w plikach konfiguracyjnych lub w repozytorium Git. Pozostałe parametry, takie jak repozytorium czy port, są definiowane bezpośrednio w pliku values.yaml dla Helm Chart, ponieważ nie stanowią wrażliwych informacji.

4.4.3 Komunikacja z bazą danych i pula połączeń

Jednym z kluczowych elementów architektury jest implementacja puli połączeń do bazy danych. Nawiązanie nowego połączenia TCP z serwerem PostgreSQL jest operacją kosztowną czasowo co przy dużej liczbie żądań HTTP znacząco degradowałoby wydajność aplikacji.

Pula połączeń została skonfigurowana z minimalną liczbą dwóch stałych połączeń i maksymalną liczbą dziesięciu równoczesnych połączeń. Minimalna liczba oznacza, że dwa połączenia są utrzymywane nawet przy braku ruchu, co eliminuje opóźnienie przy pierwszym żądaniu po okresie bezczynności - aplikacja nie musi czekać na nawiązanie połączenia, ponieważ jest ono już gotowe w puli. Maksymalna liczba połączeń została dobrana jako limit ochronny, zapobiegający nadmiernemu obciążeniu serwera bazy danych i wyczerpaniu dostępnych zasobów.

Każde żądanie HTTP pobiera połączenie z puli na początku przetwarzania i obowiązkowo zwraca je po zakończeniu.

Połączenie z bazą danych odbywa się w sieci VNet, co oznacza, że ruch nigdy nie wychodzi poza wirtualną sieć prywatną Azure. DNS wewnętrzny AKS rozwiązuje nazwę bazy danych na prywatny adres IP z podsieci klastra, a połączenie jest szyfrowane przez TLS (parametr sslmode ustawiony na require). Taka konfiguracja zapewnia bezpieczeństwo danych oraz eliminuje opóźnienia związane z routingiem przez Internet publiczny.

4.4.4 Monitoring dostępności bazy danych

Backend implementuje dedykowany mechanizm monitorowania połączenia z bazą danych w tle, niezależny od obsługi ruchu HTTP. Uruchomiony jako osobny wątek daemon, mechanizm ten wykonuje proste zapytanie testowe do bazy co sześćdziesiąt sekund (wartość konfigurowalna przez zmienną środowiskową). Zapytanie to nie pobiera żadnych danych z bazy danych - jest to najprostsze możliwe zapytanie zwracające stałą wartość, którego celem jest wyłącznie weryfikacja, czy połączenie z bazą jest możliwe.

Po każdym udanym zapytaniu wątek aktualizuje zmienną globalną z aktualnym czasem UTC. Ta informacja jest wykorzystywana przez endpoint readiness probe, który zwraca status gotowości aplikacji. Jeśli ostatnie udane połączenie miało miejsce mniej niż dwie minuty temu, aplikacja uznaje się za gotową do obsługi ruchu i zwraca kod HTTP 200. Jeśli jednak od ostatniego udanego połączenia minęło więcej niż dwie minuty, zwracany jest kod 503 (Service Unavailable), sygnalizując Kubernetes że pod nie powinien otrzymywać nowego ruchu.

Wybór dwóch minut jako progu to kompromis między szybkim wykrywaniem problemów a tolerancją na krótkotrwałe zakłócenia. Krótszy próg (np. dziesięć sekund) mógłby powodować fałszywe alarmy przy chwilowych spowolnieniach sieci, podczas gdy dłuższy (np. dziesięć minut) oznaczałby, że aplikacja przez długi czas sygnalizuje gotowość mimo że baza nie odpowiada.

Wątek monitorujący wykorzystuje mechanizm Event zamiast prostego sleep do oczekiwania między iteracjami. Event może być ustawiony z zewnątrz, co natychmiast budzi wątek - to kluczowe dla graceful shutdown, gdzie chcemy jak najszybciej zakończyć wszystkie wątki po otrzymaniu sygnału SIGTERM od Kubernetes.

4.4.5 Implementacja endpointów danych

Endpoint telemetrii podstawowej

Endpoint `/api/telemetry` stanowi źródło danych zebranych pomiarów. Obsługuje trzy parametry filtrowania przekazywane w query string: zakres czasowy w minutach (domyślnie tysiąc czterysta czterdzieści, czyli dwadzieścia cztery godziny), limit liczby rekordów (domyślnie sto tysięcy), oraz operator sieci.

Filtrowanie czasowe realizowane jest przez klauzulę WHERE z funkcją interval PostgreSQL. Zapytanie pobiera tylko rekordy, których znacznik czasu jest nowszy niż aktualna data pomniejszona o zadany interwał. Sortowanie odbywa się według daty pomiaru malejąco, co zapewnia że przy zastosowaniu limitu pobierane są najnowsze pomiary.

Ekstrakcja współrzędnych geograficznych z typu geography odbywa się przez funkcje ST_X i ST_Y. Typ geography w PostgreSQL przechowuje punkt jako binarną strukturę, więc nie można bezpośrednio odczytać wartości szerokości i długości geograficznej - konieczne jest użycie dedykowanych funkcji PostGIS.

Wyniki z bazy danych są mapowane na strukturę JSON zgodną z konwencjami JavaScript - nazwy pól w formacie camelCase zamiast snake_case używanego w SQL, konwersja timestampów na format ISO 8601 z sufiksem Z oznaczającym UTC.

Endpoint telemetrii z dopasowaniem BTS

Najbardziej zaawansowany funkcjonalnie endpoint oraz endpoint z którego korzysta frontend `/api/telemetry-with-bts` implementuje trzy-etapowy algorytm łączenia pomiarów telemetrycznych z fizycznymi lokalizacjami stacji bazowych. Problem do rozwiązania polega na tym, że identyfikator ENB (eNodeB ID) raportowany przez telefon nie jest unikalny globalnie – wiele różnych fizycznych stacji może używać tego samego identyfikatora w różnych lokalizacjach.

W pierwszym etapie algorytm pobiera dane telemetryczne analogicznie jak podstawowy endpoint, ale dodatkowo zbiera unikalne wartości identyfikatorów ENB występujących w pobranych pomiarach. Wykorzystanie struktury danych set zapewnia, że każdy identyfikator pojawia się tylko raz, niezależnie od tego ile pomiarów z nim związanych zostało pobranych. Dla tysiąca pomiarów pochodzących z pięćdziesięciu różnych stacji, zbiór będzie zawierał tylko pięćdziesiąt unikalnych identyfikatorów.

Na przykładzie punktów z sieci LTE - drugi etap wykonuje zapytanie do tabeli BTS, pobierając wszystkie stacje których identyfikator ENB znajduje się w zbiorze z pierwszego etapu. Zapytanie wykorzystuje klauzulę IN z dynamicznie generowaną liczbą placeholderów - dla pięćdziesięciu identyfikatorów zostanie wygenerowane zapytanie z pięćdziesięcioma parametrami. Biblioteka psycopg2 automatycznie podstawia wartości i escapuje je, zapobiegając SQL injection nawet przy dynamicz-

nie budowanych zapytaniach. Wyniki są grupowane w słowniku, gdzie kluczem jest identyfikator ENB, a wartością lista wszystkich stacji o tym identyfikatorze.

Trzeci etap wykonuje dopasowanie stacji do pomiarów. Dla każdego pomiaru algorytm sprawdza czy istnieją stacje o odpowiednim ENB. Jeśli istnieje tylko jedna stacja, jest ona automatycznie przypisywana do pomiaru jeśli nie jest od niego w zbyt dużej odległości. Jeśli stacji jest więcej, następuje proces filtrowania i wyboru najbardziej prawdopodobnej lokalizacji.

Pierwszy filtr wykorzystuje kod LAC (Location Area Code), który identyfikuje obszar obsługi sieci komórkowej. Telefon raportuje LAC stacji z którą jest połączony, więc jeśli pomiar ma LAC o danej wartości, musi być obsługiwany przez stację z tego samego obszaru. Stacje z niezgodnym LAC są odrzucane z listy kandydatów. Wyjątkiem są sytuacje, gdy pomiar lub stacja nie mają określonego LAC (wartość NULL w bazie) - wtedy filtr jest pomijany i decyduje wyłącznie odległość geograficzna.

Drugi filtr oblicza odległość euklidesową między współrzędnymi pomiaru a każdą z pozostałych stacji kandydujących. Zastosowano uproszczony wzór odległości na płaszczyźnie kartezjańskiej.

Spośród stacji które przeszły oba filtry, wybierana jest ta o najmniejszej odległości od pomiaru. Jeśli żadna stacja nie spełnia kryteriów, pomiar pozostaje bez przypisanej lokalizacji BTS - takie dane są nadal zwracane do frontendu, ale bez pola relatedBts.

Powyżej opisano algorytm dla punktów zebranych w sieci LTE. Algorytm przypisywania stacji dla sieci 3G polega na znalezieniu odpowiedniej stacji na podstawie wartości: RNC (Radio Network Controller) oraz btsid. Z wartości cellid wyodrębnia się RNC poprzez dzielenie przez 65536, natomiast btsid otrzymuje się na podstawie wartości cellid oraz RNC. Następnie btsid dzieli się przez 10, w celu odrzucenia ostatniej cyfry. System tworzy parę (RNC, btsid) i szuka w bazie danych rekordów tabeli bts, gdzie zarówno RNC jak i btsid się zgadzają. Po znalezieniu kandydatów, algorytm waliduje je poprzez sprawdzenie kodu LAC (Location Area Code) i wybiera stację, która jest najbliższa pomiaru.

W przypadku sieci 2G algorytm jest znacznie prostszy. Z cellid pomiaru wyodrębnia się btsid poprzez dzielenie przez 10. System szuka w bazie danych rekordów, gdzie btsid się zgadza. Podobnie jak w sieci 3G, przeprowadza walidację LAC oraz wybiera stację spośród kandydatów, która jest położona najbliżej pomiaru. Główna różnica polega na tym, że sieć 3G wymaga dwóch identyfikatorów (RNC i btsid) dla precyzyjnego dopasowania, podczas gdy sieć 2G operuje tylko jednym identyfikatorem (btsid).

4.4.6 Bezpieczeństwo i walidacja danych

Wszystkie zapytania SQL wykorzystywane w aplikacji backendowej były realizowane z użyciem parametryzacji zapewnianej przez bibliotekę `psycopg2`. Zamiast wstawiania bezpośrednio wartości pochodzących do treści zapytania, zastosowano mechanizm placeholderów. Takie podejście powoduje, że biblioteka automatycznie przeprowadza proces escapowania znaków specjalnych, co skutecznie eliminowało możliwość przeprowadzenia ataku typu SQL Injection, nawet w przypadku przekazania złośliwych danych wejściowych w parametrach zapytań HTTP.

Parametry wejściowe przekazywane do endpointów były poddawane walidacji przed ich wykorzystaniem w zapytaniach do bazy danych. Wartości liczbowe były konwertowane do typu `integer` z obsługą wyjątków, natomiast parametry tekstowe, takie jak identyfikator operatora, były przekazywane wyłącznie jako parametry, bez dodatkowej walidacji.

Obsługa błędów została zrealizowana w sposób wielopoziomowy. Błędy wynikające z niepoprawnego formatu parametrów wejściowych skutkowały zwróceniem błędu wraz z czytelnym komunikatem błędu. Problemy związane z połączeniem z bazą danych lub wykonaniem zapytań SQL

były przechwytywane w warstwie aplikacyjnej, logowane do standardowego wyjścia oraz sygnalizowane użytkownikowi końcowemu kodem HTTP 500 (Internal Server Error).

4.4.7 Health checks i integracja z Kubernetes

Mechanizmy health checków zostały zaprojektowane zgodnie z zaleceniami Kubernetes, która rozróżnia dwa podstawowe typy sprawdzeń stanu aplikacji: *liveness* (czy aplikacja działa) oraz *readiness* (czy aplikacja jest gotowa obsługiwać ruch).

Endpoint liveness probe jest maksymalnie prosty - zwraca zawsze status OK jeśli proces Flask działa. Brak odpowiedzi na ten endpoint oznacza że cały proces uległ awarii i Kubernetes powinien zrestartować kontener. Nie sprawdza on dostępności bazy danych ani innych zależności, ponieważ jego zadaniem jest wyłącznie weryfikacja czy sam serwer http działa.

Endpoint readiness probe realizuje bardziej złożoną logikę i określa gotowość aplikacji do obsługi ruchu sieciowego na podstawie stanu połączenia z bazą danych. Jeżeli ostatnie poprawne połączenie z bazą danych miało miejsce wcześniej niż dwie minuty przed wykonaniem sprawdzenia, aplikacja sygnalizowała brak gotowości, mimo że sam proces Flask nadal działa. W takiej sytuacji Kubernetes usuwa pod z listy endpointów obiektu Service, co powoduje, że nowe żądania HTTP nie są kierowane do tej instancji aplikacji, natomiast trwające połączenia mogą zostać zakończone w sposób kontrolowany.

W odpowiedzi JSON readiness probe zawiera timestamp ostatniego udanego połączenia w formacie ISO 8601 z sufiksem Z oznaczającym UTC. To pozwala na szybką diagnostykę - widząc timestamp można określić dokładnie kiedy wystąpił problem z bazą danych. Sufiks Z jest istotny dla jednoznacznej interpretacji czasu niezależnie od strefy czasowej.

Wszystkie komunikaty logowane używają parametru flush równego True przy wywołaniu funkcji print. To wymusza natychmiastowy zapis bufora do wyjścia bez oczekiwania na wypełnienie bufora lub zakończenie programu. W środowisku kontenerowym Docker/Kubernetes jest to krytyczne - logi które pozostają w buforze w momencie nagłego zakończenia procesu (SIGKILL) zostają utracone i nigdy nie trafią do kubectl logs. Flush zapewnia że każda linia logu jest natychmiast widoczna dla użytkownika.

4.4.8 Kontrolowane zakończenie działania aplikacji

Mechanizm kontrolowanego zakończenia pracy aplikacji (graceful shutdown) został zaimplementowany w celu zapewnienia poprawnego zamykania procesów podczas aktualizacji typu rolling update lub skalowania klastra Kubernetes w dół. W przypadku braku takiego mechanizmu, po wysłaniu sygnału `SIGTERM` przez Kubernetes proces aplikacji mógłby zostać wymuszony do zakończenia po upływie domyślnego okresu *grace period*, co mogłoby skutkować przerwaniem obsługiwanych żądań HTTP oraz niepoprawnym zwolnieniem zasobów.

Aplikacja rejestruje handler dla sygnałów SIGTERM (wysyłany przez Kubernetes) i SIGINT (wysyłany przy Ctrl+C w terminalu). Handler ustawia globalny Event, który jest sprawdzany przez wątek monitorujący - Event.wait zamiast prostego sleep pozwala na natychmiastowe przebudzenie wątku. Dzięki temu wątek monitorujący kończy swoją pętlę natychmiast po otrzymaniu sygnału, zamiast czekać do końca interwału.

Flask w trybie threaded automatycznie dokańcza obsługę wszystkich aktywnych żądań HTTP po zakończeniu pętli głównej. Kubernetes czeka na zakończenie procesu przez grace period, więc requesty które trwają krócej niż trzydzieści sekund zostaną obsłużone normalnie. Requesty dłuższe niż grace period zostaną przerwane.

Po zakończeniu obsługi wszystkich requestów, Flask zamyka się z kodem zero (sukces). Kubernetes widzi czysty exit i nie traktuje tego jako crash - pod jest usuwany bez restartów. Połączenia w puli są automatycznie zamykane, więc nie ma wycieku połączeń.

4.4.9 Skalowanie i wdrażanie w Kubernetes

Backend został zaprojektowany jako aplikacja bezstanowa, co jest kluczowe dla możliwości horyzontalnego skalowania. Nie przechowuje żadnego stanu sesji użytkownika - każde żądanie HTTP jest całkowicie niezależne od poprzednich. Connection pool jest lokalny dla każdego poda, więc zwiększenie liczby replik nie powoduje konfliktów o zasoby.

Aplikacja backendu jest pakowana jako obraz Docker i publikowana w Azure Container Registry pod nazwą `map-backend` z tagiem wersji. Deployment w Kubernetes został skonfigurowany do uruchomienia dwóch replik (podów) z tego samego obrazu. Każdy pod to osobna instancja aplikacji działająca w izolowanym kontenerze z własnym procesem Python/Flask oraz własną pulą połączeń do bazy danych.

Service typu ClusterIP nasłuchuje na porcie 80 i przekierowuje ruch na port 8080 wewnątrz kontenerów. Service tworzy wirtualny adres IP oraz DNS wewnętrzny, który automatycznie rozdziela ruch między dostępne pody używając mechanizmu round-robin load balancing. Jeśli jeden pod przestanie odpowiadać na readiness probe, Service automatycznie przestaje kierować do niego ruch, przekierowując wszystkie żądania do pozostałej zdrowej repliki.

4.4.10 Proces wdrażania nowej wersji

Wdrożenie nowej wersji aplikacji backendu odbywa się poprzez aktualizację tagu obrazu Docker w pliku HelmRelease monitorowanym przez Flux. Proces przebiega następująco:

1. Developer zmienia wartość `image.tag` w pliku `HelmRelease.yaml` (np. z `"1.0.15"` na `"1.0.16"`) i commituje zmianę do repozytorium Git w Azure Repos.

2. Flux, który monitoruje repozytorium co jedną minutę (parametr `interval: 1m` w GitRepository), wykrywa zmianę podczas kolejnego cyklu reconciliation.

3. Flux automatycznie wykonuje operację `helm upgrade`, przekazując nową wartość tagu do Helm Chart. Helm generuje zaktualizowany manifest Deployment z nowym obrazem i aplikuje go do klastra.

4. Kubernetes wykrywa zmianę w specyfikacji Deployment (nowy tag obrazu) i inicjuje proces rolling update zgodnie ze skonfigurowaną strategią RollingUpdate.

5. Rolling update przebiega w kontrolowany sposób:

   - Kubernetes tworzy nowy pod z obrazem `map-backend:1.0.16`
   - Czeka aż kontener się uruchomi i readiness probe zwróci status OK
   - Dodaje nowy pod do puli endpointów Service - od tego momentu nowy pod zaczyna otrzymywać część ruchu
   - Wysyła sygnał SIGTERM do jednego ze starych podów z wersją `1.0.15`
   - Stary pod przestaje przyjmować nowy ruch (usunięty z Service), ale dokańcza obsługę aktywnych żądań HTTP
   - Po zakończeniu wszystkich requestów (lub po upływie grace period trzydzieści sekund) stary pod kończy się z exit code zero

- Proces jest powtarzany dla drugiej repliki

6. w każdym momencie rolling update co najmniej jeden pod obsługuje ruch, co zapewnia zero downtime z perspektywy użytkowników końcowych. Przez krótki okres działają równocześnie stare i nowe wersje aplikacji - dlatego ważne jest aby zmiany były wstecznie kompatybilne, szczególnie w kontekście struktury danych zwracanych przez API.

Cały proces od commitu w Git do pełnego wdrożenia nowej wersji w klastrze trwa zazwyczaj parę minut - wykrycie zmiany przez Flux (interwał 5 minut), Helm upgrade i Kubernetes reconciliation, oraz rolling update obu replik z zachowaniem bezpiecznych interwałów między zmianami.

Przywracanie poprzedniej wersji

W przypadku wykrycia błędu w nowej wersji aplikacji, rollback jest realizowany przez zmianę tagu obrazu w HelmRelease z powrotem na poprzednią wartość (np. z `"1.0.16"` na `"1.0.15"`) i commit do repozytorium Git. Flux wykrywa zmianę i wykonuje analogiczny proces rolling update, ale w kierunku starszej wersji.

Zasoby i limity

W pliku values.yaml dla Helm Chart zdefiniowane są zasoby dla każdego poda:

- **Requests** - gwarantowane minimum zasobów:
  - CPU: sto dwadzieścia osiem milicores (0.128 core)
  - RAM: sto megabajtów
- **Limits** - maksymalne zużycie zasobów:
  - CPU: pięćset milicores (0.5 core)
  - RAM: dwieście pięćdziesiąt sześć megabajtów

Wartości requests określają ile zasobów Kubernetes musi zagwarantować dla poda - scheduler nie uruchomi poda na nodzie, który nie ma dostępnych stu megabajtów RAM i sto dwadzieścia osiem milicores CPU. Wartości limits określają maksymalne zużycie. Te wartości zostały dobrane eksperymentalnie.

Automatyczne skalowanie

Deployment został skonfigurowany z Horizontal Pod Autoscaler (HPA), który automatycznie dostosowuje liczbę replik w zależności od obciążenia:

- **Minimum replik**: dwa - zawsze działają co najmniej dwa pody dla zapewnienia wysokiej dostępności
- **Maximum replik**: cztery - górny limit skalowania zapobiega wyczerpaniu zasobów klastra
- **Próg CPU**: siedemdziesiąt procent średniego wykorzystania - gdy średnie zużycie CPU wszystkich podów przekroczy siedemdziesiąt procent wartości requests, HPA doda nową replikę
- **Próg RAM**: siedemdziesiąt procent średniego wykorzystania - analogicznie dla pamięci

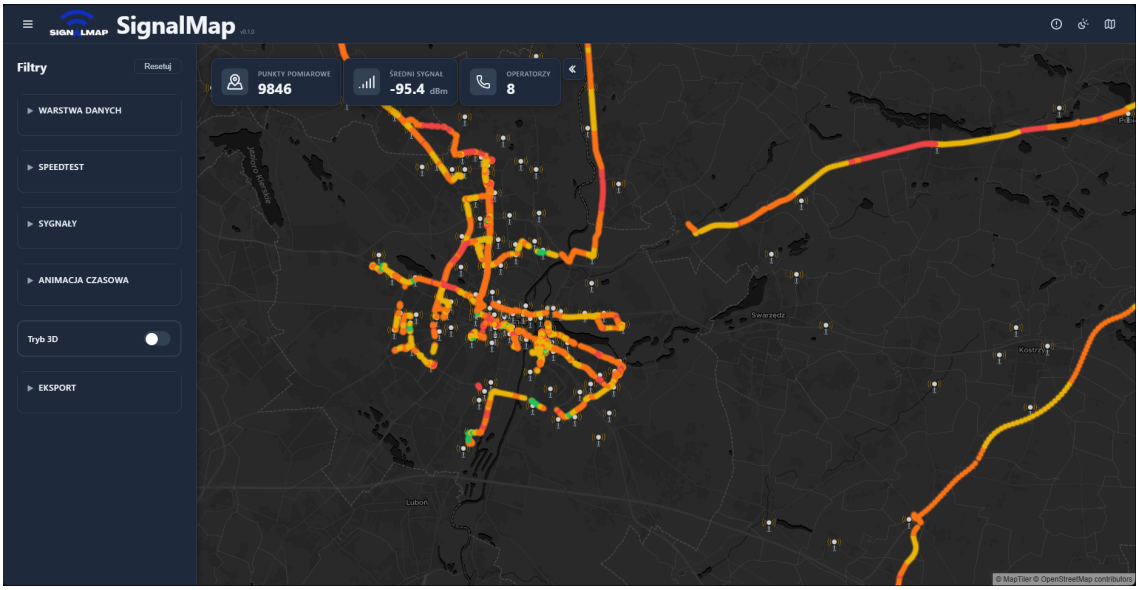
HPA sprawdza metryki co piętnaście sekund (domyślna wartość) i podejmuje decyzję o skalowaniu. Skalowanie w górę (dodanie poda) następuje po około trzydziestu sekundach ciągłego przekroczenia progu, podczas gdy skalowanie w dół (usunięcie poda) ma pięciominutową przerwe aby uniknąć niestabilności (flapping) przy zmiennym obciążeniu. [184]

Przykład działania autoskalowania: przy normalnym ruchu działają dwa pody, każdy zużywając około dwudziestu milicores CPU (piętnaście procent requests). Gdy ruch wzrasta i zużycie CPU osiąga dziewięćdziesiąt milicores na pod (siedemdziesiąt procent requests), HPA tworzy trzeci pod. Ruch jest wtedy rozdzielany między trzy pody, co obniża zużycie każdego poda. Po spadku ruchu i utrzymaniu się niskiego zużycia przez pięć minut, HPA usuwa nadmiarowy pod, wracając do dwóch replik.

Stateless charakter aplikacji sprawia, że skalowanie jest bezproblemowe - nowy pod może być dodany w dowolnym momencie bez synchronizacji stanu z istniejącymi podami. Każdy pod jest funkcjonalnie identyczny i może obsługiwać dowolne żądanie. Service automatycznie wykrywa nowe pody i zaczyna kierować do nich ruch, gdy tylko przejdą readiness probe.

4.5 Implementacja frontendu

Warstwa wizualizacji aplikacji została zaimplementowana jako Single Page Application z wykorzystaniem biblioteki React oraz biblioteki deck.gl, służącej do renderowania danych geograficznych. Jej zadaniem jest prezentacja danych pomiarowych na interaktywnej mapie, obejmujących zebrane punkty pomiarowe, informacje o prędkości połączenia internetowego (dalej nazywanych jako ”speedtest”) oraz o sile sygnału sieci komórkowej wraz z lokalizacją stacji bazowych. Dane te są prezentowane w postaci trzech niezależnych warstw na mapie, którymi użytkownik może manipulować poprzez zastosowanie odpowiednich filtrów.



Rysunek 4.10: Przykładowy wygląd aplikacji wizualizującej dane.

4.5.1 Architektura aplikacji

Architektura frontendu opiera się na komponentowym podejściu React z zarządzaniem stanem poprzez React Hooks. Interfejs użytkownika został zbudowany w oparciu o cztery główne komponenty: `Header`, `Sidebar`, `Map` oraz `Stats`, które komunikują się ze sobą za pomocą mechanizmu



Rysunek 4.11: Przykładowy wygląd aplikacji.

props oraz funkcji zwrotnych (*callback functions*). Mechanizm props umożliwia przekazywanie danych pomiędzy komponentami, natomiast funkcje zwrotne pozwalają na obsługę zdarzeń i aktualizację stanu aplikacji w odpowiedzi na interakcje użytkownika. Wizualizacja danych na mapach interaktywnych realizowana jest przy użyciu biblioteki Deck.gl w połączeniu z react-map-gl, które integrują się z usługą MapTiler zapewniającą bazowe mapy.

Kolejnym ważnym elementem systemu jest integracja frameworka Deck.gl z usługą MapTiler, pełniącą rolę dostawcy map, która jest realizowana za pośrednictwem interfejsu API. Framework DeckGL oferuje gotowe implementacje zaawansowanych technik wizualizacji przestrzennej, takich jak mapy ciepła, agregacja heksagonalna czy renderowanie trójwymiarowych słupków, które byłyby trudne do efektywnej implementacji od podstaw. MapTiler dostarcza podkłady map w ponad trzydziestu stylach wizualnych, nawigacyjnych czy satelitarnych co pozwala użytkownikowi dopasować wygląd mapy do własnych preferencji lub kontekstu analizy danych.

Warstwa prezentacji jest zbudowana jako komponent `Map` zawierający zagnieżdżone komponenty `DeckGL` oraz `Map` z biblioteki react-map-gl. Komponent DeckGL pełni rolę nadrzędnej warstwy odpowiedzialnej za renderowanie wszystkich wizualizacji danych telemetrycznych, podczas gdy react-map-gl zapewnia podkład mapy oraz obsługę podstawowych interakcji mapowych takich jak przybliżanie, przesuwanie czy obracanie widoku. Taka architektura warstwowa pozwala na niezależne zarządzanie warstwami wizualizacji danych i podkładem mapy.

4.5.2 Komunikacja z API i zarządzanie danymi

System implementuje strategię wstępnego ładowania danych przy inicjalizacji aplikacji. Wykorzystuje mechanizm React Hooks, konkretnie useEffect, który uruchamia zapytania HTTP do endpointów API. Domyślnie aplikacja pobiera dane telemetryczne z ostatnich dwóch miesięcy z limitem stu tysięcy rekordów, co uznano za kompromis między ilością danych a wydajnością renderowania.

Pobrane dane są przechowywane w stanie lokalnym komponentu App oraz w osobnej strukturze initialData, która służy jako punkt odniesienia przy operacjach resetowania filtrów. Taka architektura eliminuje konieczność ponownego odpytywania serwera przy każdej zmianie filtrów

lokalnych, znacząco poprawiając responsywność interfejsu.

Aplikacja implementuje reaktywny system filtrowania danych, który automatycznie reaguje na zmiany wprowadzane przez użytkownika. Hook useEffect nasłuchuje zmian w zakresie dat i, gdy użytkownik modyfikuje ten parametr, inicjuje nowe zapytanie do API z odpowiednimi parametrami. Mechanizm ten został zaimplementowany w sposób unikający nadmiarowych requestów poprzez sprawdzanie, czy faktycznie nastąpiła zmiana wartości filtru.

Szczególną uwagę poświęcono wydajności filtrowania danych po stronie klienta. Wykorzystano React.useMemo, który buforuje wyniki kosztownych wywołań funkcji. Dzięki temu filtrowanie tysięcy punktów według kryteriów takich jak operator, typ sieci, zakres sygnału czy prędkość transmisji odbywa się z minimalnym wpływem na responsywność interfejsu. Zapobiega to ponownemu obliczaniu tych samych wyników, gdy zmieniają się inne, niepowiązane części stanu aplikacji.

System integruje dane z trzech niezależnych źródeł: punkty pomiarowe telemetrii, dane o stacjach bazowych BTS oraz wyniki speedtestów. Każde źródło danych ma dedykowany endpoint API i posiada niezależny cykl życia w aplikacji. Dane BTS są ekstrahowane z obiektów telemetrii poprzez mapowanie relacji między punktami pomiarowymi a stacjami bazowymi, przy czym implementacja zapewnia unikalność stacji.

4.5.3 Komponent nagłówka

Komponent Header realizuje funkcję głównego paska nawigacyjnego i centrum kontroli ustawień aplikacji. Zapewnia dostęp do kluczowych funkcjonalności konfiguracyjnych poprzez możliwie intuicyjny interfejs ikon i rozwijanych opcji.

Aplikacja oferuje ponad trzydzieści różnych stylów kartograficznych MapTiler, zorganizowanych w różne kategorie tematyczne: nawigacyjne, terenowe, satelitarne, wizualizacyjne, topograficzne, artystyczne oraz inne. Menu wyboru stylu wykorzystuje wzorzec dropdown z nakładką blokującą interakcje z tłem. Aktualna selekcja jest podświetlana wizualnie, a zmiana stylu natychmiast propagowana do komponentu mapy poprzez callback onMapStyleChange.

System implementuje trzy tryby kolorystyczne interfejsu: jasny, ciemny oraz automatyczny. Tryb automatyczny dostosowuje schemat kolorów interfejsu do wybranego stylu mapy, analizując listę stylów klasyfikowanych jako jasne i odpowiednio przełączając klasę CSS dokumentu. Mechanizm ten został zrealizowany poprzez useEffect reagujący na zmiany właściwości mapStyle i themeMode.

Przełączanie motywu odbywa się poprzez manipulację klasą CSS light-mode na elemencie body dokumentu. Cała kolorystyka aplikacji została zdefiniowana przy użyciu zmiennych CSS, co umożliwia płynne przełączanie między schematami bez konieczności przeładowania komponentów. Komponenty nie muszą znać aktualnego motywu - po prostu używają zmiennych CSS, które są automatycznie aktualizowane.

4.5.4 Panel boczny z filtrami

Komponent Sidebar stanowi najbardziej złożony element interfejsu użytkownika, implementując system filtrowania wizualizacji danych. Został zaprojektowany jako komponent, gdzie cały stan filtrów jest przechowywany w komponencie nadrzędnym App, a Sidebar jedynie renderuje interfejs i emituje zdarzenia zmiany poprzez callback onFiltersChange.

Panel wykorzystuje wzorzec, w którym sekcje filtrów mogą być rozwijane i zwijane niezależnie. Taka organizacja pozwala na efektywne wykorzystanie ograniczonej przestrzeni ekranu, szczególnie na urządzeniach mobilnych. Każda sekcja akordeonowa posiada lokalny stan rozwinięcia zarządzany poprzez React useState.

Sekcja filtrów telemetrycznych umożliwia selekcję danych według następujących kryteriów: operatorzy (wielokrotnego wyboru, dynamicznie generowana lista na podstawie unikalnych wartości w zbiorze danych), typ sieci (selektor technologii sieci), zakres sygnału (dwustronne ograniczenie mocy sygnału w dBm za pomocą suwaka), limit wyświetlanych punktów, zakres dat (filtrowanie czasowe) oraz krótki kod identyfikujący punkty konkretnego użytkownika (tekstowe wyszukiwanie według identyfikatora).

Implementacja wykorzystuje dedykowane funkcje obsługi zdarzeń dla każdego typu filtru.

Panel implementuje system przełączników pozwalających na selektywne wyświetlanie trzech kategorii danych: telemetrii, stacji bazowych BTS oraz speedtestów. Każdy przełącznik modyfikuje obiekt visibleLayers w stanie filtrów, co natychmiast odzwierciedla się w widoku mapy. Dodano również opcję filtrowania punktów w celu uzyskania punktów bez przypisanych im stacji bazowych.

Sidebar oferuje kontrolę nad sposobem prezentacji danych telemetrycznych, umożliwiając wybór między czterema trybami wizualizacji: punkty (klasyczna reprezentacja każdego pomiaru jako oddzielny punkt), mapa ciepła (gęstość punktów pomiarowych), siatka heksagonalna (agregacja danych jako średniej w regularnej siatce sześciokątów) oraz słupki 3D (kolumnowa reprezentacja z wysokością proporcjonalną do mocy sygnału). Dodatkowo panel umożliwia aktywację trybu 3D, który wprowadza perspektywę do wizualizacji poprzez zmianę kąta nachylenia kamery.

Panel boczny zawiera także kontrolki animacji czasowej, która pozwala na odtworzenie chronologicznej sekwencji pomiarów. System animacji składa się z przełącznika włączenia trybu animacji, przycisku Play/Pause kontrolującego odtwarzanie, suwaka czasowego umożliwiającego nawigację do konkretnego momentu oraz kontrolki prędkości odtwarzania. Implementacja animacji opiera się na interwałowym aktualizowaniu znacznika czasu, według którego filtrowane są dane wyświetlane na mapie.

Panel udostępnia mechanizm eksportu przefiltrowanych danych w dwóch formatach: CSV oraz GeoJSON. Użytkownik może wybrać, które kategorie danych (telemetria lub speedtesty) mają zostać wyeksportowane. System generuje pliki po stronie klienta, bez konieczności angażowania serwera, co minimalizuje obciążenie infrastruktury. Format CSV jest generowany poprzez konwersje obiektów JavaScript do struktury tabelarycznej z nagłówkami kolumn, podczas gdy GeoJSON tworzy kolekcję cech geograficznych zgodną ze standardem.

4.5.5 Komponent mapy i system wizualizacji

Komponent MapView stanowi rdzeń wizualizacyjny aplikacji, integrując mapę bazową MapTiler z frameworkiem DeckGL. Inicjalizacja mapy określa centrum widoku na współrzędne geograficzne Poznania.

Aplikacja implementuje wielowarstwowy system renderowania danych przestrzennych, gdzie każda warstwa realizuje konkretny cel wizualizacyjny. Architektura oparta jest na kompozycji warstw DeckGL, które są renderowane w określonej kolejności.

Podstawowa wizualizacja telemetrii wykorzystuje dwie nakładające się warstwy ScatterplotLayer: warstwę koloru oraz warstwę umożliwiającą kliknięcie i zobaczenie szczegółów.

Kolorystyka punktów jest determinowana przez moc sygnału według, umownie przyjętej, czterostopniowej skali: zielony dla sygnału określonego jako bardzo dobry (większego lub równego minus siedemdziesiąt dBm), żółty dla sygnału dobrego (minus siedemdziesiąt do minus osiemdziesiąt pięć dBm), pomarańczowy dla sygnału średniego (minus osiemdziesiąt pięć do minus sto dBm) oraz czerwony dla sygnału słabego (mniejszego niż minus sto dBm).

Wizualizacja gęstości zebranych pomiarów, nazywanych dalej jako heatmap, wykorzystuje HeatmapLayer z biblioteki DeckGL, która agreguje gęstość występowania punktów.

HexagonLayer agreguje punkty w regularnej siatce sześciokątów, gdzie każdy heksagon reprezentuje statystyki agregowane dla punktów znajdujących się w jego obszarze. Implementacja wykorzystuje agregację tworząc średnią punktów z danego obszaru. W trybie 3D heksagony są prezentowane jako kolumny, gdzie wysokość kolumny jest proporcjonalna do średniej mocy sygnału.

Wizualizacja słupkowa wykorzystuje ColumnLayer do renderowania każdego punktu pomiarowego jako oddzielnej kolumny 3D. Wysokość słupka jest bezpośrednio mapowana na moc sygnału.

System obsługuje trzy tryby wizualizacji danych speedtest: punkty (ScatterplotLayer z kolorystyką zależną od prędkości pobierania), mapa ciepła (HeatmapLayer z wagami proporcjonalnymi do przepustowości) oraz słupki 3D (ColumnLayer z wysokością reprezentującą wybraną metrykę). Wizualizacja tych pomiarów pozwala wybrać między trzema metrykami: szybkość pobierania (download), szybkość wysyłania (upload) oraz opóźnienie (ping).

Stacje BTS są wizualizowane przy użyciu IconLayer z niestandardowym symbolem anteny w formacie SVG. Po kliknięciu w stację bazową aktywuje się warstwa LineLayer, która renderuje linie łączące wybraną stację z wszystkimi punktami pomiarowymi, które zostały zarejestrowane w jej zasięgu.

4.5.6 System interakcji użytkownika

Komponent implementuje system obsługi interakcji użytkownika z elementami mapy poprzez zdarzenia najechania kursorem oraz kliknięcia.

Najechanie kursorem na punkt pomiarowy aktywuje tooltip zawierający podstawowe informacje: operator, typ sieci, moc sygnału oraz lokalizację. Tooltip jest pozycjonowany dynamicznie względem kursora z zastosowaniem transformacji CSS zapobiegających wyjściu poza obszar widoku.

Kliknięcie w punkt pomiarowy wywołuje wyświetlenie panelu szczegółów zawierającego kompletną telemetrię pomiaru. Panel prezentuje dane pogrupowane w logiczne sekcje: sieć, parametry radiowe, informacje o komórce, lokalizacja oraz czas. Jeśli punkt posiada powiązaną stację bazową, panel zawiera również dedykowaną sekcję BTS z danymi stacji oraz przyciskiem umożliwiającym otwarcie dedykowanego panelu szczegółów stacji.

Kliknięcie w ikonę stacji lub użycie przycisku w panelu punktu otwiera panel szczegółów BTS, który prezentuje identyfikatory stacji, parametry techniczne, lokalizację oraz liczbę powiązanych pomiarów. Panel ten może współistnieć z panelem punktu, przy czym na urządzeniach desktopowych panele są pozycjonowane obok siebie, podczas gdy na urządzeniach mobilnych nakładają się.

Komponent MapView implementuje responsywność dostosowującą się do różnych rozmiarów ekranu. Na urządzeniach mobilnych panele szczegółów i filtrów są odpowiednio pozycjonowane.

4.5.7 Komponent statystyk

Komponent Stats realizuje funkcję dynamicznego panelu wyświetlającego zagregowane metryki całego zbioru danych. Panel został zaprojektowany jako zespół trzech kart prezentujących kluczowe statystyki: całkowitą liczbę załadowanych rekordów telemetrycznych, średnią ważoną mocy sygnału wyrażoną w dBm oraz liczbę unikalnych operatorów w zbiorze danych.

Panel implementuje funkcjonalność zwijania/rozwijania poprzez przycisk minimalizacji. W stanie zwiniętym widoczny jest jedynie przycisk z ikoną wykresu, co maksymalizuje przestrzeń dostępną dla widoku mapy. Przycisk pełni rolę przełącznika, gdzie każde kliknięcie zmienia stan widoczności.

4.5.8 Zarządzanie stanem i optymalizacja wydajności

Aplikacja nie wykorzystuje zewnętrznych bibliotek zarządzania stanem, zamiast tego opiera się na wbudowanym mechanizmie React hooks. Stan globalny jest zarządzany w komponencie App poprzez kompozycję wywołań useState, gdzie każde odpowiada za konkretny aspekt stanu aplikacji.

Stan aplikacji został logicznie podzielony na cztery kategorie: stan danych (data, initialData, btsData, speedtestData, loading, error), stan filtrów (obiekt zawierający wszystkie kryteria filtrowania), stan UI (sidebarOpen, mapStyle, themeMode, statsVisible) oraz stan animacji (isAnimationEnabled, isPlaying, animationTime, animationSpeed).

Komponenty potomne komunikują się z komponentem App poprzez funkcje callback przekazywane jako props. Typowy przepływ zmiany stanu wygląda następująco: użytkownik podejmuje interakcje z komponentem potomnym, komponent wywołuje callback z nowym obiektem filtrów, App aktualizuje stan, React uruchamia ponowne renderowanie wszystkich komponentów zależnych, komponenty obliczają nowe wartości poprzez useMemo.

4.5.9 Proces budowania i wdrożenia

Aplikacja wykorzystuje wieloetapowy proces budowania obrazu Docker (ang. multi-stage build), który optymalizuje rozmiar końcowego obrazu poprzez separację środowiska budowania od środowiska uruchomieniowego.

Pierwszy etap wykorzystuje obraz node 18 alpine jako bazę. W tym etapie kopiowane są pliki definicji zależności, wykonywana jest pełna instalacja zależności npm, kopiowany jest kod źródłowy aplikacji oraz uruchamiany jest proces budowania produkcyjnego.

Drugi etap wykorzystuje lekki obraz nginx 1.5 alpine i usuwa domyślne pliki konfiguracyjne Nginx, kopiuje niestandardową konfigurację serwera, kopiuje zbudowane pliki statyczne z etapu builder oraz definiuje healthcheck endpoint dla Kubernetes.

Konfiguracja implementuje mechanizm fallback dla React Router poprzez dyrektywę try files, która przekierowuje wszystkie nieistniejące ścieżki do index.html. Umożliwia to poprawne działanie client-side routingu dla Single Page Application. Serwer Nginx działa jako reverse proxy dla żądań API, przekierowując ruch z ścieżki /api/ do usługi backend w klastrze Kubernetes.

4.5.10 Wdrożenie w Kubernetes

Aplikacja jest wdrażana w klastrze Kubernetes przy użyciu Helm Charts i FluxCD. Chart został zdefiniowany z następującymi komponentami: Deployment (definicja podów aplikacji z obrazem kontenera), Service (ClusterIP udostępniający porty wewnątrz klastra), Ingress (reguły routingu ruchu zewnętrznego z nginx), HorizontalPodAutoscaler (automatyczne skalowanie od dwóch do czterech replik) oraz NetworkPolicy (reguły sieciowe kontrolujące komunikację).

Zasoby obliczeniowe podów zostały zdefiniowane z następującymi wartościami: Requests (sto megabajtów pamięci, sto dwadzieścia osiem milicores CPU) oraz Limits (dwieście pięćdziesiąt sześć megabajtów pamięci, pięćset milicores CPU). Takie ustawienie wystarczają dla stabilnego działania aplikacji przy jednoczesnej ochronie przed nadmiernym zużyciem zasobów klastra.

HorizontalPodAutoscaler został skonfigurowany do utrzymywania wykorzystania CPU poniżej siedemdziesięciu procent, wykorzystania pamięci poniżej siedemdziesięciu procent, minimum dwóch replik dla wysokiej dostępności oraz maksimum czterech replik dla ograniczenia kosztów.

Ingress wykorzystuje kontroler Nginx z automatycznym zarządzaniem certyfikatami TLS poprzez cert-manager. Konfiguracja zawiera automatyczne przekierowanie HTTP na HTTPS, cer-

tyfikat Let's Encrypt dla domeny signalmap.com.pl oraz routing wszystkich ścieżek do serwisu frontendu.

Wdrożenie jest zarządzane przez FluxCD w następującym modelu: HelmRelease definiuje źródło chart (GitRepository), Flux monitoruje repozytorium Git w interwałach pięciominutowych, każda zmiana w konfiguracji jest automatycznie synchronizowana z klastrem, co zapewnia poprawne wdrożenia.

4.5.11 Proces wdrażania nowej wersji

Wdrożenie nowej wersji aplikacji frontendu odbywa się poprzez aktualizację tagu obrazu Docker w pliku HelmRelease monitorowanym przez Flux. Należy zmienić wartość image tag w pliku HelmRelease.yaml a następnie wprowadzić tę zmianę do repozytorium Git. Flux, który monitoruje repozytorium co pięć minut, wykrywa zmianę podczas kolejnego cyklu reconciliation. Flux automatycznie wykonuje operację helm upgrade, przekazując nową wartość tagu do Helm Chart.

Kubernetes wykrywa zmianę w specyfikacji Deployment i inicjuje proces rolling update zgodnie ze skonfigurowaną strategią RollingUpdate. Rolling update przebiega w kontrolowany sposób: Kubernetes tworzy nowy pod z nowym obrazem, czeka aż kontener uruchomi się i readiness probe zwróci status OK, dodaje nowy pod do puli endpointów Service, wysyła sygnał SIGTERM do jednego ze starych podów, stary pod przestaje przyjmować nowy ruch ale dokańcza obsługę aktywnych żądań HTTP, po zakończeniu wszystkich requestów stary pod kończy się z exit code zero, proces jest powtarzany dla drugiej repliki.

W każdym momencie rolling update co najmniej jeden pod obsługuje ruch, co zapewnia zero downtime z perspektywy użytkowników końcowych. Cały proces od commitu w Git do pełnego wdrożenia nowej wersji w klastrze trwa parę minut.

4.6 CI/CD i zarządzanie kontenerami

Proces CI/CD jest nowoczesną metryka automatyzacji budowania i wdrażania aplikacji w podejściu GitOps. Całość można podzielić na dwa główne etapy: CI (Continous Integartion) oraz CD (Continous Deployment).

1. Etap CI - budowa i rejestracja obrazów
   W obu folderach aplikacji (frontendowej i backendowej) znajduję się podfolder z plikiem *azure-pipelines.yaml*, który definikuje konfigurację pipeline. Platformą, na której wykonywane są pipeline są agenty zainstalowane lokalnie na komputerach. Wykorzystano instalator znajdujący się pod adresem: `https://github.com/microsoft/azure-pipelines-agent/releases`. W panelu głównym Azure DevOps utworzona została nowa pula agentów, do której dodano agentów wcześniej zainstalowanych lokalnie na komputerze. Poszczególne kroki wykonywane w potoku przetwarzania (ang. *pipeline*):

   - Wyzwalanie - pipeline aktywował się za każdym razem, gdy następowała potwierdzona zmiana kodu (commit) na gałęzi main. Gdy zmiana kody była zatwierdzona na innej gałęzi pipeline nie uruchamiał się.
   - Wersjonowanie - konfiguracja pipeline została tak wykonana, aby mieć licznik, który tworzył unikalny tag dla obrazu. Początkowa wartość licznika to 1.0.0 - każda zmiana obrazu powodowała zwiększenie licznika o jeden.
   - Budowa obrazu - wykorzystywana jest komenda *az acr build*, agent bierze konkretne plik aplikacji i za pomocą pliku *dockerfile* buduje obraz.

- Publikacja do ACR - po poprawnym stworzeniu obrazu aplikacji agent za pomocą Service Connection stworzone i skonfigurowanego na platformie Azure DevOps łączy się z ACR i przesyła wybudowany obraz do konkretnego repozytorium obrazów.

2. Etap CD - wdrażanie aplikacji do klastra
   Za obsługę etapu CD odpowiedzialne jest narzędzie Flux. W momencie wprowadzenia zmian w kodzie odnoście wersji aplikacji, Flux wdraża ją ponowie do klastra. Poszczególne kroki działania wdrożenia przez Flux są następujące:

   - Monitorowanie – Flux posiada obiekty *ImageRepository i ImagePolicy*. Zadaniem tych obiektów jest ciągłe obserwowanie zawartości repozytorium.
   - Zmiana w Git – w momencie gdy Flux zauważy zmianę tagu obrazu, rozpoczyna mechanizm uzgadniania zgodności (ang. *reconcile*). Jego zadaniem jest połączenie się z ACR pod adresem: *inzynierka-hybud7ede0bqgaeg.azurecr.io*, aby pobrać nową wersje aplikacji i rozpocząć proces ponownego wdrażania z nowszym wydaniem.

4.7 Implementacja infrastruktury przez Terraform

W ramach projektu zaimplementowano proces wdrażania infrastruktury chmurowej w środowisku Microsoft Azure z wykorzystaniem narzędzia Terraform oraz pipeline'ów CI/CD w Azure DevOps. Proces realizowany jest na lokalnych agentach z puli *Win*, co umożliwia kontrolę nad środowiskiem uruchomieniowym oraz dostępem do zasobów sieciowych. Za pomocą Terraform zdefiniowano infrastrukturę w modelu IaC obejmującą klaster Azure Kubernetes Service (AKS), sieć wirtualną, podsieć, aplikacje serverless (Azure Functions), kolejkę komunikacyjną Azure Service Bus oraz Azure Database for PostgreSQL flexible server.

Klaster Kubernetes został utworzony jako zasób typu azurerm_kubernetes_cluster z włączonym mechanizmem automatycznego skalowania węzłów (Cluster Autoscaler). Zastosowano pulę węzłów bazującą na Virtual Machine Scale Sets (VMSS), co pozwala na dynamiczne dostosowanie liczby węzłów do aktualnego obciążenia klastra. Konfiguracja sieciowa polega na stworzeniu sieci wirtualnej (VNet) oraz podsieci przypisanej bezpośrednio do klastra AKS. Ustawiono także w Terraform natywny plugin Azure CNI.

W ramach infrastruktury wdrożono dwie aplikacje Azure Functions działające w środowisku Linux:

- aplikację typu Submit, odpowiedzialną za przyjmowanie żądań wejściowych,
- aplikację typu Worker, przetwarzającą zadania w sposób asynchroniczny.

Komunikacja pomiędzy aplikacjami realizowana jest za pomocą kolejki Azure Service Bus

Całość jest wdrażana przez pipeliny odpowiedzialne za inicjalizacje i wdrożenie usług na Azure. Pipeliny obejmują: Pierwszy pipeline Azure DevOps odpowiada za inicjalizację i wdrożenie infrastruktury Terraform. Pipeline ten:

- instaluje odpowiednią wersję Terraform,
- wykorzystuje mechanizm cache'owania pluginów Terraform,
- wykonanuje sekwencję terraform init, terraform plan oraz terraform apply tylko w przypadku braku istniejących usług

Konfiguracja Terraform tworzy zasób *azurerm_postgresql_flexible_server*, w którym określono parametry serwera, m.in. region, zasoby obliczeniowe (SKU), wersję PostgreSQL v16, rozmiar storage, retencję backupów oraz strefę dostępności. Dodatkowo utworzono bazę danych zasobu *azurerm_postgresql_flexible_server_database* z ustalonym kodowaniem UTF-8.

Drugi pipeline Azure DevOps odpowiada za inicjaliacje i wdrożenie samej bazy: Pipeline ten:

- instaluje odpowiednią wersję Terraform,
- wykorzystuje mechanizm cache'owania pluginów Terraform,
- sprawdza, czy serwer PostgreSQL już istnieje w danej Resource Group, jeśli istnieje, etap apply jest pomijany.
- konfiguruję serwis oraz bazę

W kroku wdrażania ustawiane są zmienne środowiskowe potrzebne dla providera Azure przy użyciu OIDC (ARM_USE_OIDC, ARM_CLIENT_ID, ARM_TENANT_ID, ARM_SUBSCRIPTION_ID). Hasło administratora bazy jest przekazywane jako sekret pipeline'u (DBPASSWORD) i mapowane na zmienną TF_VAR_admin_password, co pozwala utrzymać wrażliwe dane poza repozytorium.

Po utworzeniu serwera pipeline automatycznie konfiguruje parametr azure.extensions, aby włączyć rozszerzenia przestrzenne PostGIS. Dzięki temu baza jest przygotowana do pracy z danymi geograficznymi, co jest istotne w kontekście danych, które aplikacje webowa będzie przetwarzać.

W kolejnym kroku pipeline wykonuje inicjalizację struktury bazy poprzez uruchomienie pliku schema.sql za pomocą komendy *az postgres flexible-server execute*. Ponieważ pipeline działa na lokalnym agencie, a serwer PostgreSQL ma publiczny dostęp, pipeline dodaje tymczasową regułę zapory dopuszczającą publiczny adres IP agenta. Adres jest wykrywany automatycznie, po czym tworzona jest reguła firewall, wykonywany jest skrypt SQL, a następnie reguła jest usuwana. Takie podejście minimalizuje czas, w którym serwer jest dostępny z zewnątrz dla konkretnego hosta. Dodatkowo pipeline czeka na przejście serwera w stan Ready, co eliminuje ryzyko wykonywania inicjalizacji przed zakończeniem procesu provisioning.

4.8 Napotkane problemy i ich rozwiązanie

Podczas implementacji systemu SignalMap napotkano problemy, które wymagały niestandardowych rozwiązań. Poniżej opisano najważniejsze wyzwania wraz z przyjętym podejściem.

4.8.1 Problem: Niedokładne dopasowanie stacji BTS do pomiarów

Opis problemu:

Początkowa implementacja endpointu `/api/telemetry-with-bts` dopasowywała pomiary telemetryczne do stacji bazowych wyłącznie na podstawie identyfikatora ENB (eNodeB ID). Okazało się, że wiele pomiarów było błędnie przypisywanych do odległych stacji, mimo że istniały bliższe stacje z tym samym ENB.

Analiza danych wykazała, że jeden identyfikator ENB może odpowiadać paru fizycznym stacjom w różnych lokalizacjach.

Przykład problemu: Pomiar w Poznaniu był przypisywany do stacji w okolicy Jeleniej Góry mimo że w bazie istniała stacja z tym samym ENB bliżej Poznania blisko punktu pomiarowego.

Przyczyna:

Algorytm wybierał pierwszą napotkana stację z danym ENB z zapytania SQL, która niekoniecznie była najbliższa geograficznie.

Rozwiązanie:

Wprowadzono dwustopniowy algorytm filtrowania w Pythonie:

Krok 1: Filtrowanie po LAC (Location Area Code)

LAC to identyfikator obszaru obsługi w sieci komórkowej. Telefon raportuje LAC stacji, z którą się łączy.

Warunek `item["lac"] and bts["lac"]` jest konieczny, ponieważ niektóre pomiary lub stacje mogą mieć `NULL` w polu LAC (niepełne dane). W takim przypadku nie odrzucamy kandydata, tylko polegamy na odległości geograficznej.

Krok 2: Wybór najbliższej stacji geograficznie

Dla stacji które przeszły filtr LAC, obliczana jest odległość euklidesowa:

Listing 4.1: Obliczanie odległości

```
best_bts = None
best_distance = float('inf')

for bts in candidate_bts_after_lac_filter:
    if bts["lat"] and bts["lon"]:
        dist = ((item["latitude"] - bts["lat"])**2 +
                (item["longitude"] - bts["lon"])**2)**0.5

        if dist < 0.15 and dist < best_distance:
            best_distance = dist
            best_bts = bts

if best_bts:
    item["relatedBts"] = best_bts
```

Próg 0.15° został dobrany eksperymentalnie.

Wyniki:

Po implementacji algorytmu z LAC + odległość pozwoliła na eliminacje błędnych dopasowań. Możliwy jest jednak brak dopasowań ponieważ może być brak stacji o danym ENB w bazie BTS. Dodatkowo dane BTS pochodzą z zewnętrznego źródła (btsearch.pl), które nie pokrywa wszystkich stacji w Polsce oraz może zawierać błędy w numeracji ENB, duplikaty lub literówki.

4.8.2 Problem: Ograniczenia domyślnej autoryzacji platformy Azure

Opis problemu: Wstępne założenia projektowe zakładały wykorzystanie wbudowanych mechanizmów autoryzacji platformy *Azure App Service* (tzw. *Easy Auth*). Okazało się jednak, że standardowy sposób wydawania poświadczeń nie przystaje do specyfiki aplikacji *SignalMap*, która opiera się na anonimowym przesyłaniu danych pomiarowych bez klasycznego procesu logowania użytkownika.

Przyczyna: Głównym ograniczeniem okazał się sztywno zdefiniowany czas życia tokenów dostępowych, który w ekosystemie Azure domyślnie wynosi 60 minut. W przypadku architektury pozbawionej sesji użytkownika i opartej na stałym identyfikatorze *ShortCode*, tak długi interwał ważności stanowił istotny wektor ataku typu *replay*. Przechwycony token pozwalałby na nieautoryzowane przesyłanie fałszywych pomiarów przez pełną godzinę, a platforma nie oferowała wy-

starczająco elastycznych narzędzi do dynamicznego skracania tego czasu dla pojedynczych żądań bez wpływu na całą konfigurację dzierżawy (ang. *tenant*).

Rozwiązanie: Zdecydowano się na implementację mechanizmu generowania tokenów JWT bezpośrednio w ramach funkcji `fa-submit`. Zastosowano bibliotekę `PyJWT`, która umożliwiła ręczne definiowanie pola `exp` (*expiration time*) oraz pełną kontrolę nad zawartością ładunku (*payload*). Proces ten został zintegrowany z punktem końcowym `/gettoken`, który przed wydaniem poświadczenia weryfikuje poprawność klucza klienta.

Wynik: Wprowadzenie własnej logiki autoryzacji pozwoliło na osiągnięcie następujących rezultatów:

- **Redukcja okna czasowego ataku** – czas ważności tokena został skrócony z 60 do 15 minut, co o 75% ograniczyło czas, w którym przechwycone poświadczenie może zostać nadużyte.
- **Samowystarczalność** – system stał się niezależny od zewnętrznych dostawców tożsamości, co uprościło logikę odświeżania tokenów w kodzie aplikacji mobilnej (klasa *AzureClient*).

4.8.3 Migracja i ujednolicanie nieustrukturyzowanych danych BTSearch

Opis problemu: Proces zasilania bazy danych informacjami o lokalizacji stacji bazowych (BTS) napotkał trudności związane z niską jakością danych wejściowych pochodzących z serwisu zewnętrznego. Pliki w formacie CSV zawierały liczne niespójności, błędne typy danych w kolumnach numerycznych oraz niestandardowe formaty zapisu współrzędnych geograficznych, co uniemożliwiało bezpośredni import do docelowej, relacyjnej bazy danych.

Przyczyna: Zewnętrzne źródła danych (typu *crowdsourcing*) często charakteryzują się brakiem rygorystycznej walidacji. W plikach występowały puste pola w miejscach oczekiwanych liczb, znaki interpunkcyjne w polach identyfikatorów (*eNB ID*, *LAC*) oraz specyficzny format zapisu szerokości i długości geograficznej (np. `52N2445`). Próba bezpośredniego mapowania tych danych na typy `INTEGER` czy `NUMERIC` kończyła się błędami krytycznymi silnika bazy danych i przerwaniem importu.

Rozwiązanie: Zastosowano architekturę dwustopniowego importu danych z wykorzystaniem tabeli pośredniej:

1. **Tabela `bts_import`:** Utworzono tymczasową strukturę, w której każda kolumna zdefiniowana została jako typ `TEXT`. Pozwoliło to na bezbłędne wczytanie surowego pliku CSV bez ryzyka konfliktów typów danych.
2. **Mapowanie z walidacją:** Opracowano zaawansowane zapytanie `INSERT INTO ... SELECT`, które ujednolica dane (Listing 4.2).

Listing 4.2: Fragment procedury ETL ujednolicającej dane z tabeli pośredniej.

```
INSERT INTO public.bts (...)
SELECT
  -- Walidacja typow numerycznych przy uzyciu Regex
  CASE WHEN b.enbi ~ '^[0-9]+$' THEN b.enbi::INT ELSE NULL END,
  -- Konwersja formatu stopni i~minut na format dziesietny
  CASE WHEN b.lat_uk ~ '^[0-9]{2}N[0-9]{4}$' THEN
    round(substring(b.lat_uk from 1 for 2)::numeric +
          substring(b.lat_uk from 4 for 2)::numeric / 60 +
          substring(b.lat_uk from 6 for 2)::numeric / 3600, 6)
```

```
        ELSE NULL::numeric
    END,
    ...
FROM public.bts_import b
WHERE (b.enbi ~ '^[0-9]+$' OR b.stationid IS NOT NULL);
```

W rozwiązaniu wykorzystano wyrażenia regularne (ang. *Regex*) do sprawdzenia, czy tekst jest poprawną liczbą, oraz funkcje tekstowe (*substring*) do przeliczenia współrzędnych geograficznych z formatu stopniowo-minutowego na format dziesiętny.

Wynik: Wdrożenie mechanizmu pośredniego importu pozwoliło na:

- **100% skuteczności importu** – wyeliminowano błędy typu *Data Type Mismatch*, które wcześniej przerywały proces.
- **Ujednolicenie geolokalizacji** – wszystkie współrzędne zostały sprowadzone do spójnego formatu zmiennoprzecinkowego, co umożliwiło poprawne obliczanie odległości euklidesowej w algorytmie dopasowania BTS.
- **Integralność danych** – dzięki filtrowi `WHERE` w zapytaniu `SELECT`, do bazy docelowej trafiły wyłącznie rekordy posiadające kluczowe identyfikatory, co zapobiegło wprowadzania do systemu rekordów niepełnych.

4.8.4 Zapewnienie ciągłości transmisji danych w warunkach braku łączności

Opis problemu: Podczas testów terenowych systemu w warunkach dynamicznych (np. podczas przejazdów kolejowych) zaobserwowano zjawisko „dziur" w zarejestrowanej trasie pomiarowej. Mimo ciągłego działania aplikacji i poprawnego zbierania próbek przez urządzenie, na mapie wynikowej (Rys. 4.12) widoczne były wyraźne luki w miejscach, gdzie system nie odnotował żadnych rekordów.



Rysunek 4.12: Wizualizacja błędu utraty danych telemetrycznych w strefie braku zasięgu sieciowego.

Na rysunku 4.12 przedstawiono praktyczny skutek braku mechanizmu retencji danych po stronie aplikacji mobilnej. Luki w przebiegu trasy obrazują momenty, w których mimo pracy urządzenia, dane nie zostały dostarczone do serwera z powodu braku zasięgu sieci komórkowej. W pierwotnej wersji systemu brak mechanizmu kolejkowania powodował w takich sytuacjach bezpowrotną utratę rekordów. Na wizualizacji objawia się to nienaturalnymi przeskokami między zarejestrowanymi punktami, co uniemożliwia dokładne odwzorowanie rzeczywistej ścieżki poruszania się użytkownika.

Przyczyna: Standardowy mechanizm wysyłki pakietów bezpośrednio po ich wygenerowaniu okazywał się zawodny w strefach o słabym zasięgu lub całkowitym braku dostępu do Internetu. W momencie wystąpienia błędu sieciowego, pakiet danych był bezpowrotnie tracony, ponieważ aplikacja nie posiadała mechanizmu ponownej wysyłki pomiarów, których nie udało się dostarczyć do chmury w czasie rzeczywistym.

Rozwiązanie: W celu wyeliminowania strat danych wdrożono mechanizm kolejkowania pomiarów po stronie urządzenia z wykorzystaniem biblioteki *Android WorkManager*. Kluczowym elementem rozwiązania jest klasa `DataSendWorker`, która przejmuje odpowiedzialność za asynchroniczną wysyłkę danych (Listing 4.3).

Listing 4.3: Implementacja DataSendWorker wykorzystujaca mechanizm automatycznego ponawiania prob.

```
class DataSendWorker(
    appContext: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(appContext, workerParams) {
    override suspend fun doWork(): Result {
        val jsonString = inputData.getString("payload")
            ?: return Result.failure()
        return try {
            val response = AzureClient.sendToAzure(JSONObject(jsonString))
            if (response.contains("Error")) {
                Result.retry()
            } else {
                Result.success()
            }
        } catch (e: Exception) {
            Result.retry()
        }
    }
}
```

Nowa architektura opiera się na dwóch filarach:

1. **Buforowanie lokalne:** Każdy pomiar, zamiast bezpośredniej wysyłki, trafia do systemowej kolejki zadań urządzenia.

2. **Automatyczne ponawianie prób:** w przypadku niepowodzenia transmisji, system nie porzuca pakietu, lecz planuje kolejną próbę z rosnącym odstępem czasowym. Dzięki więzom `NetworkType.CONNECTED`, *WorkManager* wstrzymuje próby wysyłki do momentu odzyskania stabilnego połączenia z Internetem.

Wynik: Wprowadzenie kolejki buforującej w telefonie pozwoliło na:

- **Eliminację luk w danych** – pomiary zebrane w tunelach lub strefach braku zasięgu są automatycznie przesyłane natychmiast po odzyskaniu łączności.
- **Odporność na błędy serwera** – w przypadku tymczasowej niedostępności backendu (błędy klasy 5xx), dane pozostają bezpieczne w pamięci urządzenia do czasu przywrócenia sprawności systemu.

4.9 Testy implementacji

W niniejszym rozdziale opisano proces weryfikacji kluczowych mechanizmów systemu telemetrycznego. Testy zostały przeprowadzone w celu potwierdzenia poprawności implementacji zabezpieczeń, walidacji danych wejściowych oraz mechanizmów zapewniających spójność danych w rozproszonym środowisku chmurowym.

4.9.1 Weryfikacja procesu rejestracji i autoryzacji urządzeń

Proces rejestracji stanowi punkt wejścia do systemu, dlatego został poddany szczegółowej weryfikacji pod kątem odporności na błędne dane oraz próby nieautoryzowanego dostępu.

Walidacja identyfikatorów sprzętowych Android ID

Zgodnie z założeniami projektowymi, punkt końcowy */register* akceptuje wyłącznie 64-bitowe identyfikatory Android ID, reprezentowane jako 16-znakowe ciągi w formacie szesnastkowym (HEX). W celu zweryfikowania, czy mechanizmy obronne API działają poprawnie, przeprowadzono serię testów sprawdzających reakcję systemu na dane wejściowe. Wykorzystano do tego przygotowane skrypty PowerShell, które symulowały rzeczywiste żądania HTTP POST, przesyłając do serwera zarówno poprawne parametry, jak i celowo zniekształcone dane, mające na celu próbę obejścia walidacji. Szczegółowe zestawienie scenariuszy testowych oraz oczekiwanych odpowiedzi serwera przedstawiono w tabeli 4.1.

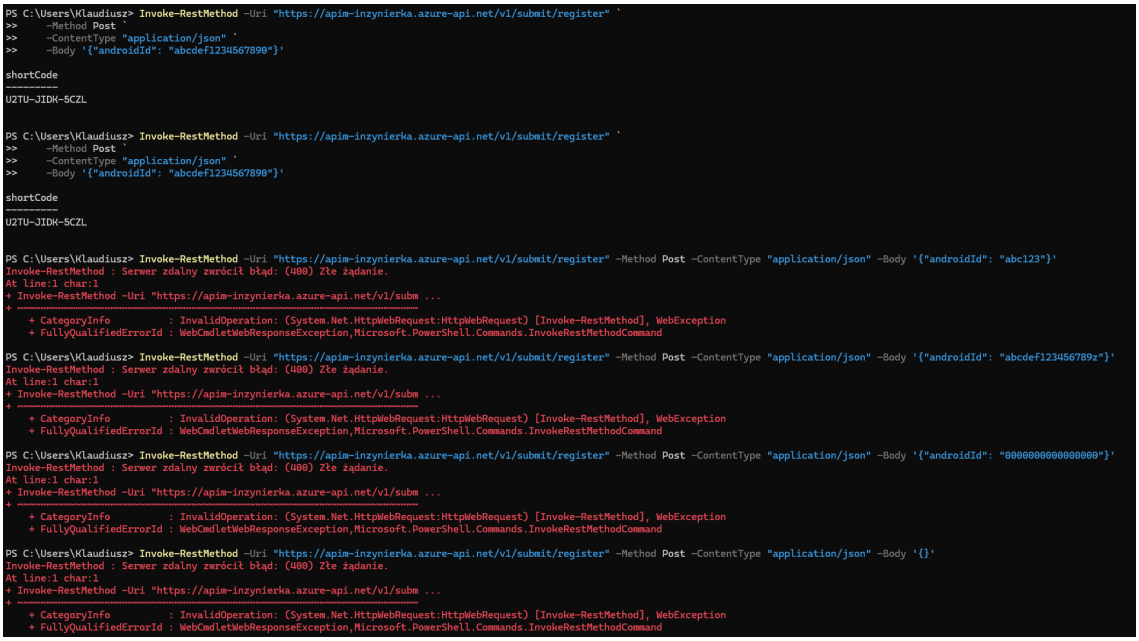
Tabela 4.1: Scenariusze testowe walidacji Android ID w punkcie /register.

ID	Opis testu	Dane wejściowe	Status HTTP
TC-01	Poprawna rejestracja nowego urządzenia	”abcdef1234567890”	201 Created
TC-02	Weryfikacja idempotencji (duplikat ID)	”abcdef1234567890”	200 OK
TC-03	Błędna długość identyfikatora (zbyt krótki)	”abc123”	400 Bad Request
TC-04	Znaki spoza alfabetu szesnastkowego	”abcdef123456789z”	400 Bad Request
TC-05	Identyfikator systemowy (same zera)	”0000000000000000”	400 Bad Request
TC-06	Próba wysłania pustego obiektu JSON	{ }	400 Bad Request

Weryfikacja poprawności walidacji została przeprowadzona przy użyciu skryptu, który przesłał sekwencję żądań odpowiadającą wszystkim zdefiniowanym przypadkom testowym (TC-01 – TC-06). Faktyczny wynik operacji, zarejestrowany bezpośrednio w konsoli systemowej, przedstawiono na rysunku 4.13.

Analiza zrzutu ekranu potwierdza pełną skuteczność zaimplementowanych filtrów:

- Pierwsze wywołanie (TC-01) zakończyło się sukcesem, a system wygenerował i zwrócił unikalny identyfikator *shortCode* o wartości `U2TU-JIDK-5CZL`.
- Drugie wywołanie (TC-02) z tym samym identyfikatorem zwróciło identyczny kod bez tworzenia nowej instancji w bazie danych, co potwierdza poprawność zaimplementowanego mechanizmu idempotencji.
- Pozostałe próby (TC-03 – TC-06), zawierające dane niezgodne ze specyfikacją, zostały natychmiastowo przerwane przez backend. Serwer zwrócił status *400 Bad Request* (widoczny w konsoli jako błąd polecenia *Invoke-RestMethod* zaznaczony kolorem czerwonym), co udowadnia, że niepoprawne dane nie są dopuszczane do dalszych warstw logiki biznesowej ani zapisu w relacyjnej bazie danych PostgreSQL.



Rysunek 4.13: Zbiorczy wynik testów walidacji punktu końcowego /register w konsoli PowerShell.

Autoryzacja wydawania tokenów

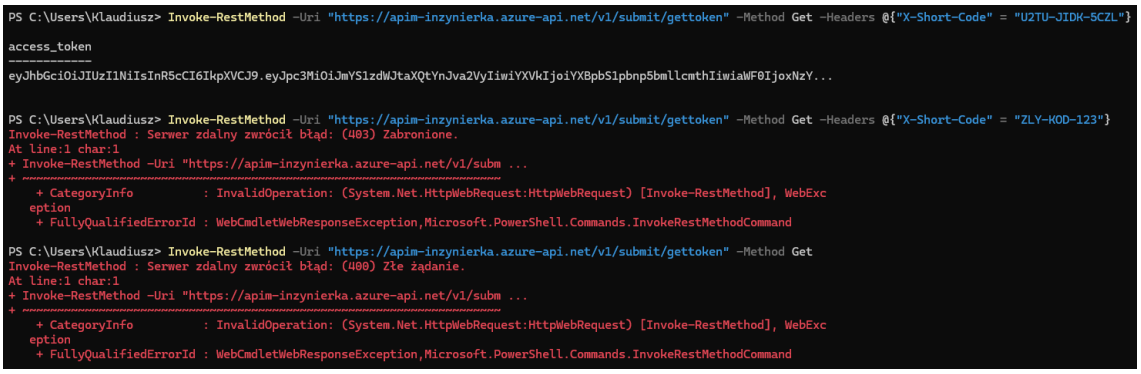
Proces *handshake* polega na wymianie kodu *shortCode* na token JWT, co stanowi implementację wzorca uwierzytelniania opartego na standardzie OAuth 2.0 [172]. Weryfikacja ta odbywa się na punkcie końcowym */gettoken*, który pełni rolę brokera wydającego poświadczenia wyłącznie zidentyfikowanym urządzeniom. Poszczególne przypadki testowe weryfikujące poprawność wydawania tokenów dostępowych zostały zestawione w tabeli 4.2.

Tabela 4.2: Scenariusze testowe mechanizmu wydawania tokenów JWT.

ID	Opis testu	Nagłówek X-Short-Code	Status HTTP
TC-07	Pobranie tokena dla poprawnego kodu	”U2TU-JIDK-5CZL”	200 OK
TC-08	Próba użycia nieistniejącego kodu	”ZLY-KOD-123”	403 Forbidden
TC-09	Próba wywołania bez nagłówka	(brak nagłówka)	400 Bad Request

Faktyczny wynik operacji zarejestrowany w konsoli PowerShell przedstawiono na rysunku 4.14. System poprawnie zidentyfikował zarejestrowane urządzenie (TC-07), zwracając obiekt JSON z klu-

czem `access_token`. Próba użycia błędnego kodu (TC-08) zakończyła się błędem *403 Forbidden*, natomiast całkowity brak nagłówka (TC-09) skutkował statusem *400 Bad Request*.



Rysunek 4.14: Wyniki weryfikacji punktu /gettoken: poprawna generacja tokena JWT oraz blokada nieautoryzowanych żądań.

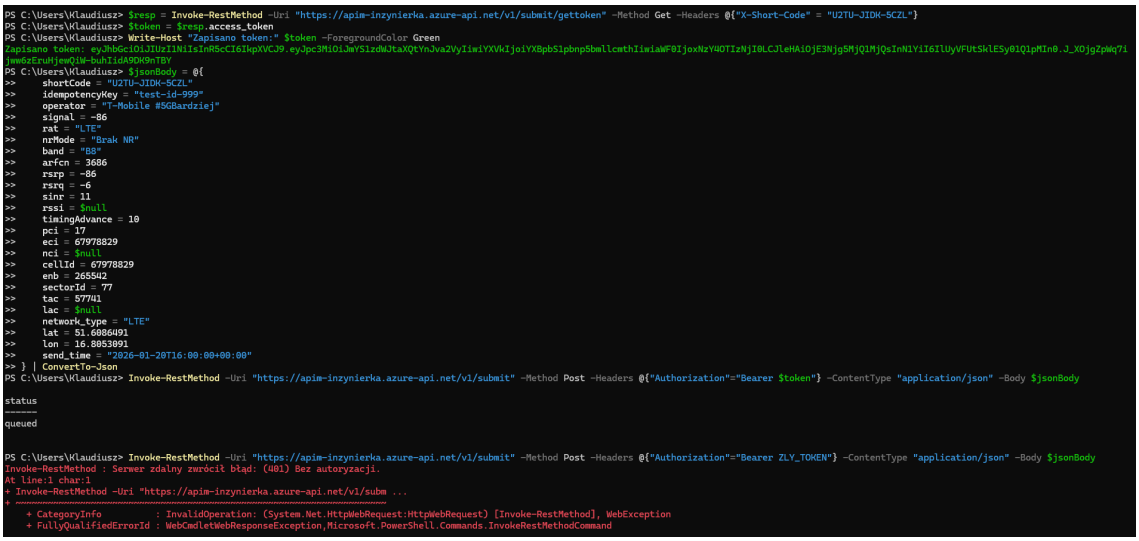
Weryfikacja autoryzacji tokenem JWT i przesyłania danych

Po pomyślnym uzyskaniu poświadczeń, przeprowadzono testy wysyłki pełnego rekordu telemetrycznego do punktu końcowego */submit*. Weryfikacja miała na celu potwierdzenie integralności danych oraz sprawdzenie odporności systemu na użycie nieważnych tokenów. Scenariusze testowe uwzględniające różne stany autoryzacji zostały zestawione w tabeli 4.3.

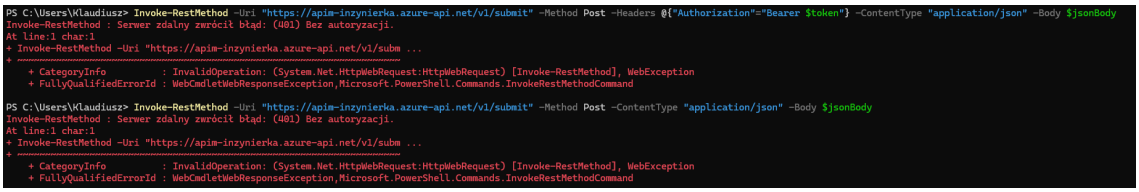
Tabela 4.3: Scenariusze testowe autoryzacji i wysyłki danych.

ID	**Opis testu**	**Parametr autoryzacji**	**Status HTTP**
TC-10	Poprawna wysyłka pomiaru	Ważny token JWT	202 Accepted
TC-11	Próba wysyłki ze zmienionym tokenem	”Bearer ZLY_TOKEN”	401 Unauthorized
TC-12	Próba wysyłki z wygasłym tokenem	Token po upływie 15 min	401 Unauthorized
TC-13	Brak nagłówka autoryzacji	(brak nagłówka)	401 Unauthorized

Analiza przeprowadzonych testów, których przebieg zarejestrowano na rysunkach 4.15 oraz 4.16, potwierdza poprawność implementacji mechanizmu autoryzacji *Bearer*. Serwer poprawnie akceptuje żądania wyłącznie z ważnym tokenem, zwracając status *202 Accepted*, co jest sygnałem przyjęcia danych do kolejki przetwarzania asynchronicznego. W przypadku wykrycia nieprawidłowości, takich jak manipulacja treścią tokena (TC-11) lub jego wygaśnięcie (TC-12), system skutecznie blokuje dostęp, zwracając kod błędu *401 Unauthorized*.



Rysunek 4.15: Zapis sesji PowerShell: Poprawny proces autoryzacji oraz odrzucenie żądania z nieprawidłowym tokenem.



Rysunek 4.16: Zapis sesji PowerShell: Odrzucenie żądań przez serwer ze względu na brak lub nieprawidłowość tokenu JWT.

Walidacja typów i zakresów danych telemetrycznych

Po pomyślnej autoryzacji tokenem JWT, system poddaje przesłany obiekt JSON walidacji typów oraz zakresów logicznych. Celem tego mechanizmu jest zapewnienie integralności danych w warstwie składowania (PostgreSQL) oraz eliminacja błędów przed przekazaniem wiadomości do kolejki asynchronicznej.

W tabeli 4.4 zestawiono scenariusze testowe weryfikujące odporność punktu końcowego */submit* na nieprawidłowe formaty danych.

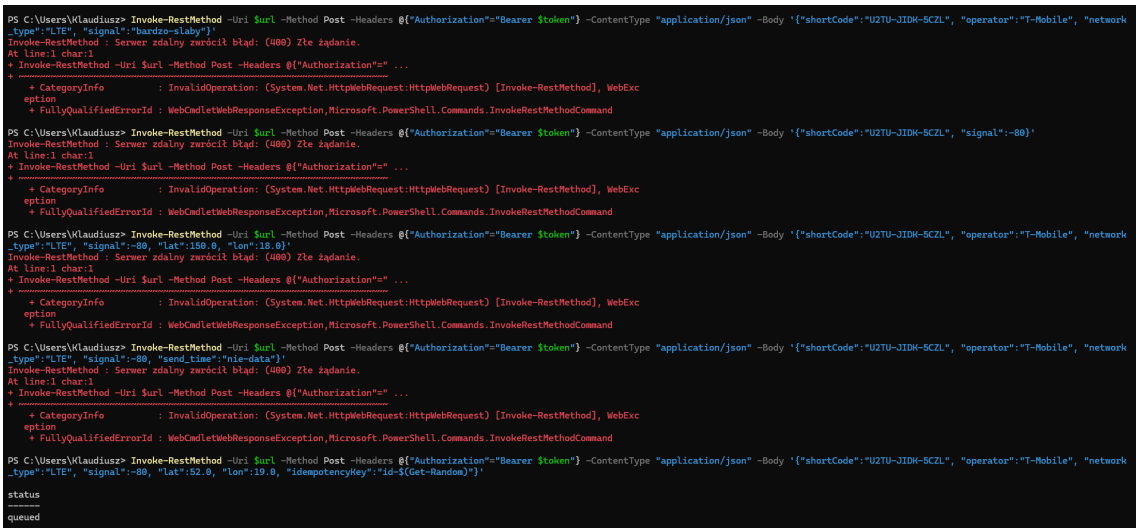
Tabela 4.4: Scenariusze testowe walidacji danych wejściowych.

ID	Opis testu	Dane wejściowe	Oczekiwany wynik
TC-14	Nieprawidłowy typ sygnału	”signal”: ”bardzo-słaby”	400 Bad Request
TC-15	Brak wymaganych pól	brak pola ”network_type”	400 Bad Request
TC-16	Współrzędne poza zakresem	”lat”: 150.0 (max 90.0)	400 Bad Request
TC-17	Niepoprawny format czasu	”send_time”: ”nie-data”	400 Bad Request

Analiza zarejestrowanej sesji testowej w środowisku PowerShell (Rysunek 4.17) potwierdza pełną operacyjność mechanizmów walidacyjnych zaimplementowanych w funkcji backendowej. Każdy z zaplanowanych scenariuszy negatywnych zakończył się zwróceniem przez serwer zdalny błędu *400 Bad Request* (widoczny na czerwono komunikat *Złe żądanie*), co wynika z natychmiastowego przerwania przetwarzania po wykryciu niezgodności ze schematem danych:

- **TC-14 (Nieprawidłowy typ):** Próba przesłania wartości tekstowej *"bardzo-slaby"* w polu przeznaczonym dla liczby całkowitej (*signal*) została odrzucona. System rygorystycznie wymusza typ numeryczny, zapobiegając błędom konwersji w bazie danych.
- **TC-15 (Brak pól):** Wysłanie obiektu pozbawionego parametrów *operator* oraz *network_type* skutkowało odmową przyjęcia zgłoszenia, co potwierdza poprawność walidacji pól obowiązkowych.
- **TC-16 (Błąd zakresu):** Wprowadzenie wartości szerokości geograficznej równej *150.0* (poza zakresem -90 do 90) zostało zidentyfikowane jako błąd logiczny, co gwarantuje spójność danych przestrzennych.
- **TC-17 (Błąd formatu czasu):** Przekazanie ciągu *"nie-data"* zamiast standardu ISO-8601 uniemożliwiło przetworzenie rekordu.

Zwieńczeniem weryfikacji jest ostatnie wywołanie widoczne na rysunku 4.17, w którym przesłano w pełni poprawny obiekt JSON. System zwrócił status `queued`, co oznacza pomyślne przejście autoryzacji oraz walidacji, a następnie przekazanie wiadomości do *Azure Service Bus*.



Rysunek 4.17: Zapis sesji testowej: odrzucenie danych telemetrycznych ze względu na błędy typów oraz poprawna akceptacja zwalidowanego żądania.

Eliminacja duplikatów i mechanizm Duplicate Detection

W systemach mobilnych, ze względu na niestabilność połączenia sieciowego, często dochodzi do retransmisji tych samych pakietów danych. Aby zapobiec duplikowaniu rekordów w bazie danych PostgreSQL, proces ten jest nadzorowany już na etapie generowania komunikatu w aplikacji na systemie Android. Każdy pakiet otrzymuje unikalny identyfikator, który pozwala na skuteczne działanie mechanizmu *Duplicate Detection* w usłudze *Azure Service Bus*. Dzięki temu, nawet jeśli telefon wyśle ten sam pomiar kilkukrotnie z powodu błędów sieci, chmura rozpozna powtórzony identyfikator i odrzuci duplikaty przed ich zapisaniem.

Do weryfikacji wykorzystano unikalne klucze *idempotencyKey* generowane przez aplikację mobilną dla każdego pomiaru. Scenariusz testowy (TC-12) obejmował:

1. Wysyłkę pomiaru z kluczem K_1.
2. Natychmiastową retransmisję identycznego pomiaru z tym samym kluczem K_1.

Skuteczność mechanizmu potwierdzają zestawione wyniki testów. Na rysunku 4.18 przedstawiono zapis sesji w konsoli PowerShell, gdzie widoczne są dwa następujące po sobie żądania do punktu końcowego */submit*. W obu przypadkach system zwrócił status `202 Accepted` z komunikatem `queued`.

Kluczowy dowód poprawności działania infrastruktury chmurowej przedstawiono na rysunku 4.19, prezentującym wynik zapytania SQL do bazy danych PostgreSQL. Mimo dwukrotnego przesłania danych do kolejki, w tabeli *telemetry* zarejestrowano wyłącznie jeden unikalny rekord o identyfikatorze `12125`. Wynika to z faktu, że usługa *Azure Service Bus* rozpoznała identyczny identyfikator wiadomości (*MessageId*) zmapowany z klucza *idempotencyKey* i po cichu odrzuciła duplikat, nie przekazując go do funkcji typu *Worker*. Mechanizm ten skutecznie chroni warstwę składowania przed redundancją danych i gwarantuje poprawność statystyk pomiarowych.

```
PS C:\Users\Klaudiusz> # 1. Przygotowanie danych
PS C:\Users\Klaudiusz> $url = "https://apim-inzynierka.azure-api.net/v1/submit"
PS C:\Users\Klaudiusz> $idKey = "unikalny-klucz-123"
PS C:\Users\Klaudiusz>
PS C:\Users\Klaudiusz> $body = @{
>>     shortCode = "U2TU-JIDK-5CZL"
>>     idempotencyKey = $idKey
>>     operator = "T-Mobile"
>>     network_type = "LTE"
>>     signal = -80
>>     lat = 52.0
>>     lon = 19.0
>> } | ConvertTo-Json
PS C:\Users\Klaudiusz>
PS C:\Users\Klaudiusz> # 2. Pierwsza wysyłka (Pierwotna)
PS C:\Users\Klaudiusz> Write-Host "Wysyłka 1:" -ForegroundColor Cyan
Wysyłka 1:
PS C:\Users\Klaudiusz> Invoke-RestMethod -Uri $url -Method Post -Headers @{"Authorization"="Bearer $token"} -ContentType "application/json" -Body $body

status
------
queued


PS C:\Users\Klaudiusz>
PS C:\Users\Klaudiusz> # 3. Druga wysyłka (Retransmisja/Duplikat)
PS C:\Users\Klaudiusz> Write-Host "Wysyłka 2 (Duplikat):" -ForegroundColor Cyan
Wysyłka 2 (Duplikat):
PS C:\Users\Klaudiusz> Invoke-RestMethod -Uri $url -Method Post -Headers @{"Authorization"="Bearer $token"} -ContentType "application/json" -Body $body

status
------
queued
```

Rysunek 4.18: Zapis sesji PowerShell: potwierdzenie akceptacji dwóch identycznych żądań przez API (status: queued).

```
209
210   select * from telemetry where send_time = '2026-01-20 16:16:14+00';

PROBLEMS 8   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POLYGLOT NOTEBOOK   POSTGRESQL QUERY RESULTS

Results   Messages                                                                                                    Open

| id | operator | network_type | signal | posit… | send_time | received_at | short_code |
|---|---|---|---|---|---|---|---|
| 12125 | T-Mobile | LTE | -80 | 0101000020E61… | 2026-01-20 16:16:14+00 | 2026-01-20 16:16:15.205645+00 | U2TU-JIDK-5CZL |
```

Rysunek 4.19: Wynik zapytania SQL: potwierdzenie obecności tylko jednego unikalnego wpisu w bazie danych po próbie duplikacji.

4.9.2 Weryfikacja mechanizmów ochrony przed przeciążeniem (Rate Limiting)

W celu zapewnienia wysokiej dostępności systemu oraz ochrony zasobów backendowych przed atakami typu *Denial of Service* (DoS) lub niekontrolowaną liczbą żądań, w usłudze *Azure API Management* (APIM) zaimplementowano polityki ograniczania przepustowości (*Rate Limiting*). Konfiguracja ta jest bezpośrednim odzwierciedleniem założeń projektowych zawartych w tabeli 4.1, gdzie zdefiniowano dopuszczalną liczbę wywołań dla poszczególnych modułów systemu.

Najbardziej restrykcyjne limity nałożono na proces rejestracji nowych urządzeń. Punkt końcowy */register* został skonfigurowany z limitem wynoszącym dokładnie 15 żądań na okres 60 sekund.

Wybór tak niskiego progu dla tej konkretnej operacji podyktowany jest faktem, że standardowe urządzenie wykonuje tę akcję tylko raz podczas cyklu życia aplikacji.

Poprawność działania mechanizmów obronnych bramy API została zweryfikowana doświadczalnie za pomocą skryptu symulującego serię 20 żądań wysłanych w odstępie milisekundowym. Wynik testu, zarejestrowany bezpośrednio w konsoli systemowej, przedstawiono na rysunku 4.20. Warto podkreślić, że w ramach weryfikacji skupiono się na tym konkretnym przypadku testowym jako najbardziej krytycznym. Pozostałe limity zdefiniowane w politykach APIM działają w oparciu o ten sam mechanizm systemowy, zatem ich poprawne funkcjonowanie jest analogiczne do zaprezentowanego wyniku.

```
PS C:\Users\Klaudiusz> $url = "https://apim-inzynierka.azure-api.net/v1/submit/register"
PS C:\Users\Klaudiusz> $body = '{"androidId": "1234567890abcdef"}'
PS C:\Users\Klaudiusz>
PS C:\Users\Klaudiusz> for ($i=1; $i -le 20; $i++) {
>>     # Użycie $($i) wymusza poprawne parsowanie zmiennej przed dwukropkiem
>>     Write-Host "Proba nr $($i): " -NoNewline
>>     try {
>>         $r = Invoke-RestMethod -Uri $url -Method Post -ContentType "application/json" -Body $body
>>         Write-Host "Sukces (201/200)" -ForegroundColor Green
>>     } catch {
>>         # Pobranie statusu numerycznego (np. 429)
>>         $status = $_.Exception.Response.StatusCode.value__
>>         Write-Host "Blad: $status - Too Many Requests" -ForegroundColor Red
>>     }
>> }
Proba nr 1: Sukces (201/200)
Proba nr 2: Sukces (201/200)
Proba nr 3: Sukces (201/200)
Proba nr 4: Sukces (201/200)
Proba nr 5: Sukces (201/200)
Proba nr 6: Sukces (201/200)
Proba nr 7: Sukces (201/200)
Proba nr 8: Sukces (201/200)
Proba nr 9: Sukces (201/200)
Proba nr 10: Sukces (201/200)
Proba nr 11: Sukces (201/200)
Proba nr 12: Sukces (201/200)
Proba nr 13: Sukces (201/200)
Proba nr 14: Sukces (201/200)
Proba nr 15: Sukces (201/200)
Proba nr 16: Blad: 429 - Too Many Requests
Proba nr 17: Blad: 429 - Too Many Requests
Proba nr 18: Blad: 429 - Too Many Requests
Proba nr 19: Blad: 429 - Too Many Requests
Proba nr 20: Blad: 429 - Too Many Requests
```

Rysunek 4.20: Zapis sesji PowerShell dokumentujący skuteczne działanie polityki Rate Limit na punkcie końcowym /register.

Analiza uzyskanych wyników pozwala na wyciągnięcie następujących wniosków:

- Pierwsze 15 zapytań (Proba nr 1 – 15) zostało przetworzonych pomyślnie, co potwierdza status `Sukces (201/200)`.
- Każde kolejne żądanie powyżej zdefiniowanego limitu (Proba nr 16 – 20) zostało natychmiastowo zablokowane przez bramę APIM, zanim obciążyło ono funkcję backendową.
- Serwer zwrócił standardowy kod błędu `HTTP 429 Too Many Requests`, informujący klienta o przekroczeniu limitu i konieczności odczekania pełnej minuty do odnowienia puli dostępnych zapytań.

Skuteczna implementacja powyższych mechanizmów gwarantuje, że nawet w przypadku awarii aplikacji mobilnej lub celowej próby ataku, infrastruktura serwerowa pozostanie stabilna dla pozostałych użytkowników systemu.

Rozdział 5

Bezpieczeństwo, autoryzacja oraz monitoring klastra

5.1 Analiza zagrożeń i strategia ochrony danych

Projektowanie systemu opartego na modelu *crowdsourcingu*, w którym tysiące rozproszonych urządzeń przesyła precyzyjne dane lokalizacyjne oraz parametry techniczne infrastruktury sieciowej, wymagało wdrożenia wielowarstwowej strategii bezpieczeństwa. Dużym wyzwaniem w projekcie *SignalMap* było zapewnienie rzetelności gromadzonych informacji przy jednoczesnym zagwarantowaniu pełnej anonimowości użytkowników.

Przed przystąpieniem do implementacji przeprowadzono analizę zagrożeń, identyfikując wektory ataków, które mogłyby wpłynąć na stabilność systemu:

- **Zatruwanie danych** – ryzyko celowego przesyłania zafałszowanych pomiarów w celu manipulacji wynikami na mapie wizualizacji pomiarów.
- **Eksploatacja punktów końcowych** – ataki typu *Denial of Service* (DoS) lub próby nieautoryzowanego zapisu danych przez pominięcie aplikacji mobilnej.
- **Naruszenie prywatności** – ryzyko deanonimizacji użytkownika poprzez wyciek identyfikatorów sprzętowych.
- **Deficyt monitoringu i audytu** – zagrożenie polegające na braku bieżącej kontroli nad stanem infrastruktury, co uniemożliwia wykrycie awarii lub wczesne rozpoznanie prób przełamania zabezpieczeń.

W odpowiedzi na powyższe wyzwania przyjęto strategię **Defense in Depth** (obrona w głąb). Zakłada ona, że bezpieczeństwo nie opiera się na jednym rozwiązaniu, lecz na synergii wielu warstw ochronnych [185]. W rozdziale szczegółowo opisano, jak poszczególne komponenty realizują tę strategię:

1. **Monitoring i Audyt** – wykorzystanie stosu *Prometheus-Grafana* do bieżącej weryfikacji stanu zdrowia klastra i wykrywania anomalii w ruchu (Sekcja 5.6).
2. **Izolacja brzegowa** – filtrowanie ruchu i ochrona przed przeciążeniem przy użyciu usługi *Azure API Management* oraz *WAF*.
3. **Autoryzacja sesji** – wdrożenie krótkotrwałych tokenów *JWT*, których zadaniem jest zagwarantowanie, że dane pochodzą wyłącznie z autoryzowanych instancji aplikacji.

4. **Pseudonimizacja** – proces deterministycznego generowania kluczy *Viewer Key*, który trwale oddziela tożsamość urządzenia od przesyłanych pomiarów.

Taka struktura rozdziału pozwala na wykazanie, że wdrożony system monitoringu nie jest jedynie narzędziem diagnostycznym, ale krytycznym elementem pętli bezpieczeństwa, umożliwiającym weryfikację skuteczności wszystkich zastosowanych barier ochronnych w czasie rzeczywistym.

5.2 Zabezpieczenie warstwy brzegowej

Pierwszą linię obrony systemu SignalMap stanowi warstwa brzegowa, której zadaniem jest filtrowanie ruchu przychodzącego z sieci publicznej oraz izolacja kluczowych komponentów backendowych od bezpośredniego kontaktu z Internetem. Realizowane jest to poprzez zintegrowane wykorzystanie usług *Azure API Management*.

Izolacja backendu

Usługa *Azure API Management* (APIM) pełni rolę punktu wejściowego (*Gateway*), który ukrywa faktyczną infrastrukturę obliczeniową (Azure Functions i klaster Kubernetes). Dzięki temu:

- **Abstrakcja adresów** – aplikacja mobilna nie zna fizycznych adresów URL funkcji procesujących dane. Komunikacja odbywa się wyłącznie przez bramę APIM.
- **Wymuszenie protokołu HTTPS** – brama APIM akceptuje wyłącznie połączenia szyfrowane za pomocą nowoczesnych wersji protokołu TLS (1.2 lub 1.3), co eliminuje ryzyko podsłuchu transmisji.
- **Zarządzanie kluczami funkcji** – brama APIM wstrzykuje niejawnie nagłówek `x-functions-key` przed przekazaniem zapytania do *Function App*. Dzięki temu klucze dostępowe do funkcji nie muszą być zaszyte w kodzie aplikacji mobilnej, co znacząco zwiększa bezpieczeństwo w przypadku próby dekompilacji aplikacji.

Ochrona punktów wejścia i kontrola przepływu

Bezpieczeństwo warstwy brzegowej zostało oparte na dwóch usługach, dostosowanych do charakteru ruchu generowanego przez różnych klientów systemu:

- **Brama API (APIM) dla aplikacji mobilnej** – pełni rolę bezpośredniego pośrednika dla urządzeń końcowych. Na tym poziomie wdrożono polityki *Rate Limiting* (np. dla punktu `/submit`), które chronią backend przed atakami typu *DoS* oraz skutkami błędów w pętli zbierania danych na telefonach. Dzięki temu infrastruktura jest odporna na próby jej przeciążenia przez zainfekowane lub wadliwie działające urządzenia mobilne.
- **Web Application Firewall (WAF) dla panelu administracyjnego** – stanowi dedykowaną barierę ochronną dla aplikacji webowej [186]. Wykorzystano podstawową implementację usługi, która filtruje ruch pod kątem typowych zagrożeń przeglądarkowych. Skonfigurowane reguły zapewniają ochronę przed atakami takimi jak: *SQL Injection*, *Cross-Site Scripting* (XSS), *Local File Inclusion* oraz blokują aktywność znanych botów [187].

Dzięki takiemu rozdzieleniu kompetencji, każdy kanał komunikacji jest chroniony narzędziem najlepiej dopasowanym do jego specyfiki, a złośliwy ładunek danych jest przerywany zanim dotrze do logiki serwerowej.

5.3 Autoryzacja dostępu i mechanizm JWT

W rozdziale opisany został mechanizm autoryzacji dostępu do punktów końcowych systemu, który bazuje na standardzie *JSON Web Token* (JWT) [188] oraz architekturze *Bearer Authentication* [189] System został zaprojektowany w taki sposób, aby próba zapisu danych telemetrycznych była poprzedzona weryfikacją tokenu i ważności sesji.

5.3.1 Proces wydawania i struktura tokena JWT

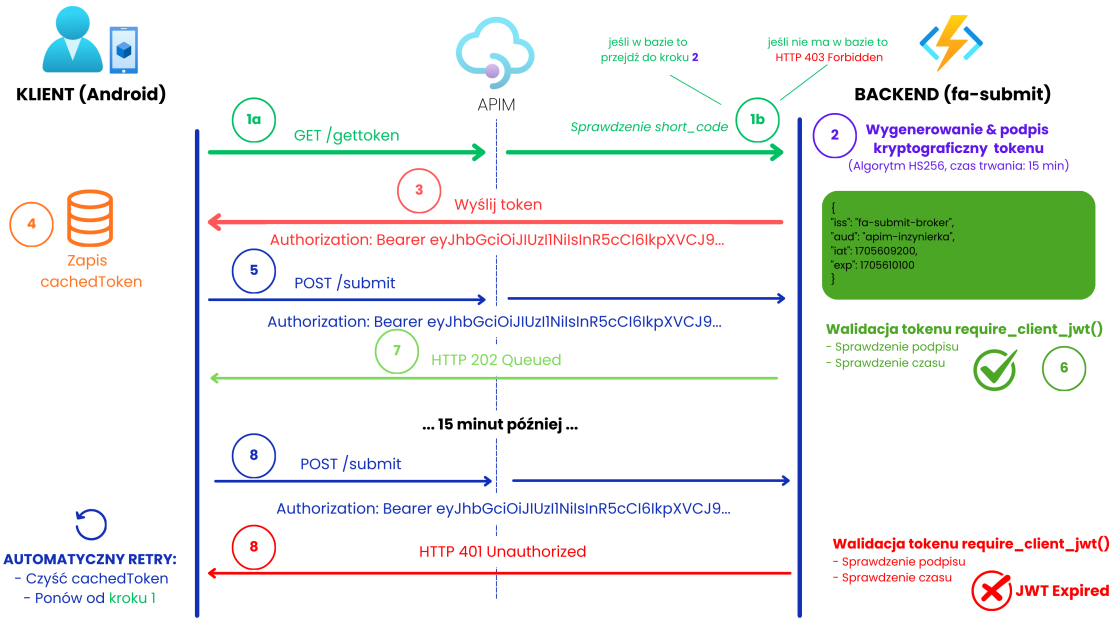
System wykorzystuje dedykowany punkt końcowy `/gettoken`, zaimplementowany w ramach usługi *Azure Functions* (`fa-submit`), do generowania poświadczeń dostępowych. Proces ten jest ściśle powiązany z tożsamością urządzenia zarejestrowanego w systemie. Odpowiedź serwera (Listing 5.1) dostarcza zakodowany token oraz metadane umożliwiające aplikacji mobilnej autonomiczne zarządzanie stanem sesji i czasem jej wygaśnięcia.

Listing 5.1: Przykładowa odpowiedź punktu końcowego /gettoken w formacie JSON.

```
HTTP/1.1 200 OK
Cache-Control: no-store
Content-Type: application/json

{
    "access_token": "eyJhbGciOiJIUzI1NiJ...[skrócono]...BwG0JU",
    "expires_in": 900,
    "expires_at": 1768769057
}
```

Proces generowania poświadczeń oraz ich późniejszego wykorzystania w cyklu życia aplikacji przedstawiono na schemacie sekwencyjnym (Rys. 5.1). Koncepcja wizualna diagramu została opracowana na podstawie schematu opublikowanych przez J. P. Media w serwisie *dev.to* [188].



Rysunek 5.1: Schemat sekwencyjny procesu autoryzacji JWT z uwzględnieniem weryfikacji identyfikatora w bazie danych oraz mechanizmu ponawiania.

Przebieg autoryzacji realizowany jest według następującej logiki:

- **Identyfikacja urządzenia i weryfikacja bazy danych** – Inicjalizacja procesu następuje poprzez żądanie `GET /gettoken`, w którym aplikacja przesyła identyfikator w nagłówku `X-Short-Code` (Krok 1a na Rys. 5.1). Serwer w kroku 1b dokonuje kluczowej weryfikacji, sprawdzając, czy przesłany `short_code` istnieje w tabeli `viewer`. Jeśli rekord zostanie odnaleziony, system przechodzi do generowania tokena (Krok 2), w przeciwnym razie proces zostaje przerwany, a serwer zwraca błąd 403 Forbidden, co uniemożliwia wydanie poświadczeń nieuprawnionym podmiotom.
- **Struktura Payloadu** – Po pomyślnej weryfikacji w bazie danych (Krok 1b), serwer tworzy ładunek zawierający wystawcę (`iss`: *fa-submit-broker*), odbiorcę (`aud`: *apim-inzynierka*) oraz precyzyjne znaczniki czasu (Krok 2 na Rys. 5.1).
- **Podpis kryptograficzny** – Token jest podpisywany algorytmem **HS256** przy użyciu klucza symetrycznego `JWT_SIGNING_KEY` (Krok 2). Wynikowy ciąg jest przesyłany do urządzenia w polu `access_token` (Krok 3).
- **Zarządzanie czasem życia (TTL)** – Serwer definiuje ważność tokena na **900 sekund** (15 minut). Parametr `expires_at` pozwala aplikacji na lokalną weryfikację ważności poświadczeń (Krok 4), co eliminuje zbędne zapytania sieciowe przed każdą wysyłką danych.
- **Ochrona przed buforowaniem** – Nagłówek `Cache-Control: no-store` gwarantuje, że poświadczenia nigdy nie zostaną utrwalone w pamięciach podręcznych serwerów pośredniczących.

5.3.2 Logika weryfikacji i zabezpieczenie punktów końcowych

Zgodnie ze schematem (Rys. 5.1), każda operacja zapisu danych (`submit`, `speedtest`) wymaga dołączenia tokena w nagłówku `Authorization: Bearer` (Krok 5). Funkcja pośrednicząca `require_client_jwt` odpowiada za dekodowanie i weryfikację podpisu (Krok 6). Pomyślna walidacja kończy się asynchronicznym przekazaniem danych do kolejki (Krok 7). W przypadku wykrycia tokena wygasłego, serwer zwraca kod **401 Unauthorized** (Krok 8), co stanowi dla klasy `AzureClient` sygnał do automatycznego odświeżenia sesji i ponowienia żądania.

Procedura weryfikacyjna obejmuje:

- Sprawdzenie obecności i formatu nagłówka autoryzacyjnego.
- Dekodowanie i weryfikację podpisu tokena przy użyciu tajnego klucza (Krok 6).
- Weryfikację zgodności pól `audience` oraz `issuer` w celu zapobieganiu atakom typu *token substitution*.
- Kontrolę czasu wygaśnięcia z uwzględnieniem 30-sekundowego marginesu błędu dla synchronizacji zegarów.

W przypadku pomyślnej walidacji, żądanie trafia do kolejki ze statusem **HTTP 202 Queued** (Krok 7). Jeśli jednak system wykryje niepoprawny lub wygasły token, zwracana jest odpowiedź **401 Unauthorized** (Krok 8), co stanowi sygnał dla aplikacji mobilnej do odświeżenia poświadczeń.

5.3.3 Implementacja klienta mobilnego i mechanizm ponawiania

Aplikacja mobilna zarządza cyklem życia tokena w sposób jawny dla użytkownika poprzez klasę `AzureClient`, co ilustruje lewa strona schematu (Rys. 5.1).

- **Buforowanie (Caching)** – Po otrzymaniu tokena (Krok 3), jest on wraz z czasem wygaśnięcia przechowywany w pamięci operacyjnej procesu jako `cachedToken` (Krok 4), co redukuje liczbę zapytań do brokera autoryzacji.
- **Weryfikacja lokalna** – Przed każdą wysyłką danych metoda `isTokenValid()` sprawdza ważność posiadanego tokena, uwzględniając 30-sekundowy bufor bezpieczeństwa.
- **Mechanizm automatycznego odświeżania (Retry)** – Jeśli po upływie czasu ważności backend zwróci błąd 401 (Krok 8), uruchamiany jest **automatyczny retry**. Metoda `postJsonWithRetry` czyści `cachedToken` i powtarza proces od Kroku 1, co czyni proces autoryzacji niemal niezauważalnym dla użytkownika.

5.3.4 Zarządzanie kluczem sygnującym

Bezpieczeństwo mechanizmu JWT w systemie SignalMap opiera się na wysokiej jakości klucza `JWT_SIGNING_KEY`. Klucz ten został wygenerowany przy użyciu narzędzia `OpenSSL` za pomocą polecenia:

```
openssl rand -base64 64
```

Zastosowanie systemowego generatora liczb losowych pozwoliło na uzyskanie ciągu o długości 512 bitów, co wielokrotnie przewyższa minimalne wymagania algorytmu HS256, co w dużej mierze wyklucza ataki typu *brute-force*.

W celu zachowania najlepszych praktyk bezpieczeństwa, klucz nie został umieszczony w kodzie źródłowym aplikacji ani w repozytorium *Git*. Zamiast tego, został on wprowadzony do zmiennych środowiskowych usługi *Azure Functions*. Taka separacja sprawia, że sekret jest dostępny jedynie dla procesu wykonawczego backendu, a dostęp do niego mają wyłącznie uprawnieni administratorzy infrastruktury chmurowej.

5.4 Prywatność i anonimizacja użytkowników

Bezpieczeństwo danych osobowych oraz prywatność użytkowników stanowią jeden z fundamentów systemu SignalMap. System został zaprojektowany w taki sposób, aby w żadnym punkcie infrastruktury chmurowej nie były przechowywane surowe identyfikatory sprzętowe (np. Android ID) ani dane pozwalające na bezpośrednią identyfikację osoby.

5.4.1 Proces deterministycznej anonimizacji

Proces przekształcania tożsamości urządzenia w anonimowy identyfikator systemowy odbywa się wewnątrz funkcji `register`. Wykorzystuje on deterministyczny mechanizm *Keyed-Hash Message Authentication Code* (HMAC) oparty na algorytmie SHA-256.

- **Identyfikator wejściowy**: Aplikacja mobilna przesyła unikalny dla systemu Android ciąg `android_id`.

- **Sekret serwerowy (Pepper)**: Do obliczeń wykorzystywany jest tajny klucz `PEPPER_HEX`, przechowywany jako bezpieczna zmienna środowiskowa backendu. Został on wygenerowany w sposób analogiczny do klucza podpisu tokenów (`JWT_SIGNING_KEY`).
- **Viewer Key**: Wynikiem funkcji `viewer_key_from_android_id` jest 32-bajtowy (256-bitowy) ciąg binarny, który stanowi unikalny identyfikator w bazie danych PostgreSQL.

Prawdopodobieństwo kolizji dla klucza o długości $n = 256$ bitów przy liczbie k użytkowników określa przybliżony wzór wynikający z paradoksu dnia urodzin [190]:

$$P(k) \approx 1 - e^{-\frac{k^2}{2 \cdot 2^n}} \tag{5.1}$$

Dla zobrazowania skali bezpieczeństwa można przyjąć, że nawet przy zarejestrowaniu miliona urządzeń ($k = 10^6$), prawdopodobieństwo wystąpienia choć jednej kolizji wynosi w przybliżeniu 10^{-65}. Wartość ta jest pomijalnie mała, co gwarantuje, że każde urządzenie otrzyma unikalny klucz binarany.

5.4.2 Generowanie identyfikatora ShortCode

Ponieważ `viewer_key` jest ciągiem binarnym nieczytelnym dla człowieka, system generuje jego reprezentację tekstową – *ShortCode*, prezentowaną w formacie `XXXX-XXXX-XXXX`. Proces ten przebiega w trzech etapach:

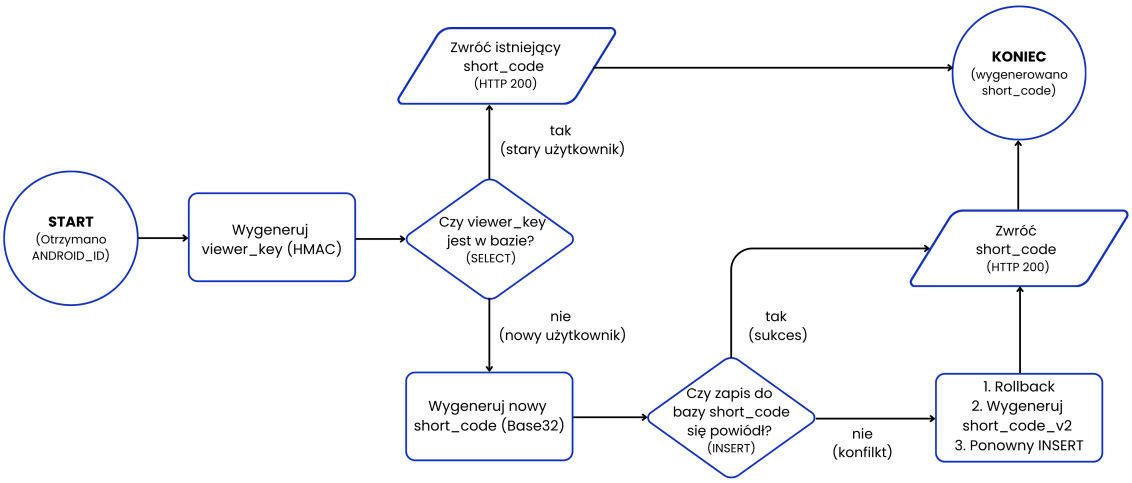
1. **Kodowanie Base32** – Pierwsze 8 bajtów klucza binarnego jest konwertowane na znaki z alfabetu `ABCDEFGHIJKLMNOPQRSTUVWXYZ234567`. Wybór tego kodowania eliminuje znaki podobne wizualnie (np. 0 i O, 1 i I), co znacząco podnosi użyteczność identyfikatora przy ręcznym przepisywaniu.
2. **Suma kontrolna Mod32** – Aby zapobiec błędom przy wpisywaniu kodu, 12. znak identyfikatora jest sumą kontrolną obliczoną z indeksów 11 poprzednich znaków modulo 32. Pozwala to na natychmiastową walidację składniową kodu już na poziomie interfejsu aplikacji mobilnej.
3. **Obsługa kolizji i transakcyjność** – Ze względu na mniejszą przestrzeń adresową kodu krótkiego ($32^{11} \approx 3,6 \times 10^{16}$), system posiada mechanizm ponowienia (*retry*). W przypadku wystąpienia błędu `UniqueViolation` w bazie danych, system wykonuje wycofanie bieżącej transakcji (ang. *rollback*), a następnie wywołuje funkcję `short_code_4x3_v2`, bazującą na innym fragmencie klucza binarnego.

5.4.3 Idempotencja i retencja danych

Logika działania punktu końcowego `/register`, obejmująca proces weryfikacji tożsamości oraz wielowarstwową obsługę kolizji, została przedstawiona na schemacie blokowym (Rys. 5.2).

Implementacja wspiera mechanizm idempotencji – przed próbą utworzenia nowego rekordu system wykonuje zapytanie `SELECT`. Jak zilustrowano na rysunku 5.2, pozwala to na natychmiastowe rozpoznanie powracającego urządzenia bez obciążania zasobów operacjami zapisu. W przypadku nowych instalacji, system przechodzi do fazy `INSERT`, gdzie ostateczną spójność gwarantują więzy unikalności bazy danych.

Dzięki takiemu podejście, system nie przechowuje danych pozwalających na bezpośrednią identyfikację osoby. Wykorzystanie pseudonimizacji poprzez identyfikatory techniczne jest zgodne z zasadą minimalizacji danych RODO i pozwala na bezpieczną agregację statystyk [46].



Rysunek 5.2: Schemat blokowy procesu rejestracji urządzenia i generowania identyfikatora ShortCode.

5.4.4 Ciągłość tożsamości po reinstalacji aplikacji

Zastosowanie deterministycznego algorytmu `viewer_key_from_android_id` pozwala na zachowanie spójności danych użytkownika bez konieczności tworzenia tradycyjnych kont opartych na adresie e-mail czy haśle.

- **Trwałość identyfikatora** – Ponieważ `android_id` jest stały dla danego urządzenia, ponowna instalacja oprogramowania skutkuje wygenerowaniem identycznego klucza `viewer_key`.
- **Automatyczne odzyskiwanie profilu** – Podczas procesu rejestracji po reinstalacji, wstępne zapytanie `SELECT` w bazie danych wykrywa istniejący `viewer_key` i zwraca przypisany do niego wcześniej `short_code`.
- **Dostęp do historii pomiarów** – Dzięki temu mechanizmowi, użytkownik automatycznie odzyskuje dostęp do swoich historycznych statystyk i pomiarów na mapie, ponieważ są one w bazie danych trwale powiązane z tym samym, niezmiennym kluczem `viewer_key`.

Rozwiązanie to łączy wygodę użytkowania (brak konieczności logowania) z wysokim poziomem prywatności, eliminując jednocześnie problem utraty danych przy zmianie wersji oprogramowania lub czyszczeniu pamięci urządzenia.

5.5 Ciągłość działania i ochrona przed utratą danych

Zapewnienie ciągłości działania systemu oraz ochrona przed utratą danych stanowią kluczowe wymagania w aplikacjach przetwarzających dane telemetryczne w środowiskach produkcyjnych i chmurowych. W przypadku systemów zbierających dane sieciowe, nawet krótkotrwała niedostępność usług backendowych, przeciążenie bazy danych lub awaria infrastruktury może prowadzić do trwałej utraty cennych informacji pomiarowych oraz zakłócenia procesów analitycznych.

W odpowiedzi na te wyzwania zaprojektowana architektura została oparta na zasadach odporności na błędy (ang. *fault tolerance*) oraz wysokiej dostępności. Kluczową rolę odgrywa tu asynchroniczne przetwarzanie danych z wykorzystaniem kolejki komunikatów, które oddziela warstwę przyjmowania danych od warstwy ich trwałego zapisu, a także mechanizmy zapewniające trwałość danych w relacyjnej bazie danych PostgreSQL działającej w środowisku Microsoft Azure.

Zastosowane rozwiązania umożliwiają skuteczne buforowanie chwilowych skoków obciążenia, minimalizację ryzyka utraty danych w przypadku awarii komponentów systemu oraz szybkie od-

tworzenie stanu bazy danych do wybranego punktu w czasie. Dzięki temu system zachowuje zdolność do nieprzerwanego działania nawet w warunkach zwiększonego ruchu, błędów operacyjnych lub awarii infrastruktury chmurowej.

5.5.1 Asynchroniczne przetwarzanie z wykorzystaniem Azure Service Bus

Zamiast bezpośredniego zapisu do bazy danych PostgreSQL, punkty końcowe `/submit` oraz `/speedtest` pełnią rolę nadawców komunikatów, którzy natychmiastowo przekazują odebrane dane do kolejki *Azure Service Bus*.

- **Separacja warstw** – Backendowa funkcja HTTP nie czeka na zakończenie operacji zapisu w bazie danych. Po pomyślnym umieszczeniu wiadomości w kolejce, użytkownik natychmiast otrzymuje status **HTTP 202 Queued**.
- **Buforowanie obciążenia** – Kolejka działa jako bufor, który przyjmuje nagłe skoki ruchu (np. masowe przesyłanie pomiarów w tym samym czasie), pozwalając procesom roboczym na przetwarzanie danych w tempie dostosowanym do wydajności bazy danych.
- **Gwarancja dostarczenia** – Wykorzystanie klasy `ServiceBusClient` z mechanizmem `with` zapewnia poprawne zamknięcie połączenia i potwierdzenie wysłania komunikatu, minimalizując ryzyko utraty pakietu na etapie przyjmowania.

5.5.2 Mechanizmy ochrony danych w relacyjnej bazie danych

Ze względu na kluczowe znaczenie gromadzonych danych telemetrycznych, wdrożono mechanizmy zabezpieczające bazę danych `inz-postgres` przed utratą informacji. Wykorzystano wbudowane funkcjonalności usługi *Azure Database for PostgreSQL flexible server*, które eliminują ryzyko utraty danych w wyniku awarii sprzętowej lub błędów ludzkich:

- **Automatyczne kopie zapasowe (Autobackup)** – usługa wykonuje pełne kopie zapasowe raz w tygodniu, kopie różnicowe dwa razy na dobę oraz ciągłe kopie logów transakcji co 5 minut [191]. Są one widoczne w panelu administracyjnym wraz z informacją o dacie ukończenia oraz terminie ich ważności.
- **Point-in-Time Restore (PITR)** – dzięki ciągłej archiwizacji logów, system rejestruje najwcześniejszy możliwy punkt przywrócenia (*Earliest restore point*). Umożliwia to administratorowi odtworzenie bazy danych do dowolnego momentu z dokładnością do sekundy w ramach określonego okresu retencji [192].
- **Trwałość i szybkie odtwarzanie** – kopie zapasowe są przechowywane w magazynie o wysokiej dostępności (LRS lub GRS), co zapewnia ich przetrwanie nawet w przypadku awarii infrastruktury. Interfejs zarządzania pozwala na natychmiastowe zainicjowanie procesu przywracania (*Fast restore*) do nowej instancji serwera, co zapewnienia ciągłości działania [193].

Integracja automatycznego backupu z architekturą opartą na kolejkach *Service Bus* tworzy kompletny łańcuch zabezpieczeń. Nawet jeśli baza danych musiałaby zostać przywrócona do stanu sprzed godziny, wiadomości zgromadzone w tym czasie w kolejce mogą zostać ponownie przetworzone, co zapewnia zerową utratę danych telemetrycznych (ang. *Zero Data Loss*).

5.6 Monitoring

Monitoring infrastruktury i aplikacji został zrealizowany przy użyciu chart'a kube-prometheus-stack wdrożonego w przestrzeni nazw monitoring klastra Kubernetes. Zapewnia on zbieranie, przechowywanie oraz wizualizację metryk zarówno systemowych, jak i aplikacyjnych.

5.6.1 Architektura systemu monitoringu

System monitoringu został zbudowany w oparciu o trzy główne komponenty: (1) Prometheus jako baza typu szeregów czasowych odpowiedzialną za zbieranie i przechowywanie metryk, (2) Grafanę jako platformę wizualizacyjną udostępniającą dashboardy oraz (3) zestaw eksporterów odpowiedzialnych za pozyskiwanie metryk z różnych źródeł danych.

Dodatkowo system integruje się z Azure Monitor, który stanowi nadrzędną warstwę analityczną gromadzącą dane telemetryczne bezpośrednio z poziomu platformy chmurowej. Azure Monitor monitoruje zasoby niedostępne dla wewnętrznych narzędzi klastra, takie jak sieć wirtualna VNet, baza danych Azure Database for PostgreSQL flexible server czy Azure Key Vault. W architekturze systemu Azure Monitor pełni rolę jednego z głównych źródeł danych dla platformy Grafana, uzupełniając metryki zbierane przez Prometheus o dane dotyczące usług w Azure.

Dokładne opisy każdego z elementów technologii są szczegółowo opisane w rozdziale 3.3. Omówienie technologii i narzędzi wykorzystanych w implementacji oraz 4.1. Konfiguracja klastra Kubernetes.

5.6.2 Prometheus - zbieranie i przechowywanie metryk

Głównym zadaniem Prometheusa jest cykliczne odpytywanie endpointów metrykowych w interwałach trzydziestosekundowych oraz zapisywanie zebranych danych w lokalnej bazie szeregów czasowych.

Prometheus działa w modelu pull, co oznacza że aktywnie pobiera metryki z aplikacji, a nie aplikacje wysyłają dane do Prometheusa. Taka architektura zapewnia lepszą kontrolę nad tym jakie metryki są zbierane oraz ułatwia diagnostykę problemów z połączeniem. Moment w którym Prometheus nie może pobrać metryk, wskazuje na nieosiągalność punktu końcowego.

Kluczowym elementem konfiguracji jest mechanizm ServiceMonitor, który definiuje z jakich usług Kubernetes mają być pobierane metryki. ServiceMonitor wykorzystuje selektor etykiet (ang. *labelSelector*) do identyfikacji podów, których metryki mają być zbierane. Prometheus automatycznie wykrywa nowe obiekty ServiceMonitor w klastrze i rekonfiguruje się, dodając nowe endpointy do swojej konfiguracji prometheus.yml. Taki mechanizm eliminuje konieczność ręcznej modyfikacji konfiguracji Prometheusa przy dodawaniu nowych aplikacji do monitoringu.

Dane z metrykami są przechowywane na trwałym wolumenie o pojemności dwudziestu gigabajtów, co zapewnia retencję danych historycznych nawet po restarcie podów Prometheusa. Wolumen został skonfigurowany z użyciem Azure Disk CSI driver ze SKU Standard LRS (Locally Redundant Storage), który oferuje wystarczającą wydajność dla operacji zapisu metryk przy jednoczesnej optymalizacji kosztów. StorageClass został skonfigurowany z polityką Delete, co oznacza że dysk Azure jest automatycznie usuwany wraz z usunięciem PersistentVolumeClaim, zapobiegając pozostawianiu nieużywanych zasobów w chmurze.

5.6.3 Grafana - wizualizacja danych

Grafana [109] stanowi warstwę prezentacyjną systemu monitoringu, umożliwiając tworzenie interaktywnych dashboardów wizualizujących dane z wielu źródeł. Platforma została skonfigurowana

z integracją dwóch źródeł danych: (1) Prometheus jako główne źródło metryk aplikacyjnych oraz klastrowych, (2) Azure Monitor jako źródło metryk usług zarządzanych Azure.

Grafana została wdrożona z trwałym przechowywaniem konfiguracji i dashboardów na wolumenie o pojemności pięciu gigabajtów. Rozwiązanie to było konieczne ze względu na problem napotkany przy użyciu domyślnej konfiguracji - gotowe dashboardy wdrażane przez Helm z chartu kube-prometheus-stack są tylko w pamięci poda i znikają po jego restarcie. Dodanie PersistentVolume dla Grafany umożliwia tworzenie własnych dashboardów oraz zachowanie wszystkich zmian konfiguracyjnych między restartami podów lub całego klastra.

Dostęp do interfejsu Grafany został zrealizowany poprzez Ingress z kontrolerem Nginx, który udostępnia platformę pod domeną grafana.signalmap.com.pl. Połączenie jest zabezpieczone certyfikatem TLS wydanym przez Let's Encrypt z automatycznym odnowieniem zarządzanym przez cert-manager. Ruch HTTP jest automatycznie przekierowywany na HTTPS, zapewniając szyfrowanie komunikacji.

Uwierzytelnianie do Grafany wykorzystuje podstawowe mechanizm username/password, gdzie hasło administratora zostało zdefiniowane w wartościach Helm Chart. W środowisku produkcyjnym powinno być ono przechowywane w Azure Key Vault i montowane jako Secret, jednak dla uproszczenia implementacji w tej pracy zostało zdefiniowane bezpośrednio w pliku values.yaml.

5.6.4 PostgreSQL Exporter - monitoring bazy danych

PostgreSQL Exporter to dedykowany agent odpowiedzialny za pozyskiwanie metryk wewnętrznych z bazy danych Azure Database for PostgreSQL flexible server i eksportowanie ich w formacie zrozumiałym dla Prometheusa. Komponent ten został wdrożony jako osobny Deployment w przestrzeni nazw monitoring z pojedynczą repliką, co jest wystarczające ze względu na jego bezstanowy charakter.

Exporter wystawia metryki na porcie 9187. Port ten jest następnie mapowany przez Service oraz wskazywany w obiekcie ServiceMonitor, który konfiguruje Prometheus do zbierania metryk co trzydzieści sekund.

Zebrane metryki pozwalają na kompleksową ocenę stanu bazy danych, identyfikację potencjalnych problemów wydajnościowych oraz monitorowanie wykorzystania zasobów.

5.6.5 Node Exporter i Kube State Metrics

Oprócz dedykowanych eksporterów dla aplikacji, stack kube-prometheus-stack automatycznie wdraża dwa kluczowe komponenty zbierające metryki na poziomie klastra Kubernetes.

Node Exporter jest wdrażany jako pod na każdym węźle klastra. Jego zadaniem jest zbieranie danych odnośnie zasobów węzłów, takich jak wykorzystanie CPU, pamięci, dysku oraz sieci.

Kube State Metrics to pod służący do translacji stanów obiektów Kubernetes do formatu rozumianego przez Prometheus. Dzięki temu Prometheus może zbierać metryki dotyczące deploymentów, podów, serwisów oraz innych zasobów klastra.

5.6.6 Ingress Nginx Controller

Dodatkowo do monitoringu został dodany ServiceMonitor dla kontrolera Ingress Nginx, który zbiera metryki dotyczące ruchu HTTP przechodzącego przez ingress. ServiceMonitor został skonfigurowany z etykietą matchLabels, dzięki czemu Prometheus może skorzystać z endpoints usługi Nginx Controller i pobrać metryki związane z tym narzędziem. Celem było zrealizowanie dashboardu w Grafanie, który przedstawia statystyki odnośnie wejść na domenę.

5.6.7 Prometheus Operator i automatyzacja konfiguracji

Całym stosem monitoringu zarządza Prometheus Operator, który jest kontrolerem Kubernetes odpowiedzialnym za automatyzację konfiguracji i cyklu życia komponentów Prometheusa.

Gdy w klastrze pojawia się nowy obiekt ServiceMonitor, Operator wykrywa tę zmianę, generuje odpowiednią sekcję w pliku prometheus.yml oraz wykonuje przeładowanie konfiguracji Prometheusa. Taki mechanizm eliminuje potrzebę restartowania Prometheusa przy dodawaniu nowych źródeł metryk, co może być szczególnie istotne w środowisku produkcyjnym żeby nie było niedostępności systemu monitoringu.

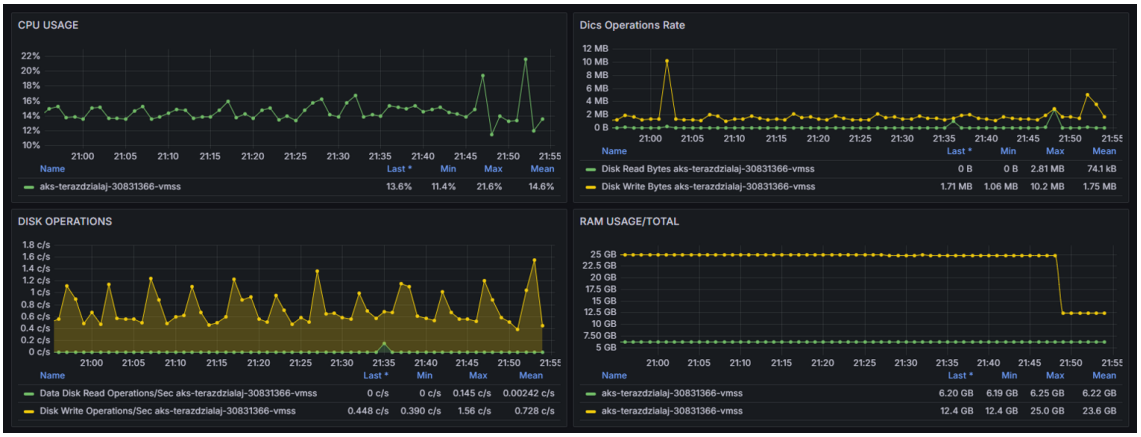
5.6.8 Przepływ danych w systemie monitoringu

Kompletny przepływ danych w systemie monitoringu przebiega następująco. Aplikacje wystawiają endpointy z metrykami na określonych portach. Prometheus, kierując się konfiguracją z obiektów ServiceMonitor, cyklicznie odpytuje te endpointy co trzydzieści sekund i zapisuje zebrane metryki w swojej bazie danych.

Grafana łączy się z Prometheusem jako źródłem danych oraz z Azure Monitor dla metryk usług zarządzanych Azure. Logowanie się do interfejsu Grafany możliwe jest poprzez przeglądarkę pod adresem https://grafana.signalmap.com.pl, gdzie można przeglądać dashboardy wizualizujące zebrane metryki w formie wykresów liniowych, słupkowych czy tabel. Dashboardy są aktualizowane w czasie rzeczywistym, co pozwala na bieżące monitorowanie stanu systemu.

W przypadku wykrycia anomalii użytkownik może zagłębić się w szczegóły konkretnej metryki, zmienić zakres czasowy wyświetlanych danych lub skorelować wiele metryk na jednym wykresie w celu diagnostyki przyczyny problemu. Wszystkie dashboardy oraz ich konfiguracje są trwale przechowywane na wolumenie Grafany, co zapewnia że wprowadzone zmiany nie zostaną utracone po restarcie systemu.

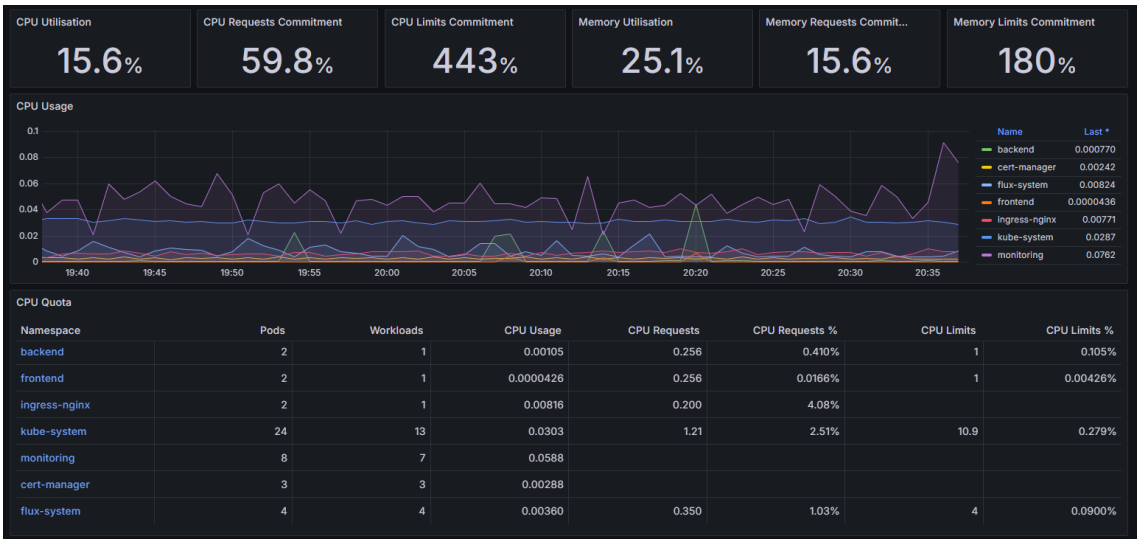
5.6.9 Przykładowe wizualizacje metryk w Grafanie



Rysunek 5.3: Metryki dotyczące VMSS

Wykresy na rys. 5.3 przedstawiają dane zebrane z *Virtual Machine Scale Set*, na których działa klaster Kubernetes uruchomiony w usłudze Azure Kubernetes Service. Przedstawiają one dane dotyczące wykorzystania zasobów sprzętowych klastra. Wykres zatytułowany *CPU USAGE* przedstawia obciążenie procesorów klastra w czasie wraz z podsumowaniem o ostatnim pomiarze, minimum, maksimum oraz o średniej z wybranego przedziału czasu. Wykres *DISK OPERATIONS* pokazuje częstotliwość operacji odczytu i zapisu na dyskach, mierzoną w operacjach na sekundę. Wykres

Dics Operations Rate przedstawia rzeczywistą liczbę danych odczytywanych i zapisywanych na dyskach w megabajtach na sekundę. Wykres *RAM USAGE/TOTAL* prezentuje zużycie pamięci operacyjnej przez poszczególne węzły klastra.



Rysunek 5.4: Metryki CPU dotyczące klastra Kubernetes

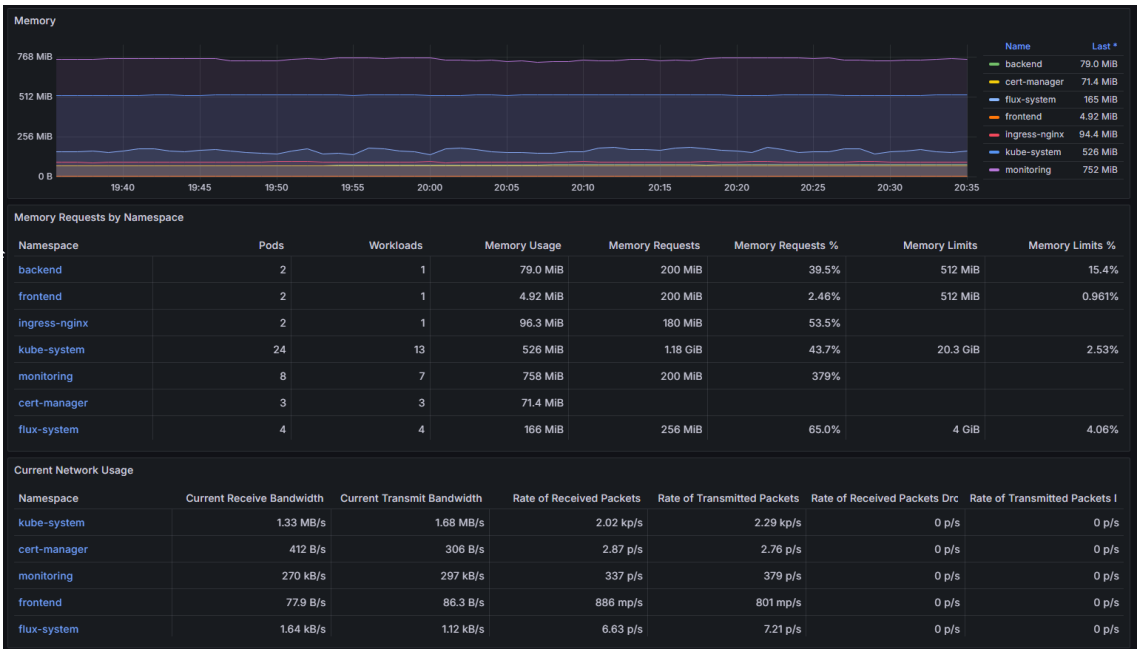
Na rys. 5.4 oraz rys. 5.5 przedstawione są dane dotyczące zużycia zasobów procesora oraz pamięci na poziomie całego klastra Kubernetes.

Metryka *CPU Utilisation* przedstawia rzeczywiste zużycie mocy obliczeniowej przez wszystkie działające pody w klastrze. Metryka *CPU Requests Commitment* pokazuje, jaki procent dostępnych zasobów CPU został zarezerwowany poprzez deklaracje requests w specyfikacjach podów. Kubernetes gwarantuje dostępność tej ilości zasobów dla aplikacji, niezależnie od ich bieżącego zużycia. Metryka *CPU Limits Commitment* przedstawia sumę maksymalnych limitów CPU. Wartość przekraczająca sto procent jest normalna, ponieważ nie wszystkie pody jednocześnie wykorzystują swoje maksymalne limity. Analogicznie dla przedstawionych metryk dotyczących pamięci. Tabela *CPU Quota* przedstawia szczegółowe informacje o zużyciu CPU dla każdej przestrzeni, liczbę podów oraz workloadów oraz procentowe wykorzystanie zarezerwowanych zasobów requests i limitów.

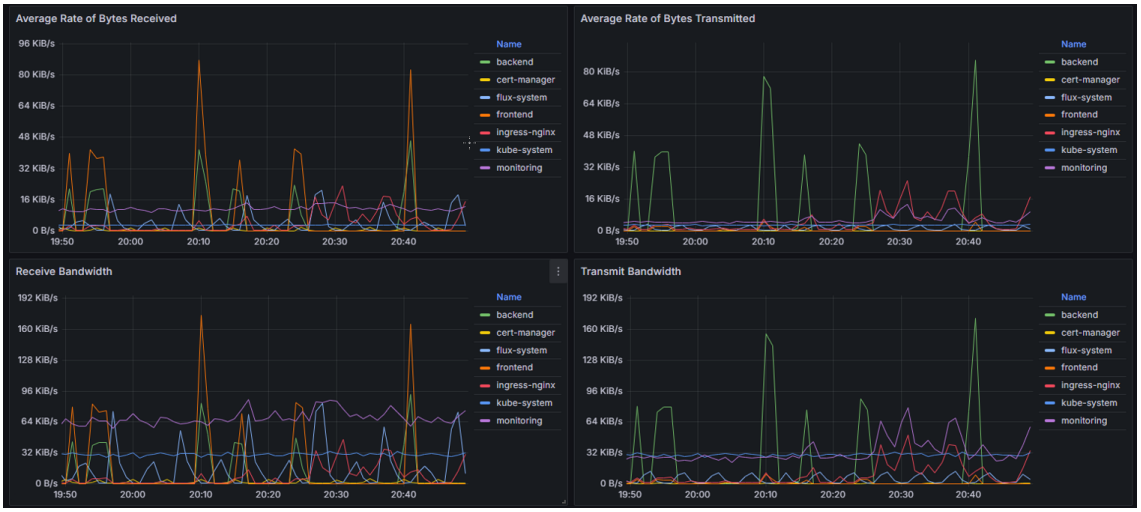
Na rys. 5.5 Prezentowane są metryki wykorzystania pamięci RAM klastra. Wykres *Memory* pokazuje zużycie pamięci przez poszczególne przesztrzenie nazw w czasie. Tabela *Memory Requests by Namespace* przedstawia szczegółowe informacje o zużyciu pamięci.

Tabela *Current Network Usage* pokazuje aktualny stan ruchu sieciowego dla każdej z przestrzeni nazw, prezentując przepustowość w bajtach na sekundę oraz liczbę pakietów.

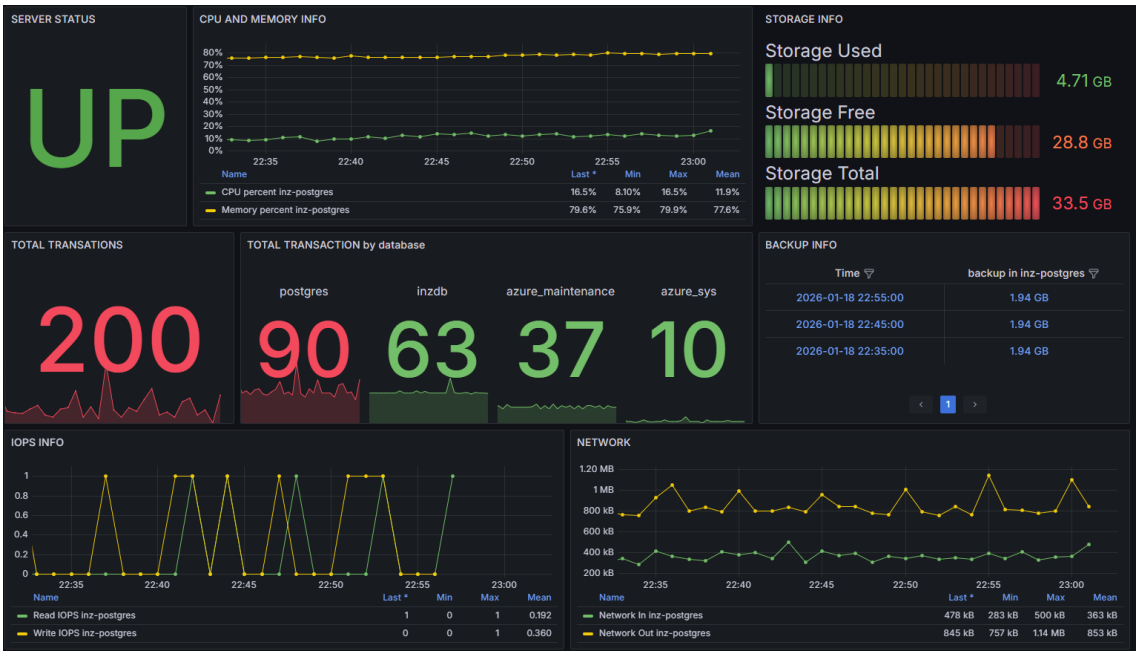
Kolejne zebrane metryki rys. 5.6 pochodzą z *Kubernetes / Networking / Cluster* i dotyczą ruchu sieciowego na poziomie przestrzeni nazw, zilustrowanych różnymi kolorami. Wykresy *Average Rate of Bytes Received* oraz *Average Rate of Bytes Transmitted* pokazują natężenie ruchu przychodzącego i wychodzącego dla poszczególnych przestrzeni nazw, co pozwala na identyfikację komponentów generujących największy ruch sieciowy. Wykresy *Receive Bandwidth* oraz *Transmit Bandwidth* prezentują całkowitą przepustowość. Widoczne skoki wartości dla frontend i backend odpowiadają momentom komunikacji API, gdy użytkownicy korzystają z aplikacji, podczas gdy w miarę stabilna wartość monitoringu świadczy o ciągłym zbieraniu metryk przez Prometheus.



Rysunek 5.5: Metryki pamięci oraz sieci dotyczące klastra Kubernetes



Rysunek 5.6: Metryki sieciowe klastra



Rysunek 5.7: Dane dotyczące bazy danych PostgreSQL

Na rysunku 5.7 zaprezetowane zostały metryki zebrane przez PostgreSQL Exporter z bazy danych. Panel *SERVER STATUS* przedstawia aktualny stan dostępności w postaci wskaźnika UP lub DOWN oznaczającego odpowiednio że baza jest osiągalna i odpowiada na zapytania, czy nie.

Panel *CPU AND MEMORY INFO* pokazuje wykres wykorzystania procesora oraz pamięci operacyjnej serwera bazy danych w czasie. Wykres *CPU percent* przedstawia obciążenie procesora wyrażone w procentach, natomiast wykres *Memory percent* pokazuje zużycie pamięci RAM.

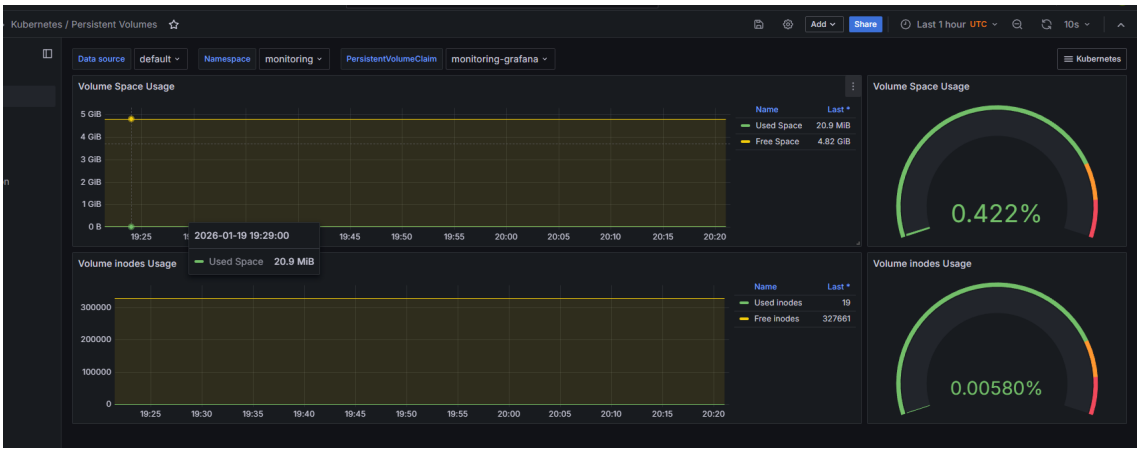
Panel *STORAGE INFO* zawiera trzy wskaźniki dotyczące przestrzeni dyskowej: *Storage Used* przedstawia ilość wykorzystanej przestrzeni dyskowej, *Storage Free* pokazuje ilość dostępnej wolnej przestrzeni, a *Storage Total* określa całkowitą pojemność dysku. Gradient kolorystyczny od zielonego przez żółty do czerwonego wizualizuje stopień zapełnienia.

Panel *TOTAL TRANSACTIONS* prezentuje całkowitą liczbę transakcji wykonanych w bazie danych wraz z wykresem tych transakcji w czasie. Panel *TOTAL TRANSACTION by database* rozbija tę metrykę na poszczególne bazy danych w instancji PostgreSQL, gdzie każda baza (postgres, inzdb, azure_maintenance, azure_sys) ma osobny wskaźnik z wizualizacją pokazującą trend aktywności transakcji.

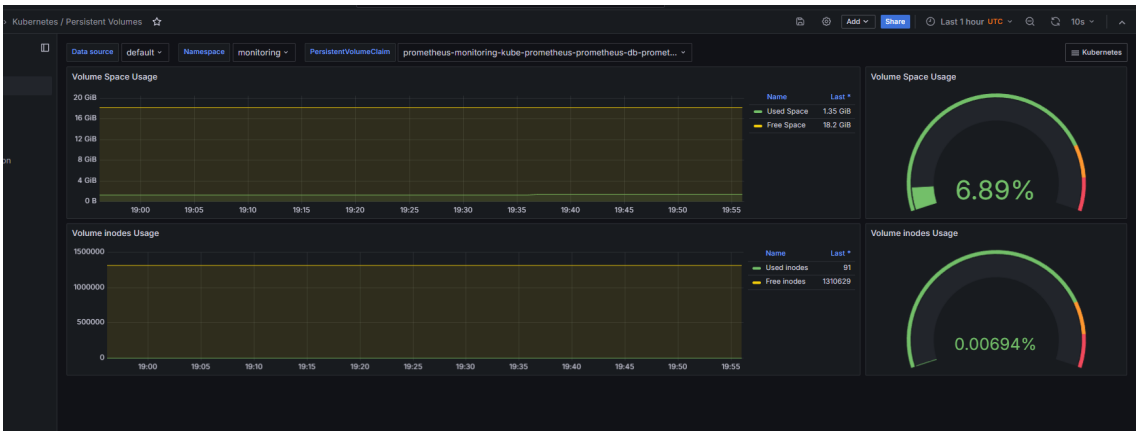
Panel *IOPS INFO* przedstawia wykresy operacji wejścia-wyjścia na dysku. Wykres *Read IOPS* pokazuje liczbę operacji odczytu na sekundę, natomiast wykres *Write IOPS* prezentuje liczbę operacji zapisu na sekundę.

Panel *NETWORK* prezentuje przepustowość sieciową serwera bazy danych. Wykres *Network In* pokazuje ruch przychodzący mierzony w kilobajtach lub megabajtach na sekundę, reprezentujący zapytania SQL wysyłane do bazy. Wykres *Network Out* przedstawia ruch wychodzący, odpowiadający wynikom zapytań zwracanym przez bazę danych do aplikacji.

Na rysunkach 5.5 oraz 5.7 przedstawione zostały metryki prezentujące informacje o podłączonych do Podów Grafany oraz Prometheusa zasobów Persistent Volume Claims (PVC). To na tych dwóch PVC zapisywane na stałe są dashboardy (w przypadku Grafany) oraz metryki (w przypadku Prometheusa). Wykres o nazwie *Volume Space Usage* wskazuje zajętość danego dysku - linia koloru zielonego oznacza zajętość dysku natomiast linia żółta wolne miejsce danego dysku. Po prawej stronie dashboardu znajduje się wykres kołowy prezentujący procentowe zużycie dysku. Wykresy w dolnym rzędzie prezentują liczbę stworzonych i-nodów [194]. Wykres po lewej prezentuje liczbę stworzonych i-nodów oraz maksymalną liczbę możliwych i-nodów. Wykres kołowy po prawej mówi o tym jakie jest procentowe wykorzystanie dostępnych i-nodów.

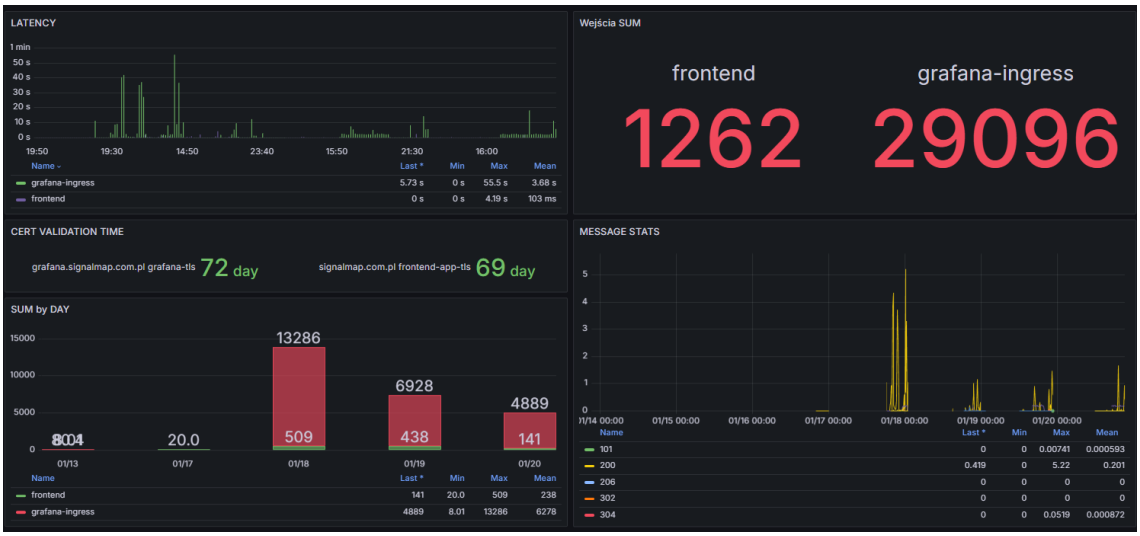


Rysunek 5.8: Dane dotyczące PVC dla narzędzia Grafana



Rysunek 5.9: Dane dotyczące PVC dla narzędzia Prometheus

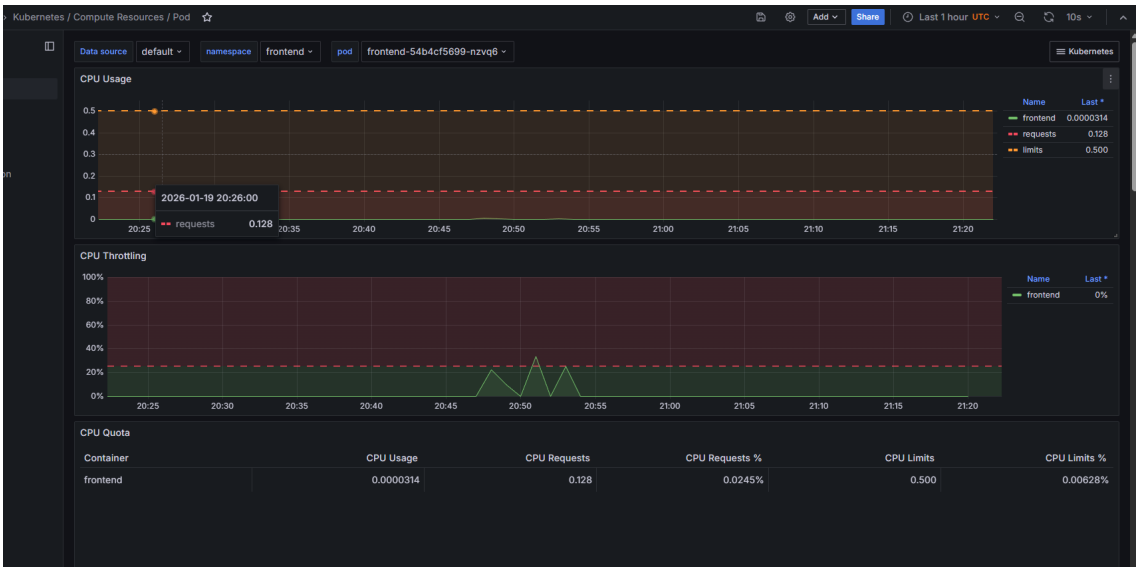
Dashboard zaprezentowany na rysunku 5.10 jest istotny w kontekście analizowania ruchu na stronie. Przedstawia on podstawowe informacje o Podach Nginx Controller. Wykres o nazwie *LATENCY* pokazuje czas odpowiedzi serwera na zapytanie. Kolor zielony pokazuje informacje na temat Ingress dla Grafany. Widać, że czas odpowiedzi porafi się zmieniać lecz średnia na poziomie 3.68s jest dobrym wynikiem przy dużej ilości danych przetwarzanych przez Grafanę. Dla Ingress związanego z aplikacja frontendową wynik jest na poziomie 103ms co jest poprawną wartością dla dobrego odbioru strony przez użytkowników. Wykres po prawej o nazwie *Wejście SUM* pokazuje sume ilości wejść na stronę w ciągu ostatniego tygodnia. Duża wartosc wejść na Grafane jest spowodowana tym, że każdy inny URI jest traktowany jako nowe wejście na stronę. Wykres *Cert Validation Time* pokazuje ile dni zostało do wygaśnięcia ważności certyfikatów TLS/SSL. Gdy ważność będzie wynosić 30 dni Cert-Manager automatycznie rozpocznie negocjację nowego certyfikatu. Ważnym z kontekście analizy ruchu jest wykres *Sum by Day*, który pokazuje rozkład wejść na stronę w przeciągu tygodnia rozbijając dane na poszczególne dni. Ostatni wykres *Message Stats* pokazuje wykres czasowy z uwzględnieniem danych odnośnie odpowiedzi jakie zawraca serwer (status HTTP). Większość odpowiedzi to kod 200 czyli poprawna obsługa żądania klienta.



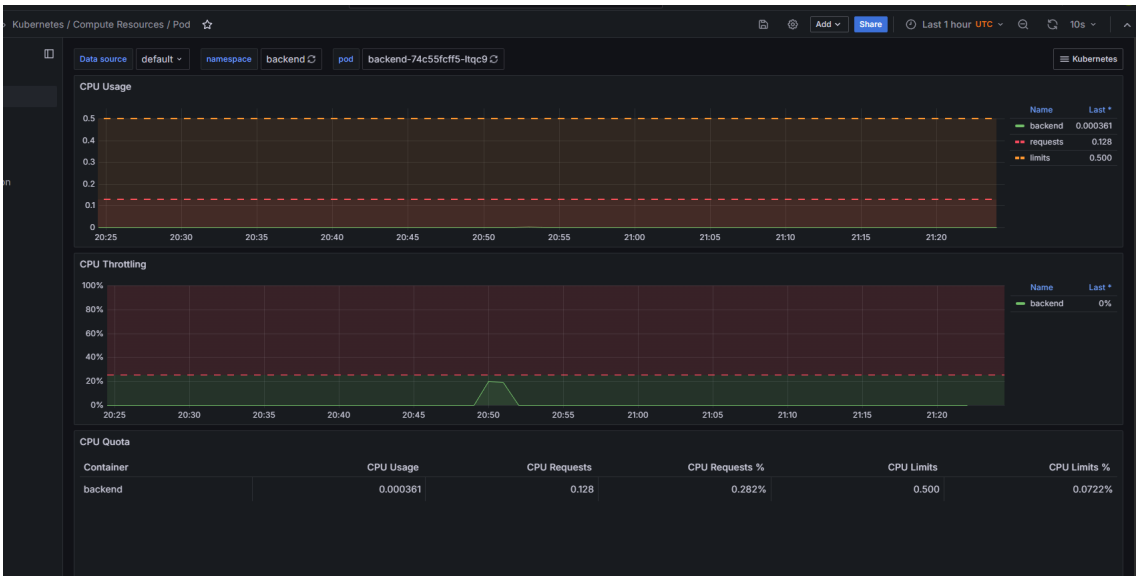
Rysunek 5.10: Dane dotyczące narzędzia Nginx Controller

Następne dwa rysunki (rys 5.11 oraz rys 5.12) przedstawiają zużycie zasobów CPU dla aplikacji frontendowej i backendowej. Wykresy przedstawiają wartość *CPU Requests 128m core* oraz *CPU Limit - 500m core*. Wartości te odzwierciadlają to co zostało ustawione podczas Deploymentu

w HelmRelease tych aplikacji. Wykresy także posiadają informacje o samym zużyciu rzeczywistym aplikacji. Jest to przedstawione przez wartość *CPU Usage* - widać, że wartości te są bardzo niskie (na poziomie 0,2-0,3 zarezerwowanego CPU). Pokazuje to, że aplikacja ma możliwość skalowalności w momencie obciążenia.



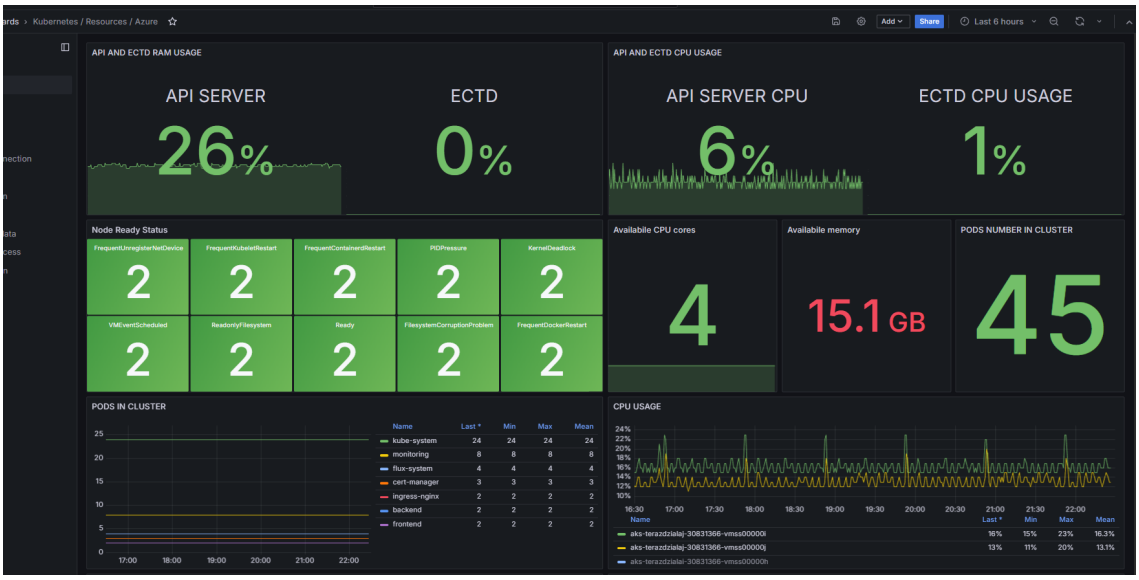
Rysunek 5.11: Dane dotyczące wykorzystywania zasobów przez Pod aplikacji frontendowej



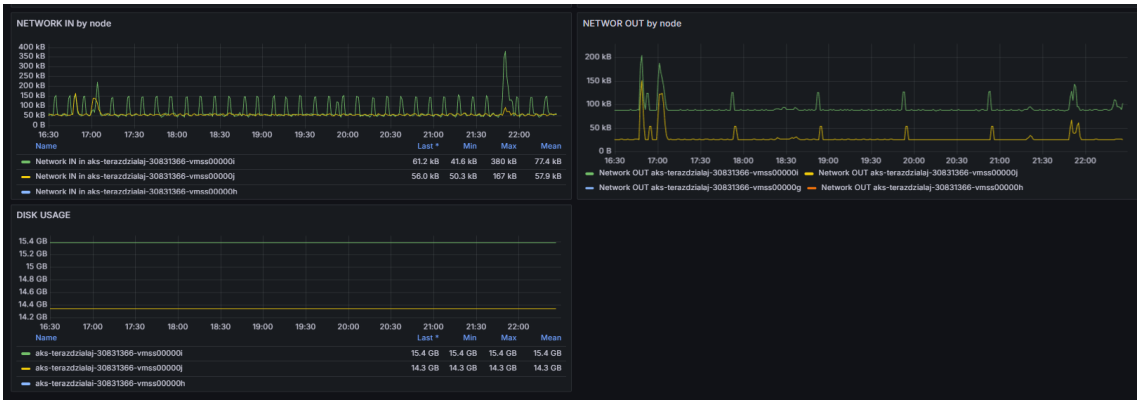
Rysunek 5.12: Dane dotyczące wykorzystywania zasobów przez Pod aplikacji backendowej

Kolejny dashboard (rys. 5.11 oraz 5.12) przedstawia informacje przekazywane przez platformę Azure odnośnie podstawowych danych związanych z usługą AKS. Pierwszy wiersz przedstawia dane na temat użycia zasobów CPU oraz RAM przez dwa podstawowa procesy Kubernetes: kubectl API oraz etcd. Wiersz niżej znajduje się tabela przedstawiająca status kluczowych komponentów działających na Nodach. Wartość 2 przy każdym komponencie oznacza, że na dwóch Nodach poprawnie działają one poprawnie. W sytuacji gdy wartość wynosi zero lub jeden może to wskazywać na problem z którymś z Nodów. Następnie po prawej stronie znajdują się wykresy zawierające informacje o ilości CPU klastra, wolnej pamięci RAM oraz suma Podów działająca w całym klastrze. W trzecim wierszu prezentowane są dane na temat liczbie poszczególnych Podów w Namespace.

Na drugim rysunku (rys. 5.14) znajdują się informacje na temat ruchu wchodzącego oraz wychodzącego z podziałem na konkretnego Noda. Ostatni wykres przekazuje dane odnośnie zajętości dysków konkretnej maszyny wirtualnej pracującej w VMSS.

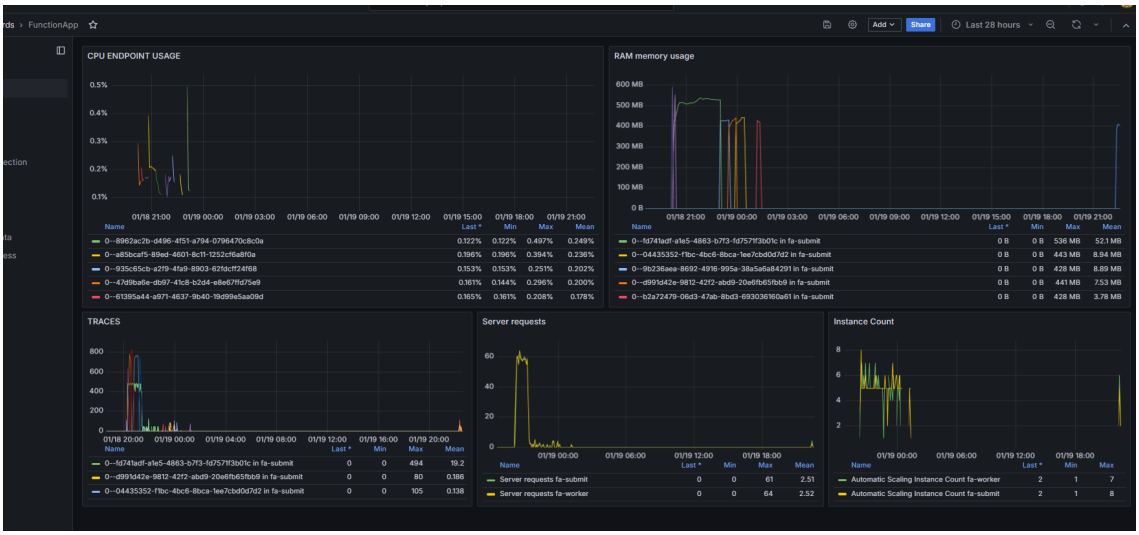


Rysunek 5.13: Dane dotyczące wykorzystywania zasobów VMSS na platformie Azure część 1



Rysunek 5.14: Dane dotyczące wykorzystywania zasobów VMSS na platformie Azure część 2

Ostatni prezentowany dashboard został przygotowany w celu śledzenia pracy usług FunctionApp. Ze względu na ustawienie tych instancji na konkretny *trigger HTTP* dane są generowane tylko w sytuacji gdy FunctionApp jest uruchamiany przez ten trigger. Wykres *CPU Endpoint USAGE* przedstawia procent zużycia zasobów CPU natomiast wykres *RAM Memory Usage* wizualizuje wartość wykorzystywanej pamięci RAM przez FunctionApp. W drugim wierszu znajdują się informacje o ilości generowanych śladów (TRACES) co jest skorelowane z ilością *Server Request* przedstawionych na wykresie obok. Ostatni wykres przedstawia liczbę instancji konkretnych FunctionApp przedstawionych na wykresie czasowym. Wizualizacja ta pokazuje, że w momencie zwiększonego ruchu Azure tworzy dodatkowe instancje do obsługi żądań.



Rysunek 5.15: Dane dotyczące wykorzystywania zasobów oraz pracy FunctionApp

Rozdział 6

Wnioski i kierunki rozwoju

6.1 Ocena rozwiązania

Opracowany system crowdsourcingowego monitoringu jakości sieci komórkowych stanowi zintegrowane rozwiązanie obejmujące mobilną aplikację pomiarową, skalowalne środowisko chmurowe oraz platformę webową do wizualizacji i analizy danych. Implementacja została oparta na architekturze chmurowej z wykorzystaniem konteneryzacji oraz orkiestracji Kubernetes, co umożliwia elastyczne zarządzanie zasobami, skalowanie obciążenia oraz zwiększenie odporności systemu na awarie.

Zaprojektowane rozwiązanie realizuje założone cele pracy, umożliwiając pozyskiwanie oraz analizę danych dotyczących jakości usług telekomunikacyjnych w warunkach rzeczywistego użytkowania. System pozwala na agregację i wizualizację parametrów sieci komórkowych, dostarczając użytkownikom narzędzi do porównawczej oceny dostępności oraz wydajności infrastruktury operatorów.

Uzyskane wyniki wskazują, że podejście oparte na crowdsourcingu może stanowić uzupełnienie profesjonalnych pomiarów wykonywanych przez operatorów telekomunikacyjnych, szczególnie w zakresie analizy doświadczenia użytkownika końcowego oraz identyfikacji obszarów o obniżonej jakości usług.

Jednocześnie należy podkreślić, że system posiada określone ograniczenia, wynikające między innymi z zależności od dostępności użytkowników końcowych, zmienności warunków radiowych oraz ograniczeń sprzętowych urządzeń mobilnych. Czynniki te wpływają na równomierność próbkowania danych oraz precyzję odwzorowania warunków sieciowych w skali całego obszaru geograficznego.

6.2 Możliwości dalszego rozwoju

Pomimo zrealizowania założonej funkcjonalności systemu, zidentyfikowano szereg obszarów, których rozwój w przyszłych iteracjach projektu pozwoliłby na podniesienie użyteczności oraz precyzji systemu *SignalMap*.

6.2.1 Aplikacja mobilna

Do najważniejszych kierunków rozwoju aplikacji mobilnej należą:

- **Lokalna historia pomiarów** – wdrożenie mechanizmu lokalnego przechowywanie danych. Rozwiązanie to pozwoliłoby na przechowywanie określonej liczby ostatnich rekordów (np. pięciu ostatnich pomiarów telemetrii oraz testów wydajnościowych) bezpośrednio w pamięci

urządzenia. Użytkownik zyskałby dzięki temu możliwość analizy trendów jakości łącza w specyficznych lokalizacjach, porównując bieżące parametry z wynikami historycznymi bez konieczności komunikacji z bazą danych w chmurze. System mógłby oferować elastyczne podejście do zarządzania tymi danymi: automatyczne nadpisywania najstarszych pomiarów lub tryb selektywny, w którym to użytkownik decyduje o trwałym zapisaniu konkretnego wyniku jako punktu referencyjnego.

- **Dynamiczna optymalizacja węzłów pomiarowych** – wprowadzenie algorytmu inteligentnego przydziału serwera do testów wydajnościowych w oparciu o aktualną geolokalizację użytkownika. Automatyczny wybór serwera o najniższym opóźnieniu pozwoliłby zminimalizować wpływ opóźnień w sieciach operatorów zewnętrznych. Dzięki temu wynik testu odzwierciedlałby rzeczywistą wydajność samego łącza radiowego bez zafałszowań wynikających z długiej trasy przesyłu danych.

- **Inteligentne zarządzanie energią i adaptacyjne próbkowanie** – wprowadzenie zróżnicowanych trybów pracy pozwoliłoby na elastyczne dostosowanie intensywności pomiarów do preferencji użytkownika, oferując wybór między gęstą siatką próbek a oszczędnością baterii. Funkcjonalność ta stanowiłaby fundament pod dalszą ewolucję algorytmów opisanych w rozdziale 4.2.1, polegającą na połączeniu decyzji użytkownika z automatyczną logiką adaptacyjną. W takim modelu system mógłby wykorzystywać historyczne dane o „martwych strefach" (ang. *dead zones*) do inteligentnego zwiększania częstotliwości pracy modułów lokalizacyjnych i radiowych tylko w obszarach o niestabilnym zasięgu. Pozwoliłoby to na precyzyjne mapowanie granic sektorów przy jednoczesnym ograniczeniu aktywności komponentów sprzętowych tam, gdzie parametry sieci są stabilne, co optymalizuje czas pracy urządzenia bez utraty kluczowych danych telemetrycznych.

- **Korelacja danych pasywnych i aktywnych** – modyfikacja silnika pomiarowego w celu automatycznego gromadzenia próbek telemetrycznych dokładnie w trakcie trwania aktywnego testu prędkości. Pozwoliłoby to na wyliczenie średnich wartości parametrów fizycznych (RSRP, SINR) dla konkretnego wyniku przepustowości, budując rzetelny model zależności wydajności od jakości sygnału.

- **Ekspansja międzyplatformowa** – opracowanie wersji aplikacji dla systemu iOS oraz innych systemów mobilnych (np. w oparciu o frameworki typu *cross-platform*). Ze względu na specyficzne restrykcje systemowe systemu iOS, ograniczające bezpośredni dostęp do szczegółowych statystyk modemu komórkowego (tzw. *Cellular Radio Information*), wersja ta koncentrowałaby się na module aktywnych testów wydajnościowych oraz danych dostępnych poprzez *NetworkExtension framework* [195]. Pozwoliłoby to na dalsze wzbogacanie bazy danych o pomiary z różnych ekosystemów, zwiększając wiarygodność generowanych map.

6.2.2 Warstwa zbierania danych i infrastruktura chmurowa

Rozwój infrastruktury serwerowej powinien koncentrować się na zwiększeniu odporności systemu na ataki, poprawie precyzji pomiarów oraz optymalizacji zarządzania danymi wrażliwymi. Do kluczowych kierunków rozwoju należą:

- **Wzmocnienie ochrony brzegowej** – wdrożenie usługi *Azure Web Application Firewall* (WAF) przed bramą API Management (APIM). Pozwoliłoby to na głęboką inspekcję pakietów i ochronę przed typowymi zagrożeniami (np. SQL Injection, XSS). Dodatkowo, rozbudowa polityk APIM o restrykcyjne limity żądań (*rate limiting*) przypisane do konkretnych

adresów IP oraz kluczy subskrypcji zapobiegałaby próbom ataków typu *Denial of Service* (DoS).

- **Izolacja i optymalizacja zasobów pomiarowych** – rozdzielenie logiki testów wydajnościowych od standardowej telemetrii poprzez dedykowane instancje *Azure Function Apps* lub kontenery (np. *Azure Container Apps*). Zapewnienie osobnych zasobów obliczeniowych dla modułu *speedtest* wyeliminowałoby ryzyko wzajemnego wpływu procesów na zajętość procesora i pamięci RAM, co bezpośrednio przełożyłoby się na wyższą wiarygodność wyników.
- **Wdrożenie protokołu mTLS** – rozszerzenie obecnego modelu autoryzacji o mechanizm **mTLS** (*mutual TLS*). Wprowadzenie weryfikacji dwustronnej przy użyciu unikalnych certyfikatów urządzeń stanowiłoby dodatkową barierę ochronną dla infrastruktury chmurowej, zgodnie ze standardem *RFC 8705* [196]. Zabezpiecza to system przed próbami wstrzykiwania danych przez nieautoryzowane skrypty emulujące ruch z aplikacji, ponieważ każde żądanie wymagałoby posiadania ważnego klucza prywatnego skojarzonego z certyfikatem zainstalowanym na urządzeniu.
- **Przeniesienie weryfikacji tokenów do warstwy bramy API** – wykorzystanie zaawansowanych funkcji *Azure APIM* (dostępnych w wyższych planach cenowych) do natywnej walidacji tokenów JWT. Takie podejście odciążyłoby funkcje backendowe, które otrzymywałyby wyłącznie wstępnie zweryfikowane i bezpieczne żądania, redukując tym samym opóźnienia i koszty operacyjne infrastruktury.
- **Zaawansowana filtracja i weryfikacja integralności danych** – wdrożenie modułów analitycznych opartych na algorytmach uczenia maszynowego w celu wykrywania anomalii i danych zakłamanych (np. nierealnych prędkości przesyłu lub błędnych koordynatów GPS). System mógłby automatycznie oznaczać rekordy odbiegające od wzorców fizycznych jako niewiarygodne, co zapobiegałoby zanieczyszczeniu bazy danych przez błędy sprzętowe urządzeń końcowych lub celowe manipulacje użytkowników.

6.2.3 Warstwa bazy danych

Dalszy rozwój warstwy składowania danych powinien koncentrować się na poprawie relacyjności modelu oraz zwiększeniu autonomii systemu w zakresie identyfikacji infrastruktury telekomunikacyjnej. Do kluczowych kierunków rozwoju należą:

- **Rozbudowa bazy o autorski rejestr stacji bazowych** – wdrożenie dedykowanej tabeli przechowującej dane o stacjach bazowych (BTS), które nie figurują w zewnętrznych rejestrach (np. *btsearch.pl*). Pozwoliłoby to na gromadzenie unikalnych informacji o nowo powstałych masztach zidentyfikowanych przez użytkowników aplikacji *SignalMap*. Taka struktura umożliwiłaby w przyszłości dwukierunkową wymianę danych z ogólnopolskimi bazami, podnosząc wartość merytoryczną całego projektu.
- **Normalizacja relacji pomiarów z infrastrukturą** – wprowadzenie dedykowanej tabeli powiązań (*mapping table*), która służyłaby do trwałego przypisywania konkretnych próbek telemetrycznych do zidentyfikowanych stacji bazowych na poziomie bazy danych, a nie tylko w logice backendu. Przeniesienie tej odpowiedzialności do warstwy bazodanowej (poprzez klucze obce i relacje) zapewniłoby wyższą integralność danych oraz umożliwiło szybsze generowanie raportów dotyczących obciążenia i zasięgu konkretnych nadajników.

- **Automatyzacja aktualizacji bazy stacji bazowych** – wdrożenie mechanizmu cyklicznego importu danych z zewnętrznych rejestrów (np. *btsearch.pl*) przy wykorzystaniu zadań harmonogramowanych (*Cron jobs* lub *Azure Function Timer Trigger*). System automatycznie wykrywałby publikację nowych zestawień infrastruktury i przeprowadzał proces ujednolicania danych do bazy docelowej. Gwarantowałoby to wysoką aktualność danych referencyjnych bez konieczności ręcznej interwencji administratora, co jest niezbędne dla poprawnego działania algorytmów dopasowujących pomiary użytkowników do rzeczywistej mapy nadajników.
- **Zaawansowana polityka ciągłości danych i retencji** – wdrożenie autorskiego systemu kopii zapasowych, uzupełniającego standardowe mechanizmy dostawcy chmurowego. Nowa polityka zakładałaby częstsze wykonywanie zrzutów danych (*snapshots*) oraz implementację skryptów automatyzujących proces przywracania bazy w przypadku awarii. Pozwoliłoby to na minimalizację parametru RPO (*Recovery Point Objective*), zapewniając niemal zerową utratę danych pomiarowych zebranych od społeczności.

6.2.4 Wizualizacja mapy i moduł analityczny

Rozwój warstwy prezentacji danych powinien dążyć do zwiększenia czytelności mapy oraz dostarczenia użytkownikom pogłębionej wiedzy o jakości usług w ich regionie. Do kluczowych kierunków rozwoju należą:

- **Normalizacja kolorystyki względem technologii sieciowej** Implementacja schematu kolorów uwzględniającego specyfikę poszczególnych standardów telekomunikacyjnych (2G, 3G, LTE, 5G). Obecny system wizualizacji stosuje jednolite progi dla wszystkich technologii, co może prowadzić do błędnej interpretacji danych – punkt pomiarowy 3G jest w tej samej skali co pomiar 5G, mimo że sposób pomiaru mocy sygnału tych technologii są znacząco różne. Wprowadzenie zróżnicowanych skal kolorystycznych pozwoliłoby na precyzyjniejszą ocenę jakości usług w kontekście wykorzystywanej technologii.
- **Agregacja punktów pomiarowych** Wdrożenie algorytmów grupowania dla gęsto rozmieszczonych punktów pomiarowych oraz anten bazowych, które zapobiegałyby ich wizualnemu nakładaniu się przy niskich poziomach przybliżenia mapy. W obecnej implementacji obszary o wysokiej gęstości pomiarów (np. centra miast) stają się nieczytelne z uwagi na wzajemne zasłanianie się punktów. Zastosowanie dynamicznej agregacji z wyświetlaniem liczby zgrupowanych obiektów oraz ich średnich parametrów poprawiłoby czytelność mapy i umożliwiło efektywną nawigację po gęsto pokrytych obszarach.
- **Optymalizacja ładowania danych przez bounding box** Ograniczenie pobierania danych telemetrycznych wyłącznie do obszaru aktualnie widocznego w oknie mapy. W obecnym rozwiązaniu aplikacja pobiera wszystkie dane z określonego przedziału czasowego, co przy dużych zbiorach punktów może skutkować nadmiernym obciążeniem pamięci przeglądarki. Implementacja parametru *bounding box* w zapytaniach do API wraz z mechanizmem dynamicznego doładowywania przy zmianie widoku znacząco poprawiłaby responsywność interfejsu oraz zmniejszyła wykorzystanie zasobów systemowych.
- **Geolokalizacja użytkownika i wyszukiwanie miejsc** Wprowadzenie funkcjonalności automatycznego centrowania mapy na podstawie lokalizacji użytkownika pozyskanej z API przeglądarki (*Geolocation API*) oraz zintegrowanego mechanizmu wyszukiwania miejscowości i adresów z podpowiedziami autouzupełniającymi opartego o usługę MapTiler Geocoding.

Usprawnienia te znacząco zwiększyłyby wygodę użytkowania poprzez eliminację konieczności ręcznego nawigowania po mapie w celu odnalezienia interesującego obszaru.

- **Ranking operatorów dla wybranych lokalizacji** Rozbudowa panelu analitycznego o moduł generowania dynamicznych rankingów dostawców usług telekomunikacyjnych dla interaktywnie wybranych obszarów (dzielnic, miast). System prezentowałby zestawienie zawierające średnią moc sygnału, przeciętną prędkość transmisji danych oraz procentowe pokrycie terenu, umożliwiając użytkownikom obiektywne porównanie jakości usług konkurencyjnych operatorów w wybranej lokalizacji.
- **Responsywność i adaptacja dla urządzeń mobilnych** Optymalizacja interfejsu użytkownika pod kątem urządzeń dotykowych poprzez zwiększenie obszarów interakcji, dostosowanie rozmiarów czcionek i elementów graficznych. Usprawnienia te mogłby być kluczowe ze względu na dominujący udział urządzeń mobilnych w dostępie do aplikacji typu crowdsourcingowej.
- **Porównanie stanu sieci przed i po modernizacji** Wprowadzenie trybu prezentacji równoległej umożliwiającego wizualizację danych z dwóch różnych przedziałów czasowych w celu obserwacji efektów modernizacji infrastruktury sieciowej przez operatorów. Funkcjonalność ta pozwoliłaby na obiektywną ocenę skuteczności rozbudowy sieci oraz identyfikację obszarów, które uległy faktycznej poprawie jakości usług.

6.2.5 Mechanizmy klastra Kubernetes

W zaimplementowanym środowisku nie zastosowano mechanizmu buforowania (*caching*) odpowiedzi generowanych przez warstwę backendową. Przy rosnącej liczbie rekordów telemetrycznych brak cache może powodować zwiększenie czasu odpowiedzi API, a w konsekwencji wydłużać czas ładowania warstwy mapowej w aplikacji webowej. Jednym z praktycznych kierunków rozwoju jest wprowadzenie pamięci podręcznej opartej o Redis, uruchomionej wewnątrz klastra Kubernetes jako wdrożenie typu *Deployment/StatefulSet* wraz z usługą *Service* udostępniającą endpoint wewnątrz sieci klastra.

W takim podejściu to **backend** (a nie aplikacja frontendowa) odpowiadałby za obsługę cache: przed wykonaniem zapytań do bazy danych aplikacja serwerowa weryfikowałaby, czy wynik dla danego zestawu parametrów (np. zakres czasu, technologia, operator, *bounding box*) jest dostępny w Redis. Jeśli dane znajdowałyby się w pamięci podręcznej, backend zwracałby odpowiedź natychmiast, ograniczając liczbę odwołań do bazy danych. W przeciwnym przypadku backend pobierałby dane z PostgreSQL, a następnie zapisywał wynik w Redis z ustalonym czasem życia wpisu (*TTL*), co poprawiałoby wydajność kolejnych zapytań o analogicznym charakterze. Uzupełnieniem tego rozwiązania powinna być obserwowalność komponentu cache, np. poprzez eksport metryk i ich wizualizację w narzędziach takich jak Prometheus i Grafana.

Kolejnym obszarem możliwej poprawy jest zwiększenie izolacji komponentów rejestru obrazów kontenerów. W niniejszej implementacji wykorzystano *Azure Container Registry* dostępny poprzez publiczny endpoint, przy czym dostęp do operacji na rejestrze został ograniczony mechanizmami RBAC. W wariancie o wyższym poziomie bezpieczeństwa możliwe jest zastosowanie *Private Endpoint* dla ACR, co umożliwia komunikację z rejestrem wyłącznie w obrębie sieci wirtualnej Azure (VNet) i eliminuje konieczność wystawiania usługi na publiczny Internet. Rozwiązanie to wiąże się jednak z dodatkowymi kosztami oraz koniecznością utrzymania spójnej konfiguracji sieciowej, dlatego w ramach niniejszego projektu przyjęto wariant podstawowy.

Ostatnim istotnym kierunkiem rozwoju jest dobór mechanizmu CNI (*Container Network Interface*) adekwatnego do docelowej skali klastra i wymagań bezpieczeństwa. W konfiguracji domyślnej *Azure CNI* przydziela Podom adresy IP z tej samej puli co węzłom (*Node*) w obrębie VNet, co w praktyce zwiększa zużycie adresów i może ograniczać skalowalność klastra w scenariuszach wymagających uruchamiania dużej liczby Podów. Alternatywą jest zastosowanie *Azure CNI Overlay*, które wykorzystuje adresację nakładkową (*overlay*) dla Podów i ogranicza zużycie adresów w VNet, lub wdrożenie rozwiązania takiego jak Calico, oferującego rozbudowane mechanizmy polityk sieciowych oraz segmentacji ruchu. Dobór CNI powinien uwzględniać zarówno wymagania skalowalności, jak i potrzeby w zakresie egzekwowania polityk bezpieczeństwa (np. *NetworkPolicy*) oraz izolacji komunikacji pomiędzy usługami. Poniższa tabela pokazuje różnice miedzy poszczególnymi CNI: [197] [198]

Tabela 6.1: Porównanie mechanizmów CNI w klastrze AKS

Cecha	**Azure CNI**	**Azure CNI Overlay**	**Calico**
Adresacja podów	Adresy IP z VNet Azure	Adresy IP z puli overlay	Adresy IP z puli overlay
Zużycie adresów IP	Wysokie (IP dla każdego poda)	Niskie	Niskie
Skalowalność klastra	Ograniczona przez pulę IP VNet	Znacznie lepsza	Bardzo dobra
Integracja z Azure	Pełna, natywna	Pełna, natywna	Częściowa
Obsługa NetworkPolicy	Podstawowa	Podstawowa	Zaawansowana
Izolacja ruchu	Ograniczona	Średnia	Wysoka
Złożoność konfiguracji	Niska	Średnia	Wyższa
Wsparcie dla polityk bezpieczeństwa	Ograniczone	Ograniczone	Rozbudowane (L3/L4, opcjonalnie L7)
Zastosowanie produkcyjne	Małe i średnie klastry	Średnie i duże klastry	Klastry o wysokich wymaganiach bezpieczeństwa
Koszt operacyjny	Niski	Niski	Średni

6.3 Podsumowanie

Celem pracy było zaprojektowanie oraz implementacja systemu monitoringu jakości sieci komórkowych, umożliwiającego gromadzenie, przetwarzanie i wizualizację danych telemetrycznych pozyskiwanych z urządzeń mobilnych użytkowników w warunkach rzeczywistego użytkowania sieci. Założony cel został zrealizowany poprzez opracowanie kompletnego rozwiązania obejmującego aplikację mobilną do wykonywania pomiarów, skalowalną infrastrukturę chmurową do przetwarzania danych oraz interaktywną aplikację webową umożliwiającą analizę wyników.

W ramach pracy przeprowadzono analizę domeny problemu oraz określono wymagania funkcjonalne i niefunkcjonalne systemu, które następnie zostały odzwierciedlone w modularnej architekturze opartej na technologiach chmurowych Microsoft Azure. Zaprojektowany system umożliwia bezpieczne zbieranie danych pomiarowych, ich asynchroniczne przetwarzanie oraz trwałe przechowywanie w relacyjnej bazie danych. Warstwa wizualizacji pozwala na analizę jakości usług telekomunikacyjnych zarówno w ujęciu przestrzennym, jak i statystycznym, zwiększając użyteczność zgromadzonych informacji dla użytkowników oraz potencjalnych interesariuszy.

Istotnym aspektem pracy było wykorzystanie orkiestracji kontenerów Kubernetes, co zapewniło wysoką dostępność usług, odporność na awarie oraz możliwość dynamicznego skalowania systemu wraz ze wzrostem liczby użytkowników i wolumenu przetwarzanych danych. Szczególną uwagę poświęcono również zagadnieniom bezpieczeństwa, obejmującym autoryzację, ochronę danych użyt-

kowników oraz zapewnienie integralności i poufności informacji telemetrycznych, co jest kluczowe w systemach opartych na modelu crowdsourcingowym.

Opracowane rozwiązanie posiada charakter praktyczny i może stanowić podstawę do dalszych badań nad jakością usług telekomunikacyjnych, jak również zostać wykorzystane jako narzędzie analityczne wspierające ocenę pokrycia, wydajności oraz niezawodności sieci komórkowych. Architektura systemu umożliwia jego dalszą rozbudowę, w szczególności o dodatkowe moduły analityczne, mechanizmy predykcyjne oraz integrację z zewnętrznymi źródłami danych.

Podsumowując, realizacja pracy umożliwiła połączenie wiedzy z zakresu sieci telekomunikacyjnych, przetwarzania danych oraz technologii chmurowych w spójne i funkcjonalne rozwiązanie inżynierskie. Opracowany system spełnia założone cele projektowe oraz tworzy solidną podstawę do dalszego rozwoju i potencjalnych wdrożeń w środowisku produkcyjnym.

Literatura

[1] L. Kowalik. Telecom operator's approach to quality of experience. *Journal of Telecommunications and Information Technology*, 2022.

[2] Metavshn. Understanding qos and qoe in telecoms: A comprehensive guide, 2025. misc article.

[3] AVSystem. Quality of experience (qoe) in telecommunication services, 2020. Blog post.

[4] Synaptique. Enhancing telecom quality of service (qos) through advanced data monitoring in competitive markets, 2025. Whitepaper / blog.

[5] Orange Wholesale International. Monitoring & reporting: Ss7, lte and ipx, 2024. Product page.

[6] Nokia Siemens Networks. Nokia siemens networks netact: Company profile & oss/bss. Technical profile, 2025.

[7] Slashdot Media. Top qoe/qos monitoring software in 2025, 2024. Product overview.

[8] Synaptique. Best practices for quality of service (qos) monitoring in telecom: Getting the most from your data, 2025. Blog / best practices.

[9] M. Fiedler et al. Crowdsourced network and qoe measurements: A white paper. Technical report, Qualinet, 2020.

[10] GSMA. The State of Mobile Internet Connectivity 2024. https://www.gsma.com/r/wp-content/uploads/2024/10/The-State-of-Mobile-Internet-Connectivity-Report-2024.pdf. Dostęp: listopad 2025.

[11] Davide Micheli, G. Muratore, A. Vannelli, A. Scaloni, M. Sgheiz, and P. Cirella. Rain effect on 4g lte in-car electromagnetic propagation analyzed through mdt radio data measurement reported by mobile phones. https://www.nokia.com/bell-labs/publications-and-media/publications/rain-effect-on-4g-lte-in-car-electromagnetic-propagation-analyzed-through-mdt-radio-data-measurement-reported-by-mobile-phones/, 2021. IEEE Transactions on Antennas and Propagation, vol. 69, no. 12, pp. 8641-8651, DOI:10.1109/TAP.2021.3090505; Dostęp: styczeń 2026.

[12] Wikipedia. Transport layer security. https://pl.wikipedia.org/wiki/Transport_Layer_Security, 2026. Dostęp: styczeń 2026.

[13] Microsoft. Rozważania dotyczące łańcucha narzędzi devops. https://learn.microsoft.com/pl-pl/azure/cloud-adoption-framework/ready/considerations/devops-toolchain, 2026. Dostęp: styczeń 2026.

[14] Microsoft. Azure devops. https://azure.microsoft.com/pl-pl/products/devops, 2026. Dostęp: styczeń 2026.

[15] The Helm Authors. Helm — the kubernetes package manager. https://helm.sh/, 2026. Dostęp: styczeń 2026.

[16] FluxCD Authors. Flux — gitops toolkit for kubernetes. https://fluxcd.io/flux/, 2026. Dostęp: styczeń 2026.

[17] FluxCD Authors. Kustomizations in flux. https://fluxcd.io/flux/components/kustomize/kustomizations/, 2026. Dostęp: styczeń 2026.

[18] Microsoft. Azure container registry. https://learn.microsoft.com/pl-pl/azure/container-registry/, 2026. Dostęp: styczeń 2026.

[19] Kubernetes Authors. Kubernetes documentation. https://kubernetes.io/docs/home/, 2026. Dostęp: styczeń 2026.

[20] Wikipedia. Microsoft azure. https://en.wikipedia.org/wiki/Microsoft_Azure. Dostęp: styczeń 2026.

[21] Wikipedia. Quality of service. https://en.wikipedia.org/wiki/Quality_of_service. Dostęp: styczeń 2026.

[22] Wikipedia. Quality of experience. https://en.wikipedia.org/wiki/Quality_of_experience. Dostęp: styczeń 2026.

[23] Orange Polska. Co to jest qos i jak monitorować jego parametry. https://www.orange.pl/poradnik-dla-firm/siec-i-transmisja/co-to-jest-qos/. Dostęp: styczeń 2026.

[24] Mohammed Alreshoodi and John Woods. Survey on qoe/qos correlation models for multimedia services. *arXiv preprint arXiv:1306.0221*, 2013. Dostęp: styczeń 2026.

[25] Android Developers. Telephonymanager — android api reference. https://developer.android.com/reference/android/telephony/TelephonyManager, 2025. Dostęp: listopad 2025.

[26] Poynting Antennas. Signal strength measure rsrp, rsrq and sinr reference for lte & 5g. https://poynting.tech/articles/antenna-faq/signal-strength-measure-rsrp-rsrq-and-sinr-reference-for-lte-cheat-sheet/. Dostęp: styczeń 2026.

[27] Eseye. Understanding signal quality: Rssi, rsrp, and rsrq. https://www.eseye.com/resources/iot-explained/understanding-signal-quality-rssi-rsrp-and-rsrq/. Dostęp: styczeń 2026.

[28] Ladislav Polák, Jan Kufa, Roman Sotner, and Tomas Fryza. Measurement and analysis of 4g/5g mobile signal coverage in a heavy industry environment. *Sensors*, 24(8):2538, 2024. Dostęp: styczeń 2026.

[29] StatCounter Global Stats. Mobile operating system market share worldwide. https://gs.statcounter.com/os-market-share/mobile/worldwide, October 2025. Dostęp: listopad 2025.

[30] Apple Developer. Coretelephony framework. https://developer.apple.com/documentation/coretelephony, 2025. Dostęp: listopad 2025.

[31] OpenSignal. Opensignal – mobile connectivity and network signal analytics app. https://en.wikipedia.org/wiki/Opensignal. Dostęp: styczeń 2026.

[32] nPerf. nperf mobile network and internet speed test application. https://www.nperf.com/en/nperf-applications. Dostęp: styczeń 2026.

[33] Google Play Store. Network cell info lite & wifi – android app. https://play.google.com/store/apps/details?id=com.wilysis.cellinfolite. Dostęp: styczeń 2026.

[34] Google Play Store. Opisy aplikacji: Opensignal, npref, network cell info. https://play.google.com/store, 2025. Dostęp: listopad 2025.

[35] Google Play Store. Opensignal – 5g, 4g speed test. https://play.google.com/store/apps/details?hl=pl&id=com.staircase3.opensignal. Zawiera informację o liczbie pobrań (>10,000,000), dostęp: styczeń 2026.

[36] Google Play Store. nperf – network and internet speed test. https://play.google.com/store/apps/details?hl=pl&id=com.nperf.tester. Zawiera informację o liczbie pobrań (>10,000,000), dostęp: styczeń 2026.

[37] Giorgio Brajnik and Silvia Gabrielli. A review of misc advertising effects on the user experience. *International Journal of Human–Computer Interaction*, 26(10):971–997, 2010. Dostęp: styczeń 2026.

[38] LTE Physical Cell ID Reference. Lte physical cell id is an identification of a cell at the physical layer. https://www.sharetechnote.com/html/Handbook_LTE_PCI.html. Dostęp: styczeń 2026.

[39] Telecom Hall. What is the formula for cell id (eci) in lte networks? https://www.telecomhall.net/t/what-is-the-formula-for-cell-id-eci-in-lte-networks/93. Dostęp: styczeń 2026.

[40] Wikipedia. Cell identifier. https://pl.wikipedia.org/wiki/Cell_Identifier. Dostęp: styczeń 2026.

[41] Mustafa Kaplan, Kadir Keşir, and Orhan Coşkun. A comparative and measurement-based study on real-time network kpi extraction methods for 5g and beyond applications. *arXiv preprint arXiv:2504.16039*, 2025. Dostęp: listopad 2025.

[42] Wikipedia. Anonimizacja danych. https://pl.wikipedia.org/wiki/Anonimizacja_danych. Dostęp: styczeń 2026.

[43] IBM. What is data privacy? https://www.ibm.com/think/topics/data-privacy. Dostęp: styczeń 2026.

[44] Wikipedia. Cloud storage. https://en.wikipedia.org/wiki/Cloud_storage. Dostęp: styczeń 2026.

[45] GDPR-info.eu. Art. 17 gdpr – right to erasure ('right to be forgotten'). https://gdpr-info.eu/art-17-gdpr/. Dostęp: styczeń 2026.

[46] GDPR-info.eu. Art. 5 gdpr – principles relating to processing of personal data. https://gdpr-info.eu/art-5-gdpr/. Dostęp: styczeń 2026.

[47] Data Protection Commission. Principles of data protection: Lawfulness, fairness and transparency. https://www.dataprotection.ie/en/individuals/data-protection-basics/principles-data-protection. Dostęp: styczeń 2026.

[48] OpenSignal. Data privacy charter. https://www.opensignal.com/apps/data-privacy-charter, 2025. Dostęp: listopad 2025.

[49] nPerf. Privacy policy. https://www.nperf.com/en/privacy-policy, 2025. Dostęp: listopad 2025.

[50] IBM. What is cloud architecture? https://www.ibm.com/think/topics/cloud-architecture. Dostęp: styczeń 2026.

[51] Coursera Articles. What is client-server architecture? https://www.coursera.org/articles/client-server-architecture. Dostęp: styczeń 2026.

[52] OVHcloud. What is high availability (ha)? https://www.ovhcloud.com/pl/learn/what-is-high-availability/. Dostęp: styczeń 2026.

[53] Amazon Web Services. Aws global infrastructure. https://aws.amazon.com/about-aws/global-infrastructure/. Dostęp: styczeń 2026.

[54] SecurityTrails. Dns and infrastructure data for opensignal.com. https://securitytrails.com/domain/opensignal.com, November 2025. Dostęp: listopad 2025.

[55] Sedai. Azure api management overview & key concepts. https://sedai.io/blog/azure-api-management-overview-key-concepts. Dostęp: styczeń 2026.

[56] Microsoft. Czym jest infrastruktura jako kod (infrastructure as code). https://learn.microsoft.com/pl-pl/devops/deliver/what-is-infrastructure-as-code. Dostęp: styczeń 2026.

[57] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, 3rd edition, 2012.

[58] Microsoft. About api management. https://learn.microsoft.com/en-us/azure/api-management/api-management-key-concepts. Dostęp: styczeń 2026.

[59] Microsoft. What is azure virtual network? https://learn.microsoft.com/en-us/azure/virtual-network/virtual-networks-overview. Dostęp: styczeń 2026.

[60] Microsoft. Azure private endpoint. https://learn.microsoft.com/en-us/azure/private-link/private-endpoint-overview. Dostęp: styczeń 2026.

[61] Mozilla. What is https? https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview. Dostęp: styczeń 2026.

[62] Microsoft. Ip addresses in azure functions. https://learn.microsoft.com/pl-pl/azure/azure-functions/ip-addresses?tabs=portal#find-outbound-ip-addresses. Dostęp: styczeń 2026.

[63] Microsoft. Czym jest paas (platform as a service). https://azure.microsoft.com/pl-pl/resources/cloud-computing-dictionary/what-is-paas, 2026. Dostęp: styczeń 2026.

[64] Microsoft. Używanie opcji platformy jako usługi (paas) — usługi zarządzane. https://learn.microsoft.com/pl-pl/azure/architecture/guide/design-principles/managed-services. Dostęp: styczeń 2026.

[65] Microsoft. Azure service bus messaging overview. https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-messaging-overview. Dostęp: styczeń 2026.

[66] Microsoft. Azure private dns — overview. https://learn.microsoft.com/en-us/azure/dns/private-dns-overview. Dostęp: styczeń 2026.

[67] Microsoft. Czym jest azure kubernetes service (aks). https://learn.microsoft.com/pl-pl/azure/aks/what-is-aks, 2026. Dostęp: styczeń 2026.

[68] Microsoft. Azure functions documentation. https://learn.microsoft.com/en-us/azure/azure-functions/functions-overview. Dostęp: styczeń 2026.

[69] P. Castro, V. Ishakian, V. Muthusamy, and A. Slominski. The rise of serverless computing. *Communications of the ACM*, 62(12):44–54, 2019.

[70] Microsoft. Azure database for postgresql - flexible server documentation. https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/. Dostęp: styczeń 2026.

[71] E. F. Codd. *The Relational Model for Database Management: Version 2*. Addison-Wesley Longman Publishing Co., Inc., 1990.

[72] Refractions Research. Postgis introduction. https://postgis.net/documentation/. Dostęp: styczeń 2026.

[73] PostGIS Developers Group. Postgis 3.4.2 manual. https://postgis.net/docs/manual-3.4/. Dostęp: styczeń 2026.

[74] PostGIS Developers Group. Postgis geometry accessors. https://postgis.net/docs/reference.html#Geometry_Accessors. Dostęp: styczeń 2026.

[75] Daniele Varrazzo. Psycopg 2.9 documentation. https://www.psycopg.org/docs/. Dostęp: styczeń 2026.

[76] G. Hohpe and B. Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2003.

[77] Microsoft. Service bus message settlement: Peek-lock. https://learn.microsoft.com/en-us/azure/service-bus-messaging/message-transfers-locks-settlement#peeklock. Dostęp: styczeń 2026.

[78] Microsoft. Storage queues and service bus queues - compared and contrasted. https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-azure-and-service-bus-queues-compared-contrasted. Dostęp: styczeń 2026.

[79] S. Ahmed. Implementing event-driven architectures with kafka, rabbitmq, or azure service bus. https://www.c-sharpcorner.com/article/implementing-event-driven-architectures-with-kafka-rabbitmq-or-azure-service-b/, 2024. Dostęp: styczeń 2026.

[80] Microsoft. Kontrola dostępu oparta na rolach (rbac) w azure. https://learn.microsoft.com/pl-pl/azure/role-based-access-control/overview, 2026. Dostęp: styczeń 2026.

[81] Microsoft. Azure devops boards. https://azure.microsoft.com/pl-pl/products/devops/boards, 2026. Dostęp: styczeń 2026.

[82] Microsoft. Azure devops repos. https://azure.microsoft.com/pl-pl/products/devops/repos, 2026. Dostęp: styczeń 2026.

[83] Microsoft. Azure devops pipelines. https://azure.microsoft.com/pl-pl/products/devops/pipelines, 2026. Dostęp: styczeń 2026.

[84] HashiCorp. Terraform documentation. https://developer.hashicorp.com/terraform, 2026. Dostęp: styczeń 2026.

[85] Kubernetes Authors. Pody — kubernetes. https://kubernetes.io/pl/docs/concepts/workloads/pods/, 2026. Dostęp: styczeń 2026.

[86] Kubernetes Authors. Kubernetes nodes. https://kubernetes.io/docs/concepts/architecture/nodes/, 2026. Dostęp: styczeń 2026.

[87] The Helm Authors. Helm charts. https://helm.sh/docs/topics/charts/, 2026. Dostęp: styczeń 2026.

[88] OWASP Foundation. Sql injection prevention cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. Dostęp: styczeń 2026.

[89] OWASP Foundation. Cross site scripting (xss) prevention cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html. Dostęp: styczeń 2026.

[90] Microsoft. What is azure web application firewall on azure application gateway? https://learn.microsoft.com/en-us/azure/web-application-firewall/ag/ag-overview. Dostęp: styczeń 2026.

[91] Microsoft. Advanced request throttling with azure api management. https://learn.microsoft.com/en-us/azure/api-management/api-management-sample-flexible-throttling. Dostęp: styczeń 2026.

[92] Microsoft. Versions and revisions in azure api management. https://learn.microsoft.com/en-us/azure/api-management/api-management-get-started-publish-versions. Dostęp: styczeń 2026.

[93] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. Fundamenty architektury REST i zarządzania zasobami.

[94] NGINX. Nginx kubernetes ingress controller. https://github.com/nginx/kubernetes-ingress, 2026. Dostęp: styczeń 2026.

[95] cert-manager Authors. cert-manager. https://cert-manager.io/, 2026. Dostęp: styczeń 2026.

[96] Let's Encrypt. Dokumentacja let's encrypt. https://letsencrypt.org/pl/docs/, 2026. Dostęp: styczeń 2026.

[97] Let's Encrypt. How it works — let's encrypt. https://letsencrypt.org/how-it-works/, 2026. Dostęp: styczeń 2026.

[98] Microsoft. Azure key vault. https://learn.microsoft.com/pl-pl/azure/key-vault/, 2026. Dostęp: styczeń 2026.

[99] Kubernetes SIGs. Secrets store csi driver. https://secrets-store-csi-driver.sigs.k8s.io/, 2026. Dostęp: styczeń 2026.

[100] Microsoft. What is azure private link? https://learn.microsoft.com/en-us/azure/private-link/private-link-overview. Dostęp: styczeń 2026.

[101] Microsoft. What is an azure private endpoint? https://learn.microsoft.com/en-us/azure/private-link/private-endpoint-overview. Dostęp: styczeń 2026.

[102] Microsoft. What is azure private dns? https://learn.microsoft.com/en-us/azure/dns/private-dns-overview. Dostęp: styczeń 2026.

[103] A. Verma. *Cloud Native Infrastructure: Patterns for Scalable Infrastructure and Applications in a Dynamic Environment*. O'Reilly Media, 2022.

[104] Microsoft. Azure monitor — podstawy. https://learn.microsoft.com/pl-pl/azure/azure-monitor/fundamentals/overview, 2026. Dostęp: styczeń 2026.

[105] Prometheus Community. kube-prometheus-stack helm chart. https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack, 2026. Dostęp: styczeń 2026.

[106] Prometheus Operator Authors. Prometheus operator. https://prometheus-operator.dev/, 2026. Dostęp: styczeń 2026.

[107] Prometheus Authors. Prometheus — introduction and overview. https://prometheus.io/docs/introduction/overview/, 2026. Dostęp: styczeń 2026.

[108] Wikipedia. Time series database. https://en.wikipedia.org/wiki/Time_series_database, 2026. Dostęp: styczeń 2026.

[109] Grafana Labs. Grafana documentation. https://grafana.com/docs/, 2026. Dostęp: styczeń 2026.

[110] Prometheus Authors. node_exporter. https://github.com/prometheus/node_exporter, 2026. Dostęp: styczeń 2026.

[111] Kubernetes Authors. kube-state-metrics. https://github.com/kubernetes/kube-state-metrics, 2026. Dostęp: styczeń 2026.

[112] Prometheus Community. Postgresql exporter for prometheus. https://github.com/prometheus-community/postgres_exporter, 2026. Dostęp: styczeń 2026.

[113] Google. Android developers: Platform architecture. https://developer.android.com/guide/platform. Dostęp: styczeń 2026.

[114] Google. Android developers: Telephonymanager reference. https://developer.android.com/reference/android/telephony/TelephonyManager. Dostęp: styczeń 2026.

[115] S. Kohlbrecher. *High Performance Android Apps: Improve Context-Awareness, Power Use, and Memory Management*. O'Reilly Media, 2021. Sekcja: Battery Life and Background Processing.

[116] AWS. What is a restful api? https://aws.amazon.com/what-is/restful-api/. Dostęp: styczeń 2026.

[117] https://flask.palletsprojects.com/en/stable/. Dostęp: styczeń 2026.

[118] developer.mozilla.org. Spa (single-page application). https://developer.mozilla.org/en-US/docs/Glossary/SPA. Dostęp: styczeń 2026.

[119] https://react.dev/. Dostęp: styczeń 2026.

[120] https://deck.gl/. Dostęp: styczeń 2026.

[121] https://www.maptiler.com/. Dostęp: styczeń 2026.

[122] What is css. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/What_is_CSS. Dostęp: styczeń 2026.

[123] https://create-react-app.dev/. Dostęp: styczeń 2026.

[124] Gcore. Managed vs self-managed kubernetes. https://gcore.com/blog/managed-vs-self-managed-k8s, 2023. Dostęp: styczeń 2026.

[125] Microsoft Azure. Configure autoscale for virtual machine scale sets in the azure portal. https://learn.microsoft.com/en-us/azure/virtual-machine-scale-sets/virtual-machine-scale-sets-autoscale-portal, 2024. Dostęp: styczeń 2026.

[126] Microsoft Azure. Aks node minimum pods configuration. https://azure.github.io/PSRule.Rules.Azure/en/rules/Azure.AKS.NodeMinPods/, 2024. Dostęp: styczeń 2026.

[127] Microsoft Azure. Azure networking. https://learn.microsoft.com/en-us/azure/aks/core-aks-concepts, 2024. Dostęp: styczeń 2026.

[128] BTSearch. Btsearch – baza stacji bazowych w Polsce. https://www.btsearch.pl/. Dostęp: styczeń 2026.

[129] IETF. The application/json media type for javascript object notation (json). https://www.rfc-editor.org/rfc/rfc8259. Dostęp: styczeń 2026.

[130] OWASP. Input validation cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html. Dostęp: styczeń 2026.

[131] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns*. Addison-Wesley, 2003.

[132] IBM. What is batch processing? https://www.ibm.com/docs/en/zos-basic-skills?topic=jobs-what-is-batch-processing. Dostęp: styczeń 2026.

[133] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[134] Wikipedia. Base64. https://pl.wikipedia.org/wiki/Base64. Dostęp: styczeń 2026.

[135] Kubernetes. Custom resources. https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/. Dostęp: styczeń 2026.

[136] cert-manager. Issuer. https://cert-manager.io/docs/concepts/issuer/. Dostęp: styczeń 2026.

[137] cert-manager. Certificate. https://cert-manager.io/docs/usage/certificate/. Dostęp: styczeń 2026.

[138] cert-manager. Certificaterequest. https://cert-manager.io/docs/usage/certificaterequest/. Dostęp: styczeń 2026.

[139] Raghu K. Ganti, Fan Ye, and Hui Lei. Mobile crowdsensing: Current state and future challenges. *IEEE Communications Magazine*, 2011.

[140] GeeksforGeeks. Mvvm (model view viewmodel) architecture pattern in android. https://www.geeksforgeeks.org/android/mvvm-model-view-viewmodel-architecture-pattern-in-android/. Dostęp: styczeń 2026.

[141] Dmitry Jemerov and Svetlana Isakova. *Kotlin in Action*. Manning Publications, 2017. Rozdział: Data classes and immutability.

[142] JetBrains. Data classes | kotlin documentation. https://kotlinlang.org/docs/data-classes.html. Dostęp: styczeń 2026.

[143] Google Developers. Jetpack compose - android developers. https://developer.android.com/compose. Dostęp: styczeń 2026.

[144] Google Developers. Stateflow i sharedflow | kotlin | android developers. https://developer.android.com/kotlin/flow/stateflow-and-sharedflow?hl=pl. Dostęp: styczeń 2026.

[145] Google. Fused location provider api | google for developers. https://developers.google.com/location-context/fused-location-provider. Dostęp: styczeń 2026.

[146] JetBrains. Coroutines overview | kotlin documentation. https://kotlinlang.org/docs/coroutines-overview.html#coroutine-context-and-behavior. Dostęp: styczeń 2026.

[147] Google Developers. Threading in android. https://developer.android.com/guide/components/processes-and-threads. Dostęp: styczeń 2026.

[148] Google Developers. Power management restrictions | android developers. https://developer.android.com/topic/performance/power/power-details. Dostęp: styczeń 2026.

[149] Google Developers. Foreground services | android developers. https://developer.android.com/guide/components/foreground-services. Dostęp: styczeń 2026.

[150] Google Developers. Broadcasts overview | android developers. https://developer.android.com/guide/components/broadcasts. Dostęp: styczeń 2026.

[151] Google Developers. Save key-value data | android developers. https://developer.android.com/training/data-storage/shared-preferences. Dostęp: styczeń 2026.

[152] Google Developers. Datastore | android developers. https://developer.android.com/topic/libraries/architecture/datastore, 2024. Dostęp: styczeń 2026.

[153] Google Developers. Telephonydisplayinfo | android developers. https://developer.android.com/reference/android/telephony/TelephonyDisplayInfo, 2024. Dostęp: styczeń 2026.

[154] Google Developers. Cellidentity | android developers. https://developer.android.com/reference/android/telephony/CellIdentity. Dostęp: styczeń 2026.

[155] JetBrains. callbackflow | kotlinx-coroutines-core. https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/callback-flow.html. Dostęp: styczeń 2026.

[156] JetBrains. Reflection | kotlin documentation. https://kotlinlang.org/docs/reflection.html. Dostęp: styczeń 2026.

[157] Google Developers. Locationrequest | google play services. https://developers.google.com/android/reference/com/google/android/gms/location/LocationRequest. Dostęp: styczeń 2026.

[158] Google Developers. Locationrequest.builder | android developers. https://developer.android.com/reference/android/location/LocationRequest.Builder. Dostęp: styczeń 2026.

[159] GISGeography. Gps accuracy: Hdop, pdop and gdop. https://gisgeography.com/gps-accuracy-hdop-pdop-gdop/. Dostęp: styczeń 2026.

[160] Wikipedia. Histereza. https://pl.wikipedia.org/wiki/Histereza. Dostęp: styczeń 2026.

[161] G. Almes, S. Kalidindi, and M. Zekauskas. A round-trip delay metric for ippm. RFC 2681, IETF, 1999. Dostęp: styczeń 2026.

[162] Netia. Jitter – co to jest i jaki ma wpływ na połączenie z internetem? https://www.netia.pl/pl/blog/jitter-co-to-jest. Dostęp: styczeń 2026.

[163] DNV. Terminology explained: P10, p50 and p90. https://www.dnv.com/article/terminology-explained-p10-p50-and-p90-202611/. Dostęp: styczeń 2026.

[164] NIST/SEMATECH. e-handbook of statistical methods: Percentiles. https://www.itl.nist.gov/div898/handbook/prc/section2/prc262.htm. Dostęp: styczeń 2026.

[165] Rob J. Hyndman and Yanan Fan. Sample quantiles in statistical packages. *The American Statistician*, 50(4):361–365, 1996. Dostęp: styczeń 2026.

[166] M. Allman, V. Paxson, and E. Blanton. Tcp congestion control. RFC 5681, IETF, 2009.

[167] Rohan S. Jahagirdar. Android http requests in kotlin with okhttp. https://medium.com/@rohan.s.jahagirdar/android-http-requests-in-kotlin-with-okhttp-5525f879b9e5. Dostęp: styczeń 2026.

[168] Microsoft Azure. Azure north europe region - data center locations. https://www.azurespeed.com/Information/AzureRegions/NorthEurope. Dostęp: styczeń 2026.

[169] JetBrains. Sealed classes and interfaces - kotlin documentation. https://kotlinlang.org/docs/sealed-classes.html. Dostęp: styczeń 2026.

[170] Edward R. Tufte. *Beautiful Evidence*. Graphics Press LLC, Cheshire, Connecticut, 2006.

[171] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, Reading, Massachusetts, 1994.

[172] IETF. The oauth 2.0 authorization framework - access tokens. https://oauth.net/2/access-tokens/. Dostęp: styczeń 2026.

[173] Riyas Pullur. Caching in android with kotlin. https://medium.com/@riyaspullurofficial/caching-in-android-with-kotlin-1ff2889ed4ad. Dostęp: styczeń 2026 (źródło archiwalne).

[174] Android Developers. Keeping your app responsive: Anrs. https://developer.android.com/topic/performance/anrs/diagnose-and-fix-anrs?hl=pl. Dostęp: styczeń 2026.

[175] Android Developers. Guide to app architecture: Single activity architecture. https://developer.android.com/topic/architecture. Dostęp: styczeń 2026.

[176] Cloudinary Ltd. Cloudinary kotlin sdk integration and usage guide. https://cloudinary.com/documentation/kotlin_integration. Dostęp: styczeń 2026.

[177] Android Developers. Webview - software architecture and engine. https://developer.android.com/reference/android/webkit/WebView. Dostęp: styczeń 2026.

[178] Microsoft Learn. Azure api management policy reference: Set http header. https://learn.microsoft.com/en-us/azure/api-management/set-header-policy. Dostęp: styczeń 2026.

[179] Mozilla Developer Network. Http headers: Cache-control. https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Cache-Control. Dostęp: styczeń 2026.

[180] Microsoft Learn. Azure api management policy reference: Rewrite url. https://learn.microsoft.com/en-us/azure/api-management/rewrite-uri-policy. Dostęp: styczeń 2026.

[181] Wikipedia. Operator wirtualny. https://pl.wikipedia.org/wiki/Operator_wirtualny. Dostęp: styczeń 2026.

[182] Telecompedia. Ran sharing: Architecture, benefits and challenges. https://telecompedia.net/ran-sharing/. Dostęp: styczeń 2026.

[183] PostgreSQL Global Development Group. Postgresql documentation: Chapter 8. data types. https://www.postgresql.org/docs/current/datatype.html. Dostęp: styczeń 2026.

[184] https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/#algorithm-detail. Dostęp: grudzień 2026.

[185] Murugiah Souppaya and Karen Scarfone. Guide to enterprise telework, remote access, and bring your own device (byod) security. Technical Report NIST SP 800-46 Rev. 2, National Institute of Standards and Technology (NIST), 2021.

[186] Microsoft. Web application firewall on azure application gateway. https://learn.microsoft.com/en-us/azure/web-application-firewall/ag/ag-overview. Dostęp: styczeń 2026.

[187] Microsoft. Web application firewall drs and crs rule groups and rules. https://learn.microsoft.com/en-us/azure/web-application-firewall/ag/application-gateway-crs-rulegroups-rules?tabs=drs21%2Cowasp32. Dostęp: styczeń 2026.

[188] J. P. Media. Jwt explained in 4 minutes with visuals. https://dev.to/jaypmedia/jwt-explained-in-4-minutes-with-visuals-g3n. Dostęp: styczeń 2026 (adaptacja na potrzeby projektu SignalMap).

[189] SmartBear Software. Bearer authentication | swagger docs. https://swagger.io/docs/specification/v3_0/authentication/bearer-authentication/. Dostęp: styczeń 2026.

[190] Bruce Schneier. *Applied Cryptography: Protocols, Algorithms, and Source Code in C*. John Wiley & Sons, New York, 2015.

[191] Microsoft Azure Documentation. Backup and restore in azure database for postgresql - flexible server. https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/concepts-backup-restore. Dostęp: styczeń 2026.

[192] Microsoft Azure Documentation. Business continuity and disaster recovery (bcdr) in azure database for postgresql. https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/concepts-business-continuity. Dostęp: styczeń 2026.

[193] Microsoft Azure Documentation. Reliability in azure database for postgresql - flexible server. https://learn.microsoft.com/en-us/azure/reliability/reliability-postgresql-flexible-server. Dostęp: styczeń 2026.

[194] Wikipedia. Inode. https://en.wikipedia.org/wiki/Inode, 2025. Dostęp: styczeń 2026.

[195] Apple Inc. Networkextension framework documentation: Hotspothelper and network analysis. https://developer.apple.com/documentation/networkextension. Dostęp: styczeń 2026.

[196] B. Campbell, J. Bradley, N. Sakimura, and T. Lodderstedt. OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens (RFC 8705), 2020.

[197] Microsoft. Azure kubernetes service networking concepts — azure cni overview. https://learn.microsoft.com/pl-pl/azure/aks/concepts-network-cni-overview, 2024. Dostęp: styczeń 2026.

[198] Tigera. Project calico — networking and network security for kubernetes. https://docs.tigera.io/calico/latest/about/, 2024. Dostęp: styczeń 2026.

Dodatek A

Repozytorium kodu

Repozytorium kodu dostępne jest pod adresem:
https://github.com/LobsterInSheol/SignalMap.



© 2026 Franciszek Mazoń, Jakub Medyński, Klaudiusz Zdrojewski

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu LaTeX na platformie Overleaf.